EECE 435L / Software Tools Lab

FINAL PROJECT SPRINT I

Security Incident Detection and Response Management System

Backend implementation and testing

Project team

Christophe ElChabab / Threats, Vulnerabilities, Incidents
Joey Saade / Users, Assets, Notifications
Karl Khouri / Audit Logging, Playbooks, Reports

Submission date: 20/10/2026

9 SERVICES 4 USER ROLES REST APIs + SQLITE

Includes the implementation, API documentation, database design and 180 captures of API execution, stored records and Git contributions.

Contents

1 Introduction (Christophe ElChabab, Joey Saade) 1

2 System Architecture (Christophe ElChabab, Joey Saade, Karl Khouri) 1

3 Implementation Details (Christophe ElChabab, Joey Saade, Karl Khouri) 2

3.1 Users, Assets and Notifications (Joey Saade) 2

3.2 Threats, Vulnerabilities and Incidents (Christophe ElChabab) 3

3.3 Audit Logging, Playbooks and Reports (Karl Khouri) 5

3.4 Incident, remediation and response workflows (Christophe ElChabab, Karl Khouri) . 7

4 Database Design (Christophe ElChabab, Joey Saade, Karl Khouri) 7

4.1 Identity, assets and notifications (Joey Saade) 8

4.2 Threats, vulnerabilities and incidents (Christophe ElChabab) 9

4.3 Audit and response records (Karl Khouri) 9

5 Error Handling and Validation (Christophe ElChabab, Joey Saade, Karl Khouri) 10

6 API Documentation (Christophe ElChabab, Joey Saade, Karl Khouri) 11

6.1 Threats, Vulnerabilities and Incidents (Christophe ElChabab) 11

6.2 Users Assets and Notifications (Joey Saade) 12

6.3 Audit Logging Playbooks and Reports (Karl Khouri) 13

6.4 Inputs, outputs and Postman files (Christophe ElChabab, Joey Saade, Karl Khouri) 14

6.5 Three API examples from the captured run (Christophe ElChabab) 16

6.6 Importable collections and live documentation (Christophe ElChabab, Joey Saade, Karl Khouri) 17

7 Authentication and Authorization (Christophe ElChabab, Joey Saade, Karl Khouri) 17

8 GitHub and Version Control (Christophe ElChabab, Joey Saade, Karl Khouri) 18

8.1 CREATES feedback (Christophe ElChabab, Joey Saade, Karl Khouri) 23

9 References (Christophe ElChabab, Joey Saade, Karl Khouri) 23

A Postman and Database Evidence (Christophe ElChabab) 24

B Postman and Database Evidence (Joey Saade) 119

C Postman and Database Evidence (Karl Khouri) 152

1 Introduction (Christophe ElChabab, Joey Saade)

Project overview. A reported security problem needs more than a description: the team must know which asset is affected, who is investigating it, what has already been done and whether the issue is resolved. Our backend keeps those records together and controls who can read or change them.

The system covers the nine services in the Sprint I handout. Employees report incidents, analysts investigate assigned incidents, administrators manage records and assignments, and auditors review the information without changing it. Threat indicators and vulnerabilities give context to investigations. Notifications, audit records and reports make the resulting activity visible to the appropriate users.

Scope and objectives. This sprint is backend-only; there is no frontend application. It extends the API, database, documentation and Git work from the earlier labs into a team project. Our main objectives are to:

- provide the required REST endpoints with each owner's full name in the route;
- store related records with primary keys, foreign keys and validation checks;
- authenticate callers and enforce both role and record-level permissions;
- test valid requests, invalid input and forbidden operations in Postman;
- demonstrate service integration and record each member's work in Git.

2 System Architecture (Christophe ElChabab, Joey Saade, Karl Khouri)

Overall design. Each service is a separate FastAPI process with its own HTTP port. The local implementation uses one SQLite file so references to users and assets remain consistent across the services. Services also communicate over HTTP: they deliver audit events and notifications, and the Reports service calls the other services' list APIs. This is a shared-database local design, not nine independently deployed databases.

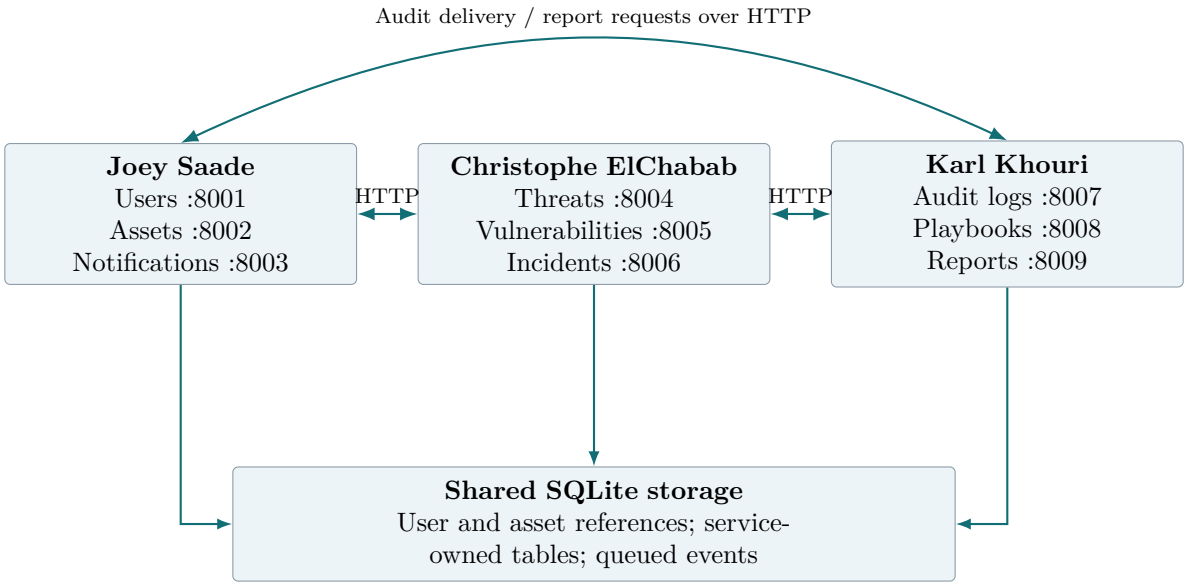

Figure 1. Christophe ElChabab - System architecture - Nine service processes communicate through APIs and share local storage. The ports identify the process; the route prefix identifies its owner.

Owner	Responsibility	Route prefix
Joey Saade	Account identity, organizational assets and user notifications	`/api/Joey-Saade`
Christophe ElChabab	Threat indicators, asset vulnerabilities and incident investigations	`/api/Christophe-ElChabab`
Karl Khouri	Security audit records, ordered response actions and security reports	`/api/Karl-Khouri`

Communication between services. Users log in through Joey's Users service. The returned bearer token is sent with protected requests. A change that needs an audit entry or notification first saves that event in the `outbox` table together with the record change. After the transaction commits, delivery calls the destination API. If delivery fails, the saved event can be retried. A unique event ID prevents a retry from creating a second logical event.

Karl's reports request lists from the owning APIs using the caller's token. For example, the vulnerability report uses Christophe's vulnerability list rather than directly counting his table. This keeps access checks at the service boundary and makes a failed source visible instead of reporting a misleading zero.

3 Implementation Details (Christophe ElChabab, Joey Saade, Karl Khouri)

The service explanations below follow the handout's responsibilities and required operations. Section 6 gives the endpoint reference and input/output contracts. The linked evidence appendices contain the actual Postman and database captures.

3.1 Users, Assets and Notifications (Joey Saade)

The Users, Assets and Notifications services run on ports 8001–8003 under `/api/Joey-Saade`. They share authentication code and SQLite storage. The implementation is in users.py, assets.py, notifications.py and common.py; schema.sql defines the tables.

3.1.1 Users and Identity (Joey Saade)

Public registration accepts a name, username, email, password and department. The public route creates only a Regular Employee; an attempt to submit an administrator or other privileged role is rejected. Usernames and emails are unique without regard to case. Passwords are checked for length and letter/digit content, then stored as PBKDF2-SHA256 hashes with individual random salts. Responses exclude the hash.

Login accepts a username or email and password, then returns a signed bearer token with a one-hour expiry. Protected requests verify the signature and expiration and reload the current account, so deleting, suspending or deactivating an account removes its access without waiting for the token to expire. Administrators and auditors can list users. A user may inspect and update their own profile; administrator-only role and status changes use the extra `PATCH /users/{id}/admin` route. The administrator cannot delete their own account. Foreign-key references from assets and notifications prevent deletion of a user still in use.

Registration and administrator bootstrap reserve IDs in `user_id_allocations`. Issued IDs remain reserved after a user is deleted. This prevents both reuse of an old token subject and collision with the retained `welcome-user-{id}` outbox event. Initialization seeds an older database from current users and historical welcome and user-audit events. A regression test covers register, delete and

register again, plus rejection of the deleted account's token.

3.1.2 Asset Management (Joey Saade)

An asset records a name, type, IP address, department, owner, location, criticality, status and optional operating system. Creation requires an existing owner and validates the type, IP address and enumerated values before insertion. Administrators create, update and delete assets. Administrators and auditors list assets; administrators, analysts and auditors can retrieve and search them. Employees have no asset read access in the implemented permission policy.

Search combines supplied filters for name, type, IP address, department and criticality. The name filter matches a substring. Each create, update and delete writes an `asset_history` snapshot with the original asset ID, change timestamp and actor. History remains available after asset deletion. The database restricts deletion while an incident or vulnerability still references the asset; the API returns 409 for a referenced asset.

3.1.3 Notification and Alert Management (Joey Saade)

Notifications store recipient ID, title, message, type, severity, read status and timestamps. Administrators and authenticated peer services can create them; a peer uses `X-Service-Key` and supplies an event ID. Repeating the same event ID returns the existing notification, which makes delivery retries idempotent. The recipient can retrieve and mark their own notification read. Administrators can search and delete notifications, auditors have read-only access, and analyst searches are confined to the analyst's own records.

Registration queues a Welcome notification in the same database transaction as the new user. User, asset and notification mutations also queue audit events. After commit, the outbox worker sends notification and audit events over HTTP to the respective services. Failed deliveries stay in the outbox for retry. The ordered Postman collection reads records before cleanup and deletes the sample asset and its notifications before deleting the sample user, preserving foreign-key integrity.

3.1.4 Recorded checks and captured results (Joey Saade)

The saved focused test output records seven passing tests, including the deleted-user ID regression. A saved Newman run of Joey's exported collection against nine local service processes records 27 requests, 29 assertions and zero failures. Appendix B includes 27 individual Postman request/response captures and five database views. The API images show bodies or query parameters, responses and status codes. The database queries show the updated user role, asset history, notification state, queued events and cleanup results from an earlier run; their IDs are separate from the later API captures.

3.2 Threats, Vulnerabilities and Incidents (Christophe ElChabab)

The Threat Intelligence, Vulnerability Management and Incident Management services run as separate FastAPI processes on ports 8004, 8005 and 8006. Their 18 business endpoints use the prefix `/api/Christophe-ElChabab`. The source files are `christophe/threats.py`, `christophe/vulnerabilities.py`, `christophe/incidents.py` and `christophe/schema.sql`.

Joey Saade's Users service authenticates users and issues bearer tokens. Each request checks the authenticated account and its current role. Users and Assets are shared database entities; Christophe's services reference them instead of creating separate copies. Mutations queue audit and notification events in the same database transaction. After commit, delivery calls Karl Khouri's Audit service and Joey Saade's Notifications service over HTTP. Failed deliveries remain queued for retry. Karl's Reports service reads the list APIs over HTTP.

3.2.1 Threat Intelligence (Christophe ElChabab)

Administrators create indicators with a type, value, description, severity and source. Reads return IDs and UTC creation/update timestamps alongside those fields. List and search return JSON arrays; detail returns one object. Search accepts type, value, severity and source, combining supplied conditions with AND. Updates accept description, severity and source. Deletion returns a confirmation message.

IP addresses, domain syntax and hexadecimal file hashes are validated according to their indicator type. Normalization precedes the unique type/value check, so repeated indicators return 409. The captured duplicate, invalid address/hash/domain and missing-field requests demonstrate rejection. Only administrators can mutate threats; analysts and auditors can read/search. The professor confirmed that auditors may use GET /threats/search.

Two synthetic indicators were used in the real local execution: a Domain indicator and IP indicator 198.51.100.42. They are test inputs, not assertions of real malicious activity. The domain was initially High and then updated to Critical with source Investigation; the IP was Critical with source SOC. These classifications exercise the stored severity/source fields and filtering. Before Cleanup, the database and threat report both contained two Critical indicators. Successful deletion and the subsequent 404 for the domain are captured separately.

3.2.2 Vulnerability Management (Christophe ElChabab)

Administrators create vulnerabilities with title, description, CVE ID, CVSS score, explicit severity, affected asset ID and source. Creation returns the stored object with status Open. Reads and searches return the stored fields and server timestamps. Search supports CVE ID, severity, asset ID and status, combining supplied filters with AND.

CVSS must be a finite number from 0 to 10; booleans are not valid scores or IDs. CVE validation checks the CVE-YYYY-NNNN syntax with at least four final digits; it does not verify a vulnerability against an external registry. The affected asset must exist. Captures demonstrate malformed CVEs, invalid CVSS and an unknown asset returning 422. Administrators and analysts can update permitted fields; auditors cannot update, and only administrators can delete.

The required remediation sequence is Open -> Under Investigation -> In Progress -> Resolved -> Closed. Skipping a state returns 409. The server sets the resolution timestamp at Resolved and retains it at Closed. Every transition is captured.

The first test record starts at CVSS 8.1/High, then is updated to 9.1/Critical and reaches Closed. The second remains 5.0/Medium/Open. The Critical/High/Medium values are explicit test classifications; they demonstrate priority differences rather than verified production risk. Before Cleanup, the report and database agree on one Critical Closed record and one Medium Open record, with average CVSS $(9.1 + 5.0) / 2 = 7.05$.

3.2.3 Incident Management (Christophe ElChabab)

Administrators and employees can report an incident with title, description, category, severity, reporter ID and asset ID. The reporter and asset must exist. An employee cannot submit under another user's identity. Creation returns a new Open incident; assignment requires an active Security Analyst and administrator permission.

The professor confirmed that GET /incidents is available to all four roles. Administrators and auditors see all incidents, analysts see only their assignments, and employees see only their submissions. These restrictions derive from authenticated identity. Forged reporter, assignee or role query parameters do not broaden access. The screenshots show employee 4 seeing records 1 and 2, while the other employee's record 3 is excluded; analyst 2 sees only assigned record 1.

The sequence is Open -> Under Investigation -> Mitigated -> Resolved -> Closed. Only administrators or the assigned analyst may change status. Skips are rejected with 409. The captured workflow includes all transitions and retention of the resolution timestamp.

Authorized staff add a nonblank investigation note. The server records the authenticated author, incident ID and creation timestamp, rather than accepting a forged author from the request. The stored note is linked to incident 1 and analyst 2 and appears in the retrieved incident. No incident DELETE endpoint is added because the handout does not request one.

The three test incidents use High, Critical and Medium severity. High represents the primary suspicious-message investigation, Critical the higher-priority comparison scenario, and Medium the other reporter's lower-priority scenario. These are test classifications, not evidence that an actual compromise occurred. The different reporters and severities exercise priority storage and identity-based access. At the end of the captured workflow, incident 1 is Closed/High and assigned to analyst 2; incident 2 is Open/Critical and incident 3 is Open/Medium.

3.3 Audit Logging, Playbooks and Reports (Karl Khouri)

Karl Khouri's three FastAPI processes run on ports 8007–8009 and expose the required routes under `/api/Karl-Khouri`. They reuse the team's bearer-token verification and the shared SQLite database, so audit and playbook records hold foreign keys to existing users. The source files are `karl/audit_logs.py`, `karl/playbooks.py`, `karl/reports.py`, `karl/common.py` and `karl/schema.sql`. The request and response contract is documented in `docs/karl-api.md`.

3.3.1 Security Audit Logging (Karl Khouri)

An audit entry records an event ID, actor, action, description, severity, related service, subject ID, the time the event occurred and the time it was stored. Severity is Information, Warning, Error or Critical. A `source` field records whether the entry came from a peer service, an administrator or the audit service itself.

Peer services send events with an `X-Service-Key` header holding `AUDIT_SERVICE_AUTH_SECRET`. This key must differ from the notification key, so a credential meant for notifications cannot write audit records. Service events must carry an event ID and related service. Repeating an event ID returns the entry already stored instead of creating a second one, which makes delivery retries safe. This holds even when two deliveries of the same event arrive at once. An administrator can also add a manual entry, but the actor and time are taken from the token and the server clock. A request that tries to name a different actor or set its own time is rejected with 422.

Events are produced automatically by the other services: successful and failed logins, user creation, update and deletion, role and status changes, asset and notification changes, incident, threat and vulnerability changes, playbook activity, report generation and every request refused for lack of permission (403). Peer services first store each event in the transactional `outbox` table, then deliver it over HTTP. If the audit service is unavailable, the event stays queued for retry and the original request still succeeds. A failed login has no authenticated actor, so it is stored with a null actor ID and the attempted username as the label.

Administrators and auditors can list every entry and read the history of any user through `GET /logs/user/{user_id}`. Analysts can retrieve a single entry and search, as the handout allows; employees have no access to the audit APIs. There is no self-access exception: an analyst or employee asking for their own history also receives 403. A user ID with no recorded activity returns an empty list. Search combines actor, action, severity, related service, subject and a start/end date range. Only administrators can delete an entry, and each deletion writes a permanent `audit_log_deleted` entry naming the removed record. These deletion records cannot themselves be deleted (409).

Entries cannot be edited. A database trigger rejects any UPDATE of an audit row. The only

change it allows is the foreign key setting `actor_user_id` to null when the user is deleted. `original_actor_user_id` and `actor_label` keep the history readable after that. The audit service writes its own denials and deletions directly to its table instead of sending them to itself over HTTP, so logging cannot loop.

3.3.2 Security Playbook and Response Automation (Karl Khouri)

A playbook has a unique name, description, category, severity, creator and timestamps. Categories are Malware, Phishing, Ransomware, Unauthorized Access, Data Breach, Denial of Service, Insider Threat, Vulnerability Remediation and Other; severity is Low, Medium, High or Critical. The creator is taken from the token. Only administrators create and delete playbooks, as the handout specifies. Administrators and analysts add response actions and update their status; auditors can only read playbooks; employees have no access.

Each response action is an ordered step with a description and a step number, the handout's required information; a title and an assignee are optional. Steps must be numbered 1, 2, 3 and so on without gaps: a gap returns 422, and reusing a number returns 409. The database also enforces one step per number in each playbook. An assignee must be an active Security Analyst or Security Administrator. Assigning a step, at creation or later, sends that person a notification through the outbox.

`PUT /actions/{id}/status` accepts the handout's three updates: status, assigned user and completion date. A step's status moves only from Pending to In Progress to Completed. Skipping In Progress or moving backwards returns 409, a step cannot start until every earlier step is Completed, and an unassigned step cannot change status. A completion date may accompany the move to Completed; it cannot be in the future or before the step started. A completed step cannot be reassigned. The start time, completion time and completing user are stored on the step. Each change also adds a row to `action_history` with the old and new status, the user and an optional note. A playbook's detail shows its steps in order and an overall execution state of Not Started, In Progress or Completed. A playbook with a step in progress cannot be deleted. After deletion, its execution history remains readable through `GET /playbooks/{id}/history`.

3.3.3 Security Reports (Karl Khouri)

Six reports are available to administrators and auditors: summary, incidents, assets, threats, vulnerabilities and activity. Reports do not read other services' tables. Each one calls the owning service's list API over HTTP and forwards the requesting user's own token, so the upstream service applies its own permissions again. The summary draws on all the handout's data sources (Users and Identity, Assets, Incidents, Threat Intelligence and Audit Logging) plus Vulnerabilities. Upstream addresses are configured by environment variables, not hard-coded table access.

The summary's users section gives the total number of users by role, status and department. The incident report gives totals by status and severity, the number still open and the number unassigned. The asset report groups by type, criticality, status and department and counts active High or Critical assets. The threat report groups by type and severity. The vulnerability report groups by severity and status and gives the open count and average CVSS score. The activity report, built from the audit log, counts events by action, severity and service, failed logins, denied requests and the most active users, and lists the five most recent security activities. Every report accepts optional `start_date` and `end_date` filters and records a `report_generated` audit event.

If an upstream service is unreachable, the report returns 503; if it replies with an error or an unexpected format, the report returns 502. No zero values are substituted. The summary report lists each source it could not reach under `unavailable`, shows that section as null, and sets `complete` to false. A complete summary therefore shows that every figure came from real stored records.

3.3.4 Recorded checks and captured results (Karl Khouri)

The saved focused test output records the original seven passing tests for the three services (`docs/evidence/karl-pytest-2026-10-18.txt`); two regression tests were added on 20 October 2026. They cover audit delivery from the outbox, deduplication, forged keys, actor spoofing, immutability, user deletion, playbook ordering and transitions, and report reconciliation and upstream failure. A saved six-process live HTTP check (`docs/evidence/karl-live-http-2026-10-18.txt`) and a saved Newman run of Karl's exported collection against nine local service processes (`docs/evidence/karl-postman-newman-2026-10-20.txt`) record the services working over real HTTP. The Newman run had 57 requests, 77 assertions and zero failures. Capturing the screenshots exposed one defect: after a playbook was deleted, SQLite reused its step IDs, the new step's notification event ID collided with the old one, and creating the next playbook failed with HTTP 500. AUTOINCREMENT keys and a startup migration fixed it, and a regression test covers it. The Postman and database screenshots appear in the evidence appendix.

3.4 Incident, remediation and response workflows (Christophe ElChabab, Karl Khouri)

The handout gives different lifecycles to incidents and vulnerabilities. In particular, an incident is Mitigated, while a vulnerability moves to In Progress. We keep those values separate instead of using one shared status list. Response actions have their own shorter sequence. A request cannot skip a state or move backwards.

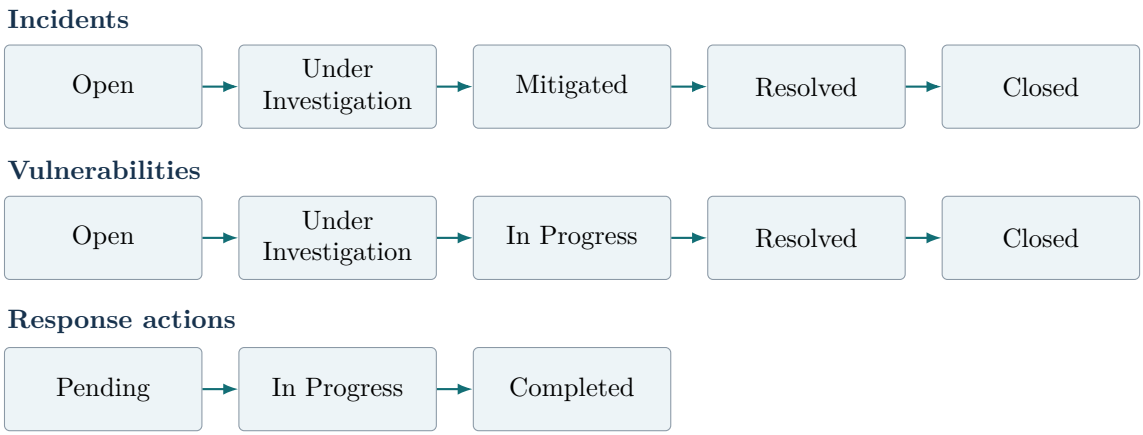

Figure 2. Christophe ElChabab - Required status workflows - Incident and vulnerability updates follow every arrow. Karl Khouri's response actions also require all earlier steps to be completed before the next step starts.

For incidents, the server records the resolution time at Resolved and retains it at Closed. Vulnerabilities follow the same default, but also accept a valid timezone-aware resolution date between creation and the current time on a Resolved/Closed record. Response actions record start/completion times and who made each change.

4 Database Design (Christophe ElChabab, Joey Saade, Karl Khouri)

Design decisions. One shared SQLite file holds identity and asset records referenced by the other services. Every main entity has an integer primary key. Foreign keys stop a record from referring to a missing user, asset, incident or playbook. CHECK constraints limit status/severity values and CVSS ranges; unique constraints prevent duplicate usernames, threat indicators and playbook step

numbers.

The three schema diagrams below divide the database by service ownership while repeating shared tables where necessary. Arrows point from the table holding a foreign key to the table it references. The actual field definitions are available in the linked Joey schema, Christophe schema and Karl schema.

4.1 Identity, assets and notifications (Joey Saade)

Table	Main fields and relationships
`users`	ID, full name, username, email, password hash, department, role, account status, creation/last-login times. Username and email are unique.
`assets`	ID, name, type, IP address, operating system, department, location, criticality, status, creation/update times. `owner_id` references Users.
`notifications`	ID, recipient, title, message, type, severity, Unread/Read status, creation/read times and optional unique event ID. `user_id` references Users.
`asset_history`	Original asset ID, nullable live asset ID, actor, time, action and before/after snapshots. The history survives asset deletion.
`outbox`	Unique event ID, destination, payload, creation/delivery times, attempts and last error. Stores events waiting for HTTP delivery.
`user_id_allocations`	Retains every issued user ID after deletion so a later account cannot inherit an old token's identity.

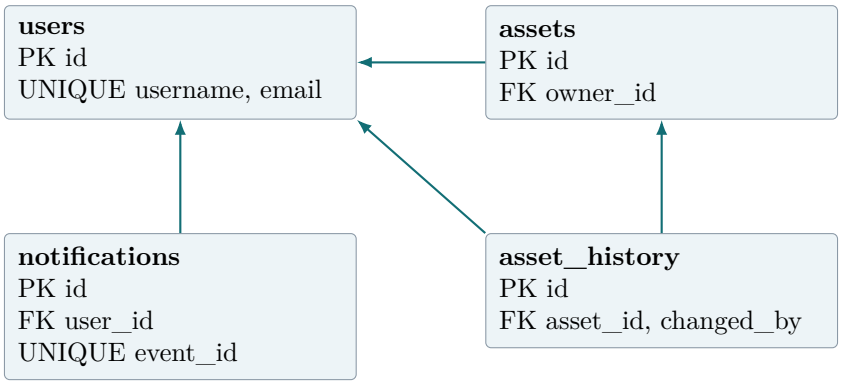

Figure 3. Christophe ElChabab - Identity and asset relationships - Joey Saade's Users table is referenced by owned assets, notifications and asset-history actors. Outbox and issued-ID tables have no foreign-key arrows.

One user can own many assets and receive many notifications. Deleting a referenced user is restricted. Deleting an asset sets its live history reference to null, while the original ID and snapshots remain available. Joey's database captures are in Appendix B.

4.2 Threats, vulnerabilities and incidents (Christophe ElChabab)

Table	Main fields and relationships
`threats`	ID, type, value, description, severity, source, creation/update times. The pair (type, value) is unique.
`vulnerabilities`	ID, title, description, CVE, CVSS, severity, source, status, creation/update/resolution times; `asset_id` references Assets.
`incidents`	ID, title, description, category, severity, status, creation/update/resolution times; reporter, assignee and affected asset reference Users/Assets.
`investigation_notes`	ID, content, incident, analyst and creation time. One incident has many notes; each note has one authenticated author.

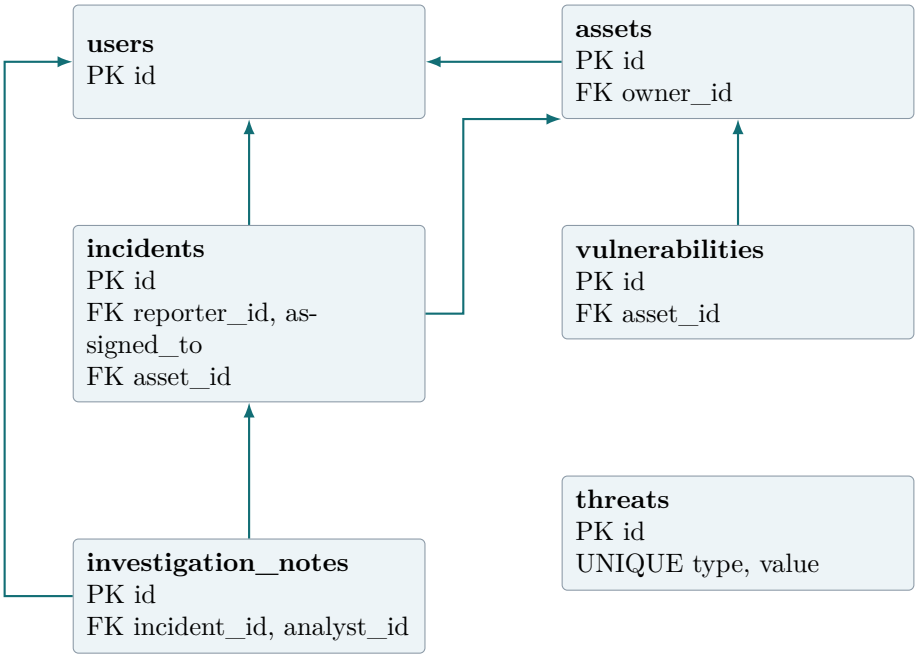

Figure 4. Christophe ElChabab - Incident and vulnerability schema - Shared users and assets link investigations to their reporter, analyst and affected asset. Threat indicators are independent records.

One asset can have several vulnerabilities and incidents. Users/assets cannot be deleted while these records reference them. Investigation notes use ON DELETE CASCADE for their incident reference. Severity, CVSS and status checks also run at database level. Actual table definitions and populated records appear in Appendix A.

4.3 Audit and response records (Karl Khouri)

Karl Khouri's four tables are defined in `karl/schema.sql` and created after the users table they reference; the Reports service stores no tables. All use integer primary keys. `audit_logs`, `playbooks` and `response_actions` use AUTOINCREMENT, so a deleted entry's ID is never given to a new row. Deletion records, execution history and notification event IDs keep pointing at the right record. Databases created before this change are rebuilt on startup with their rows kept.

`audit_logs` stores one row per event with a unique `event_id`. `actor_user_id` references `users.id` with ON DELETE SET NULL, while `original_actor_user_id` and `actor_label` keep the actor after the account is deleted. CHECK constraints limit severity to Information, Warning, Error and Critical and the source to service, user or audit. Indexes on the original actor and the occurrence time support user activity lookups and date-range searches. The `audit_logs_immutable` trigger aborts any UPDATE except that foreign-key nulling.

`playbooks` has a case-insensitive unique name, category and severity CHECK constraints, and `created_by` referencing `users.id`. `response_actions` references `playbooks.id` with ON DELETE CASCADE and `users.id` through `assigned_to` (nullable until a user is assigned) and `completed_by`. Its status is limited to Pending, In Progress and Completed, and `UNIQUE(playbook_id, step_order)` prevents two steps sharing a number. `action_history` references the action, playbook and changing user with ON DELETE SET NULL. It also stores the original action and playbook IDs, so the execution history survives deletion. One playbook has many response actions, one action has many history rows, and one user can create many playbooks, be assigned many actions and appear in many audit entries. A user who created a playbook or is assigned a step cannot be deleted while those references remain.

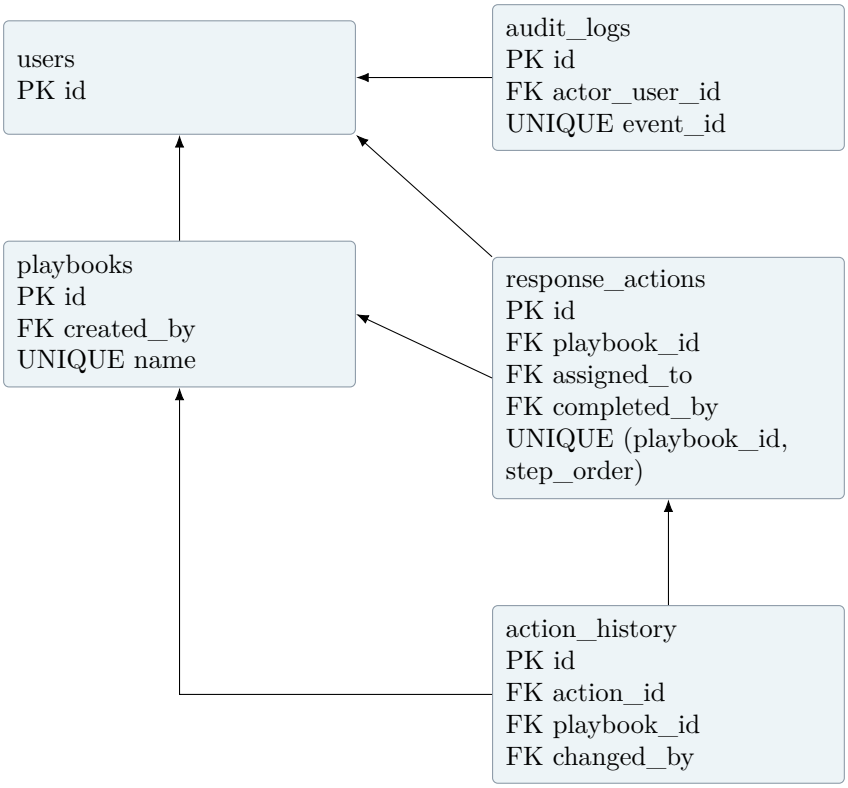

Figure 5. Karl Khouri - Audit and playbook schema - Arrows point from foreign-key tables to referenced tables. Users belong to Joey Saade's service.

5 Error Handling and Validation (Christophe ElChabab, Joey Saade, Karl Khouri)

Error handling approach. Requests are checked before a change is written. A failed operation returns a useful status and JSON message rather than a successful response with missing data. The shared response format is:

```
{"status": "error", "message": "..."}
```

This is the response format, not a captured execution. Field-validation errors can also include an `errors` array. Actual rejected requests are shown in the evidence appendices.

HTTP	Meaning	Example handled by the backend
401	Authentication required	Incorrect password; absent, invalid or expired token; invalid audit service key.
403	Permission refused	Employee reading threats; auditor updating a vulnerability; unassigned analyst changing an incident.
404	Record not found	Unknown user, asset, threat, vulnerability, incident, audit entry or playbook.
409	Conflicting operation	Duplicate indicator, referenced deletion, duplicate step number or skipped lifecycle state.
422	Invalid input	Invalid email/IP/CVE/CVSS, missing fields, unknown asset, invalid assignee or malformed date.
502/503	Report source failure	An upstream API returns unusable data or cannot be reached.

Validation rules. Users require a valid email, unique username and a password meeting the configured length/content rules. Assets require valid IP, type, criticality and status values. Threats validate their value according to indicator type and reject duplicate normalized indicators. Vulnerabilities validate CVE syntax, CVSS in 0–10 and the affected asset. Boolean CVSS/IDs, empty updates and unknown body fields are rejected in Christophe's services. Incidents and response actions enforce the workflows in Figure 2.

Security checks. SQL values are passed as bound parameters, not joined into query text. The server chooses permitted filter columns and checks role/ownership before returning a record. Role changes cannot be requested through public registration or ordinary profile updates. Audit actors and investigation-note authors come from authenticated identity, so a request cannot simply name another author. Forbidden requests are recorded by the audit integration.

Checking the result against the database. In Christophe's populated run, the stored records and reports agree on two Critical threats, two vulnerabilities and three incidents. One vulnerability is Closed/Critical at 9.1 and the other is Open/Medium at 5.0, giving mean CVSS 7.05. Incident 1 is Closed/High; the other two are Open/Critical and Open/Medium. Later Cleanup screenshots show deletion of the test threats/vulnerabilities, so those later responses are not inconsistent with the earlier populated database images.

6 API Documentation (Christophe ElChabab, Joey Saade, Karl Khouri)

The tables retain the handout's HTTP methods and endpoint suffixes. Prefix each suffix with the named owner's route prefix and use the port in Figure 1. Unless stated otherwise, protected requests send `Authorization: Bearer <token>`; JSON requests use `Content-Type: application/json`.

6.1 Threats, Vulnerabilities and Incidents (Christophe ElChabab)

Prefix: `/api/Christophe-ElChabab`. Administrator, Analyst, Auditor and Employee refer to the four roles explained in Section 7.

Method	Suffix	Purpose	Authorized roles
POST	/threats	Create indicator	Administrator
GET	/threats	List indicators	Administrator, Analyst, Auditor
GET	/threats/{id}	Retrieve indicator	Administrator, Analyst, Auditor

Method	Suffix	Purpose	Authorized roles
GET	/threats/search	Search indicators	Administrator, Analyst, Auditor
PUT	/threats/{id}	Update threat information	Administrator
DELETE	/threats/{id}	Delete indicator	Administrator
POST	/vulnerabilities	Create vulnerability	Administrator
GET	/vulnerabilities	List vulnerabilities	Administrator, Analyst, Auditor
GET	/vulnerabilities/{id}	Retrieve vulnerability	Administrator, Analyst, Auditor
GET	/vulnerabilities/search	Search vulnerabilities	Administrator, Analyst, Auditor
PUT	/vulnerabilities/{id}	Update fields/remediation state	Administrator, Analyst
DELETE	/vulnerabilities/{id}	Delete vulnerability	Administrator
POST	/incidents	Report incident	Administrator, Employee
GET	/incidents	List within authenticated scope	All four roles, restricted
GET	/incidents/{id}	Retrieve incident	Administrator, Auditor, assigned Analyst, original Employee
PUT	/incidents/{id}/assign	Assign analyst	Administrator
PUT	/incidents/{id}/status	Change lifecycle state	Administrator, assigned Analyst
POST	/incidents/{id}/notes	Add investigation note	Administrator, assigned Analyst

New threats, vulnerabilities, incidents and notes return 201. Reads, searches, updates and deletions return 200. List/search results are JSON arrays; detail responses are objects, and deletions return a message. An empty valid search is `[]`. See Section 5 for errors and Appendix A for actual request/response captures.

6.2 Users Assets and Notifications (Joey Saade)

All suffixes in this table follow `/api/Joey-Saade`. Administrator, Analyst and Auditor refer to the corresponding Security roles; Employee means Regular Employee. The first 18 rows are the required routes. The last two document implemented supporting routes.

Method	Suffix	Function	Access
POST	/users/register	Register an Employee	Public
POST	/users/login	Issue a bearer token	Public
PUT	/users/{id}	Update permitted profile fields	Administrator, self
DELETE	/users/{id}	Delete an unreferenced user	Administrator
GET	/users	List users	Administrator, Auditor
GET	/users/{username}	Retrieve a permitted profile	Administrator, Auditor, self
POST	/assets	Create an asset	Administrator
PUT	/assets/{id}	Update an asset	Administrator
DELETE	/assets/{id}	Delete an unreferenced asset	Administrator
GET	/assets	List assets	Administrator, Auditor

Method	Suffix	Function	Access
GET	/assets/{id}	Retrieve an asset	Administrator, Analyst, Auditor
GET	/assets/search	Filter assets	Administrator, Analyst, Auditor
POST	/notifications	Create a notification	Administrator, service key
GET	/notifications/user/{user_id}	List a recipient's notifications	Recipient, Administrator, Auditor
GET	/notifications/{id}	Retrieve a notification	Recipient, Administrator, Auditor
PUT	/notifications/{id}/read	Mark notification read	Recipient, Administrator
GET	/notifications/search	Filter notifications	Administrator, Auditor, Analyst (own)
DELETE	/notifications/{id}	Delete a notification	Administrator
PATCH	/users/{id}/admin	Change role or status	Administrator
GET	/assets/{id}/history	Read asset history	Administrator, Auditor

Creation returns 201; successful reads, updates and deletions return 200. Missing or invalid authentication returns 401, insufficient permission returns 403, absent records return 404, duplicate or referenced records return 409, and invalid input returns 422. Error bodies carry a status and message, with field errors for request-shape validation. The importable collection and credential-free template are `postman/joey.collection.json` and `postman/joey.environment.template.json`. Credentials and bearer tokens are entered only in a private local environment.

6.3 Audit Logging Playbooks and Reports (Karl Khouri)

All suffixes in this table follow `/api/Karl-Khouri`. Administrator, Analyst and Auditor refer to the corresponding Security roles; Employee means Regular Employee. The first 18 rows are the required routes. The last row documents an implemented supporting route.

Method	Suffix	Function	Access
POST	/logs	Store an audit event; repeated event IDs return the original	Service key, Administrator
GET	/logs	List audit entries, newest first	Administrator, Auditor
GET	/logs/{id}	Retrieve an audit entry	Administrator, Analyst, Auditor
GET	/logs/search	Filter by actor, action, severity, service, subject and date range	Administrator, Analyst, Auditor
DELETE	/logs/{id}	Delete an entry and record the deletion	Administrator
GET	/logs/user/{user_id}	List one user's activity; empty list if none	Administrator, Auditor
POST	/playbooks	Create a playbook with optional ordered steps	Administrator

Method	Suffix	Function	Access
GET	/playbooks	List playbooks with steps and progress	Administrator, Analyst, Auditor
GET	/playbooks/{id}	Retrieve a playbook, steps and execution state	Administrator, Analyst, Auditor
POST	/playbooks/{id}/actions	Add an ordered step (description, step number; optional assignee)	Administrator, Analyst
PUT	/actions/{id}/status	Update status, assigned user or completion date	Administrator, Analyst
DELETE	/playbooks/{id}	Delete a playbook not in progress	Administrator
GET	/reports/summary	Combined figures: users plus the five sources below	Administrator, Auditor
GET	/reports/incidents	Incidents by status and severity, open, unassigned	Administrator, Auditor
GET	/reports/assets	Assets by type, criticality, status, department	Administrator, Auditor
GET	/reports/threats	Threats by type and severity	Administrator, Auditor
GET	/reports/vulnerabilities	Vulnerabilities by severity and status, average CVSS	Administrator, Auditor
GET	/reports/activity	Audit activity by action, severity, service and user	Administrator, Auditor
GET	/playbooks/{id}/history	Read step creation and status history	Administrator, Analyst, Auditor

Creation returns 201; successful reads, searches, status changes and deletions return 200, and a repeated audit event ID returns 200 with the stored entry. Missing or invalid authentication, or a wrong service key, returns 401. Insufficient permission returns 403, absent records return 404, and duplicate step numbers, invalid status transitions and protected deletions return 409. Invalid input, out-of-order steps and invalid date filters return 422. A report whose upstream service is unreachable returns 503, and one whose upstream replies with an error returns 502. Error bodies carry a status and message. The importable collection and credential-free template are `postman/karl.collection.json` and `postman/karl.environment.template.json`. The administrator's credentials and the audit service key are entered only in a private local environment.

6.4 Inputs, outputs and Postman files (Christophe ElChabab, Joey Saade, Karl Khouri)

Path parameters such as `id`, `user_id` and `username` identify the target record. List and search GET requests have no JSON body; the filters below are query parameters. DELETE requests identify the record through the path. A successful detail request returns one JSON object, while list/search requests return an array. The returned objects include the stored ID and applicable timestamps.

6.4.1 Users, Assets and Notifications contracts (Joey Saade)

Users. Registration requires `full_name`, `username`, `email`, `password` and `department`. The `role` defaults to Regular Employee; if supplied publicly, it must still be that role. The response excludes the password/hash. Login accepts `username_or_email` and `password`; its response contains `access_token`, `token_type` and `expires_in`. Profile PUT accepts a nonempty selection of `full_name`, `email` and `department`. The supporting admin PATCH accepts `role` and/or `status`. GET returns public user fields; DELETE returns a confirmation. Unknown usernames return 404, duplicates 409 and invalid public role requests 403.

Assets. POST requires `name`, `type`, `ip_address`, `department`, `owner_id`, `location` and `criticality`; `operating_system` and `status` are optional. PUT accepts changes to these fields. Search accepts `name`, `type`, `ip_address`, `department` and `criticality`. Records return these fields with ID and creation/update times. The history route returns create/update/delete snapshots. Missing owners or invalid values return 422; deletion of an asset referenced by an incident/vulnerability returns 409.

Notifications. POST requires `user_id`, `title`, `message`, `type` and `severity`. A peer-service call also requires `event_id` and the `X-Service-Key` header. Records return recipient, content, type, severity, status, creation/read times and event ID. The read update needs no JSON body. Search filters are `user_id`, `type`, `severity` and `status`. A recipient-specific list may be empty; accessing another user's records without permission returns 403. Repeated event IDs return the previously stored notification.

6.4.2 Threats, Vulnerabilities and Incidents contracts (Christophe ElChabab)

Threats. POST requires `type`, `value`, `description`, `severity` and `source`. PUT accepts a nonempty subset of `description`, `severity` and `source`, without changing indicator identity. Search filters are `type`, `value`, `severity` and `source`, combined with AND. A threat object contains those five fields plus `id`, `created_at` and `updated_at`. Duplicate type/value returns 409; malformed indicators return 422. DELETE returns a message confirming deletion.

Vulnerabilities. POST requires `title`, `description`, `cve_id`, `cvss_score`, `severity`, `asset_id` and `source`. PUT accepts `description`, `severity`, `cvss_score`, `status` and `resolution_date`. A supplied resolution date must be timezone-aware, between creation and the current time, and associated with a Resolved/Closed record. Search accepts `cve_id`, `severity`, `asset_id` and `status`. Responses add ID, status and creation/update/resolution times to the submitted fields. Missing assets/invalid CVSS or CVE return 422; skipped transitions return 409. DELETE returns a confirmation message.

Incidents. POST requires `title`, `description`, `category`, `severity`, `reporter_id` and `asset_id`. Assignment PUT takes `analyst_id`; status PUT takes `status`; adding a note takes `content`. An incident object adds `id`, `assigned_to`, `status` and timestamps. Detail/assignment/status responses also include `notes`; each note has ID, incident ID, author ID, content and creation time. List GET has no client-controlled ownership filter: the token determines scope. Invalid references/assignees return 422; an unassigned analyst is refused with 403; a lifecycle skip returns 409. No incident DELETE is requested by the handout or added here.

6.4.3 Audit, Playbooks and Reports contracts (Karl Khouri)

Audit logs. Service POST requires `event_id`, `action` and `related_service`. It may include `actor_user_id`, `actor_label`, `description`, `occurred_at`, `severity` and `subject_id`. It sends the separate audit `X-Service-Key`. Administrator-created entries take actor/time from the authenticated user and server. Responses include the stored event, original actor identity, recorded time and source. Search accepts `actor_user_id`, `action`, `severity`, `related_service`, `subject_id`, `start_date` and `end_date`. A repeated event returns the original object with 200 rather than creating a second entry. Protected deletion records return 409 if deletion is attempted.

Playbooks. POST requires `name`, `description`, `category` and `severity`, with an optional `steps` array. Adding an action requires `description` and `step_order`; `title` and `assigned_to` are optional. The step number must be the next consecutive number. Status PUT accepts `status` and/or `assigned_to`, with optional `note`. A valid `completion_date` may accompany a transition to Completed. An action may be created unassigned, but needs an assignee before its status changes. List filters are `category` and `severity`. Detail responses include `actions`, `execution_status`, `completed_steps` and `total_steps`. The supporting history route returns recorded changes. In-

valid assignees, gaps and invalid completion dates return 422; duplicate numbers, skipped states, unassigned status transitions and blocked deletions return 409.

Reports. All six GET routes accept optional `start_date` and `end_date`; no request body is needed. Dates accept a calendar date or ISO-8601 datetime. Incident output groups by severity/status; assets by type, criticality, status and department; threats by type/severity; vulnerabilities by severity/status with mean CVSS; activity by action/severity/service with failed-login and denied-request counts and the five most recent events. Summary combines these sections with user totals grouped by role, status and department, and includes `complete` and `unavailable`. An unavailable summary section is null, not a fabricated zero. Individual reports use 502/503 for failed sources; invalid dates return 422.

6.5 Three API examples from the captured run (Christophe ElChabab)

The request values and selected response fields below come from the actual screenshots linked beside each example. Response excerpts intentionally show only the fields being discussed; the screenshot contains the full response. They do not replace execution evidence.

6.5.1 Create a threat indicator (Christophe ElChabab)

`POST /api/Christophe-ElChabab/threats`, as an administrator. The collection expands its run-tag variable to a unique test domain. In the captured run, the request creates a High-severity Domain indicator from SOC:

```
{
  "type": "Domain",
  "value": "cc-1792442003645.example",
  "description": "Phishing indicator investigated by Christophe ElChabab",
  "severity": "High",
  "source": "SOC"
}
```

The captured response is HTTP 201 with `id: 1`, the same type/value, severity High, source SOC and server creation/update timestamps. Repeating the normalized type/value is rejected with 409. See Figure 12 and the duplicate-indicator capture that follows it.

6.5.2 Register an asset vulnerability (Christophe ElChabab)

`POST /api/Christophe-ElChabab/vulnerabilities`, as an administrator. The asset variable resolves to existing asset 1:

```
{
  "title": "Christophe ElChabab gateway finding",
  "description": "Missing security update",
  "cve_id": "CVE-2026-12345",
  "cvss_score": 8.1,
  "severity": "High",
  "asset_id": 1,
  "source": "Scanner"
}
```

The captured response is HTTP 201 with `id: 1`, `status: "Open"`, `asset_id: 1`, `cvss_score: 8.1` and `resolution_date: null`. Later updates change severity/CVSS and follow every remediation state. See Figure 31. The CVE is a test identifier used to check format, not a claim about a

real registered vulnerability.

6.5.3 Report an incident as an employee (Christophe ElChabab)

`POST /api/Christophe-ElChabab/incidents` links the new record to the authenticated reporter and an existing asset. In the captured case, employee 4 reports incident 1 at High severity on asset 1. Creation returns 201 and status Open; the administrator later assigns analyst 2. The analyst adds a note and follows the complete lifecycle until Closed.

```
{
  "title": "Christophe ElChabab phishing incident",
  "description": "Suspicious message received",
  "category": "Phishing",
  "severity": "High",
  "reporter_id": 4,
  "asset_id": 1
}
```

The detail response preserves reporter/asset IDs, the assignment, status, resolution date and the note's authenticated author. Employee 4 can retrieve this submission but cannot read the other employee's incident 3. See Figure 53 and the later assignment, note and scoped-list captures.

6.6 Importable collections and live documentation (Christophe ElChabab, Joey Saade, Karl Khouri)

Owner	Requests	Credential-free environment
Christophe ElChabab	Collection JSON	Environment template
Joey Saade	Collection JSON	Environment template
Karl Khouri	Collection JSON	Environment template

Import the collection and its own environment template, set private credentials locally, then run the folders in order with Cleanup last. Christophe's and Karl's collections use service-specific URL variables; Joey's uses literal localhost ports 8001–8003 and retains variables for credentials and record IDs. They do not use the single Lab 5 `base_url`. Do not submit private environments or reusable credentials. With a service running, its `/docs` page gives the detailed OpenAPI schema; for example, Incident API documentation on port 8006.

7 Authentication and Authorization (Christophe ElChabab, Joey Saade, Karl Khouri)

Authentication mechanism. Passwords are stored as salted PBKDF2-SHA256 hashes, not plaintext. Login verifies the password and issues a signed bearer token valid for one hour. Each protected request verifies the signature/expiry and reloads the current user, so a deleted, inactive or suspended account loses access. Service-to-service audit/notification calls use distinct private keys instead of a user's password.

Role	Access in the implementation
Security Administrator	Manages users, assets, assignments, threats and vulnerabilities; reviews audits/reports and manages response playbooks. Destructive operations still respect referenced-record and workflow checks.
Security Analyst	Investigates assigned incidents and adds notes; reads threat information and updates vulnerabilities; works on permitted response actions. Does not gain administrator deletion rights.
Regular Employee	Manages their own profile, reports incidents, views only their submitted incidents and handles their own notifications. Cannot browse threat/vulnerability records or organizational assets.
Security Auditor	Reads permitted users, assets, incidents, threats, vulnerabilities, audit records and reports without mutating them.

Record-level permissions. The professor clarified that auditors may search threats and that all four roles may call `GET /incidents`. Administrators/auditors see all incidents, analysts see their assignments and employees see their submissions. These restrictions use the token's user ID even if a request supplies misleading reporter, assignee or role query parameters. The same principle limits notifications to their recipient where required.

The shared login evidence in Appendix A shows both a valid administrator login and an incorrect password returning 401. Employee threat access returns 403 while an allowed reader succeeds; auditor vulnerability updates and unauthorized incident changes are rejected. Joey's API captures and Karl's protected API captures provide additional access-control evidence. The handout's two permission clarifications are applied in the endpoint tables above.

For audit records, analysts may retrieve an entry or search, but only administrators and auditors may list all entries or retrieve a user's history. There is no own-history exception: an analyst or employee requesting their own history still receives 403. Playbook creation and deletion are administrator-only; analysts may add response actions and update their status or assignment. Karl's updated captures in Appendix C show these permission checks.

8 GitHub and Version Control (Christophe ElChabab, Joey Saade, Karl Khouri)

Team repository: ChristopheElChabab-JoeySaade-KarlKhouri.

The ownership table in Section 2 describes the three-service split. The repository keeps source code, schemas, Postman collections, tests and report sources together. Later fixes are recorded as new commits, and teammate updates are merged without replacing their commits.

Member	Commit	Recorded work
Joey Saade	`5b067bb`	Users, Assets and Notifications implementation.
Christophe ElChabab	`537e769`	Threats, Vulnerabilities and Incidents implementation.
Karl Khouri	`2cf2a2e`	Audit Logging, Playbooks and Reports implementation.
Christophe ElChabab	`2c8a917`	Professor-confirmed permissions, regression checks and collection updates.
Karl Khouri	`ee55ad5`	Report sections, ID-reuse fix and regression tests.
Joey Saade	`b56ebf6`	Report screenshot manifest, guide and ten screenshots.

The captured Git history and author-filtered views identify `chrs-005` (Christophe ElChabab), `JS9872` (Joey Saade) and `karlkhoury` (Karl Khouri). These snapshots end at `e236431`; later updates are in the live repository history.

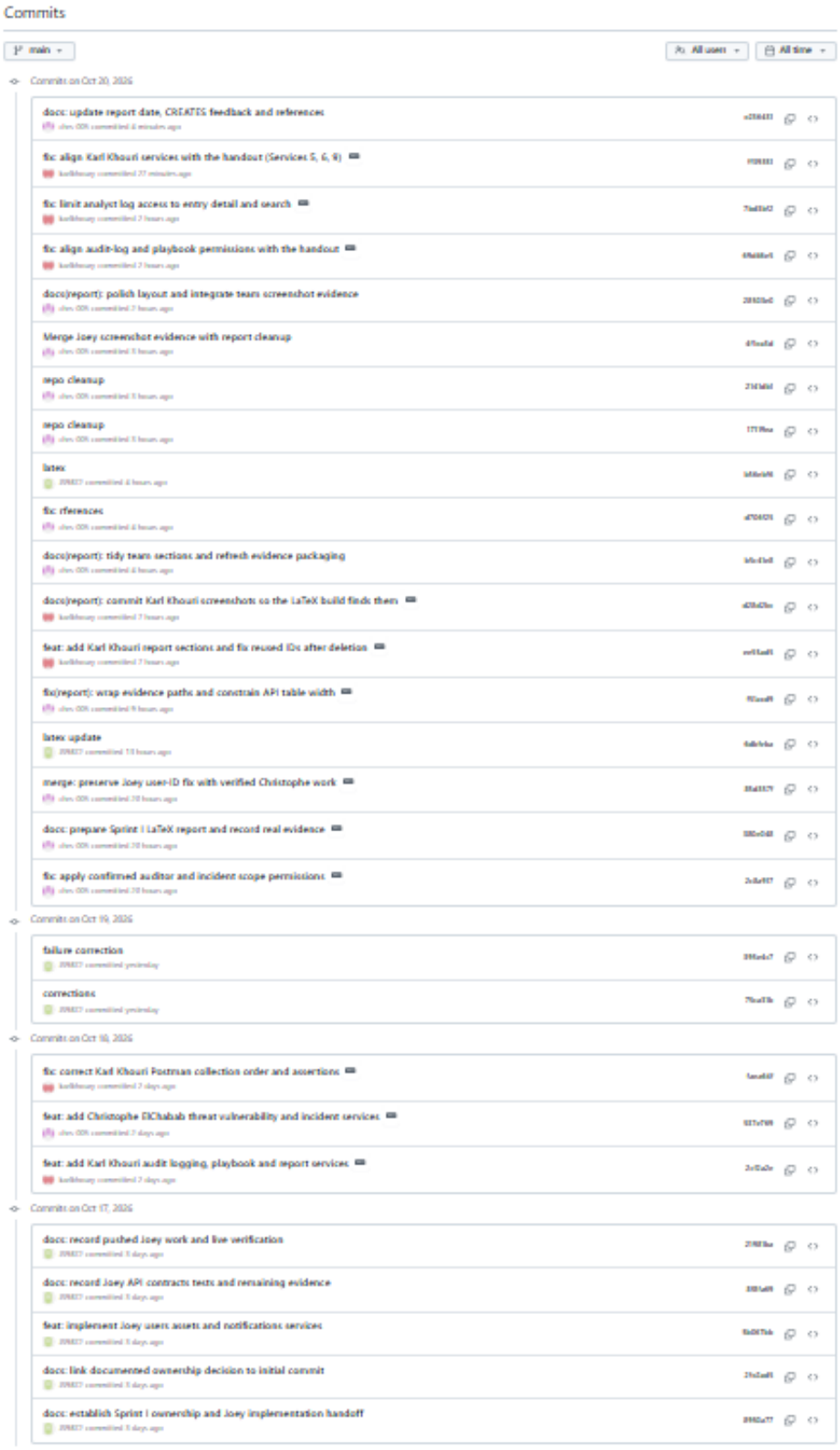

Figure 6. Christophe ElChabab, Joey Saade, Karl Khouri - Team commit history - The captured main-branch history shows implementation, corrections and report contributions by the three accounts.

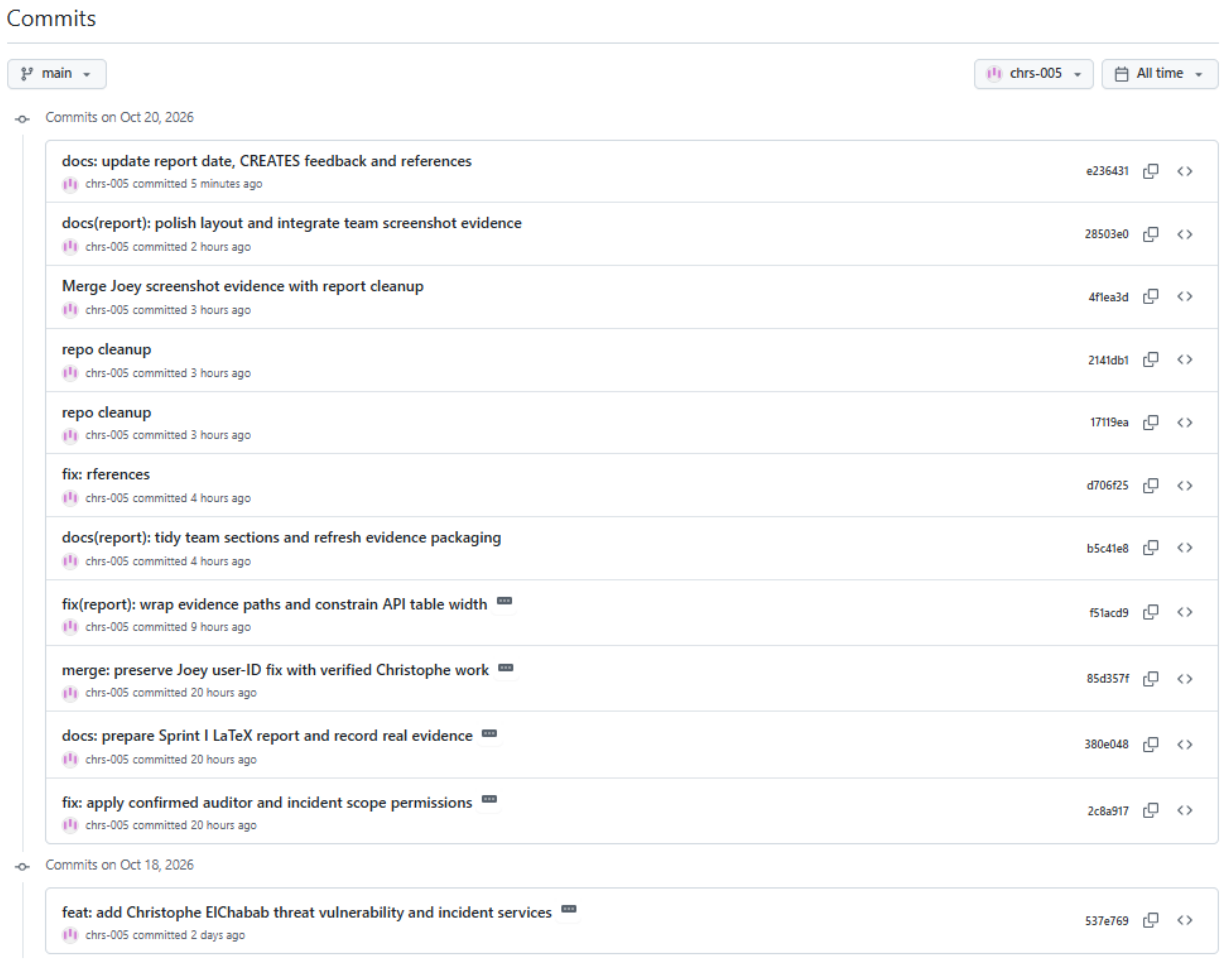

Figure 7. Christophe ElChabab - GitHub contributions - The author-filtered history shows the Threats, Vulnerabilities and Incidents implementation, permission fixes and report work.

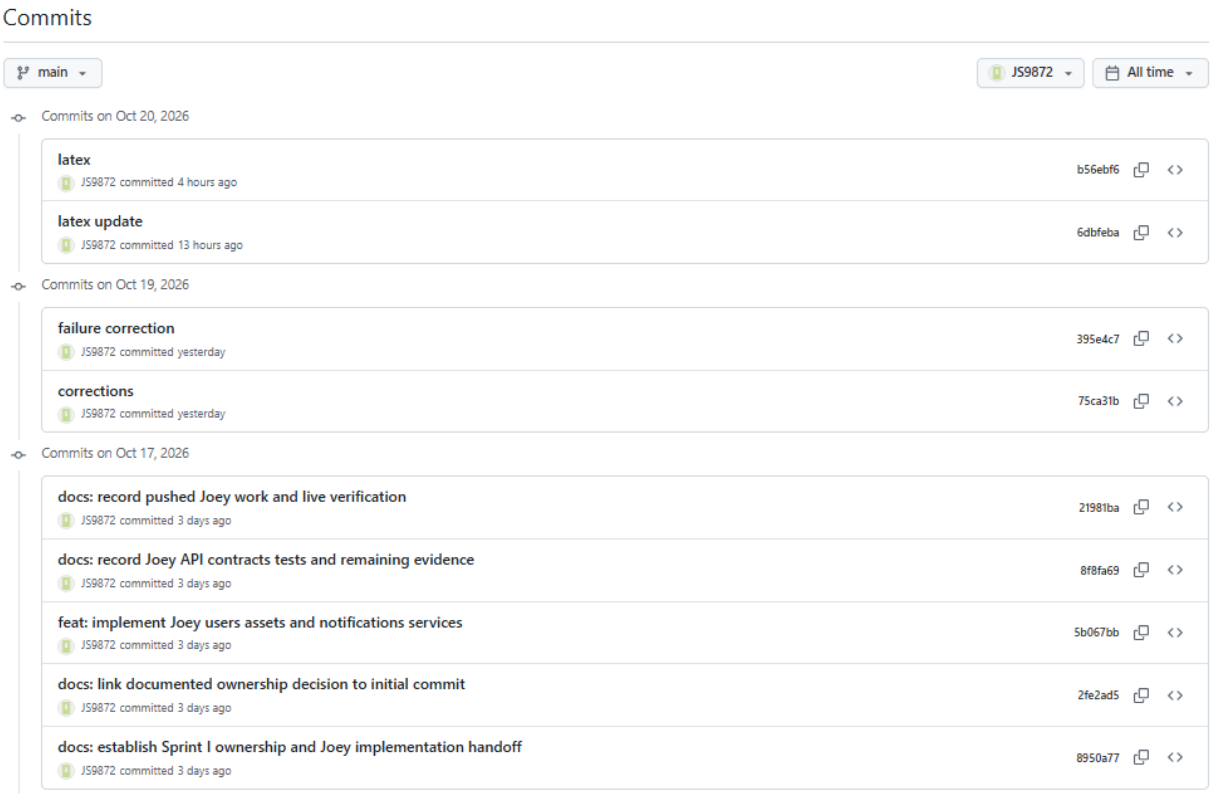

Figure 8. Joey Saade - GitHub contributions - The author-filtered history shows the Users, Assets and Notifications implementation, corrections and documentation work.

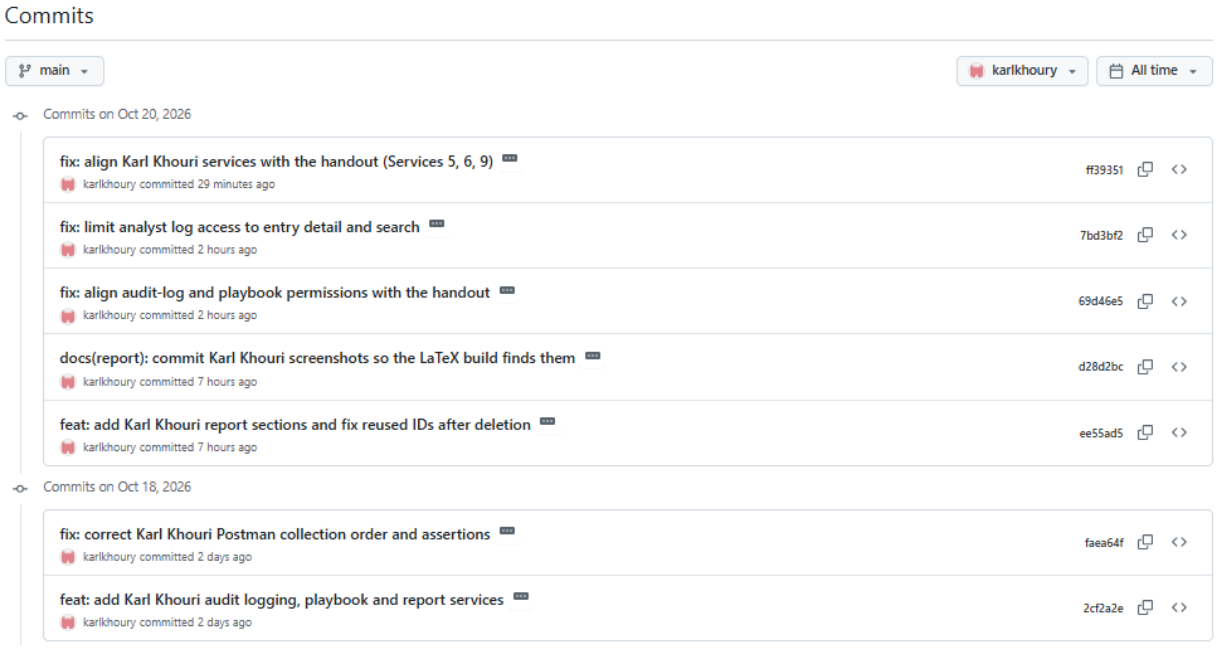

Figure 9. Karl Khouri - GitHub contributions - The author-filtered history shows Audit Logging, Playbooks and Reports, permission fixes and evidence contributions.

8.1 CREATES feedback (Christophe ElChabab, Joey Saade, Karl Khouri)

Our team met with CREATES on Friday, 16 October 2026, at 11:30 AM. We discussed good and bad Git practices and how to correct repositories with README files that were too brief or too complicated, or with a missing `requirements.txt`. We also discussed how to structure a project report. The CREATES associate reviewed our repository and confirmed that it looked good. The session helped us check that the repository instructions were clear and that the report followed an organized structure.

9 References (Christophe ElChabab, Joey Saade, Karl Khouri)

We used the instructor's *Final Project Fall 2026-2027–Sprint 1* Word handout to identify the required services, API operations, role permissions, database relationships, execution evidence and report sections. We also referred to the previous EECE 435L labs for database work, API testing with Postman, Git commands and project documentation. These materials helped us apply the practices covered in the labs to the common project.

We used Codex (GPT 6.1) and Claude Code (Opus 5.5) to help with coding and trouble shooting errors. We wrote a good chunk of the code and remain responsible for everything in our submission, given that it was all checked and overviewed humanly in the final submission form. AI tools also helped us create the Postman collections.

A Postman and Database Evidence (Christophe ElChabab)

These 94 unchanged captures include two shared-authentication requests, 84 API/integration requests and eight database views. Database records were captured before Cleanup. Shared login and integration calls retain Joey Saade's and Karl Khouri's routes because those APIs demonstrate dependencies, not additional Christophe services.

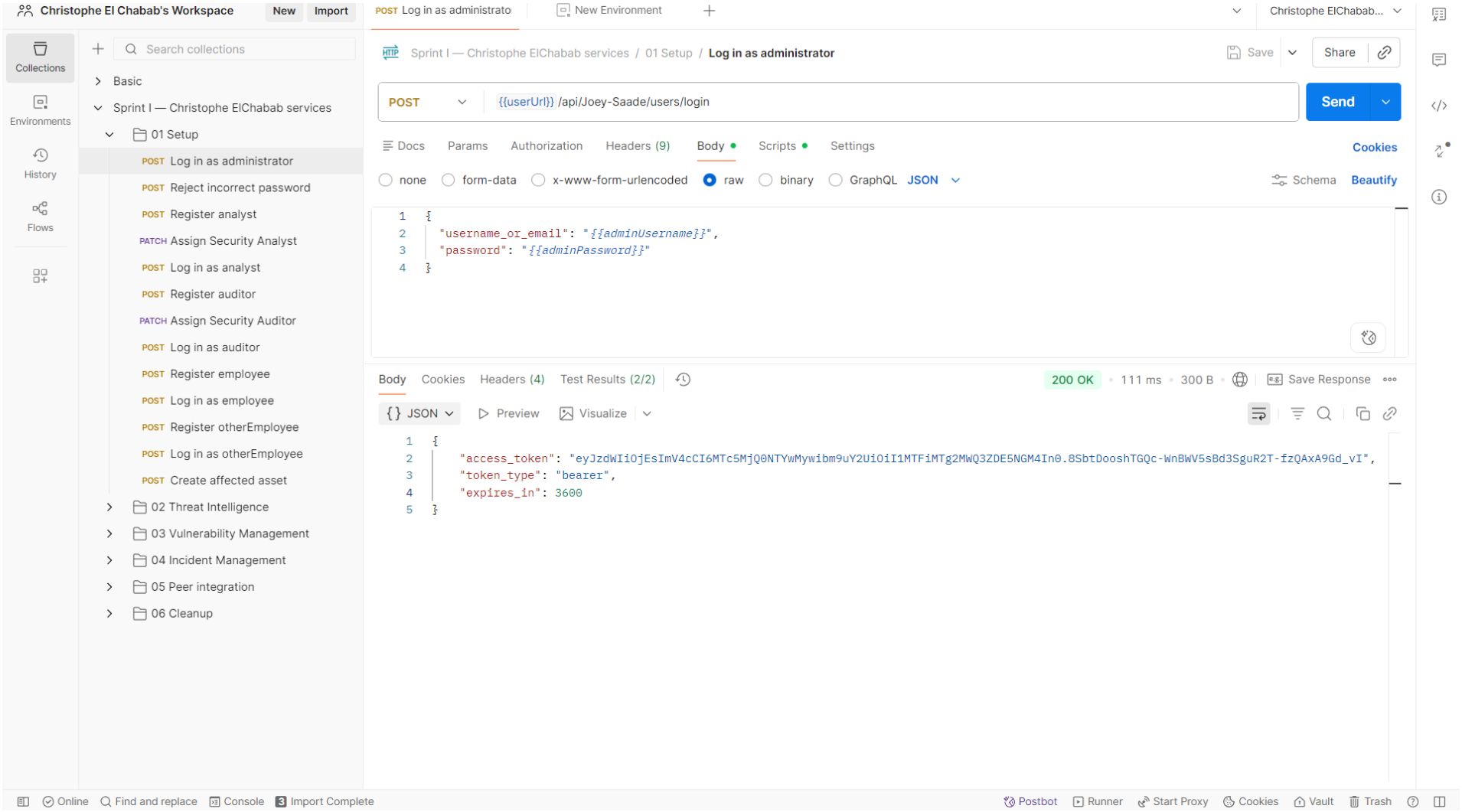

Figure 10. Christophe ElChabab - Successful authentication - Joey Saade's login API returns an access token for a valid administrator.

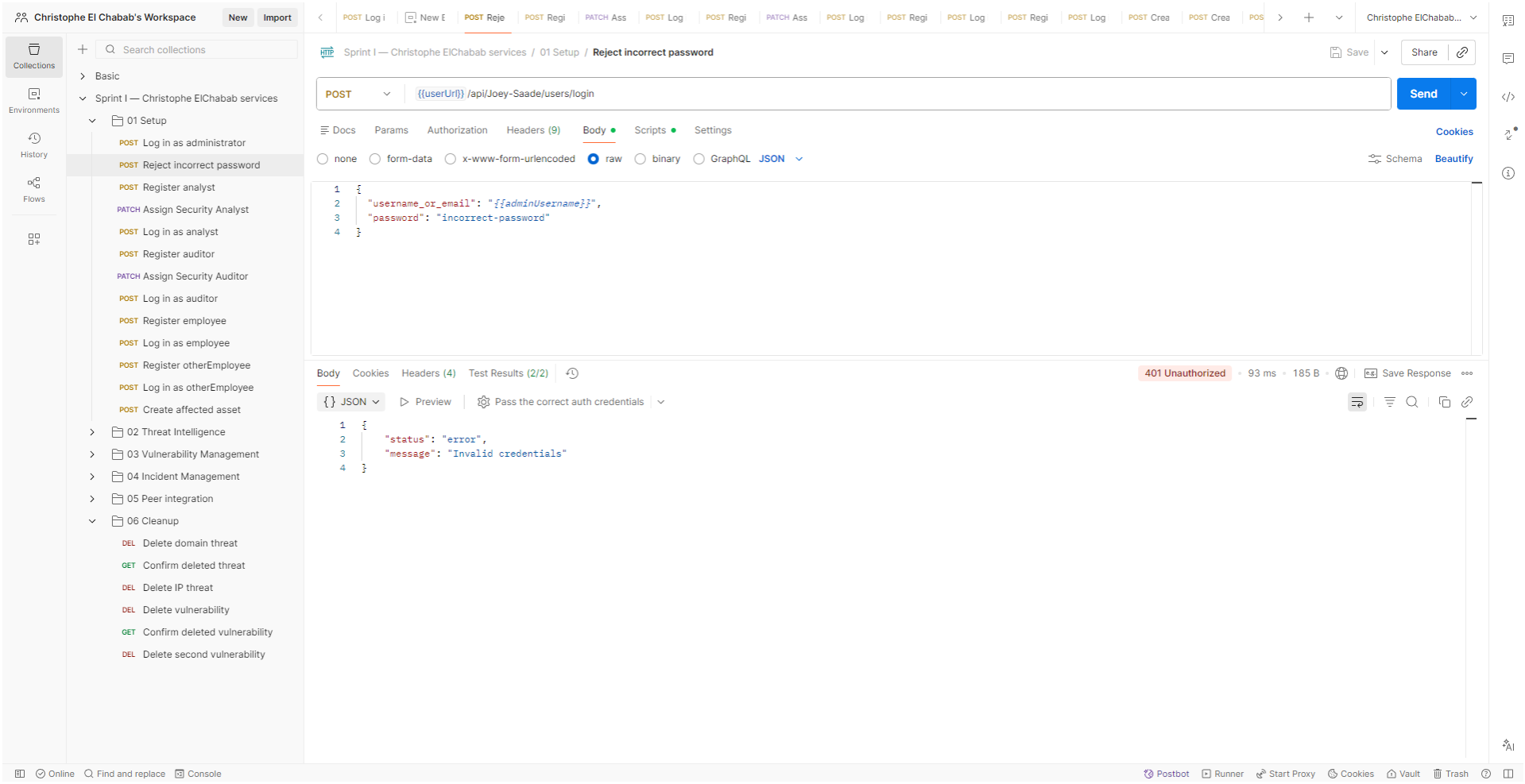

Figure 11. Christophe ElChabab - Failed authentication - Joey Saade's login API rejects an incorrect password with HTTP 401.

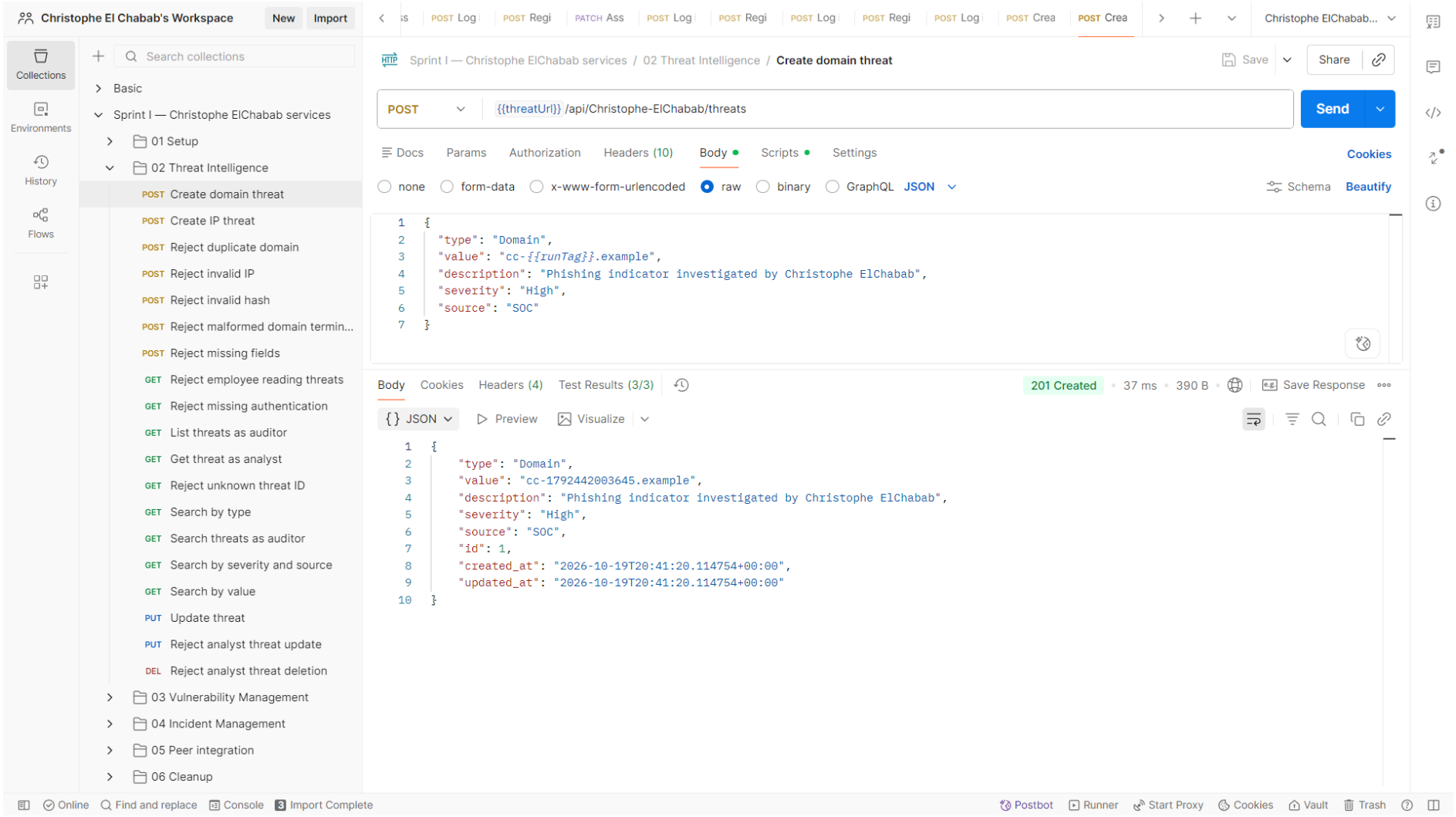

Figure 12. Christophe ElChabab - POST Create Domain Threat - Shows the submitted request and the newly created record returned with HTTP 201.

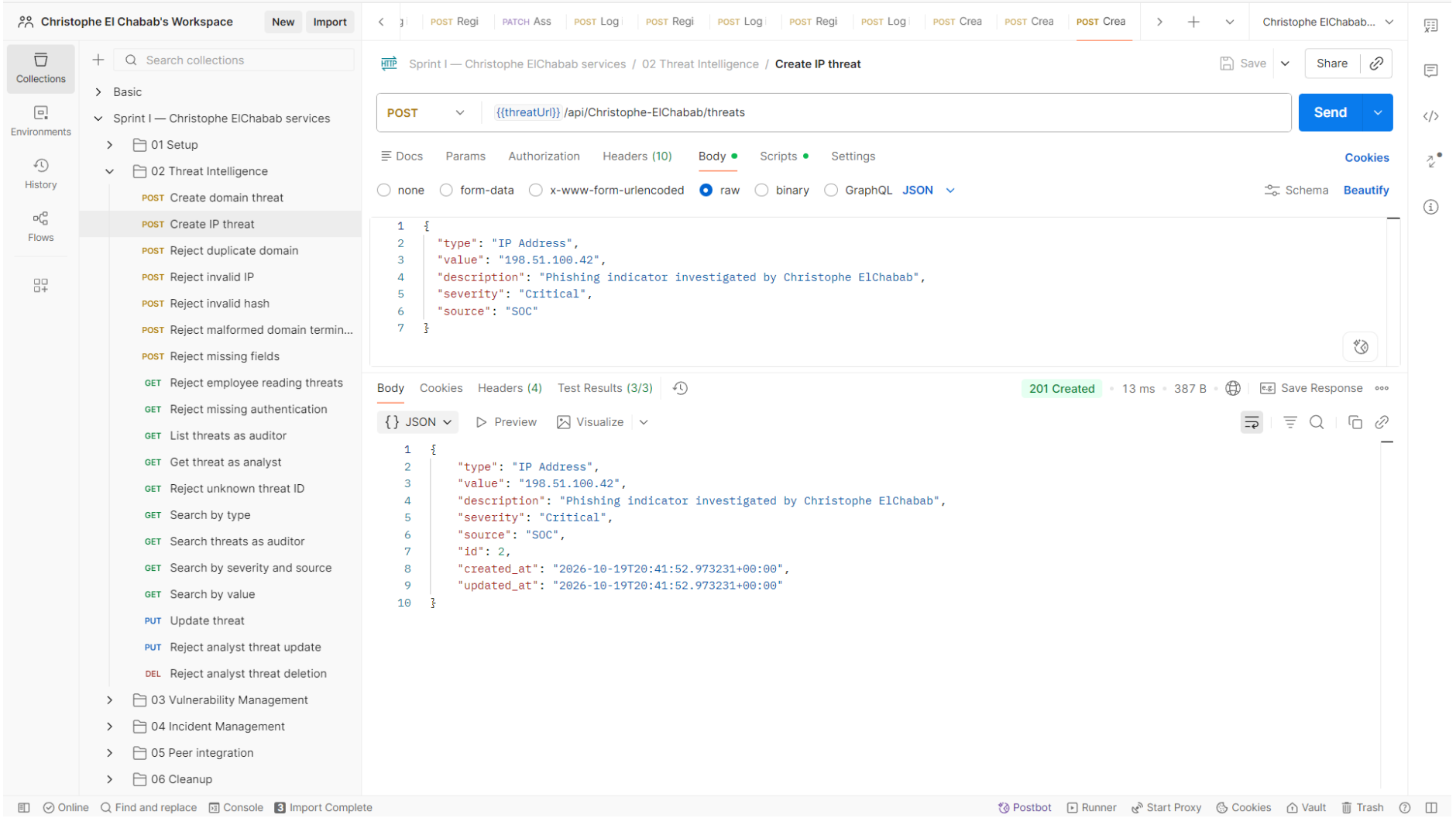

Figure 13. Christophe ElChabab - POST Create IP Threat - Shows the submitted request and the newly created record returned with HTTP 201.

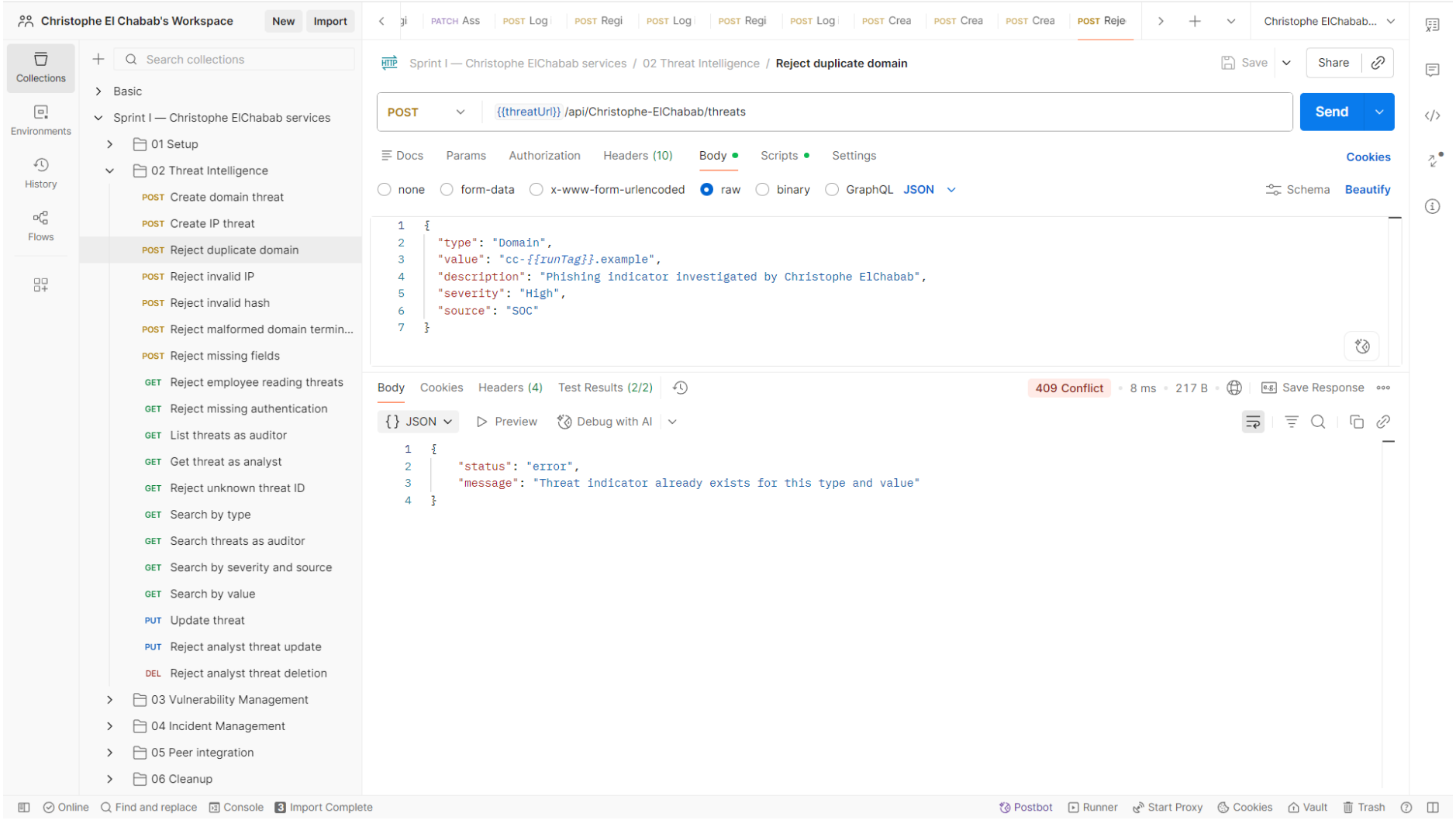

Figure 14. Christophe ElChabab - POST Reject Duplicate Domain - Shows that a duplicate indicator or invalid lifecycle change is rejected with HTTP 409.

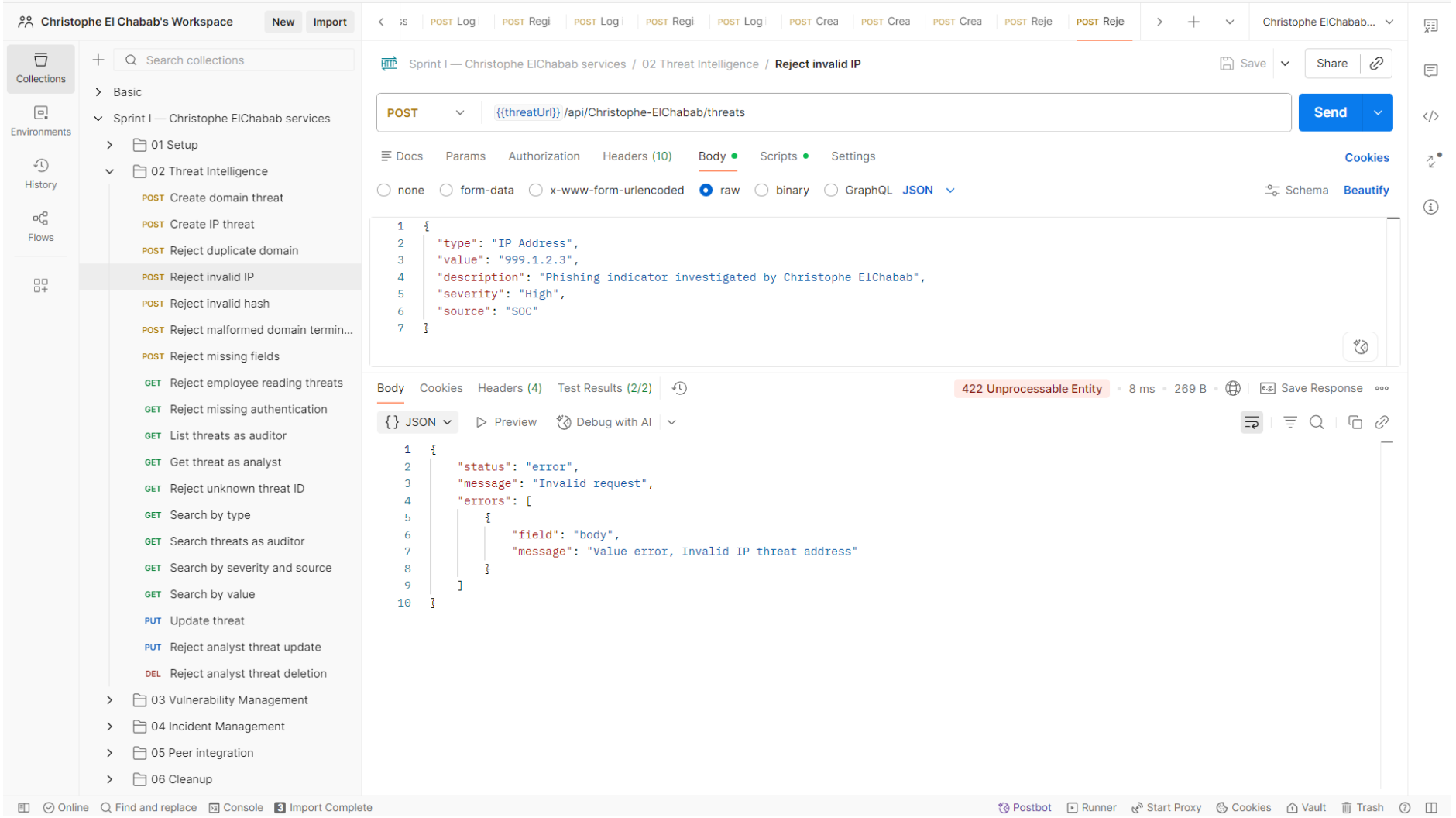

Figure 15. Christophe ElChabab - POST Reject Invalid IP - Shows that invalid input or a reference is rejected with HTTP 422.

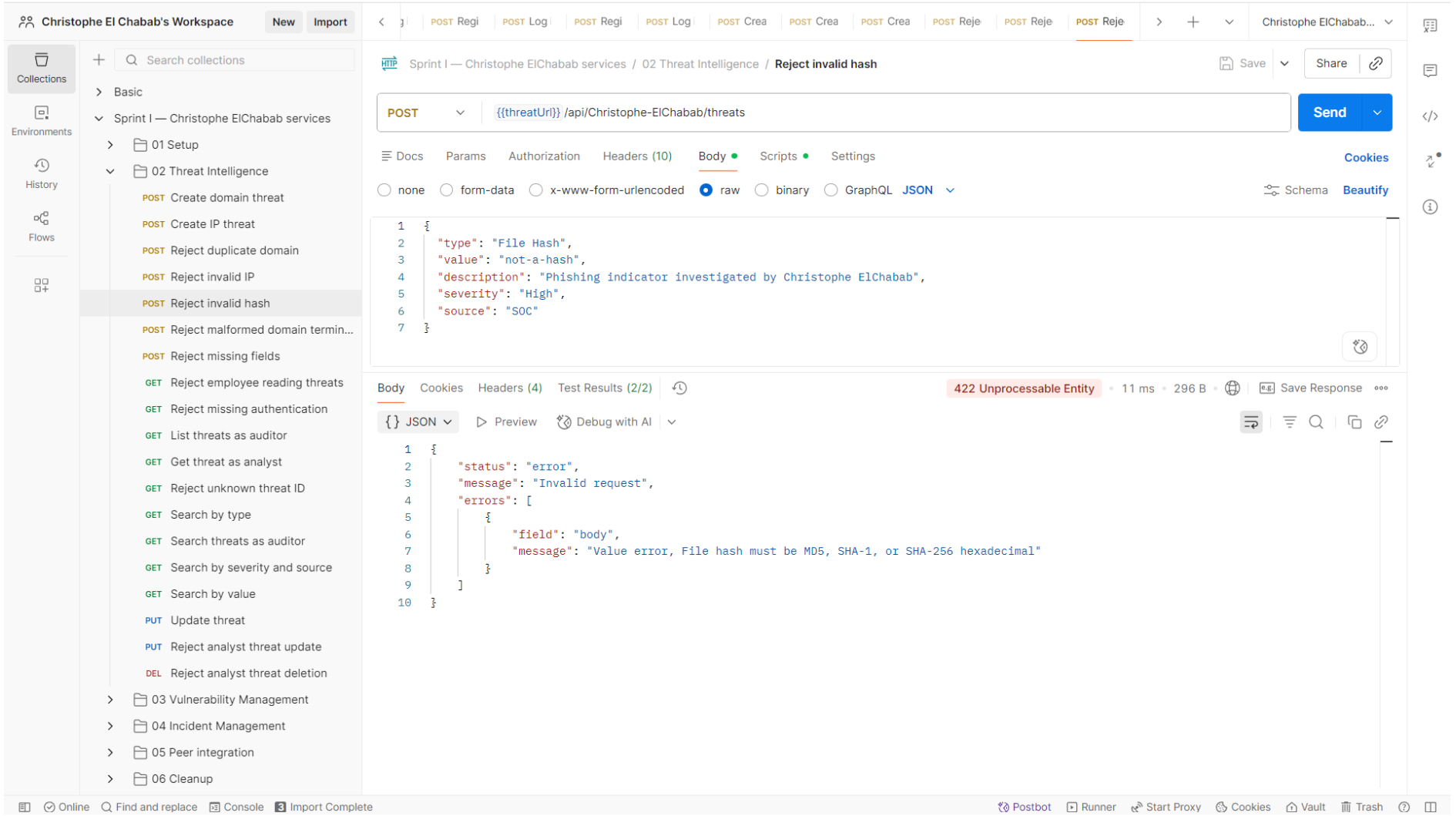

Figure 16. Christophe ElChabab - POST Reject Invalid Hash - Shows that invalid input or a reference is rejected with HTTP 422.

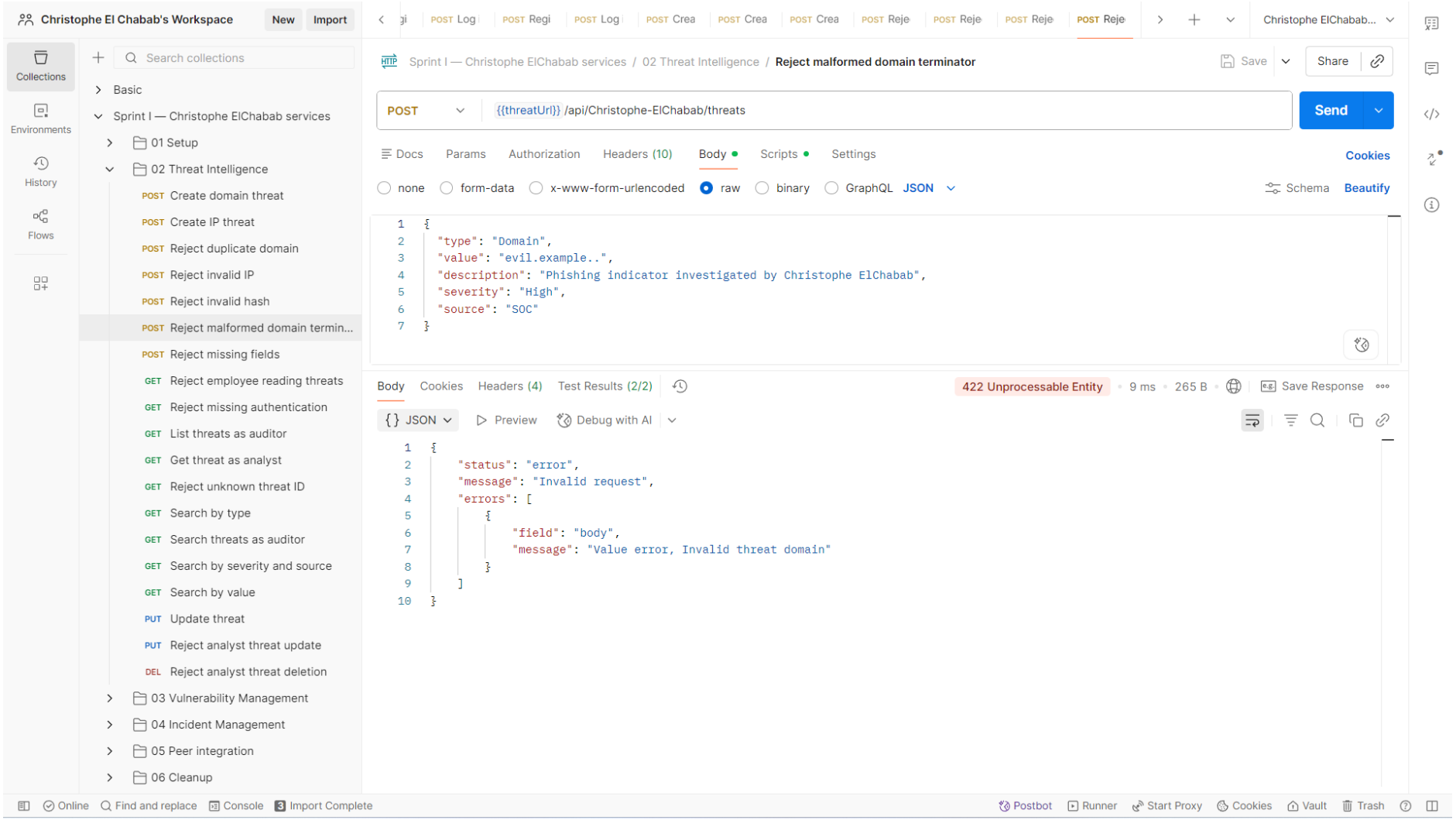

Figure 17. Christophe ElChabab - POST Reject Malformed Domain - Shows that invalid input or a reference is rejected with HTTP 422.

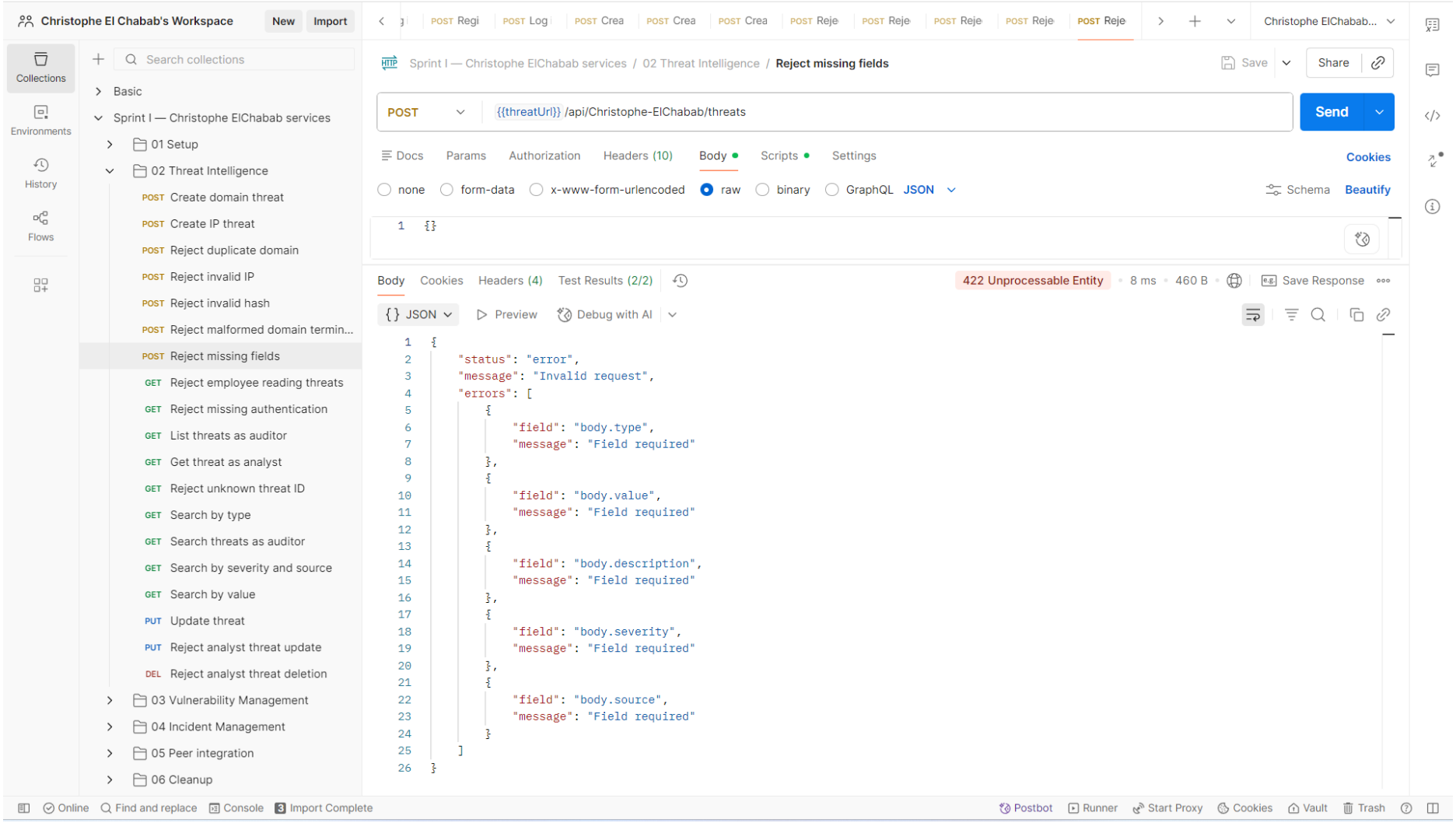

Figure 18. Christophe ElChabab - POST Reject Missing Fields - Shows that invalid input or a reference is rejected with HTTP 422.

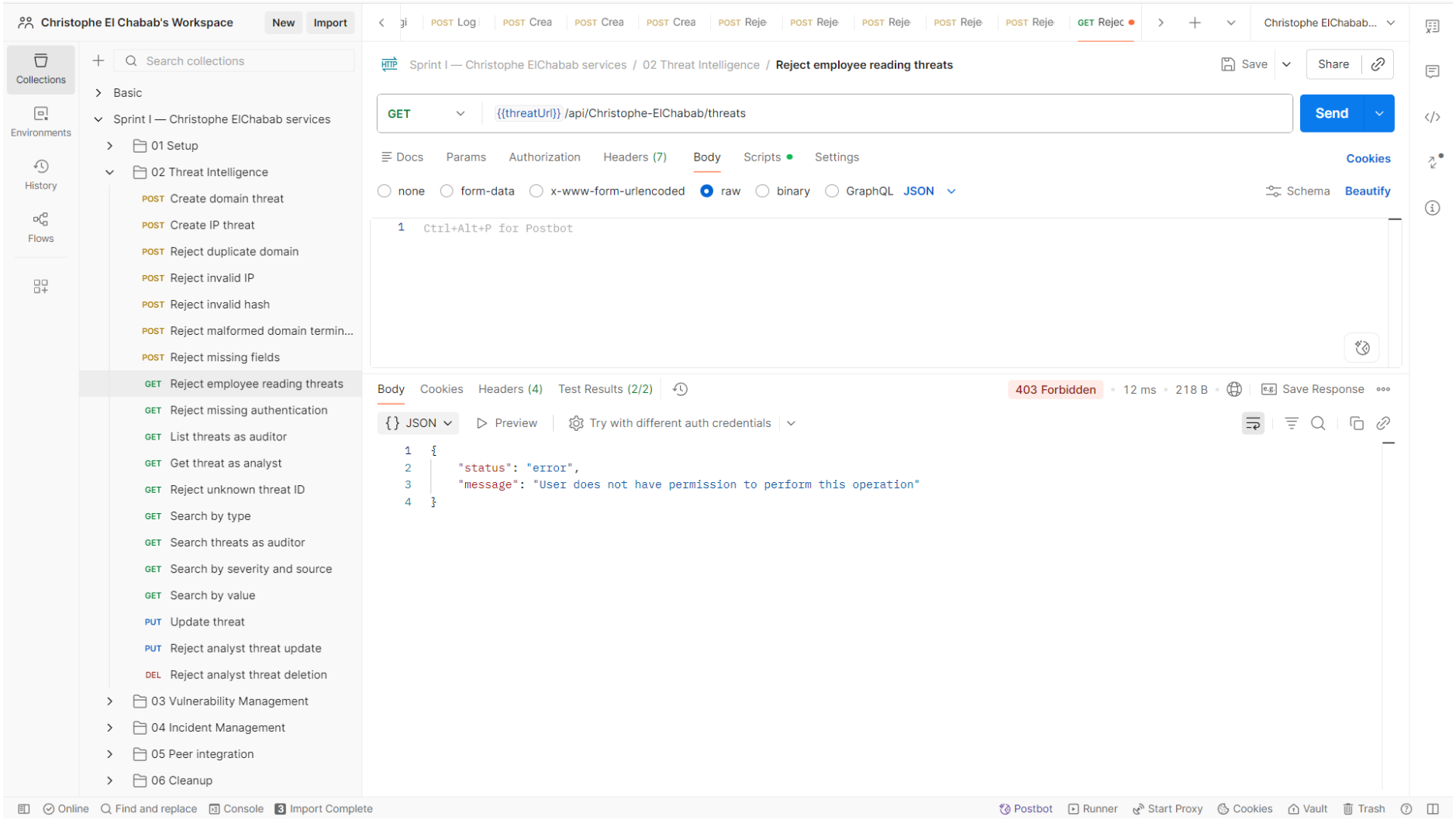

Figure 19. Christophe ElChabab - GET Reject Employee Threat Access - Shows that the unauthorized operation is refused with HTTP 403.

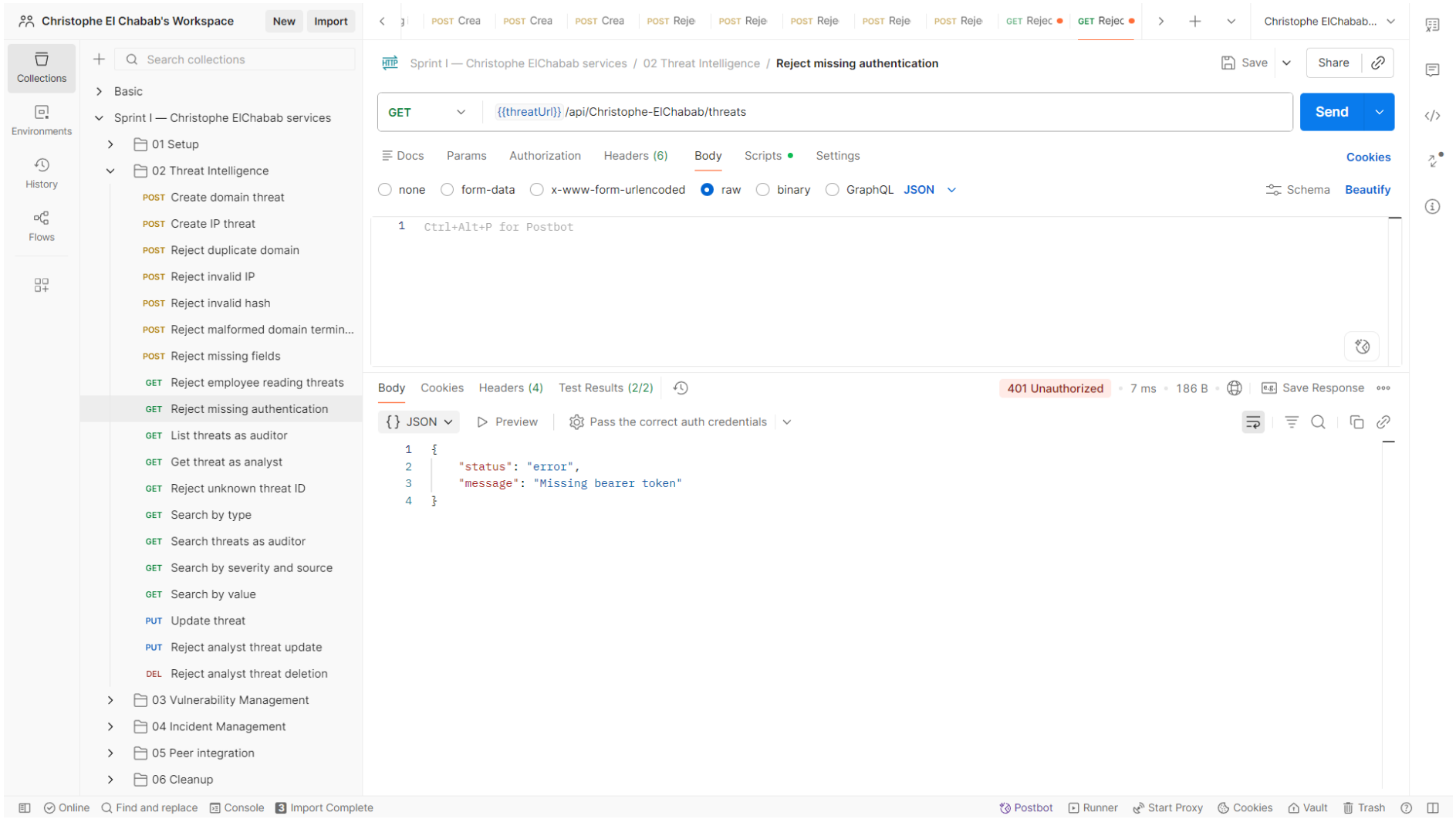

Figure 20. Christophe ElChabab - GET Reject Missing Authentication - Shows that the unauthenticated request is rejected with HTTP 401.

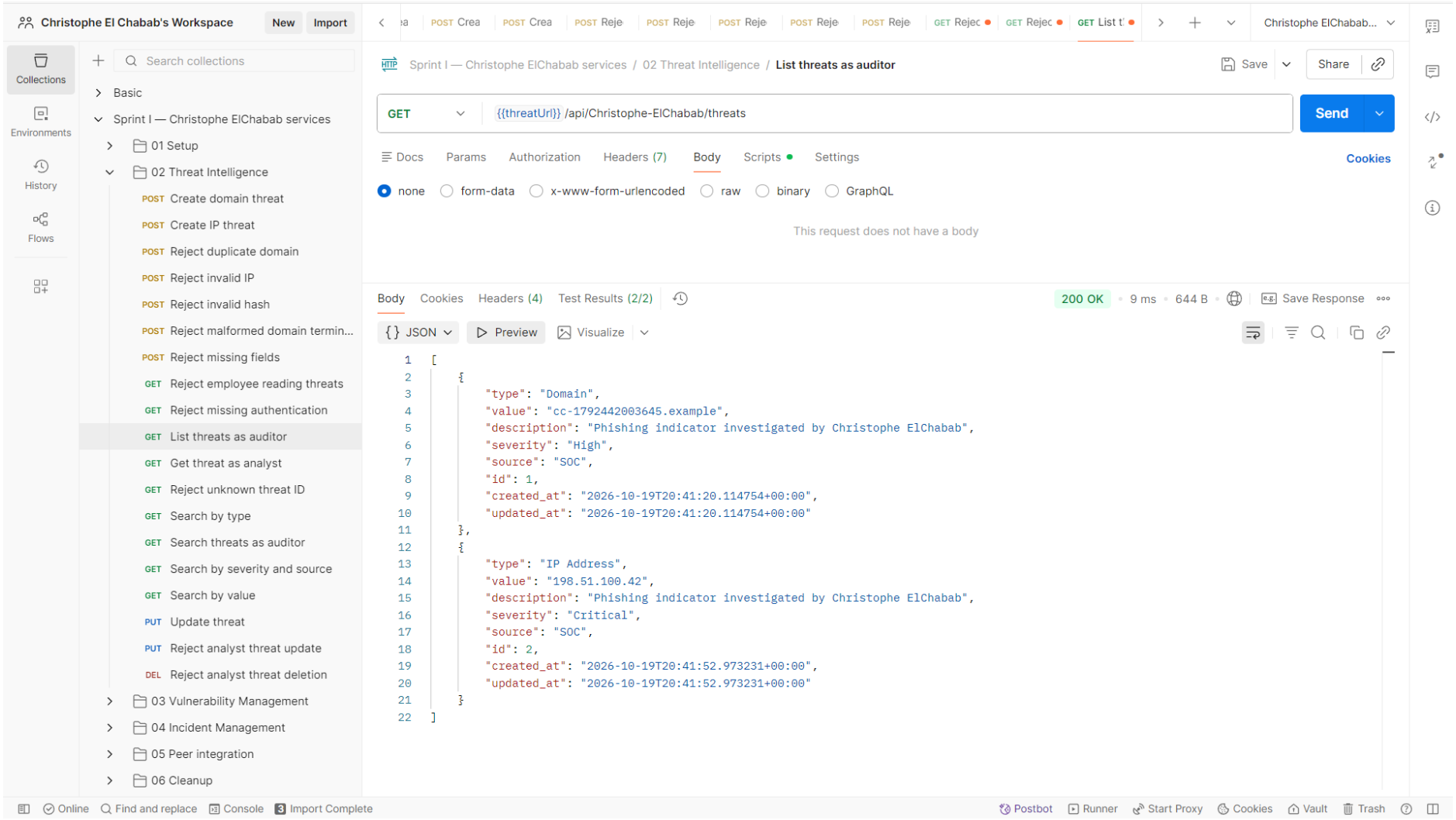

Figure 21. Christophe ElChabab - GET List Threats as Auditor - Shows the named request and its successful HTTP 200 result.

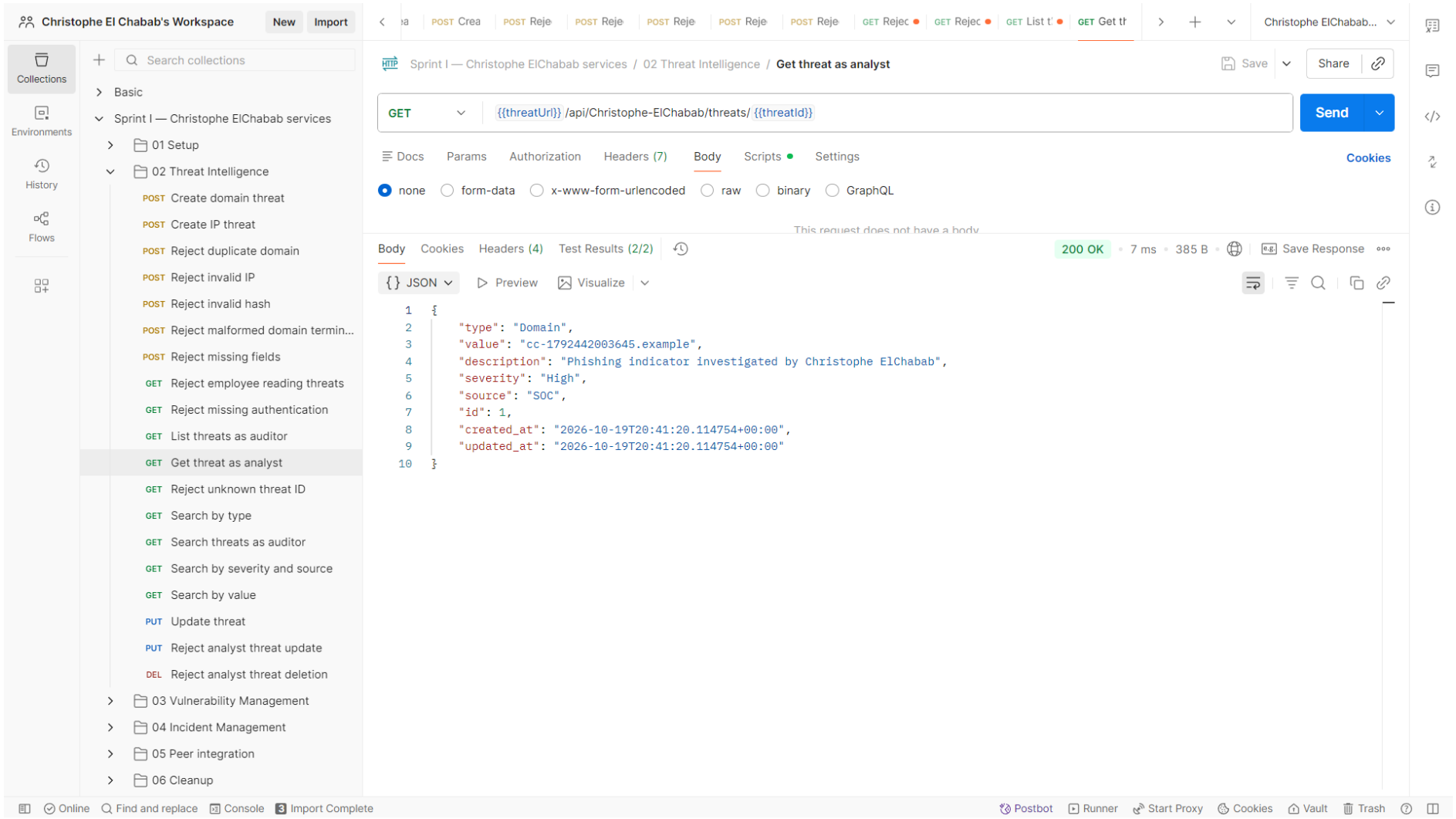

Figure 22. Christophe ElChabab - GET Get Threat as Analyst - Shows the named request and its successful HTTP 200 result.

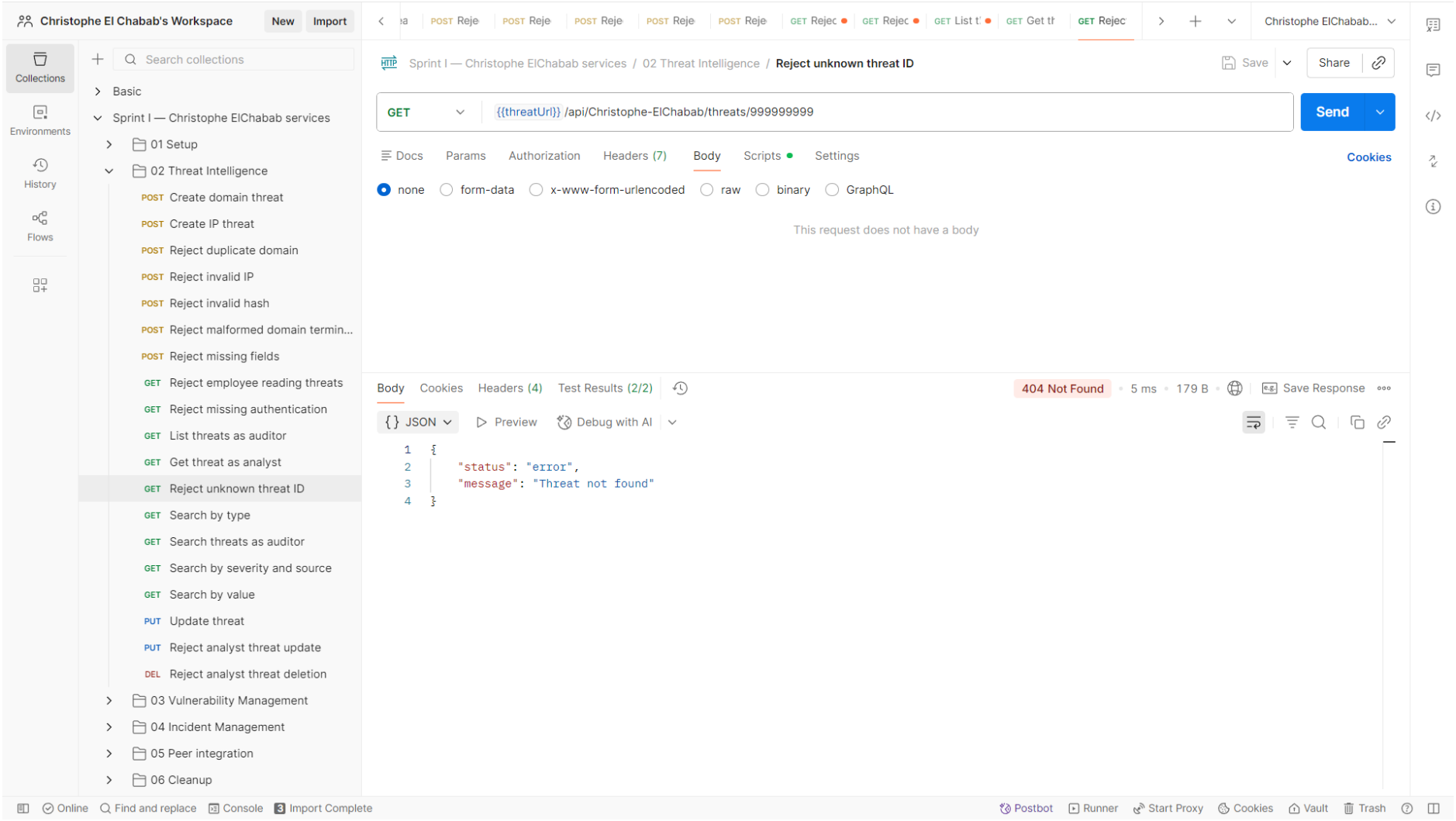

Figure 23. Christophe ElChabab - GET Reject Unknown Threat ID - Shows that an unknown record returns HTTP 404.

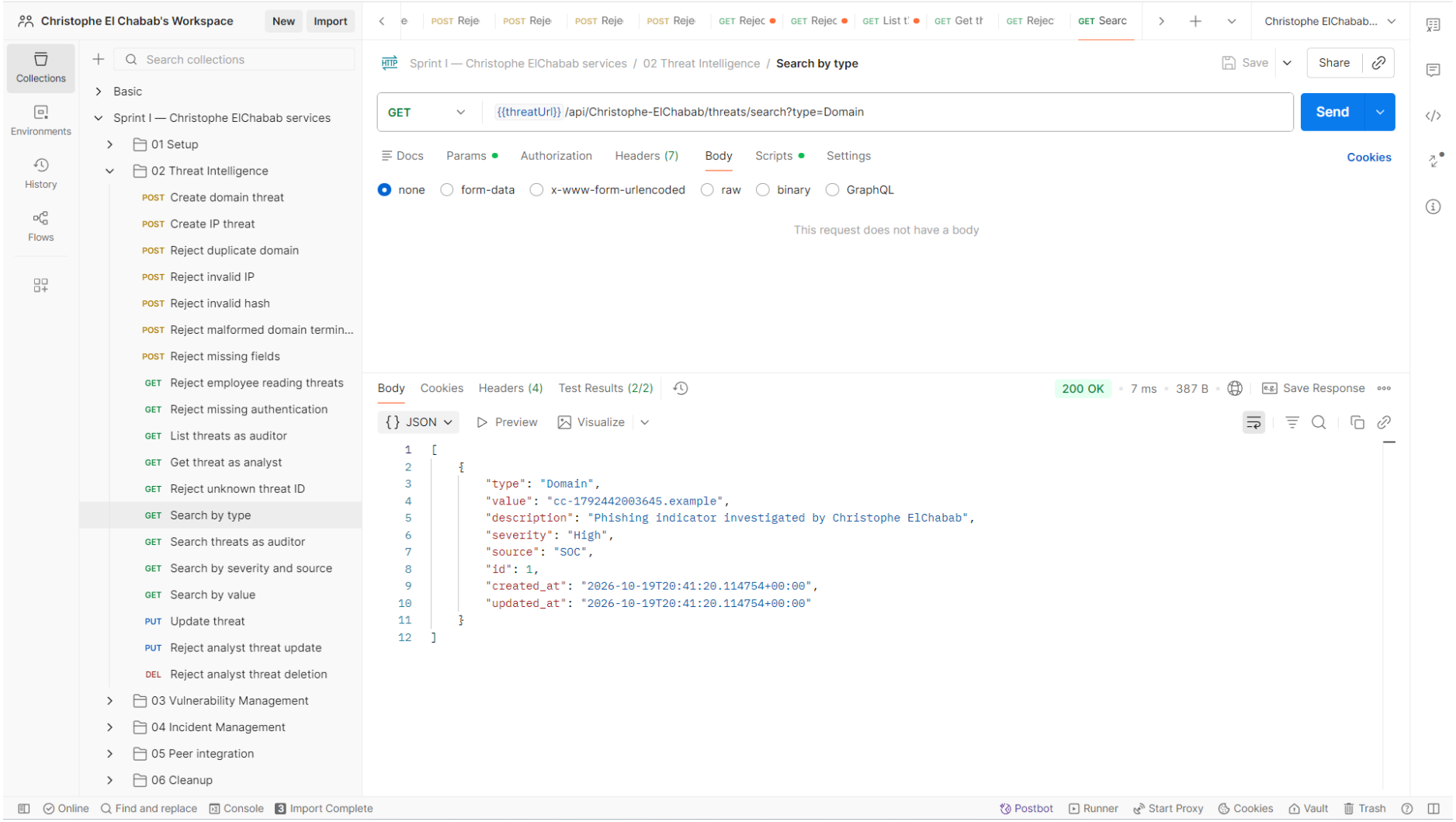

Figure 24. Christophe ElChabab - GET Search Threats by Type - Shows the named request and its successful HTTP 200 result.

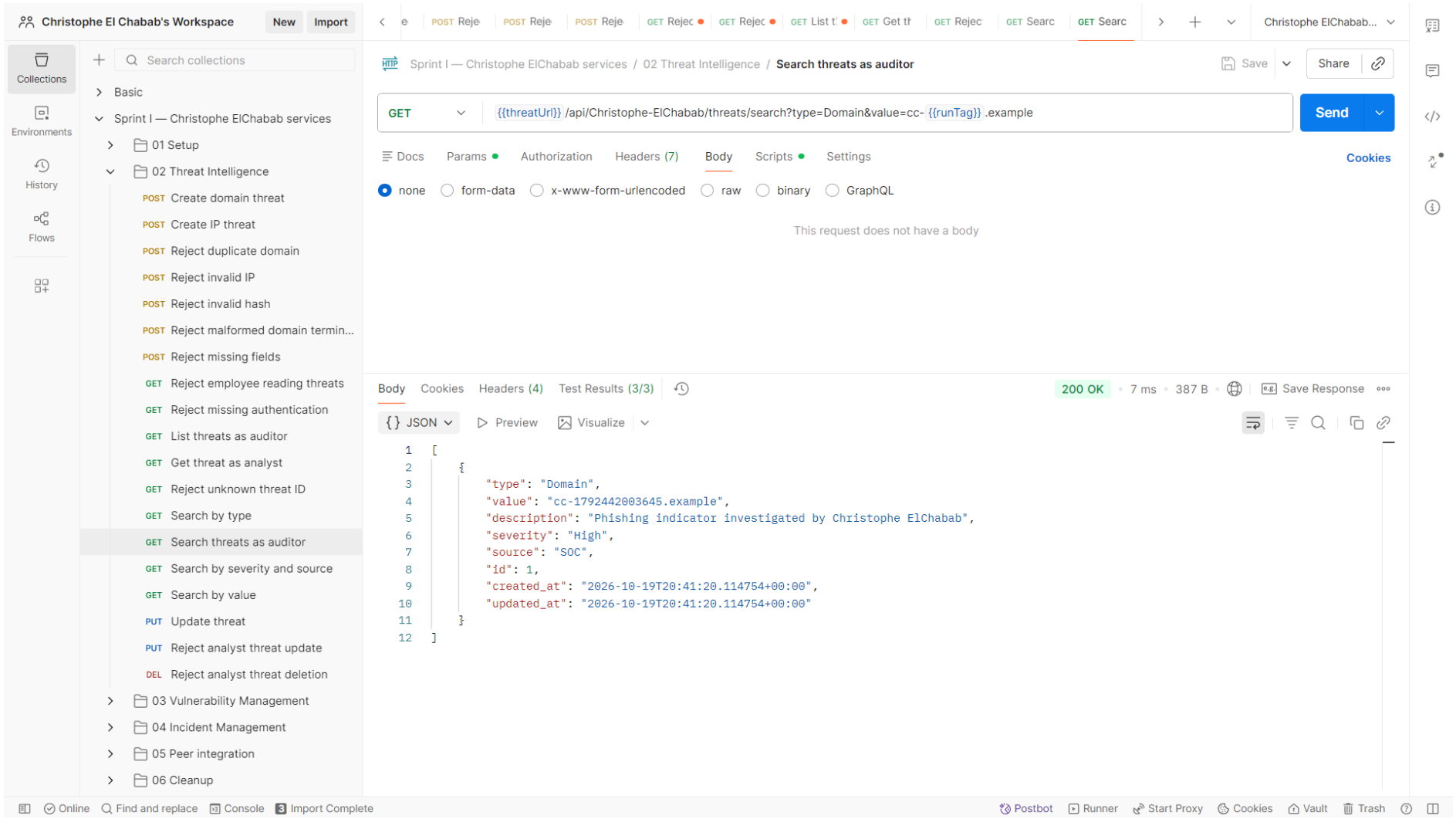

Figure 25. Christophe ElChabab - GET Search Threats as Auditor - Shows the named request and its successful HTTP 200 result.

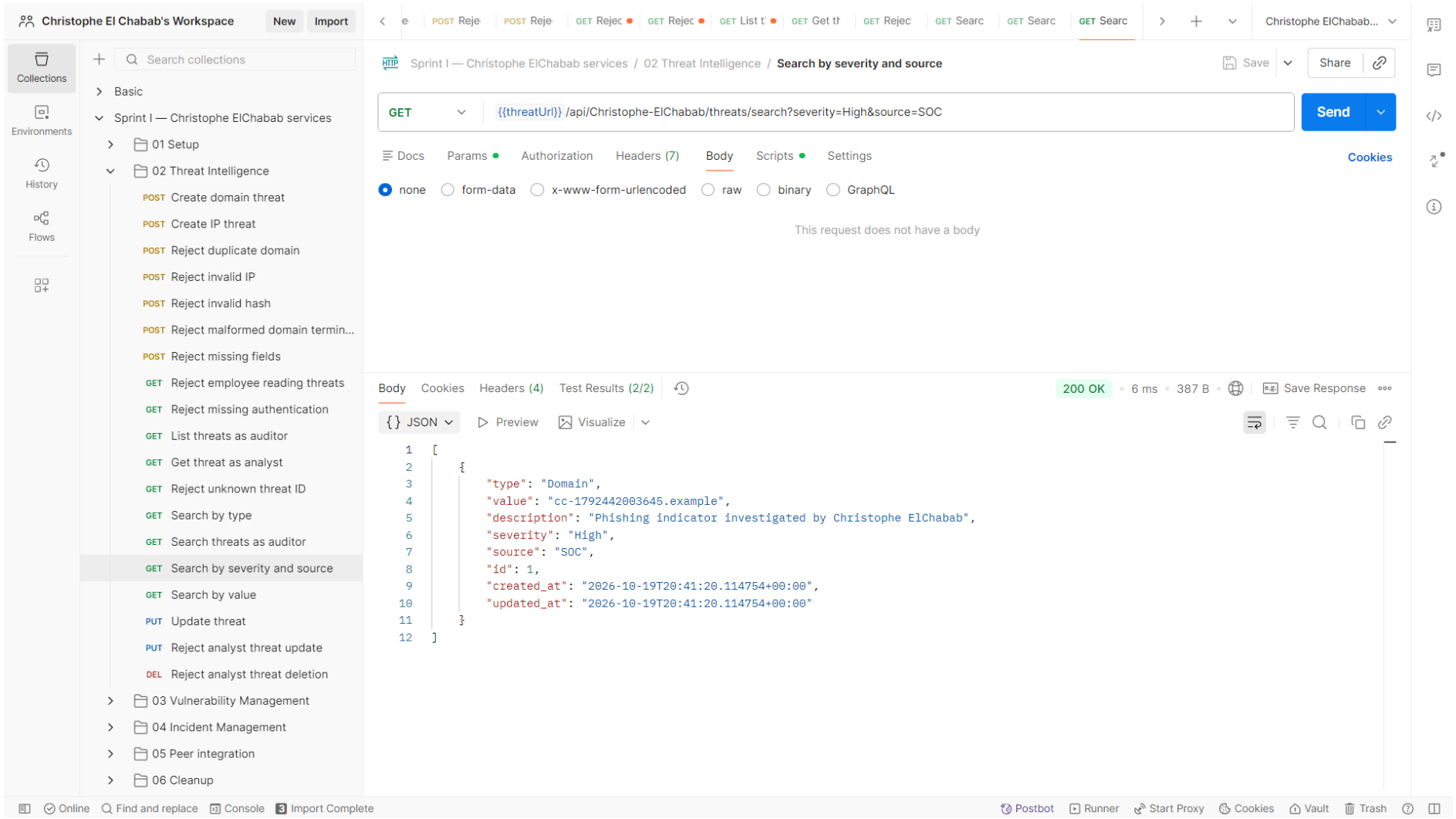

Figure 26. Christophe ElChabab - GET Search by Severity and Source - Shows the named request and its successful HTTP 200 result.

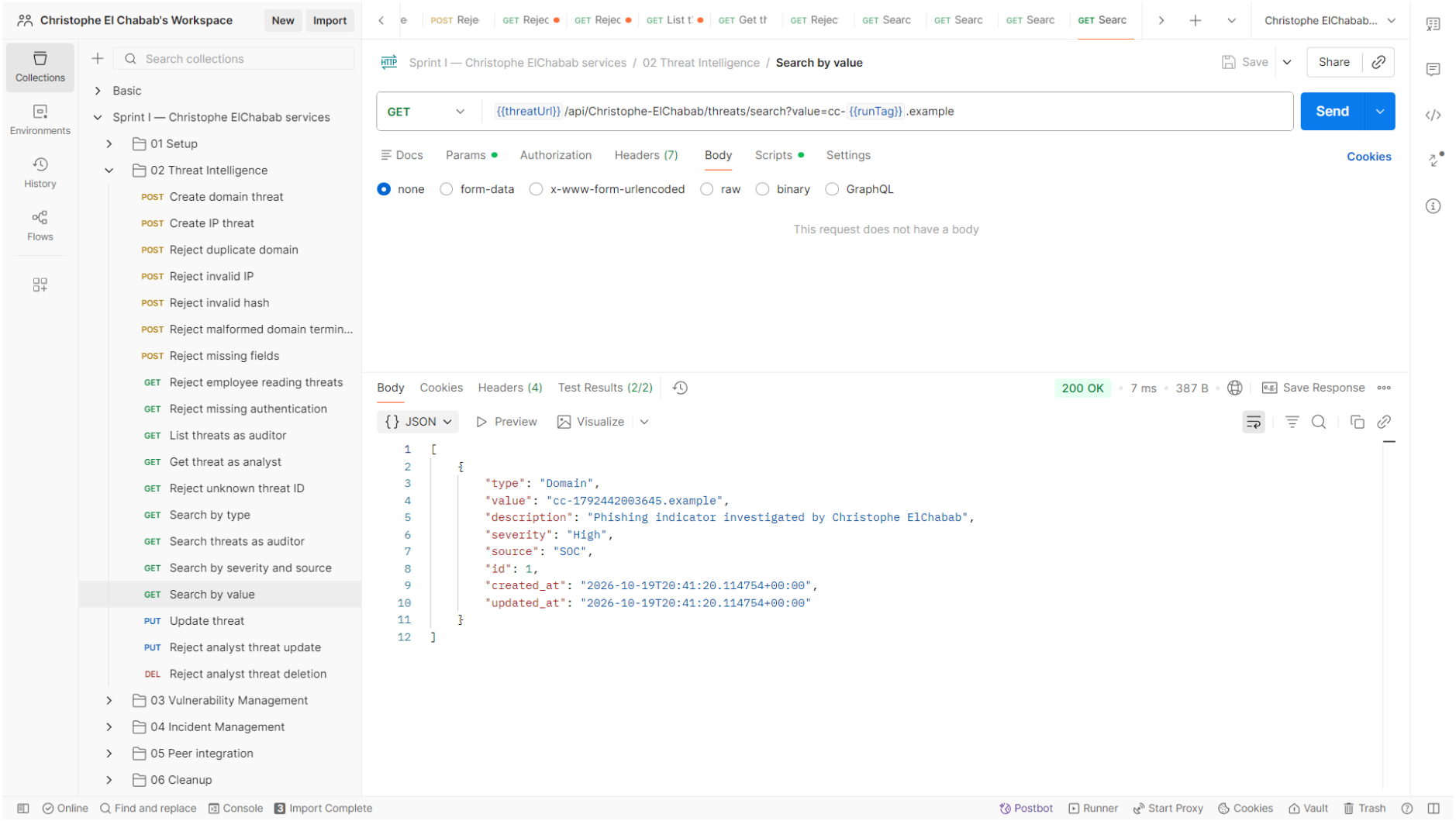

Figure 27. Christophe ElChabab - GET Search Threats by Value - Shows the named request and its successful HTTP 200 result.

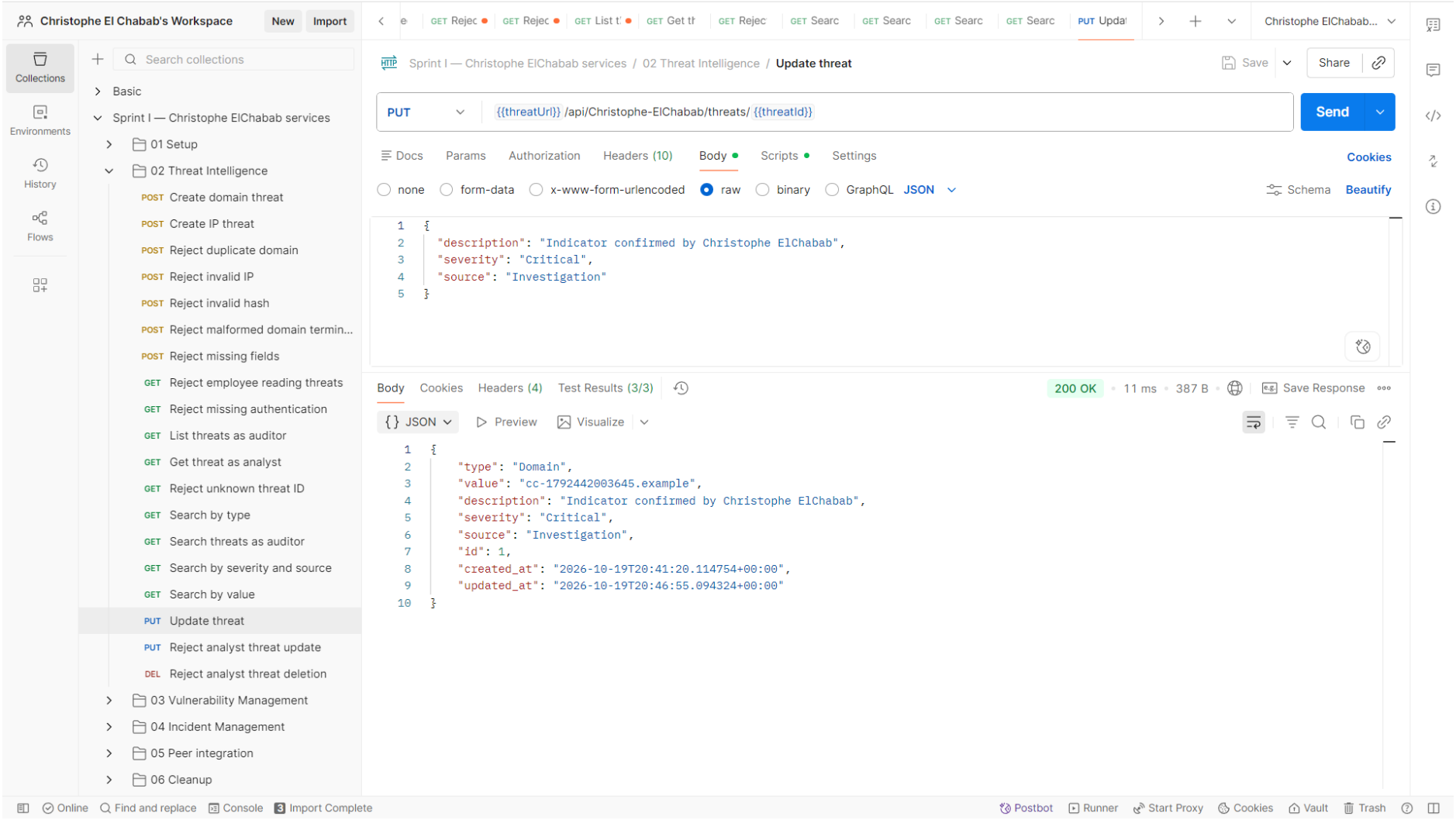

Figure 28. Christophe ElChabab - PUT Update Threat - Shows the named request and its successful HTTP 200 result.

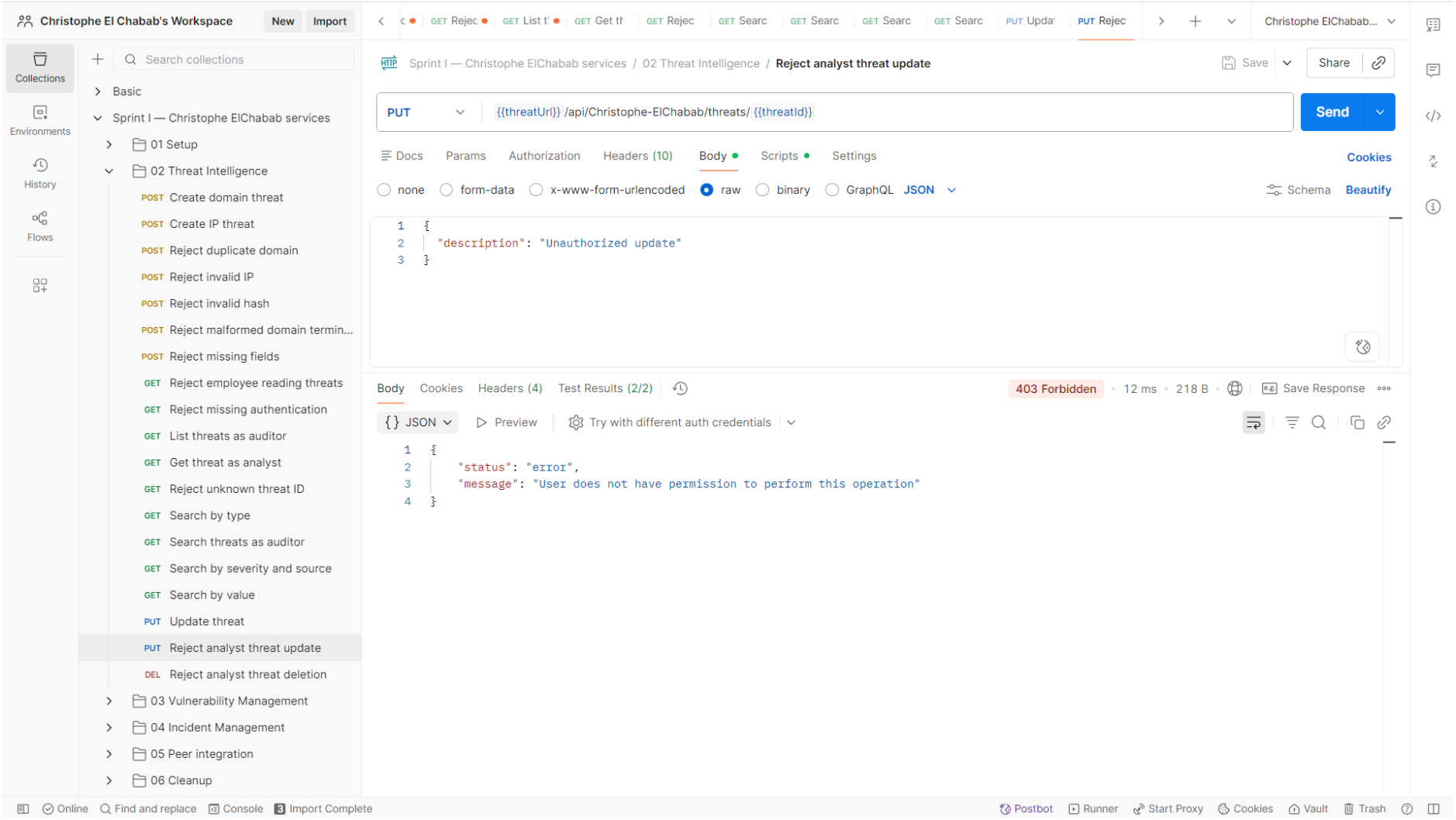

Figure 29. Christophe ElChabab - PUT Reject Analyst Threat Update - Shows that the unauthorized operation is refused with HTTP 403.

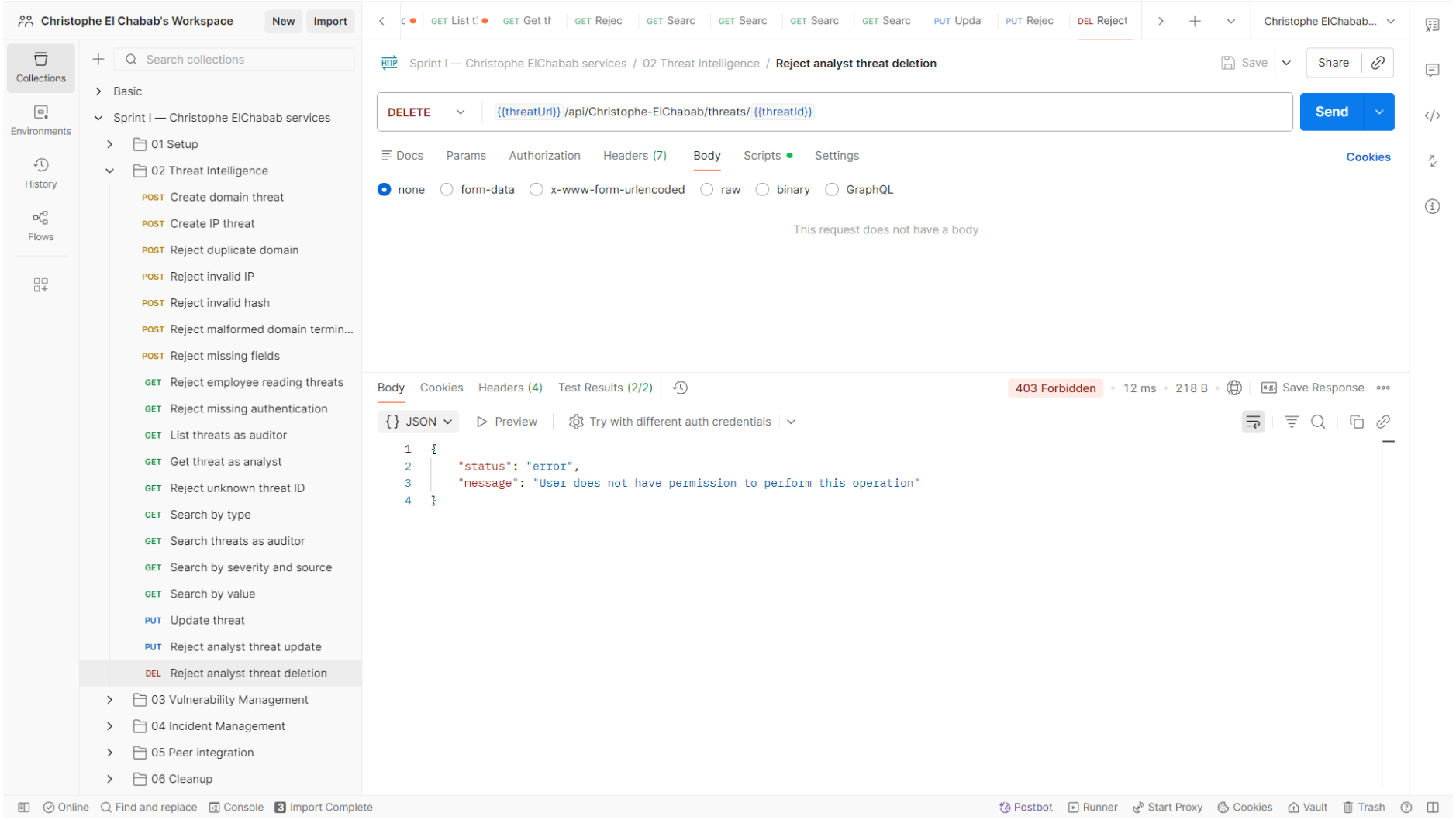

Figure 30. Christophe ElChabab - DELETE Reject Analyst Threat Deletion - Shows that the unauthorized operation is refused with HTTP 403.

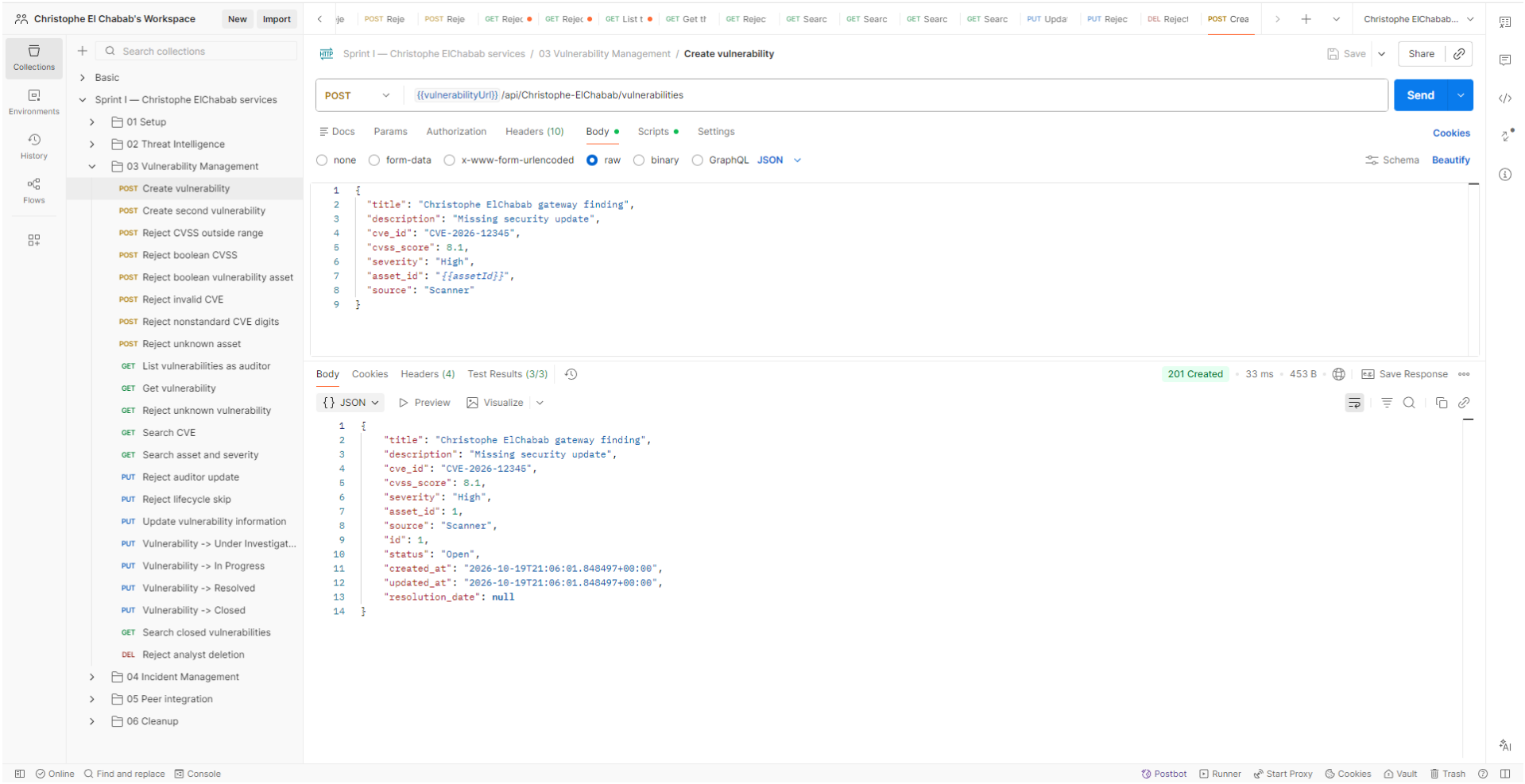

Figure 31. Christophe ElChabab - POST Create Vulnerability - Shows the submitted request and the newly created record returned with HTTP 201.

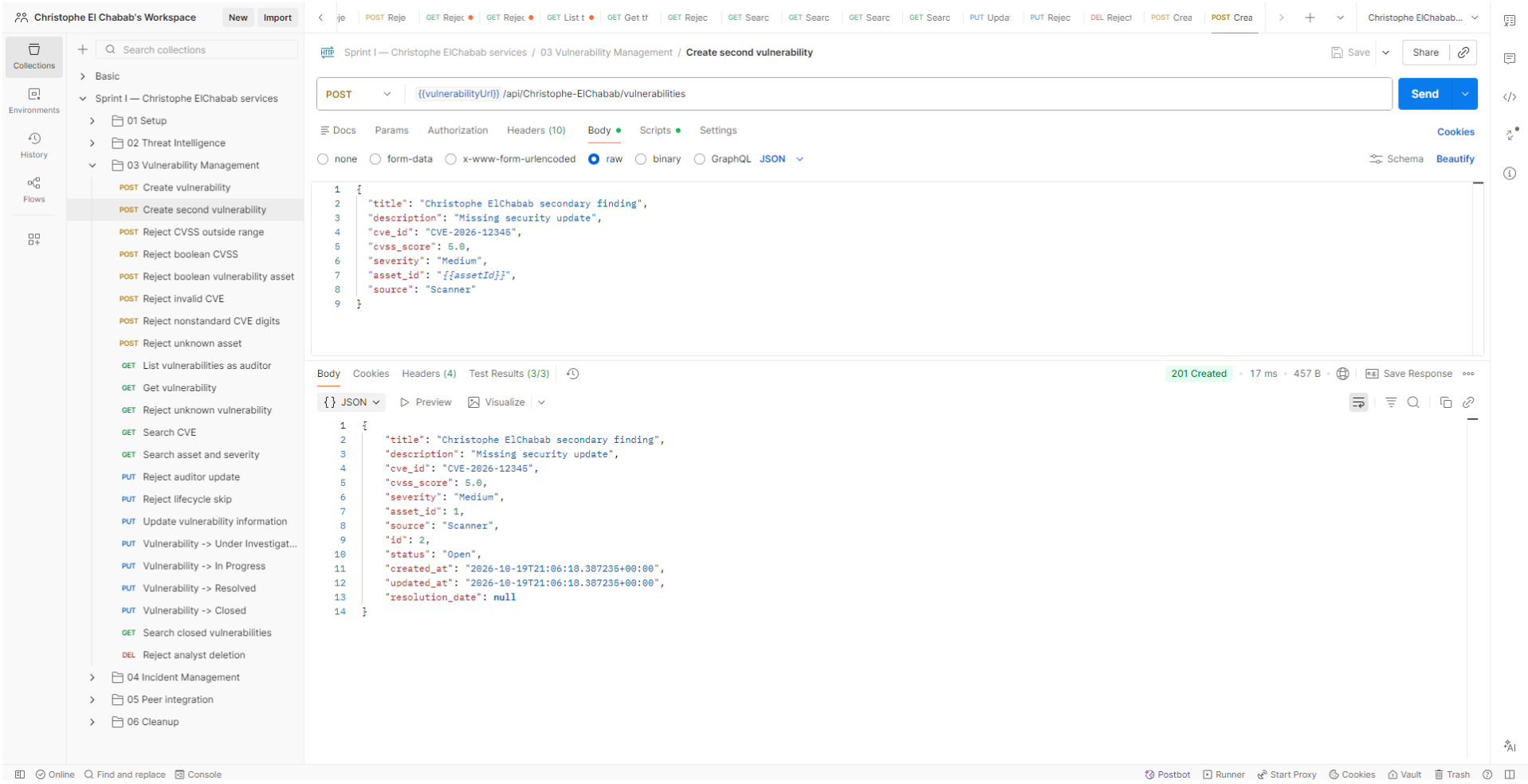

Figure 32. Christophe ElChabab - POST Create Second Vulnerability - Shows the submitted request and the newly created record returned with HTTP 201.

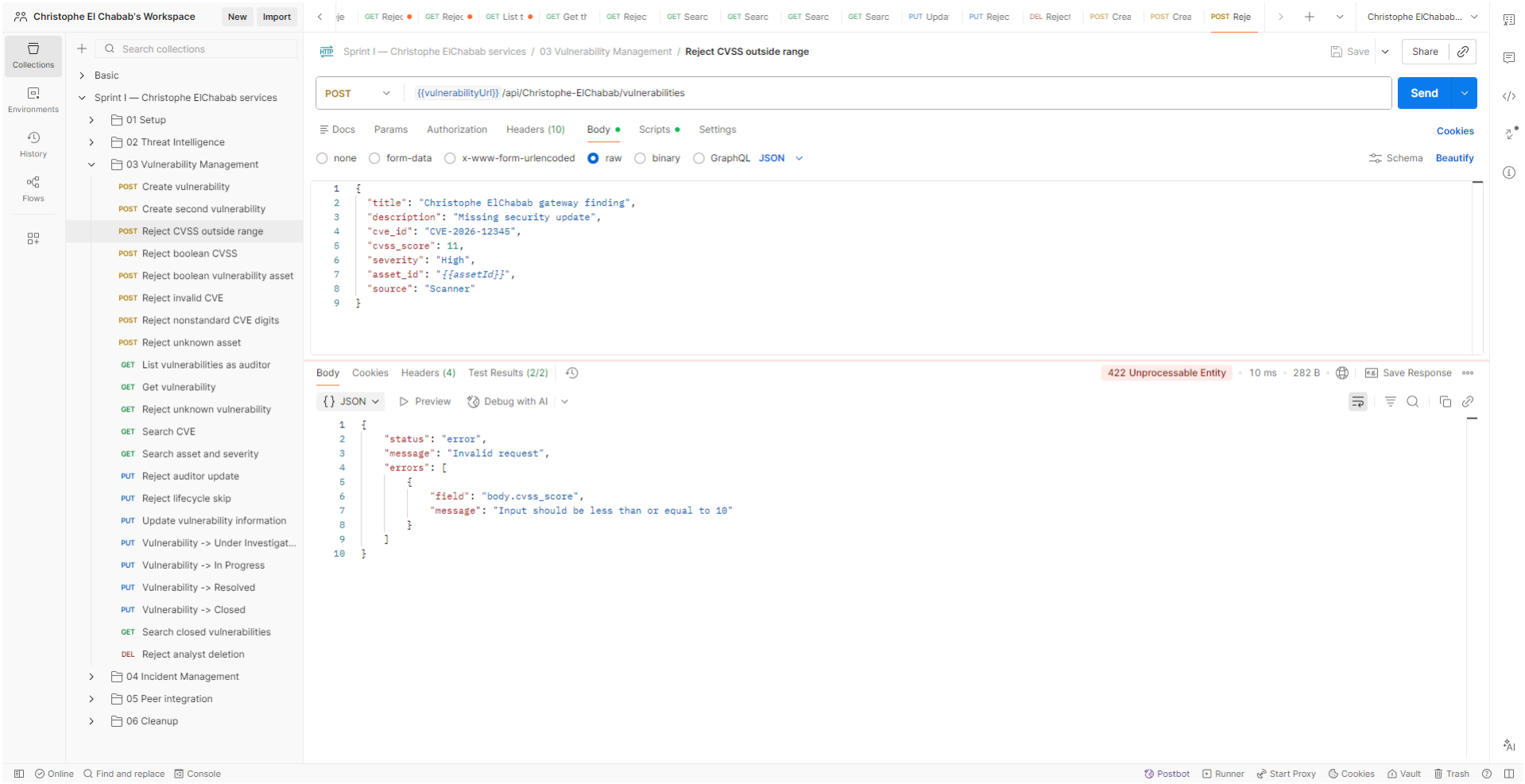

Figure 33. Christophe ElChabab - POST Reject Out of Range CVSS - Shows that invalid input or a reference is rejected with HTTP 422.

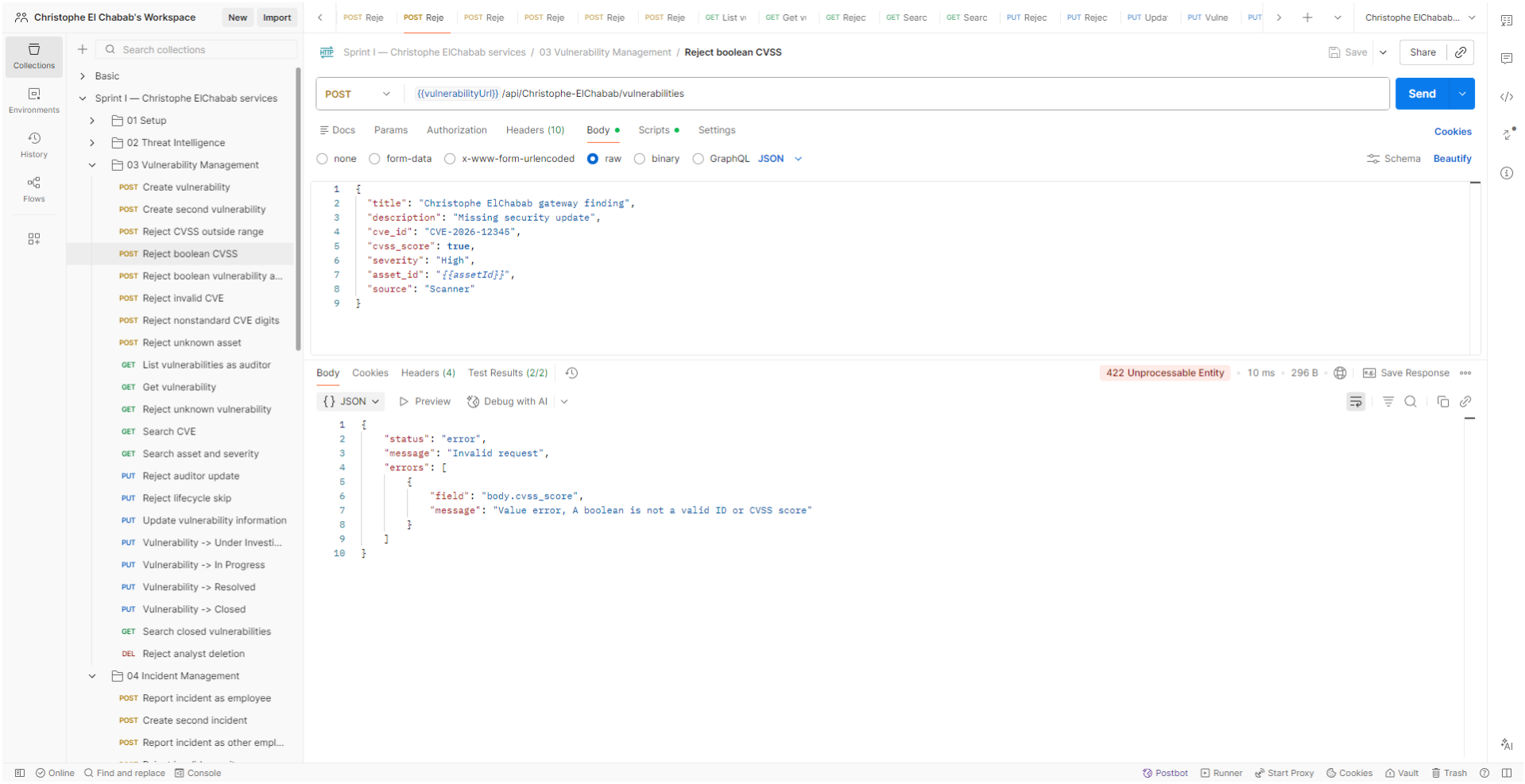

Figure 34. Christophe ElChabab - POST Reject Boolean CVSS - Shows that invalid input or a reference is rejected with HTTP 422.

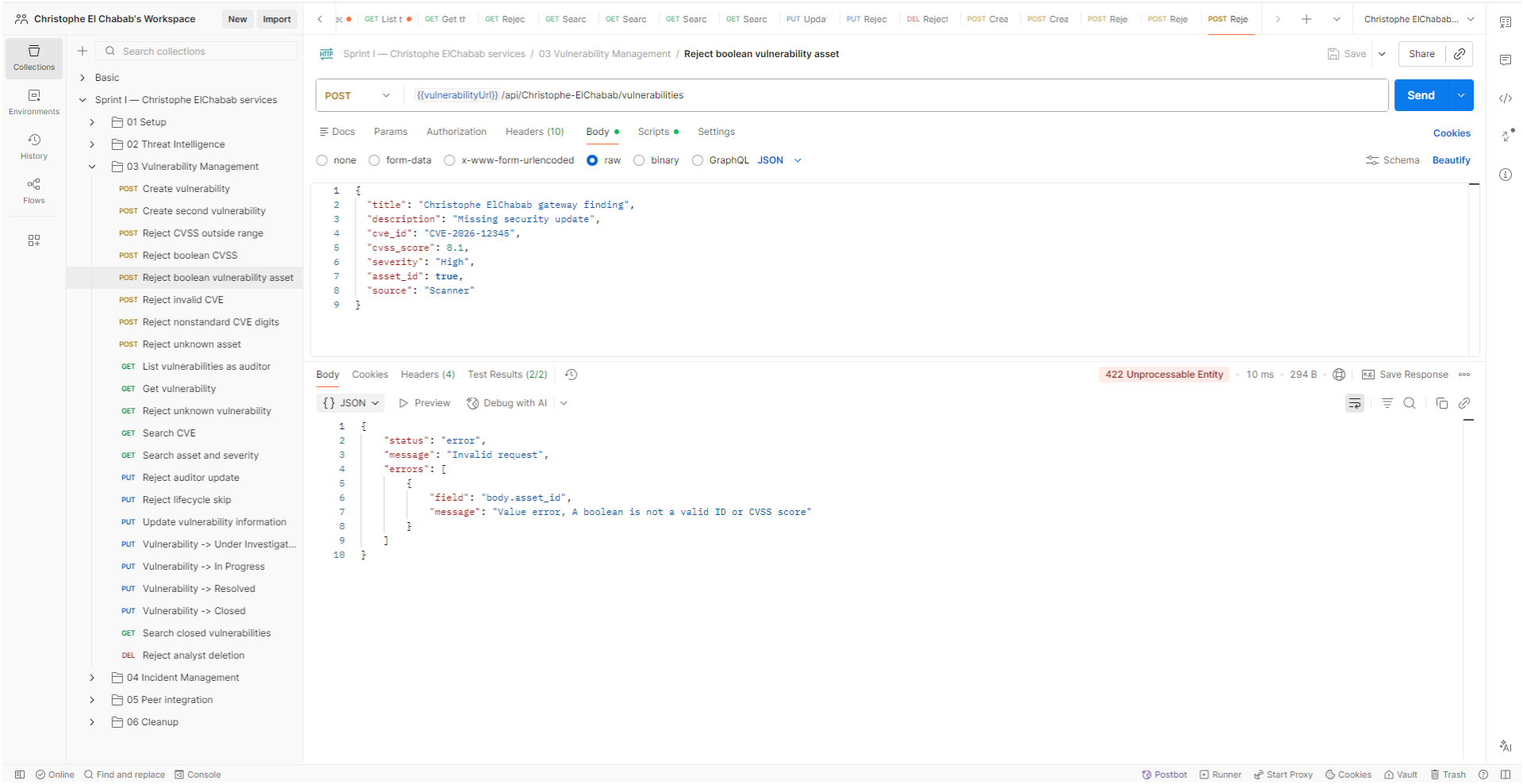

Figure 35. Christophe ElChabab - POST Reject Boolean Asset ID - Shows that invalid input or a reference is rejected with HTTP 422.

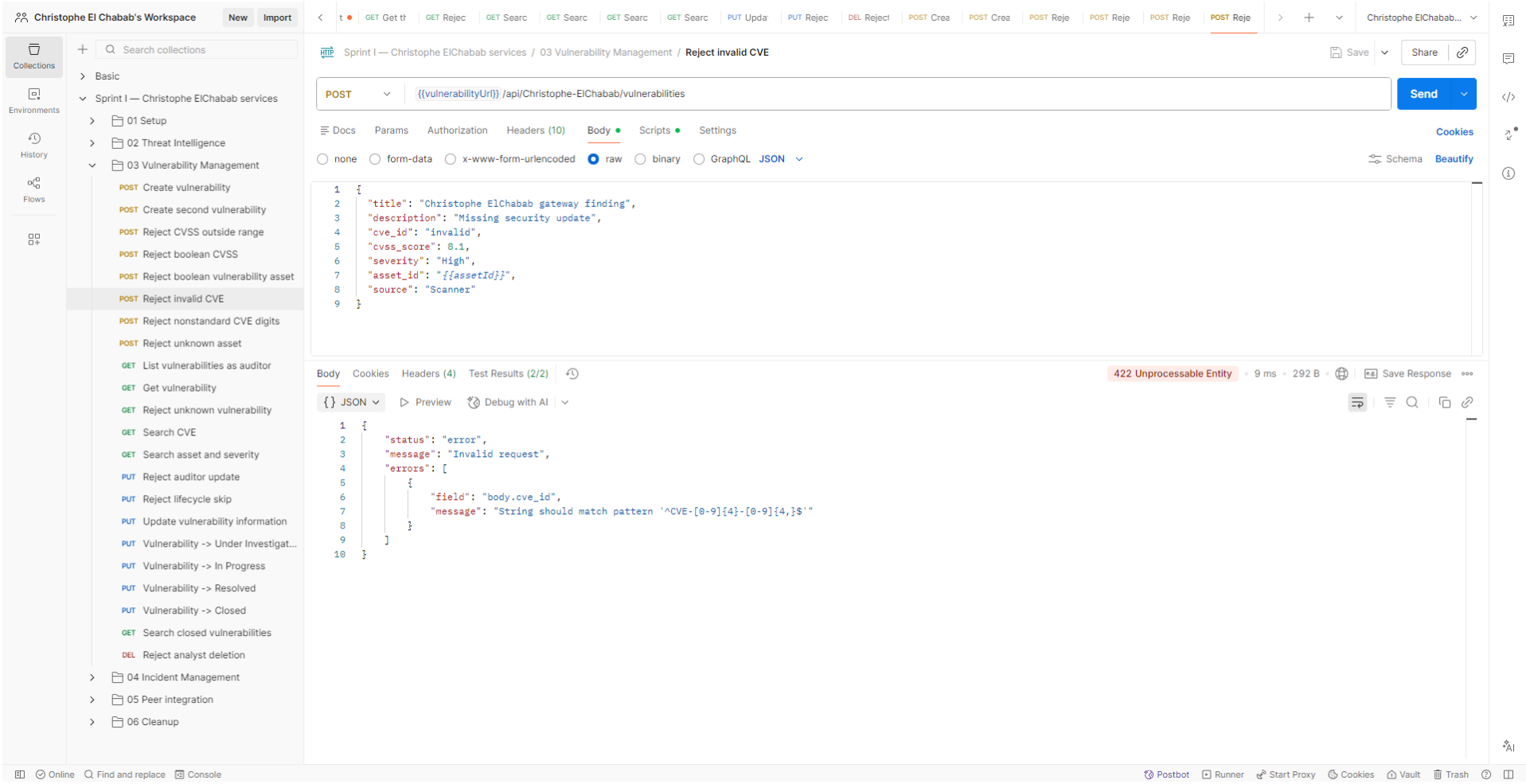

Figure 36. Christophe ElChabab - POST Reject Invalid CVE - Shows that invalid input or a reference is rejected with HTTP 422.

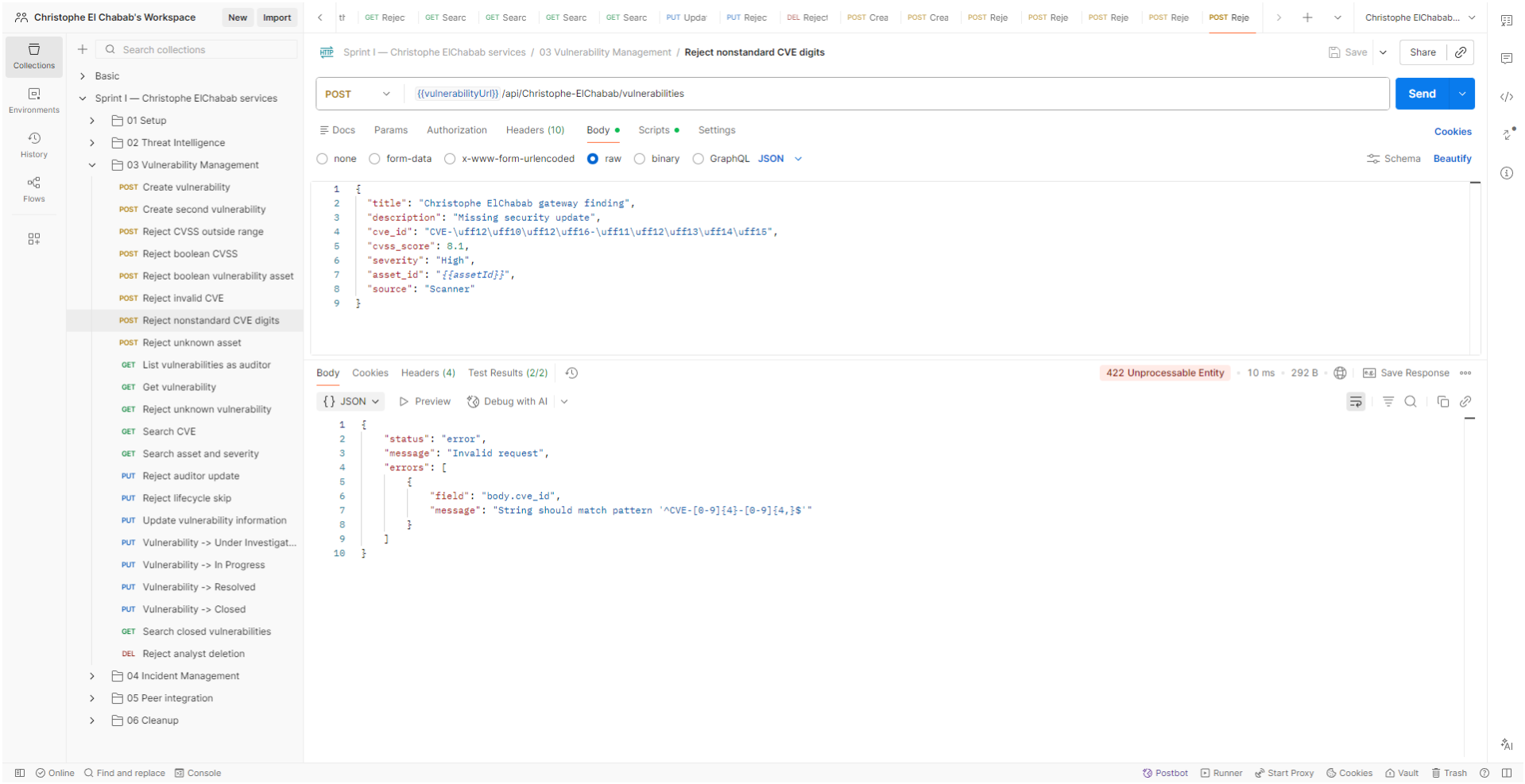

Figure 37. Christophe ElChabab - POST Reject Nonstandard CVE Digits - Shows that invalid input or a reference is rejected with HTTP 422.

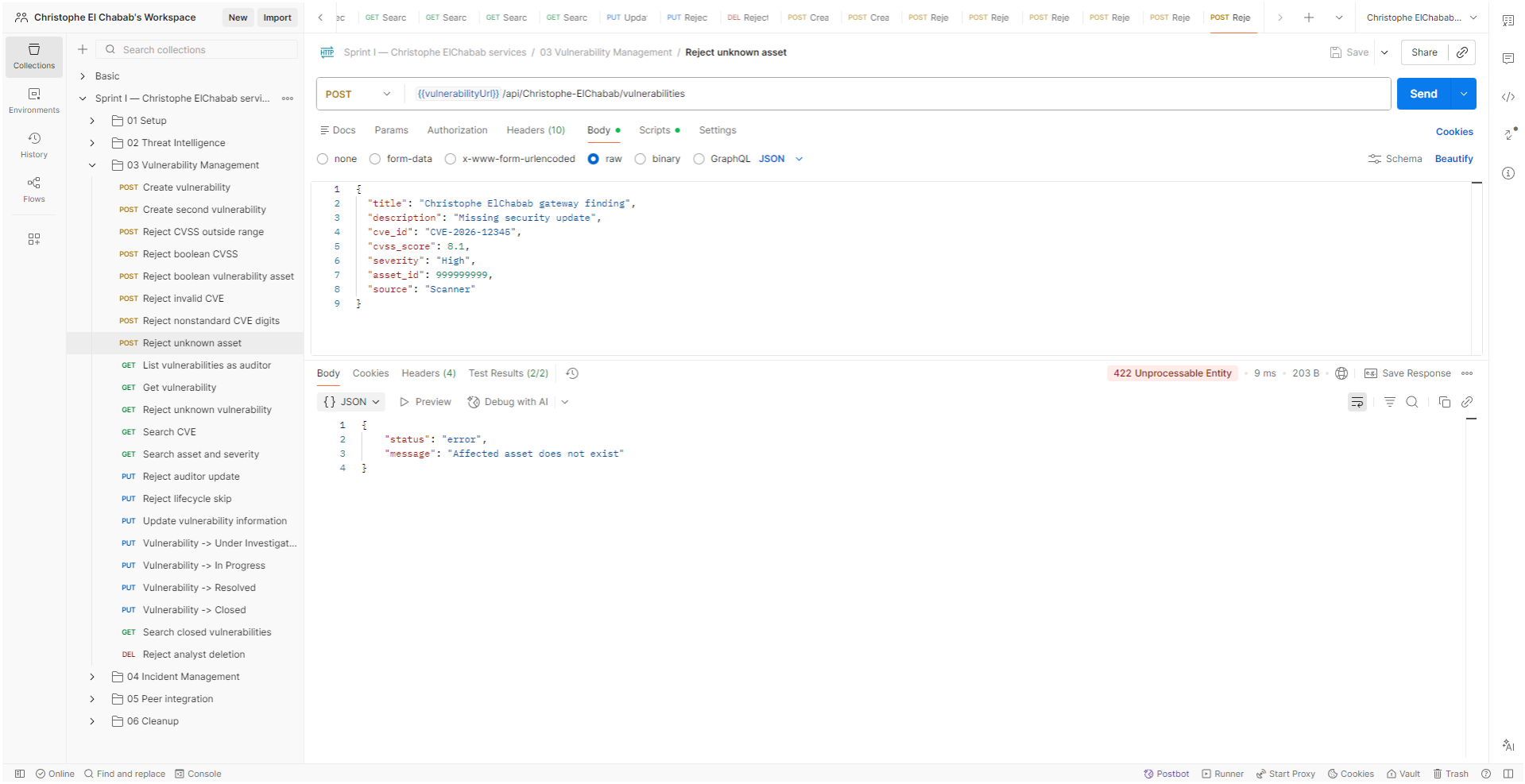

Figure 38. Christophe ElChabab - POST Reject Unknown Asset - Shows that invalid input or a reference is rejected with HTTP 422.

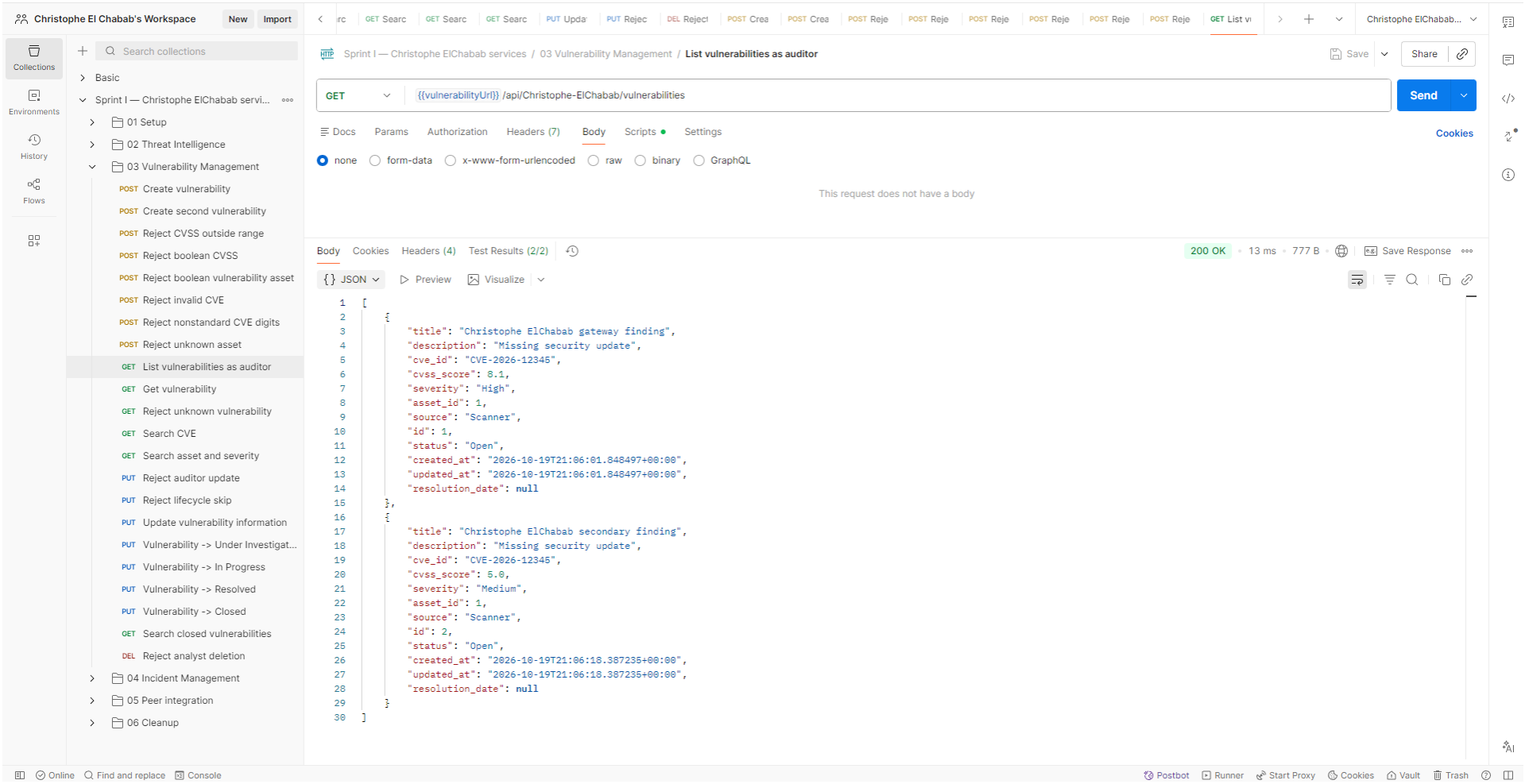

Figure 39. Christophe ElChabab - GET List Vulnerabilities as Auditor - Shows the named request and its successful HTTP 200 result.

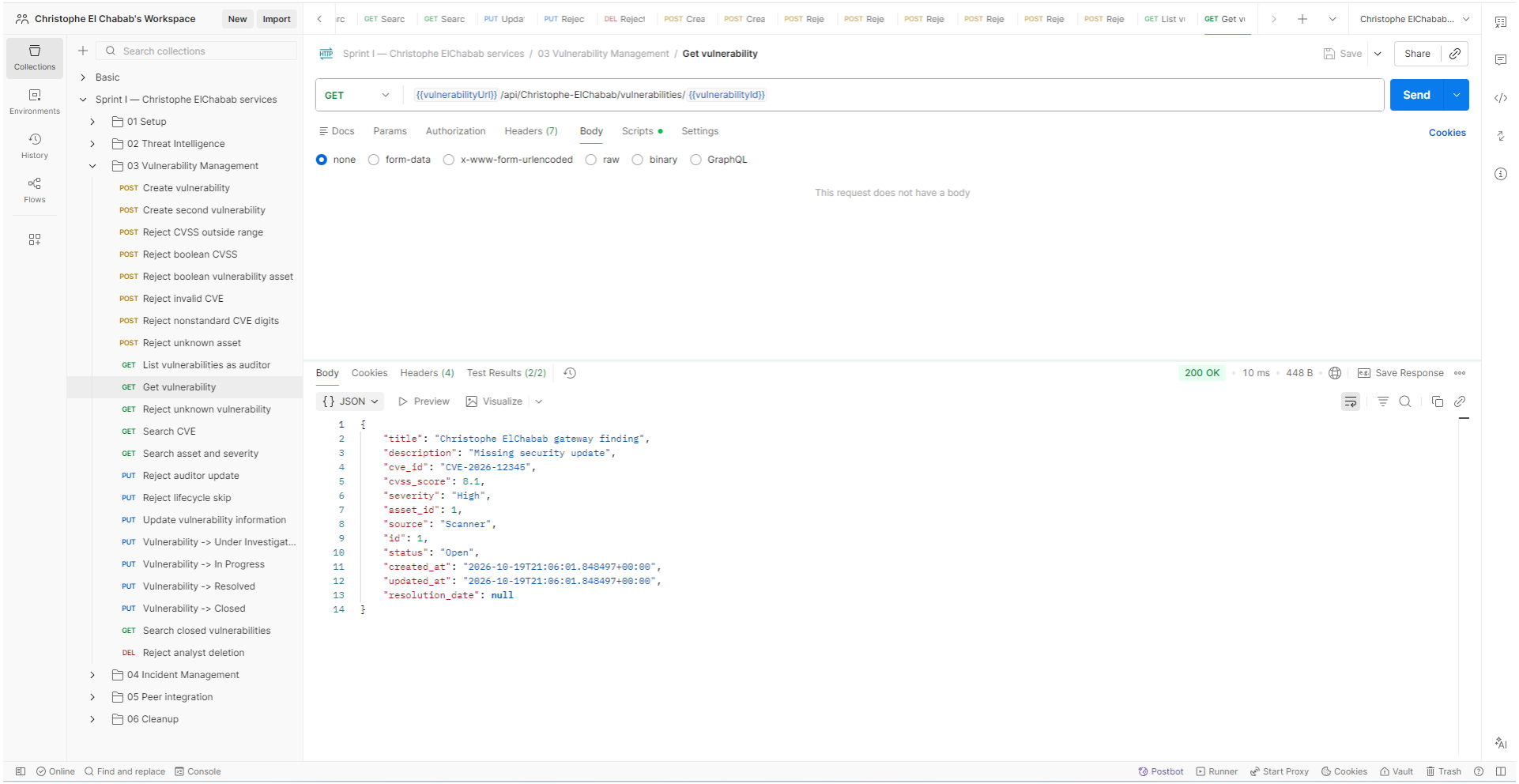

Figure 40. Christophe ElChabab - GET Get Vulnerability - Shows the named request and its successful HTTP 200 result.

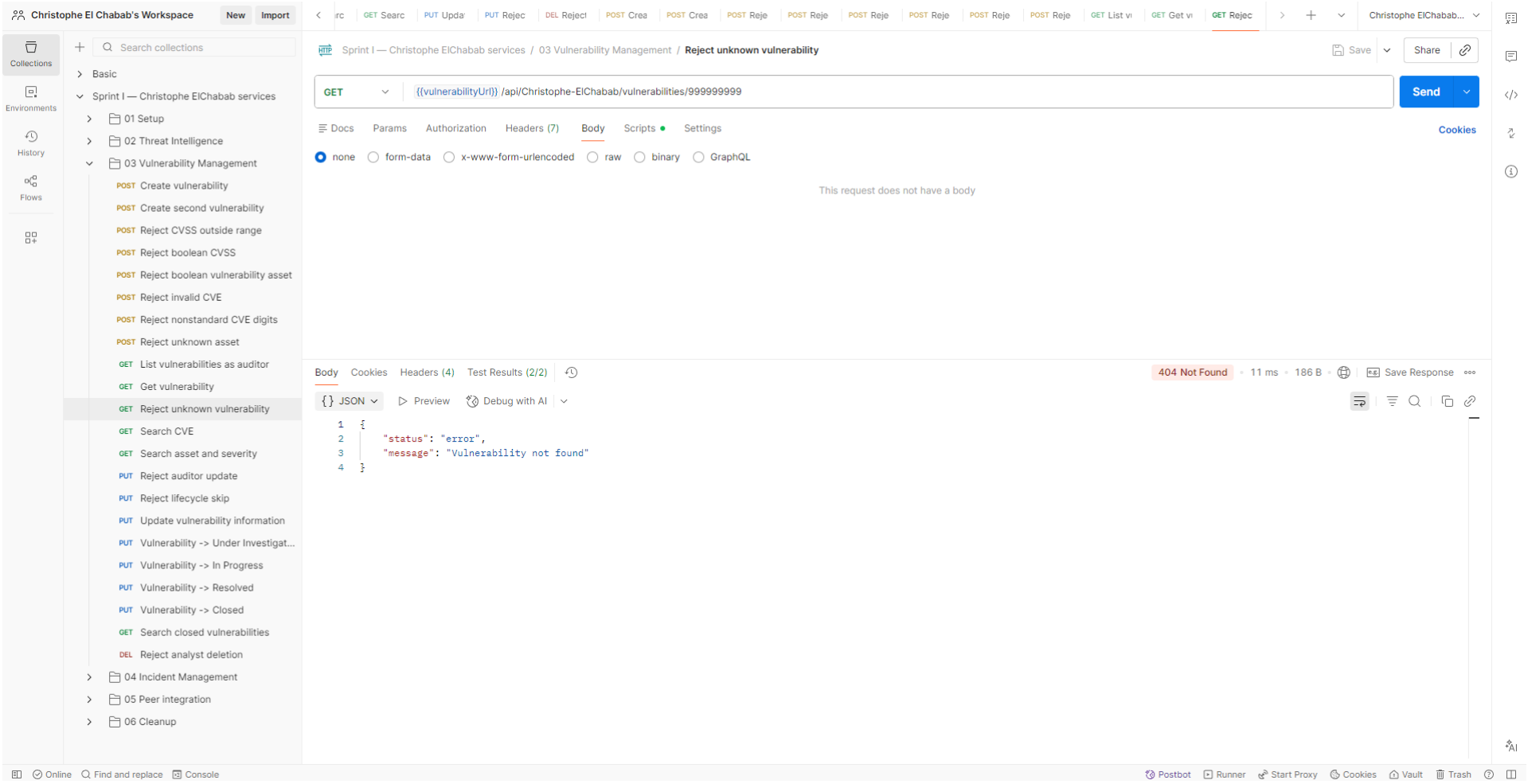

Figure 41. Christophe ElChabab - GET Reject Unknown Vulnerability ID - Shows that an unknown record returns HTTP 404.

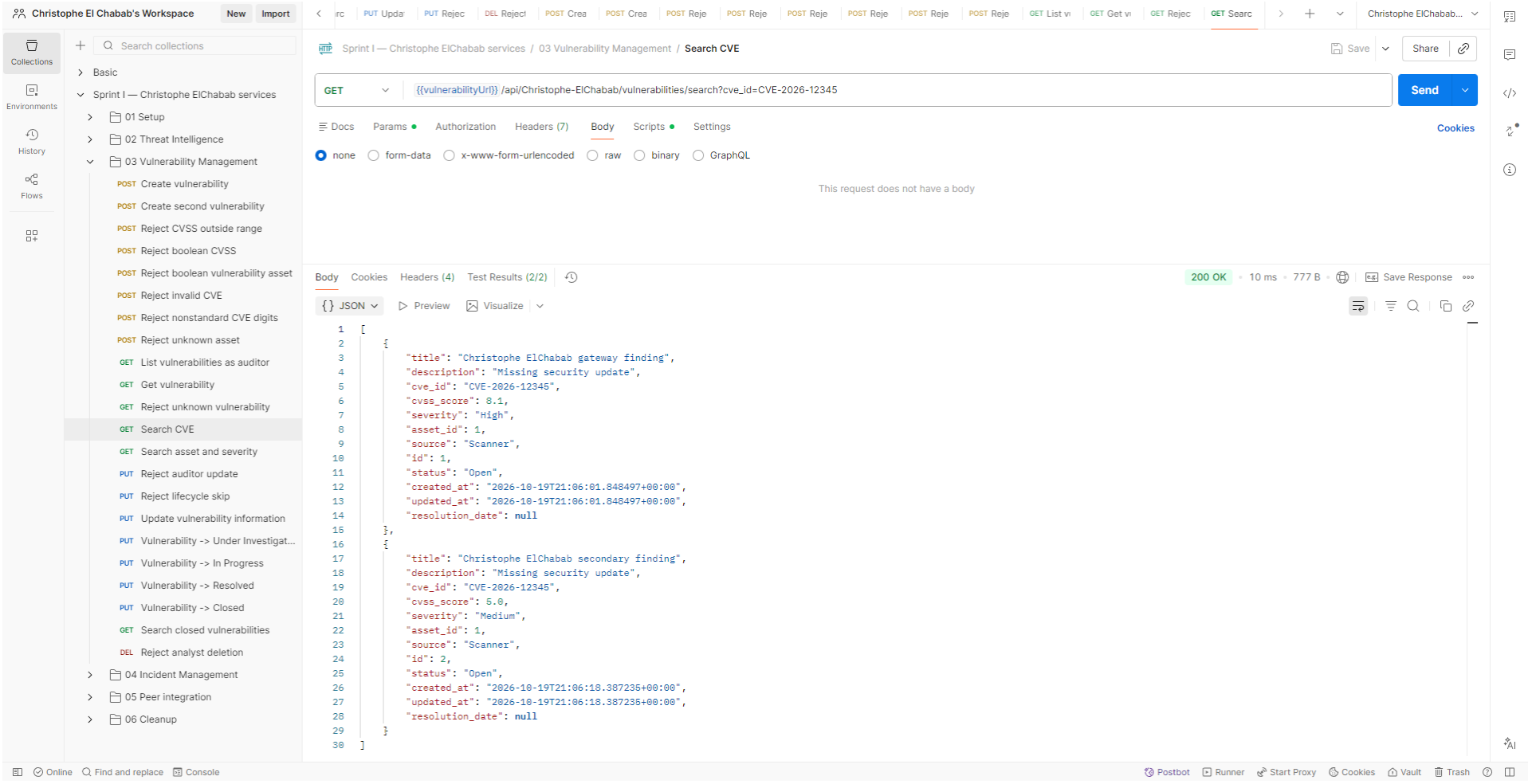

Figure 42. Christophe ElChabab - GET Search by CVE - Shows the named request and its successful HTTP 200 result.

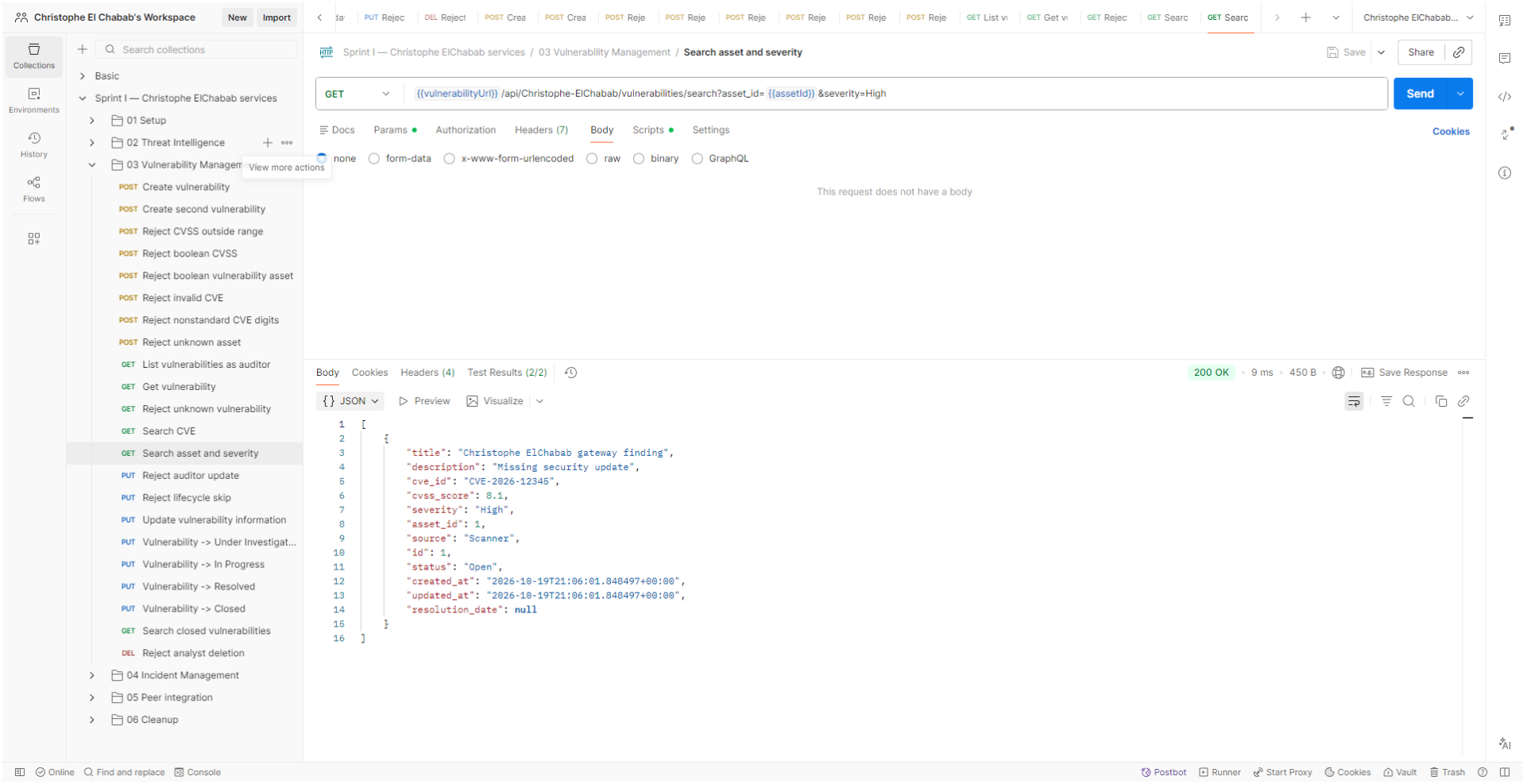

Figure 43. Christophe ElChabab - GET Search by Asset and Severity - Shows the named request and its successful HTTP 200 result.

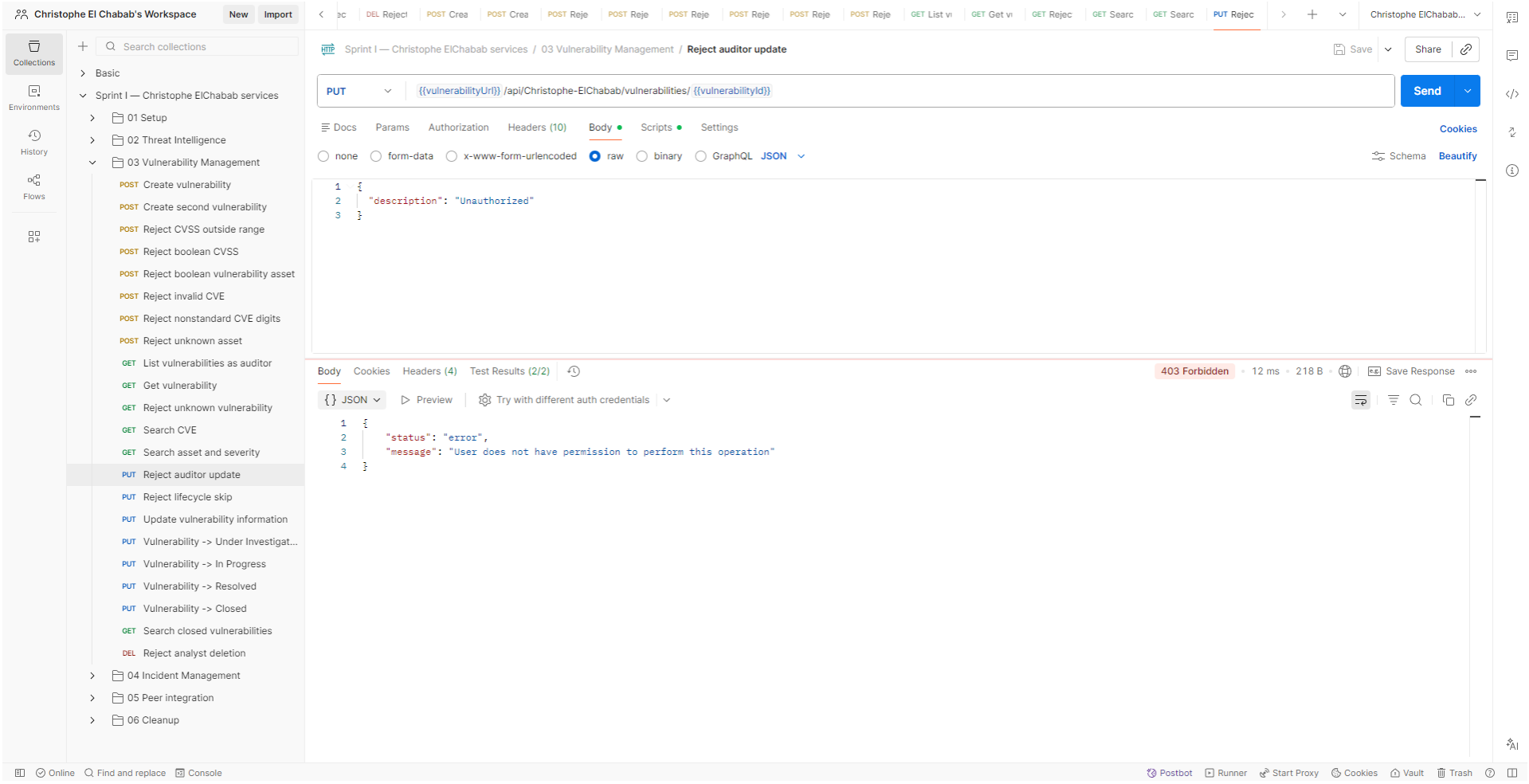

Figure 44. Christophe ElChabab - PUT Reject Auditor Update - Shows that the unauthorized operation is refused with HTTP 403.

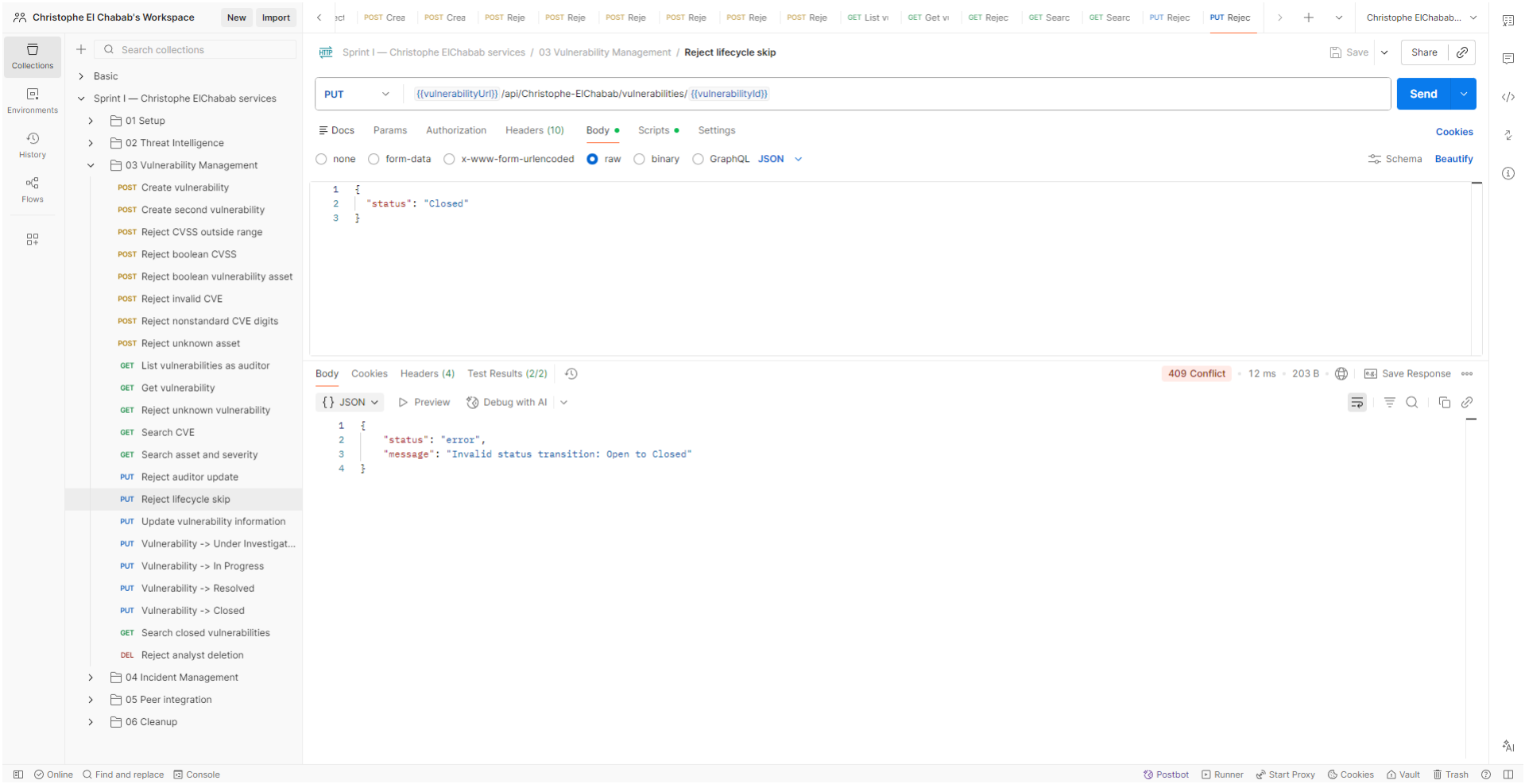

Figure 45. Christophe ElChabab - PUT Reject Lifecycle Skip - Shows that a duplicate indicator or invalid lifecycle change is rejected with HTTP 409.

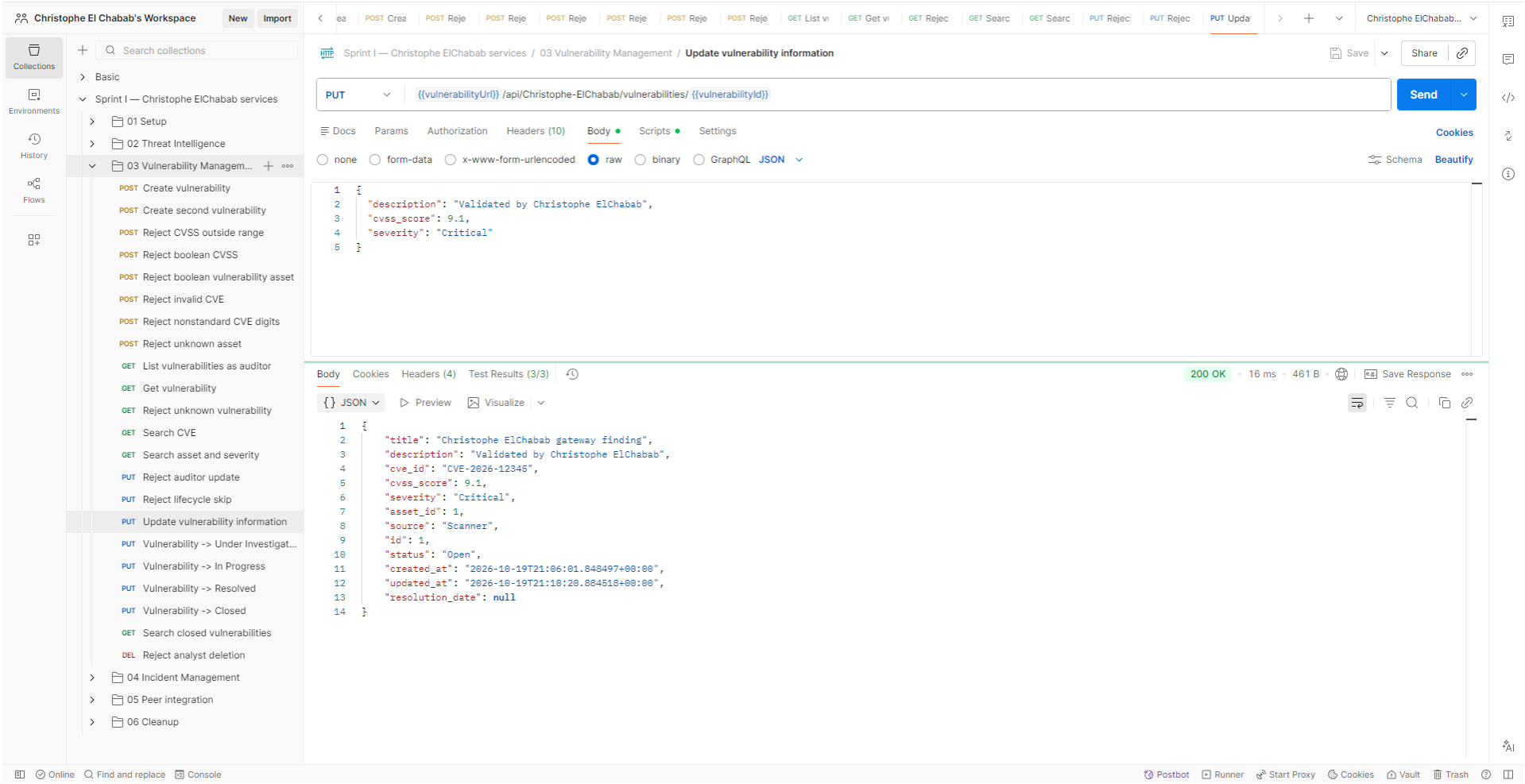

Figure 46. Christophe ElChabab - PUT Update Vulnerability - Shows the named request and its successful HTTP 200 result.

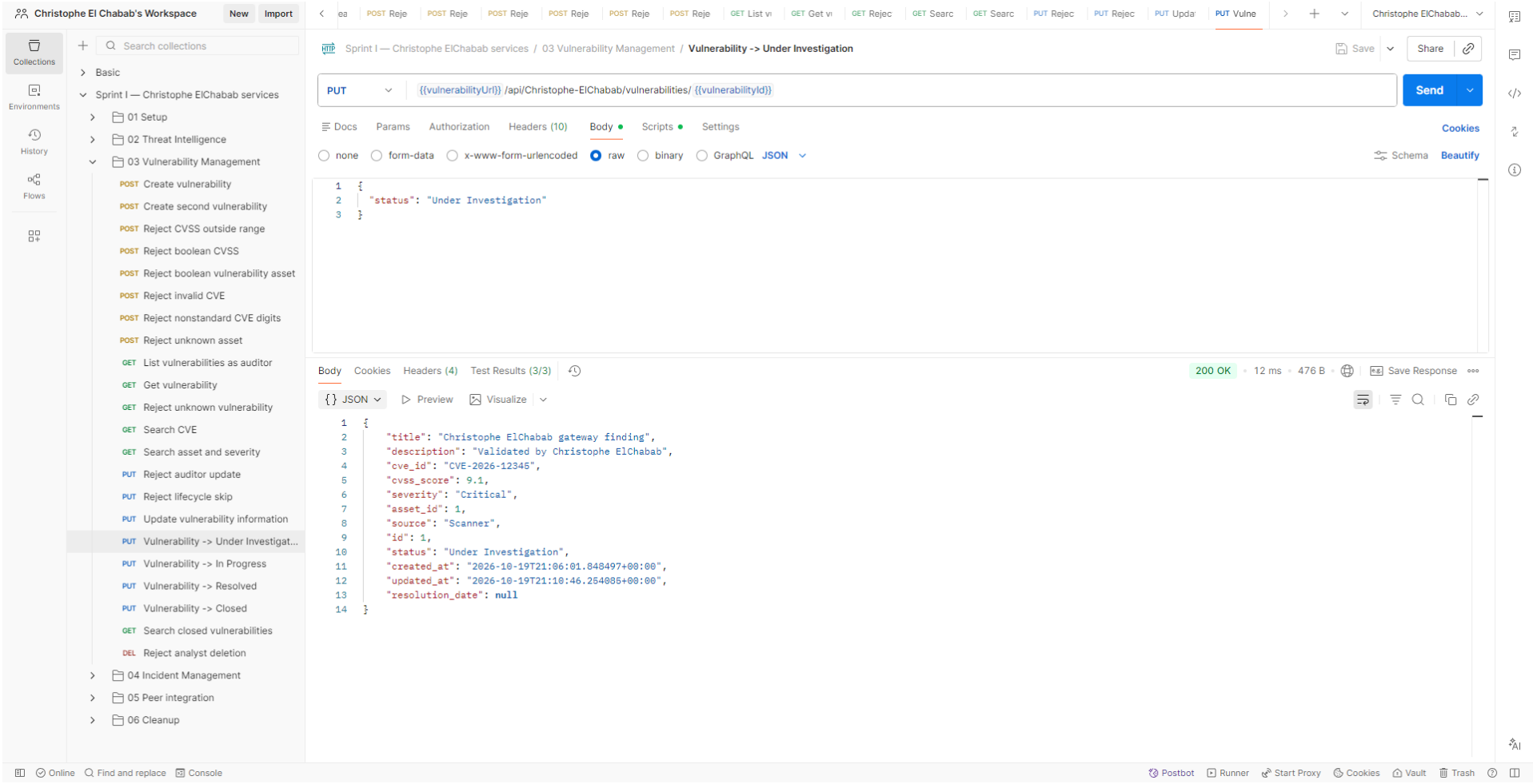

Figure 47. Christophe ElChabab - PUT Status Under Investigation - Shows the named request and its successful HTTP 200 result.

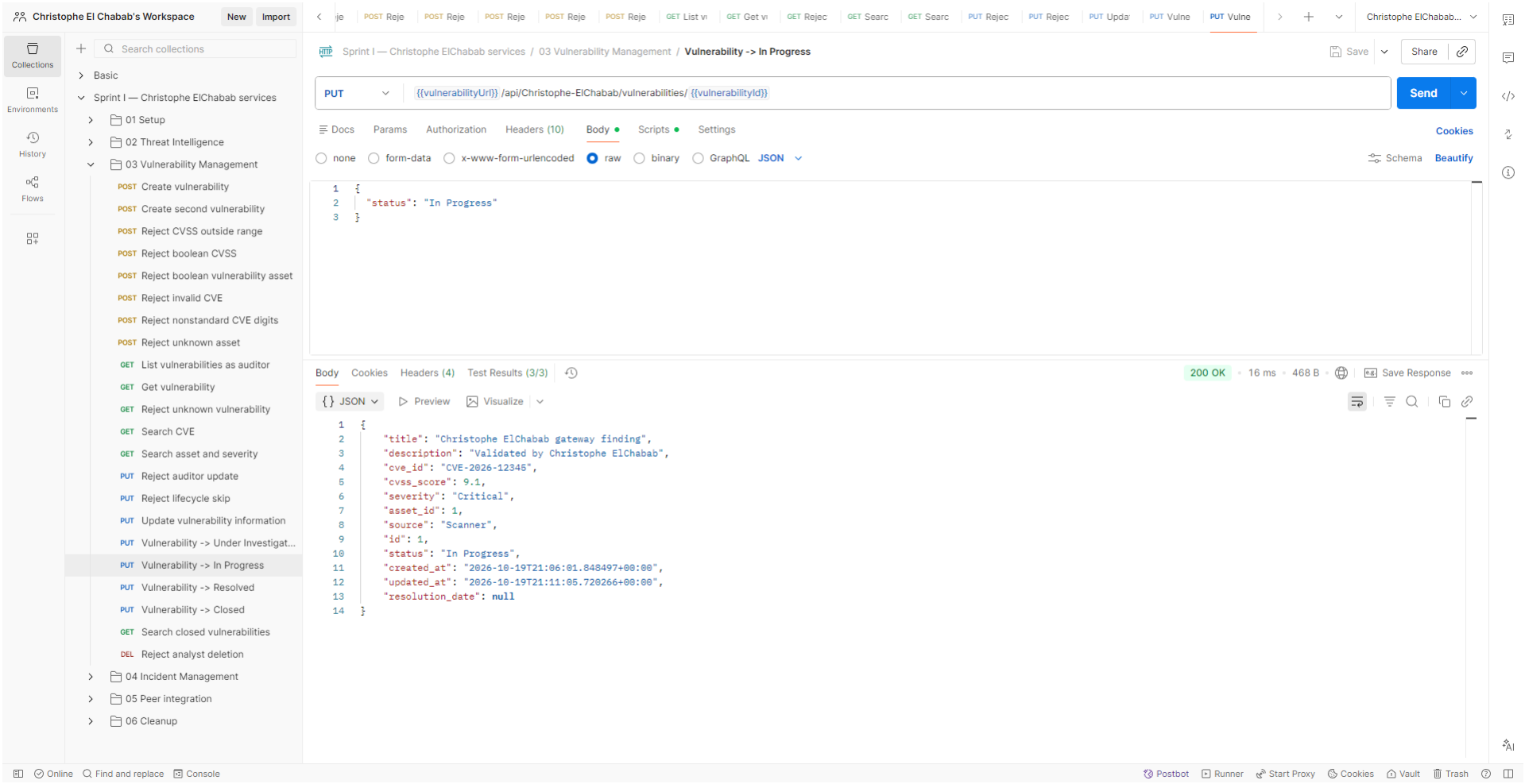

Figure 48. Christophe ElChabab - PUT Status In Progress - Shows the named request and its successful HTTP 200 result.

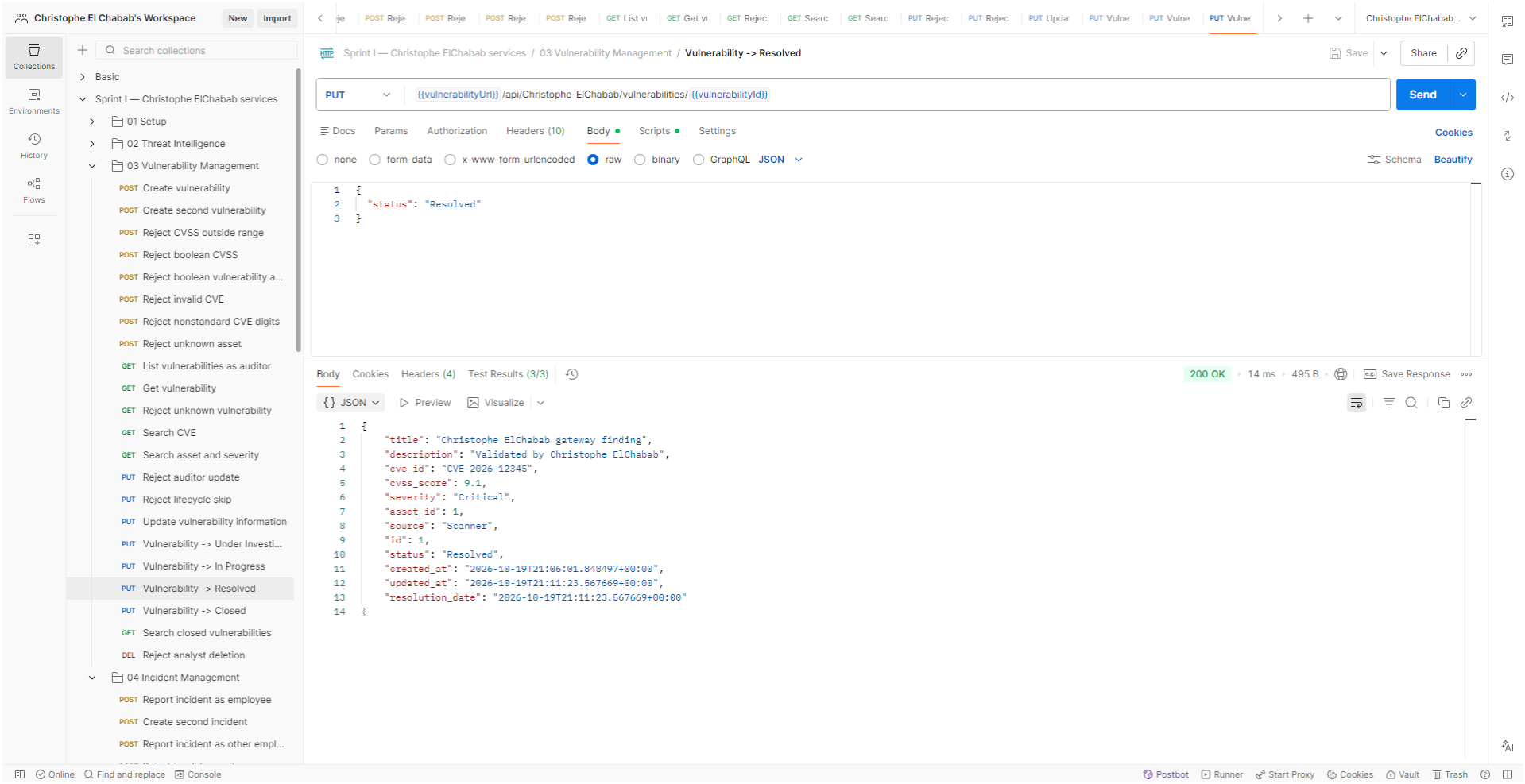

Figure 49. Christophe ElChabab - PUT Status Resolved - Shows the named request and its successful HTTP 200 result.

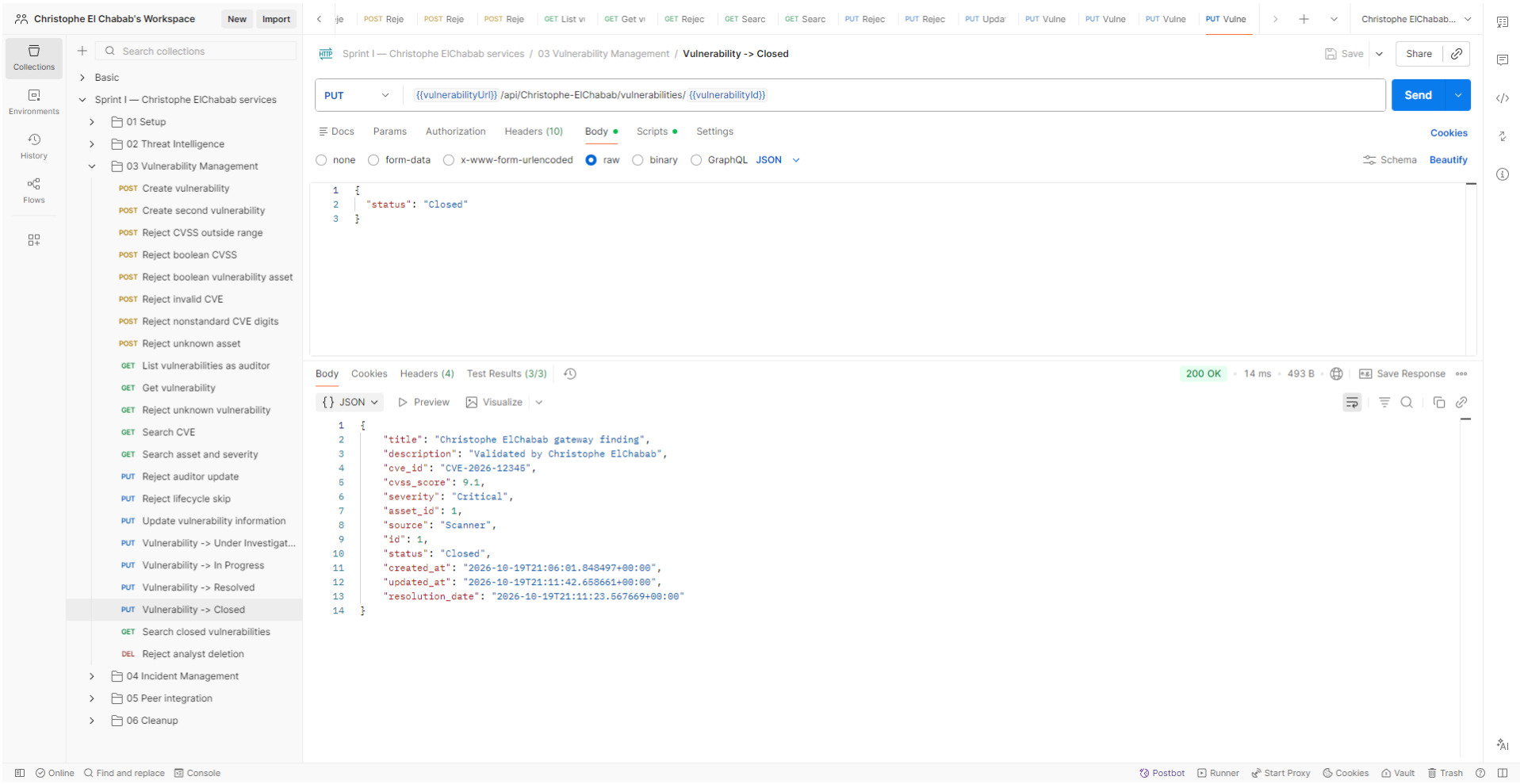

Figure 50. Christophe ElChabab - PUT Status Closed - Shows the named request and its successful HTTP 200 result.

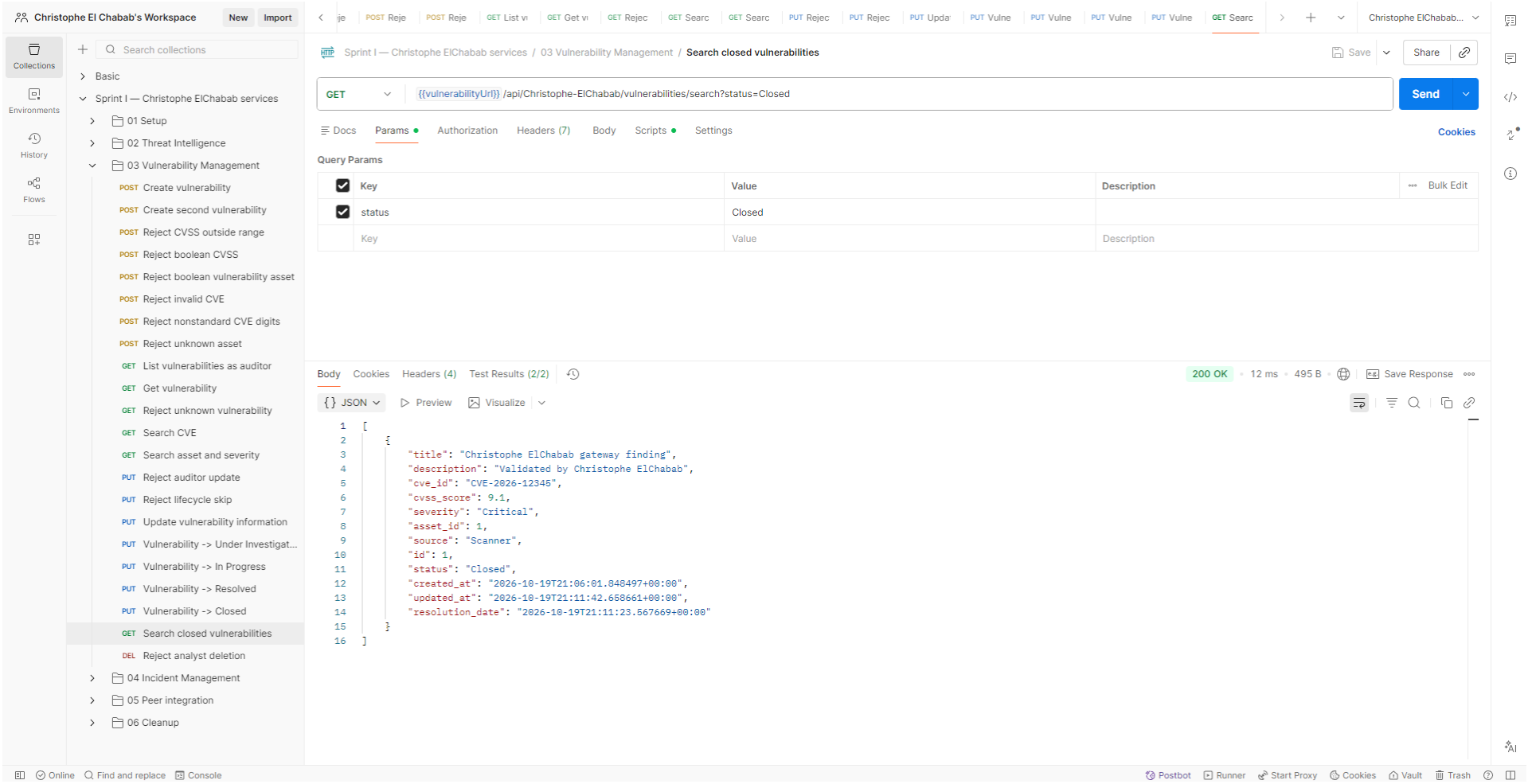

Figure 51. Christophe ElChabab - GET Search Closed Vulnerabilities - Shows the named request and its successful HTTP 200 result.

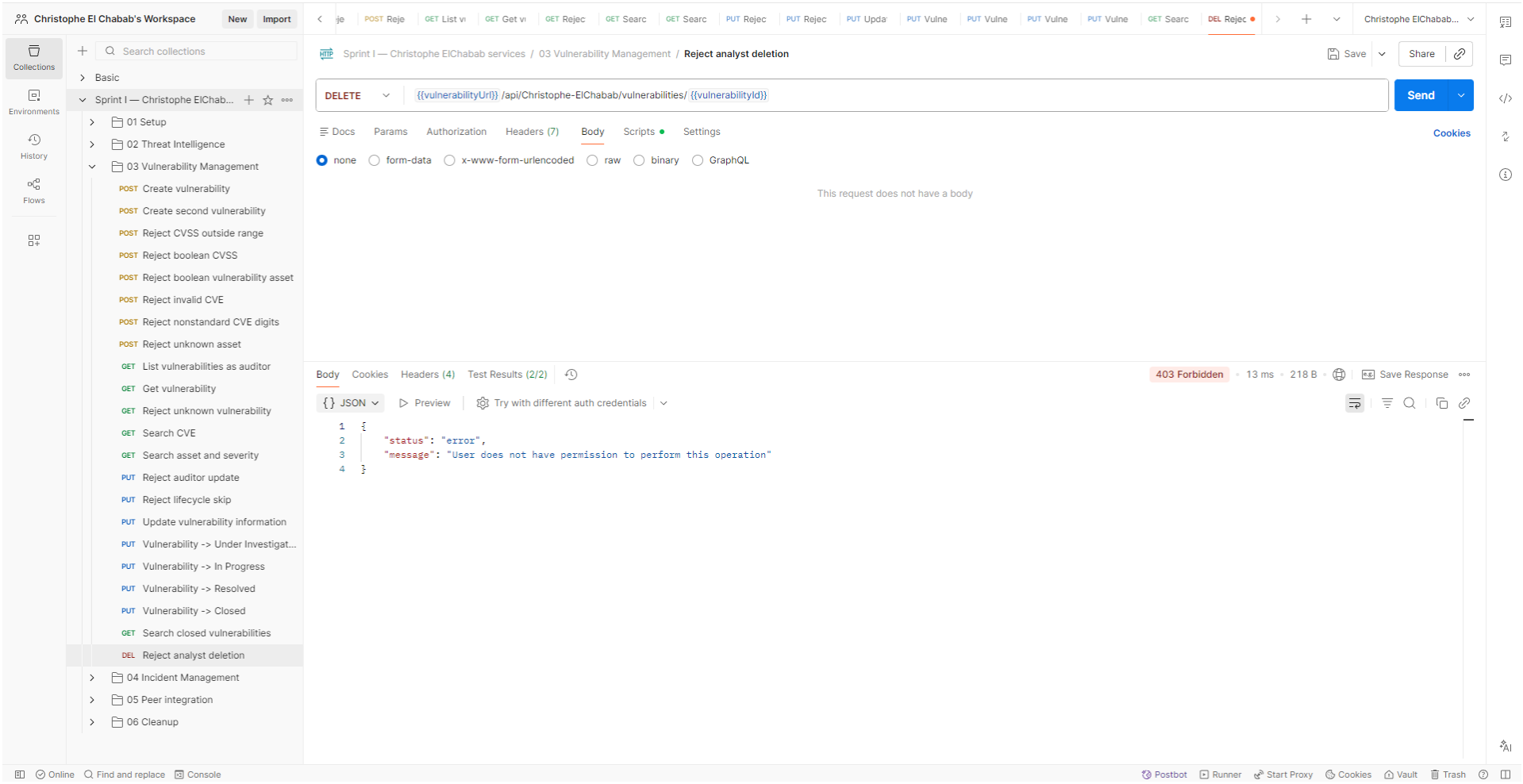

Figure 52. Christophe ElChabab - DELETE Reject Analyst Deletion - Shows that the unauthorized operation is refused with HTTP 403.

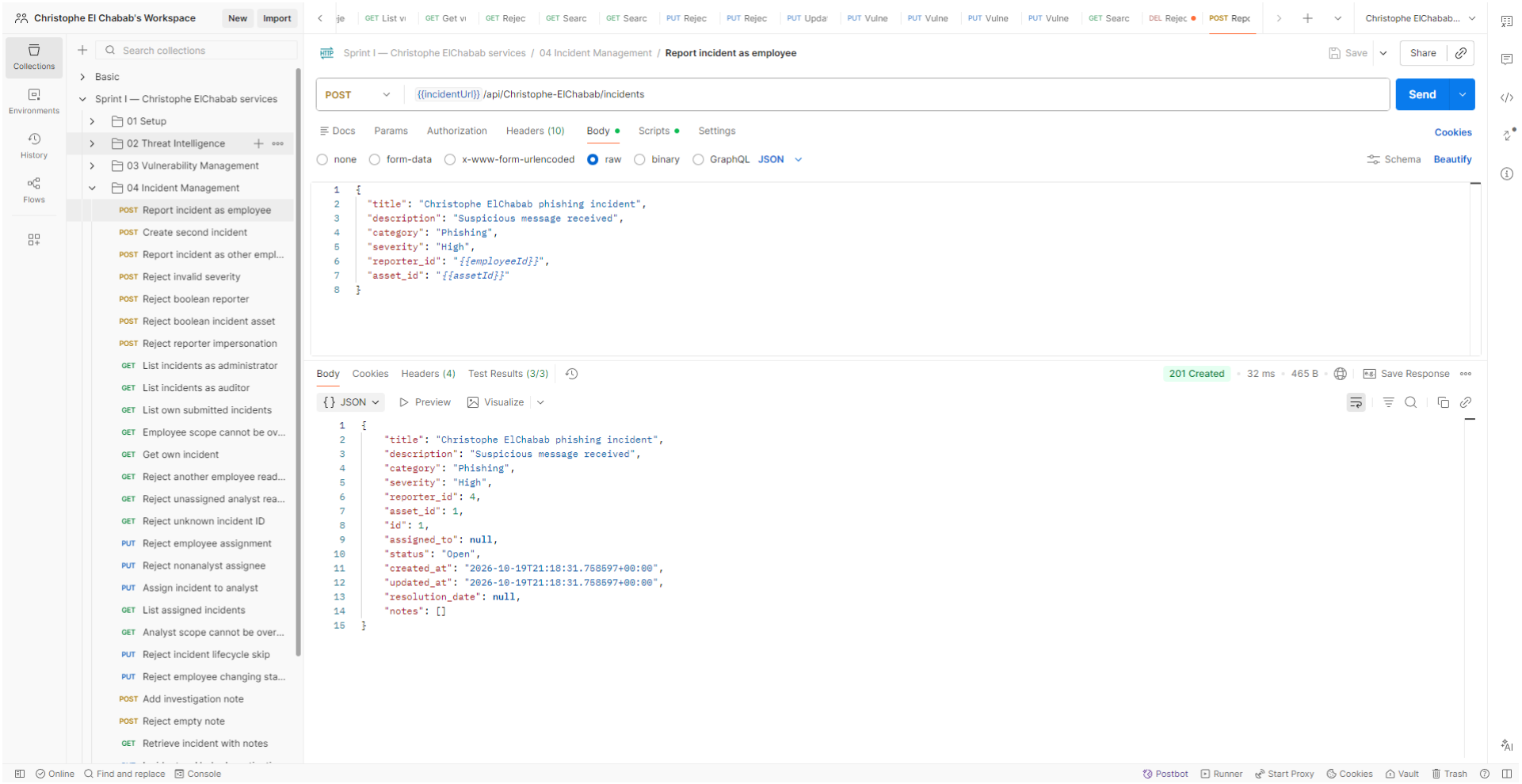

Figure 53. Christophe ElChabab - POST Report incident as employee - Shows the submitted request and the newly created record returned with HTTP 201.

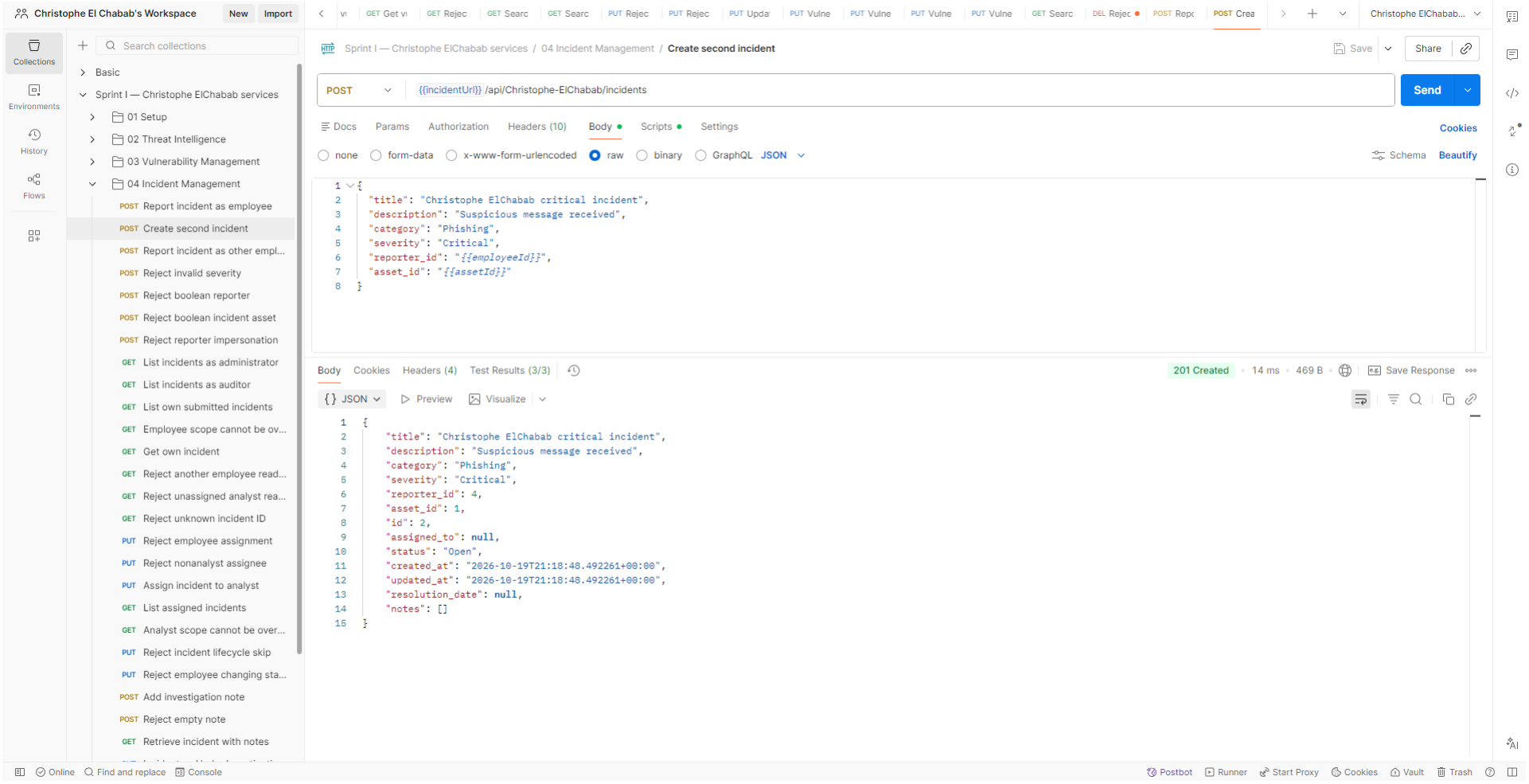

Figure 54. Christophe ElChabab - POST Create second incident - Shows the submitted request and the newly created record returned with HTTP 201.

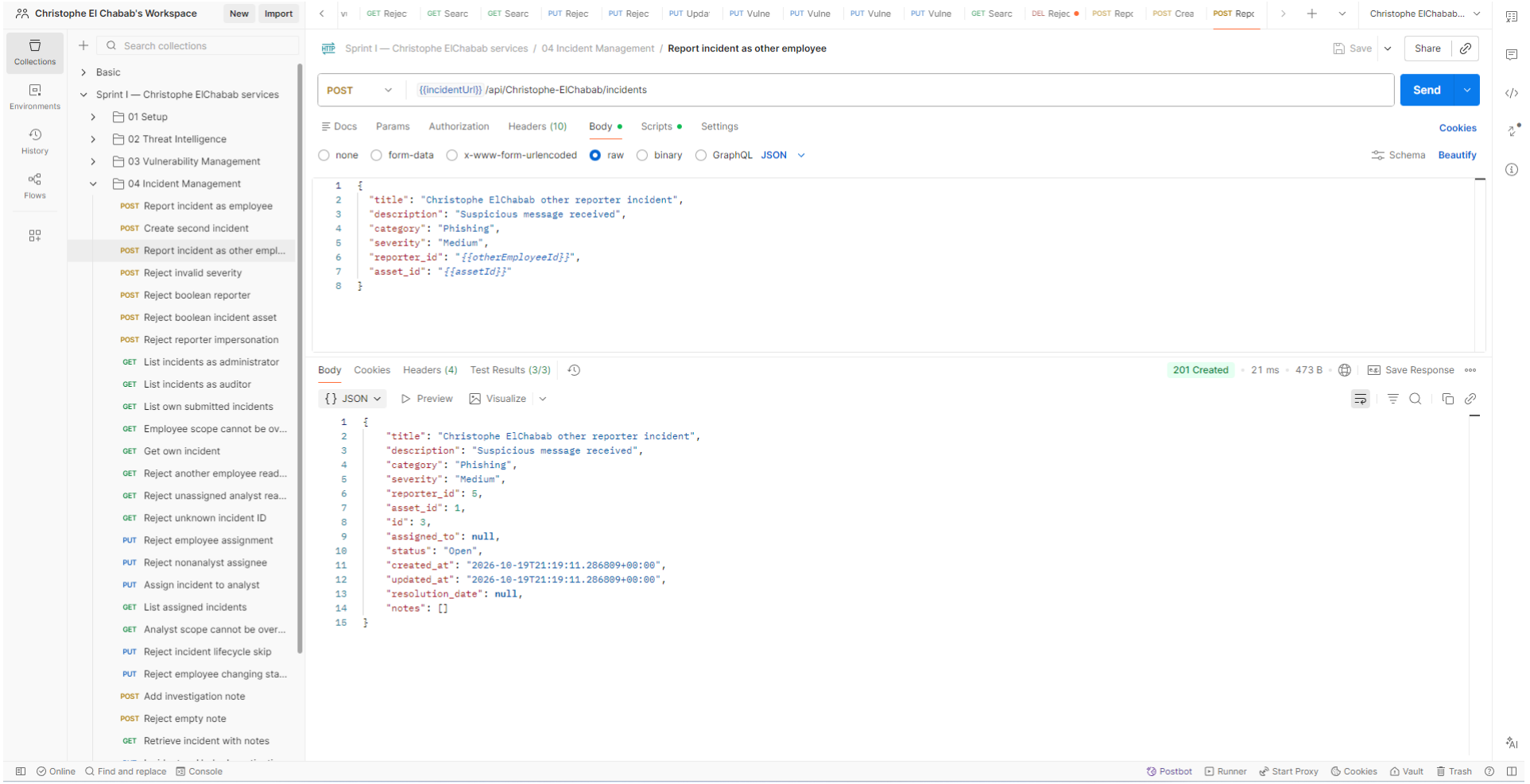

Figure 55. Christophe ElChabab - POST Report incident as other employee - Shows the submitted request and the newly created record returned with HTTP 201.

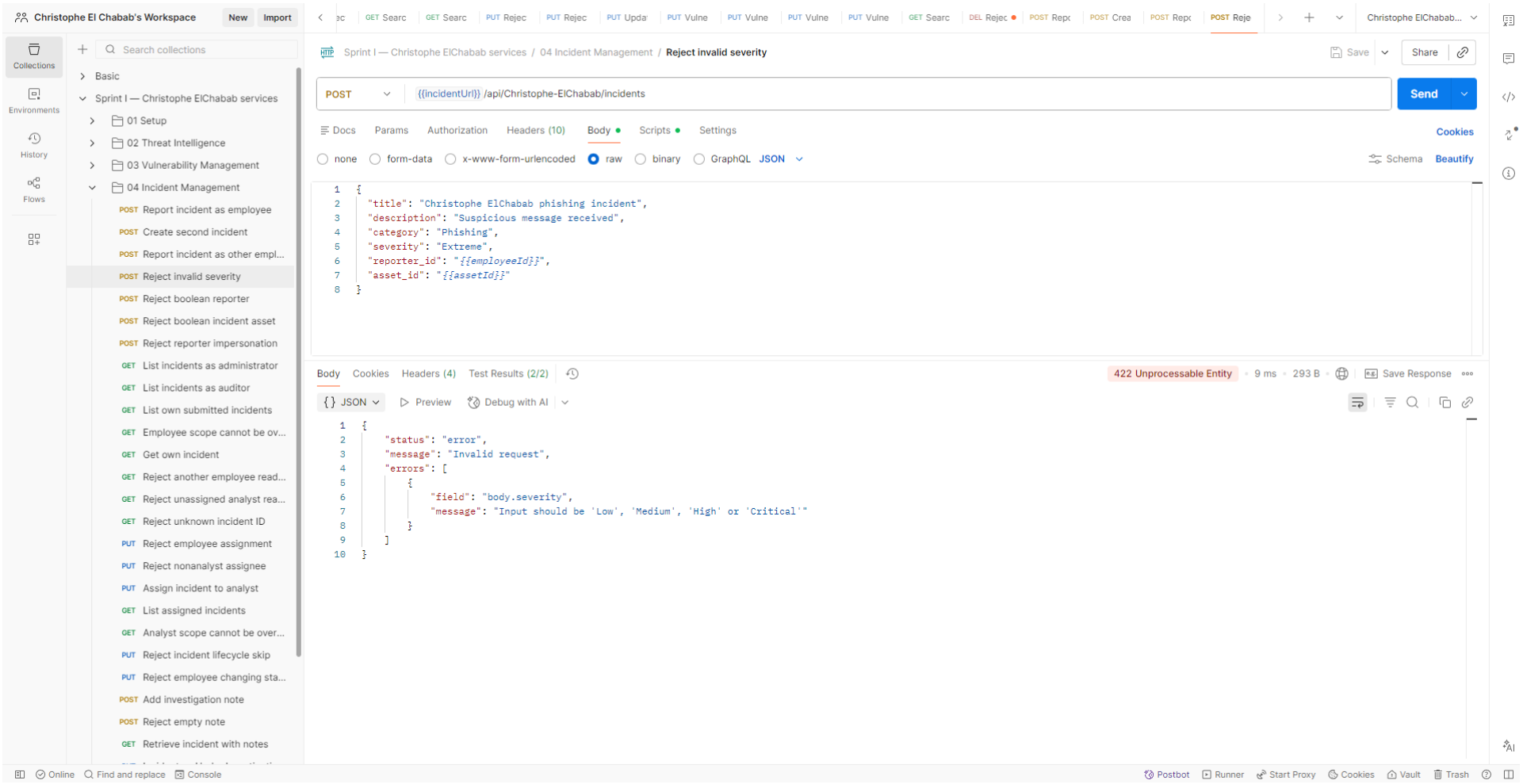

Figure 56. Christophe ElChabab - POST Reject invalid severity - Shows that invalid input or a reference is rejected with HTTP 422.

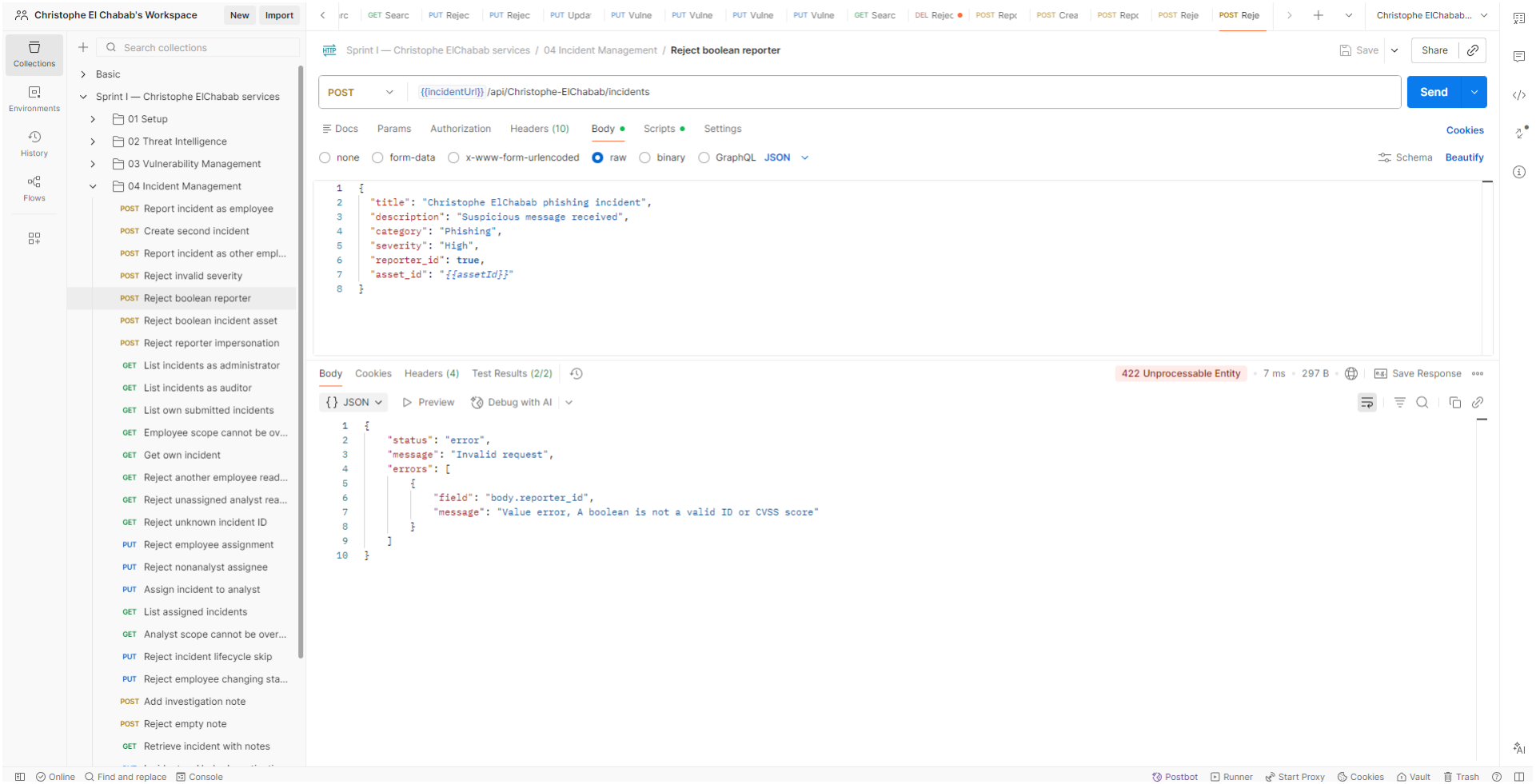

Figure 57. Christophe ElChabab - POST Reject boolean reporter - Shows that invalid input or a reference is rejected with HTTP 422.

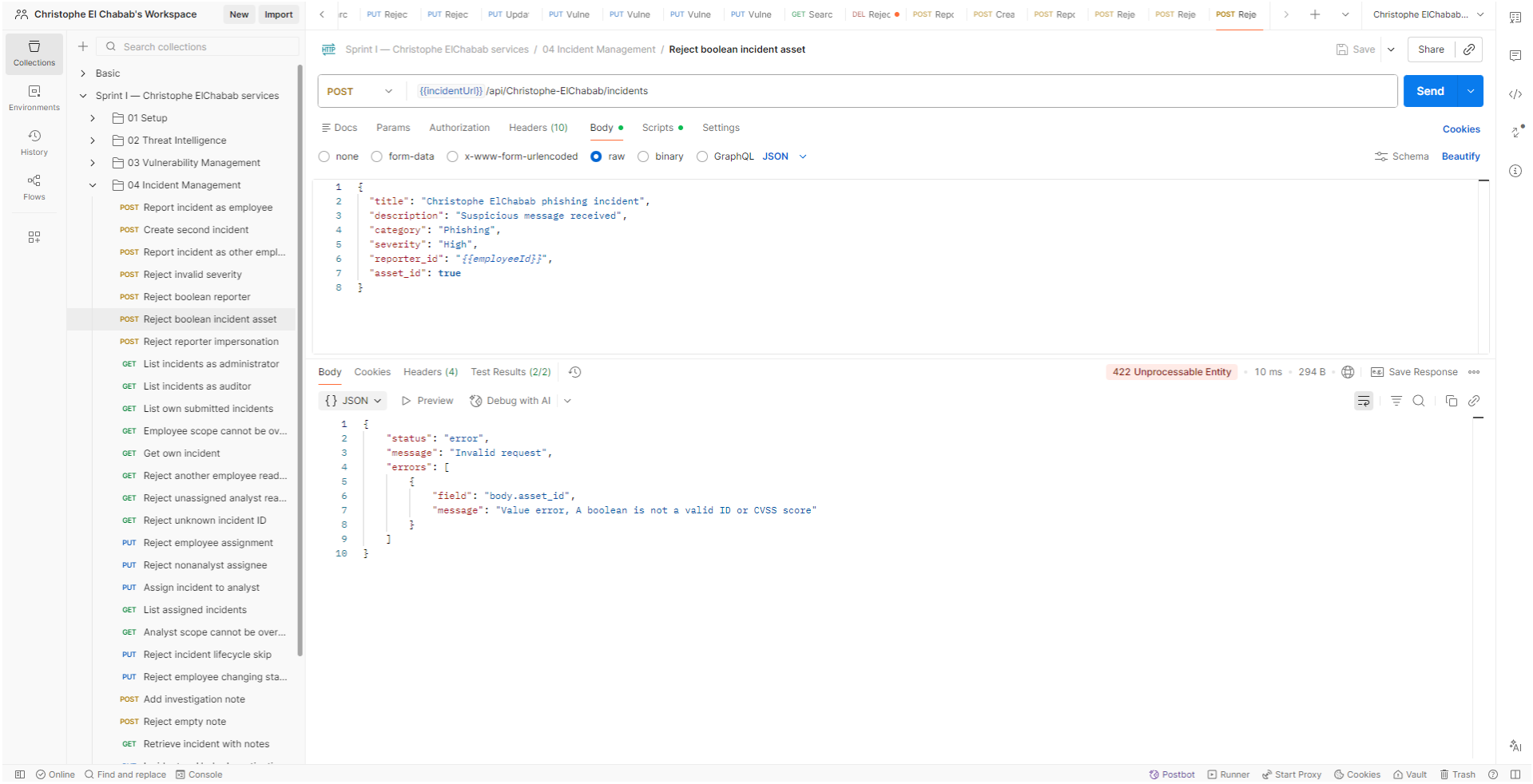

Figure 58. Christophe ElChabab - POST Reject boolean incident asset - Shows that invalid input or a reference is rejected with HTTP 422.

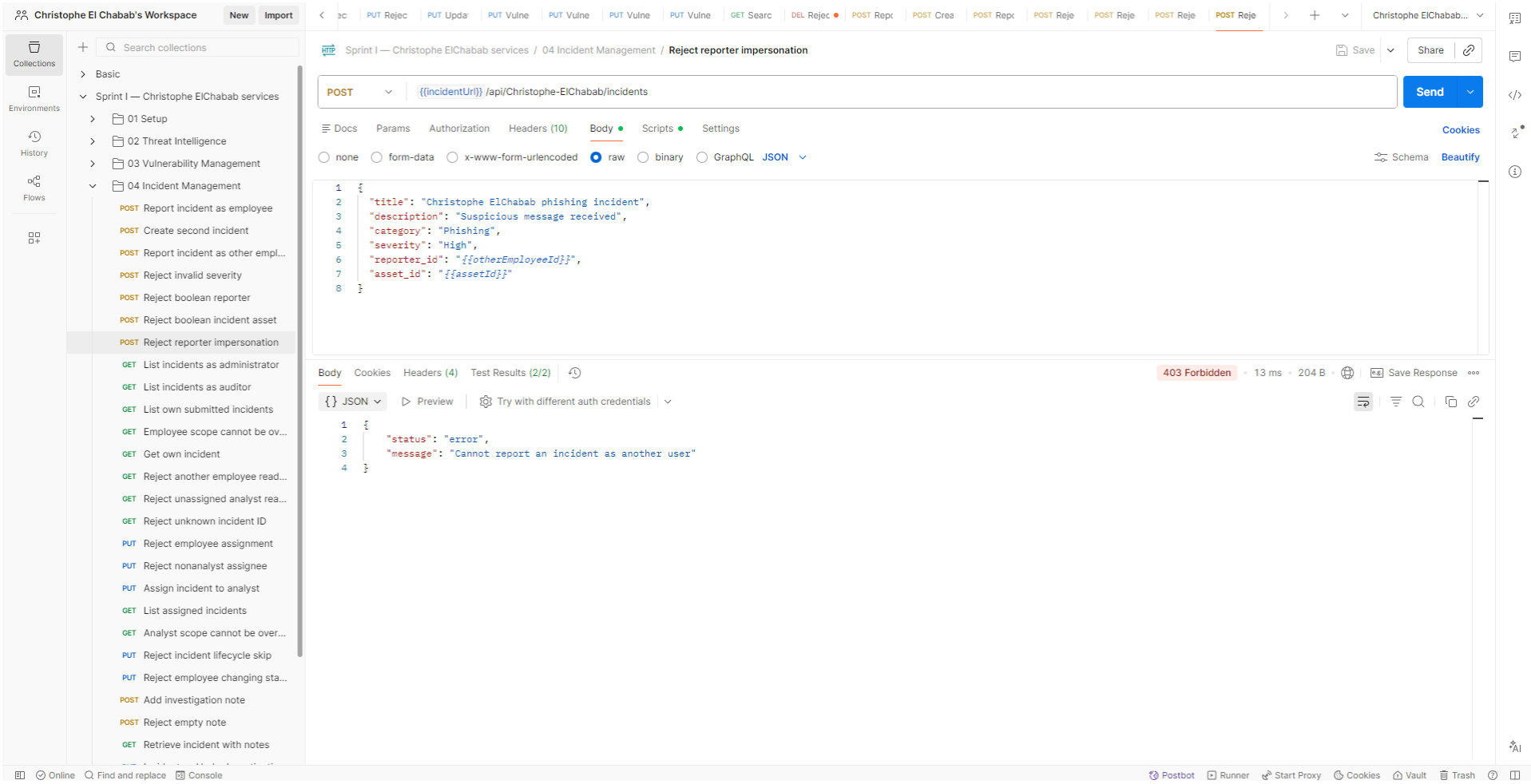

Figure 59. Christophe ElChabab - POST Reject reporter impersonation - Shows that the unauthorized operation is refused with HTTP 403.

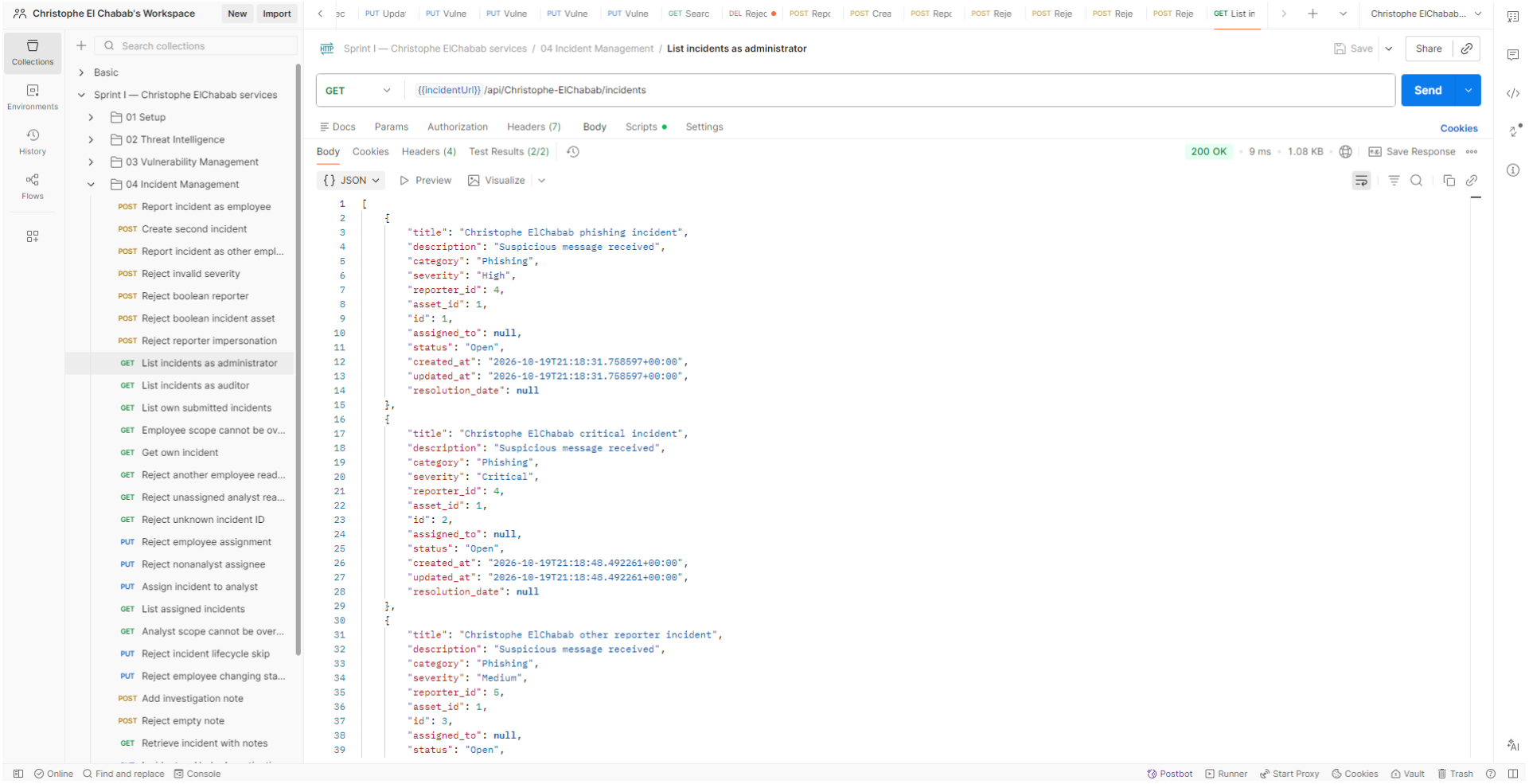

Figure 60. Christophe ElChabab - GET List incidents as administrator - Shows the named request and its successful HTTP 200 result.

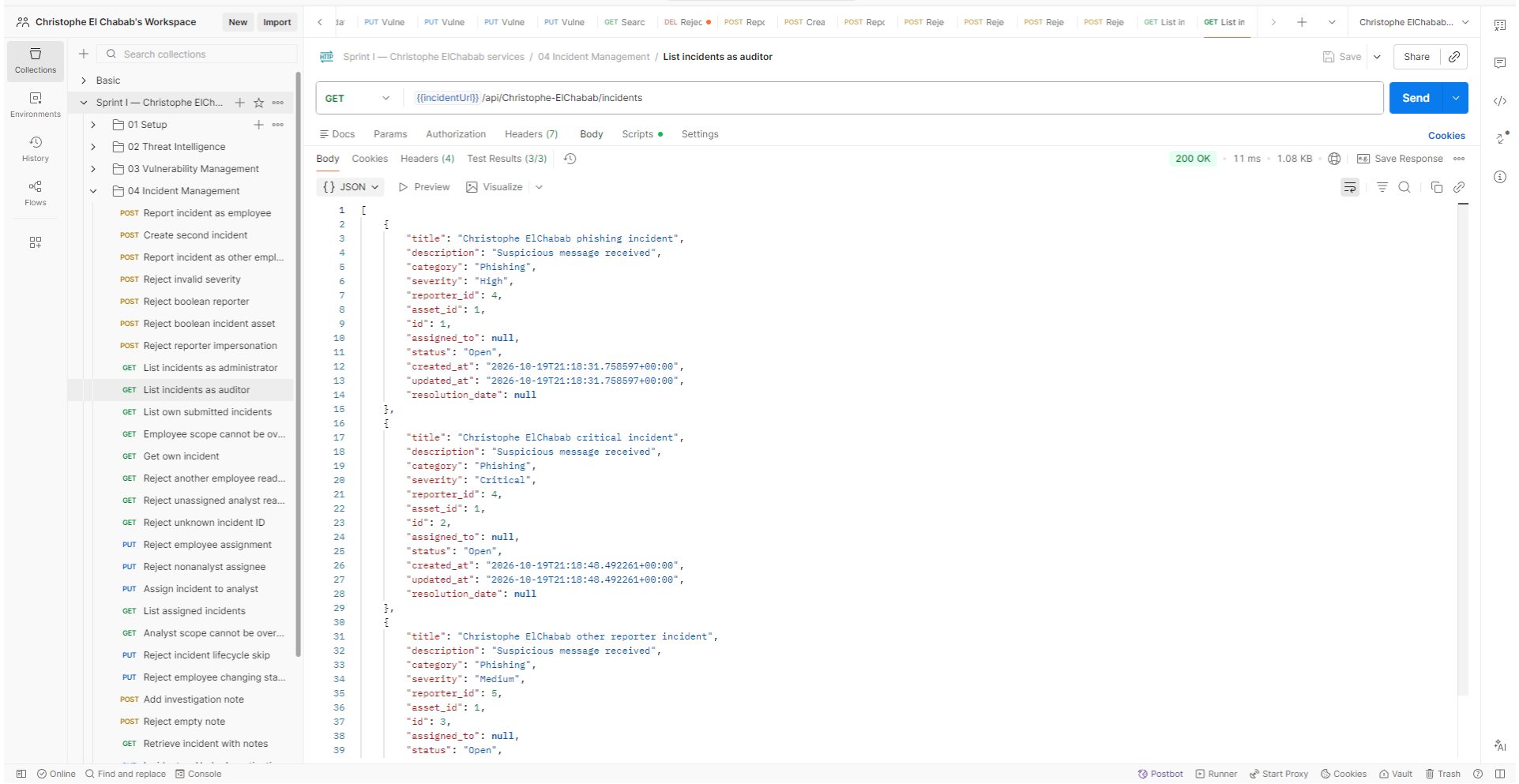

Figure 61. Christophe ElChabab - GET List incidents as auditor - Shows the named request and its successful HTTP 200 result.

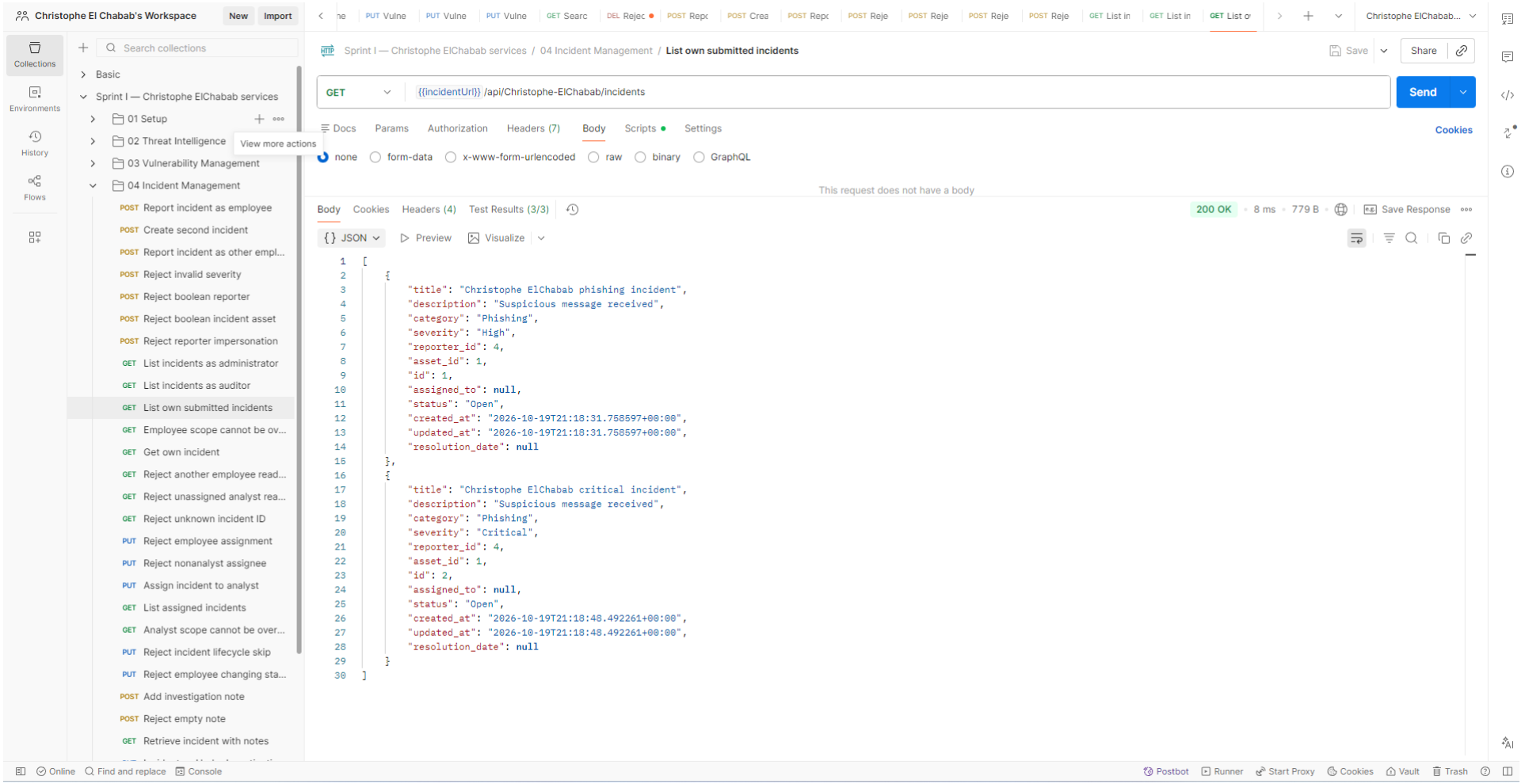

Figure 62. Christophe ElChabab - GET List own submitted incidents - Shows the named request and its successful HTTP 200 result.

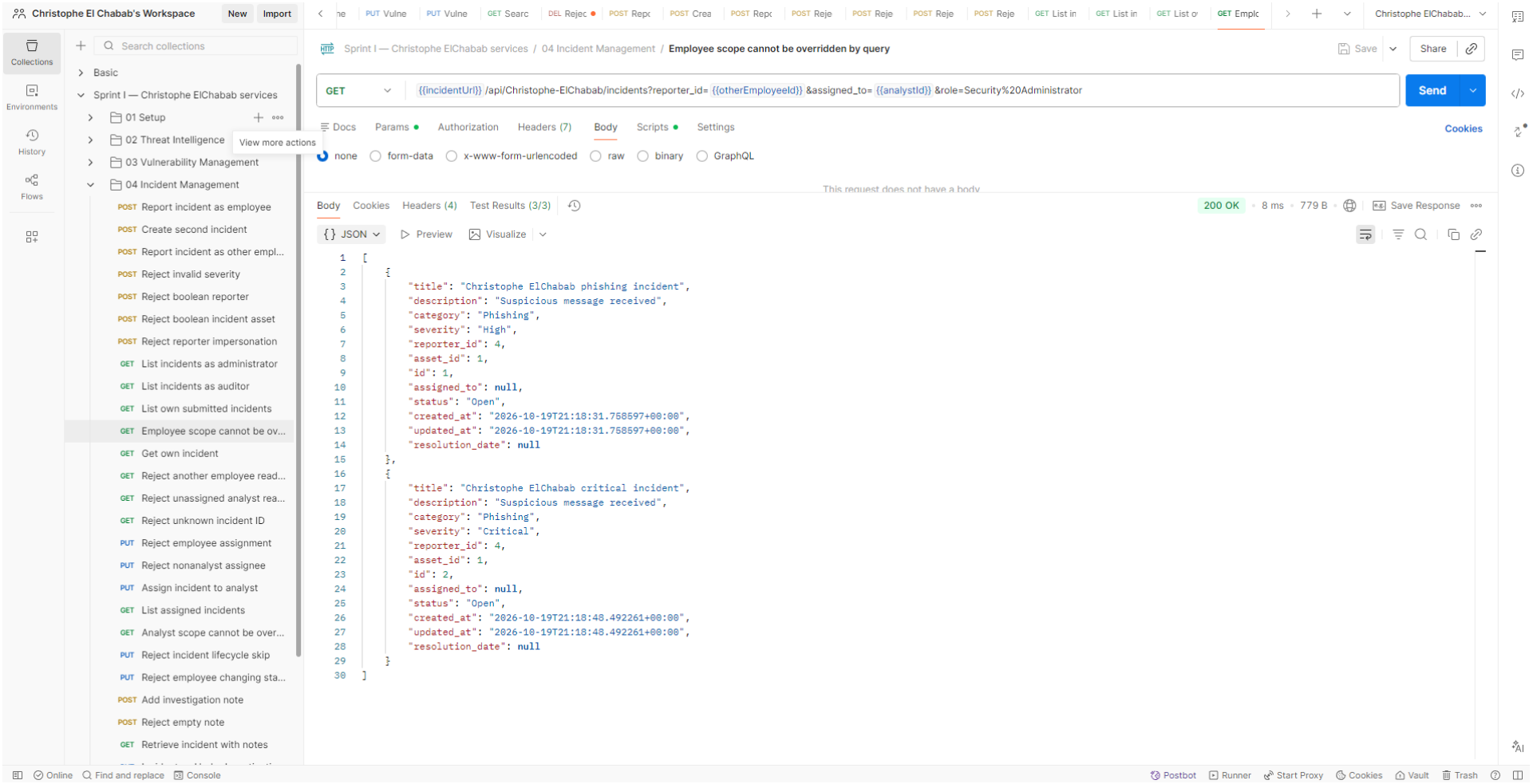

Figure 63. Christophe ElChabab - GET Employee scope cannot be overridden by query - Shows that forged query parameters do not expand the authenticated user's incident scope.

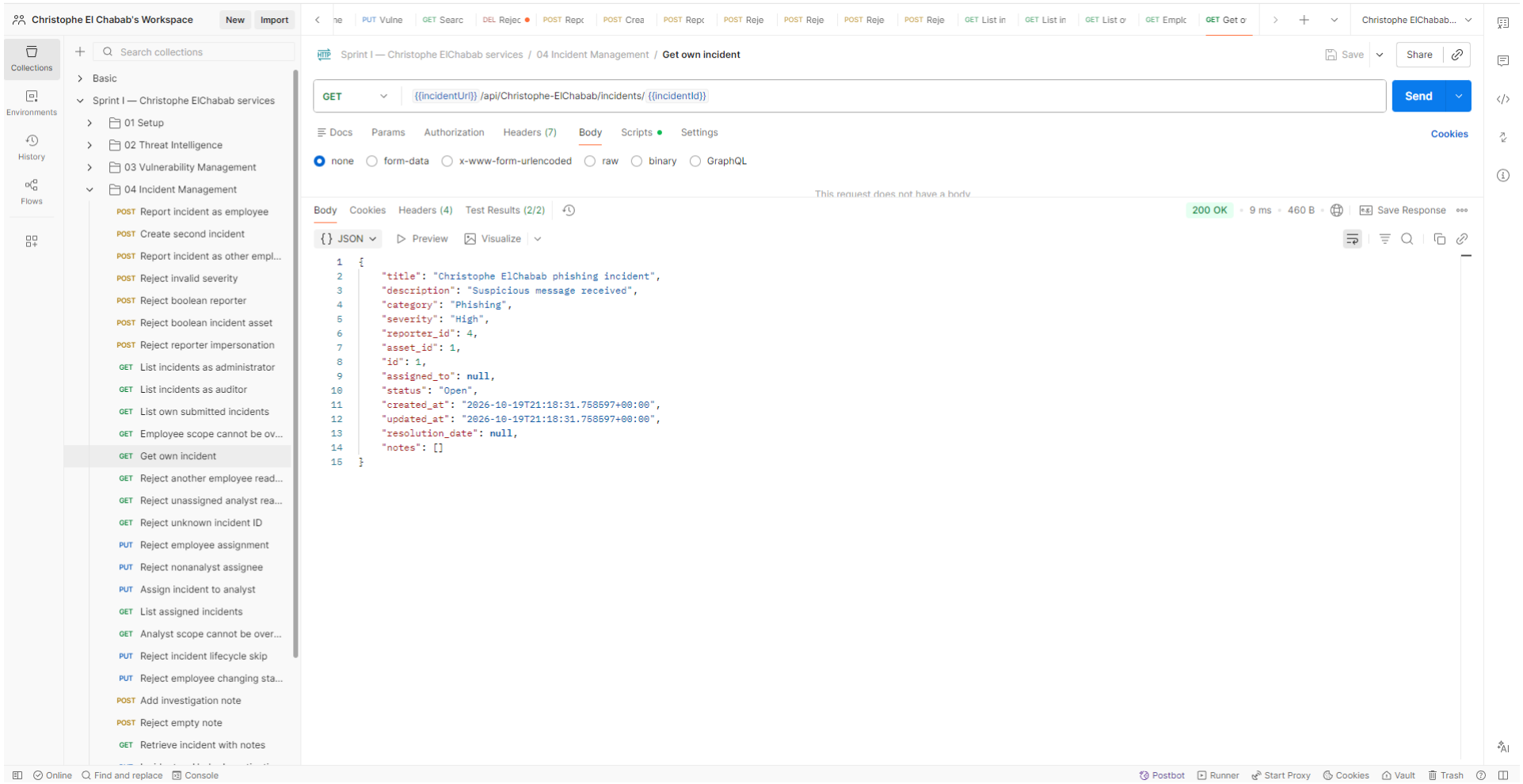

Figure 64. Christophe ElChabab - GET Get own incident - Shows the named request and its successful HTTP 200 result.

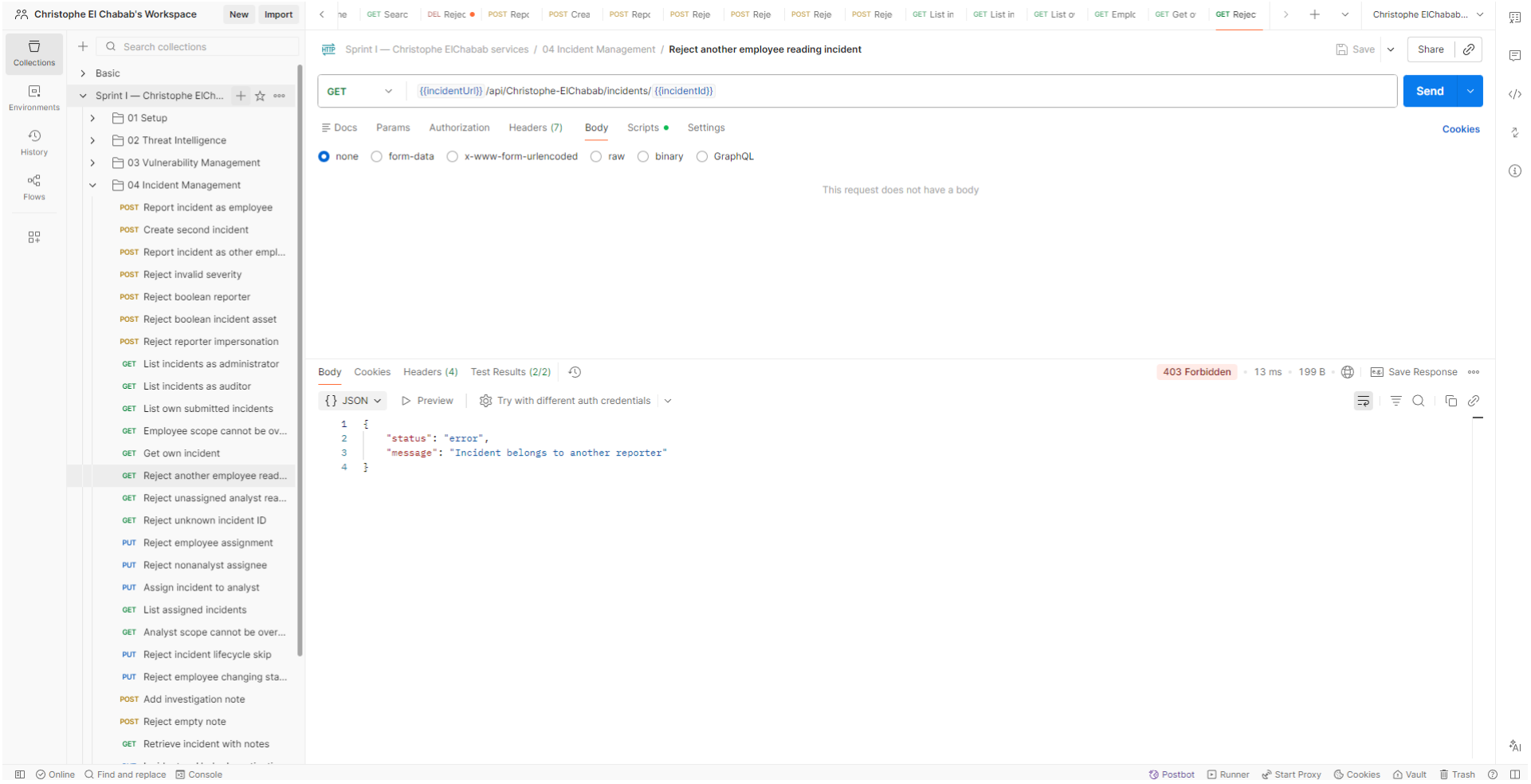

Figure 65. Christophe ElChabab - GET Reject another employee reading incident - Shows that the unauthorized operation is refused with HTTP 403.

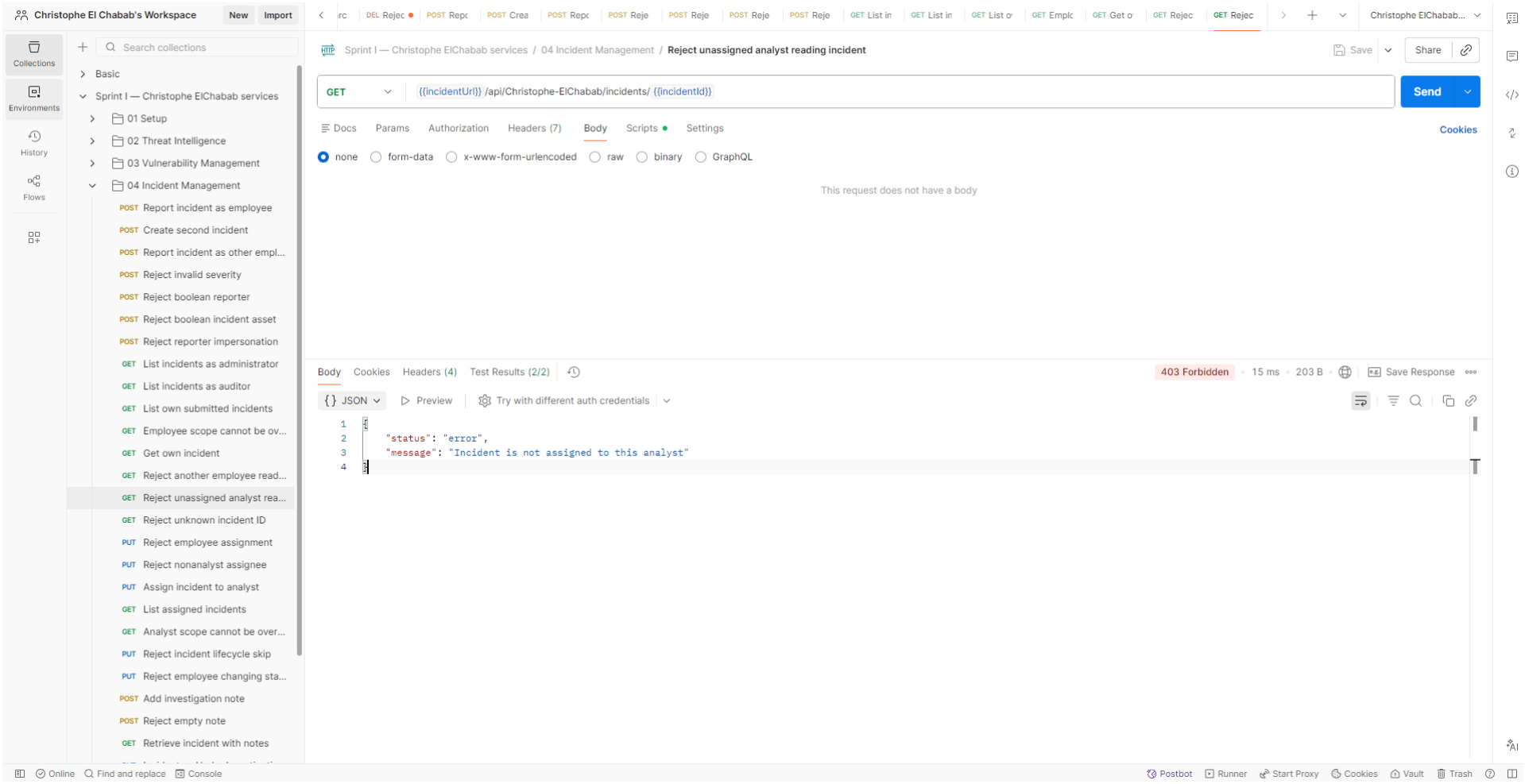

Figure 66. Christophe ElChabab - GET Reject unassigned analyst reading incident - Shows that the unauthorized operation is refused with HTTP 403.

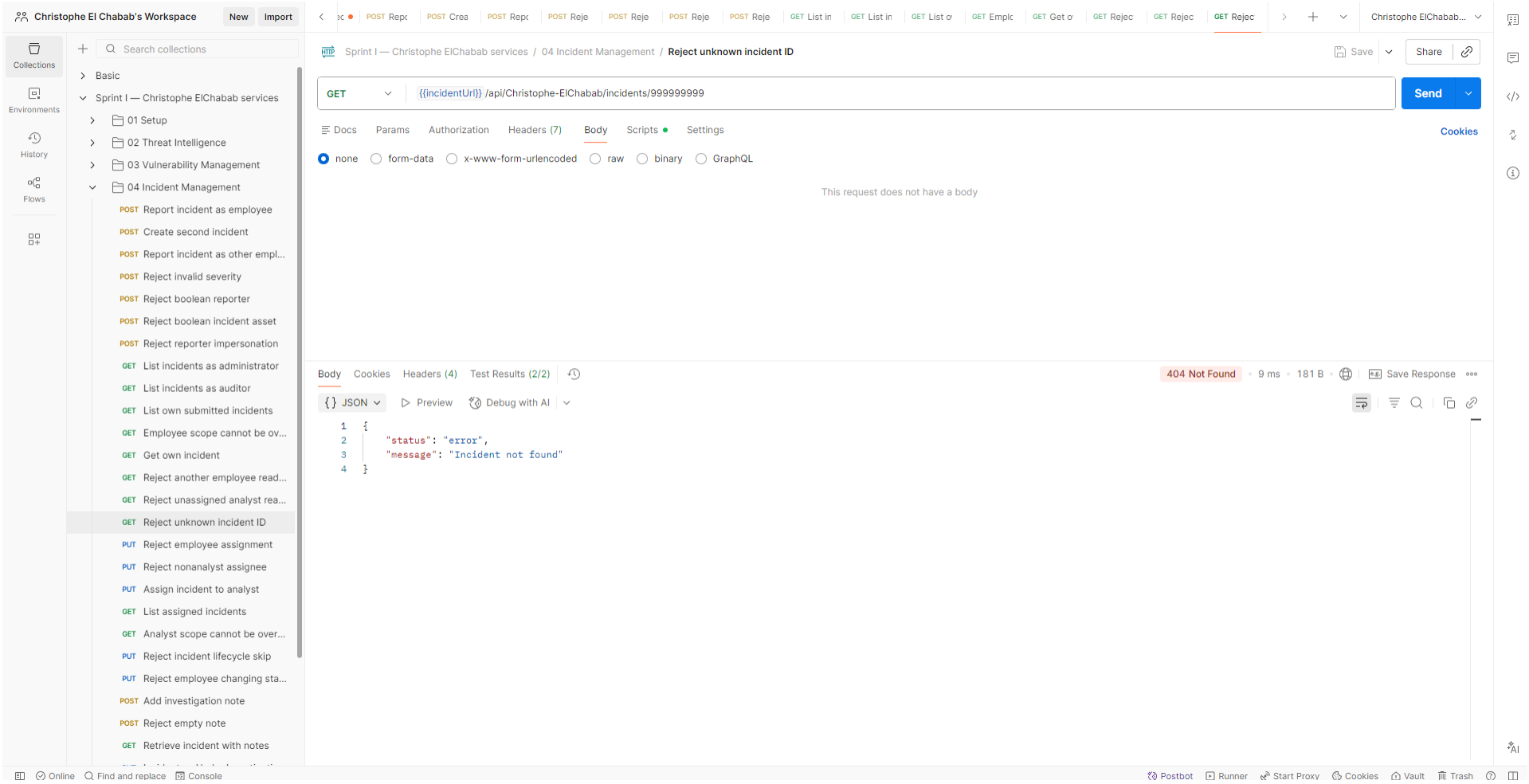

Figure 67. Christophe ElChabab - GET Reject unknown incident ID - Shows that an unknown record returns HTTP 404.

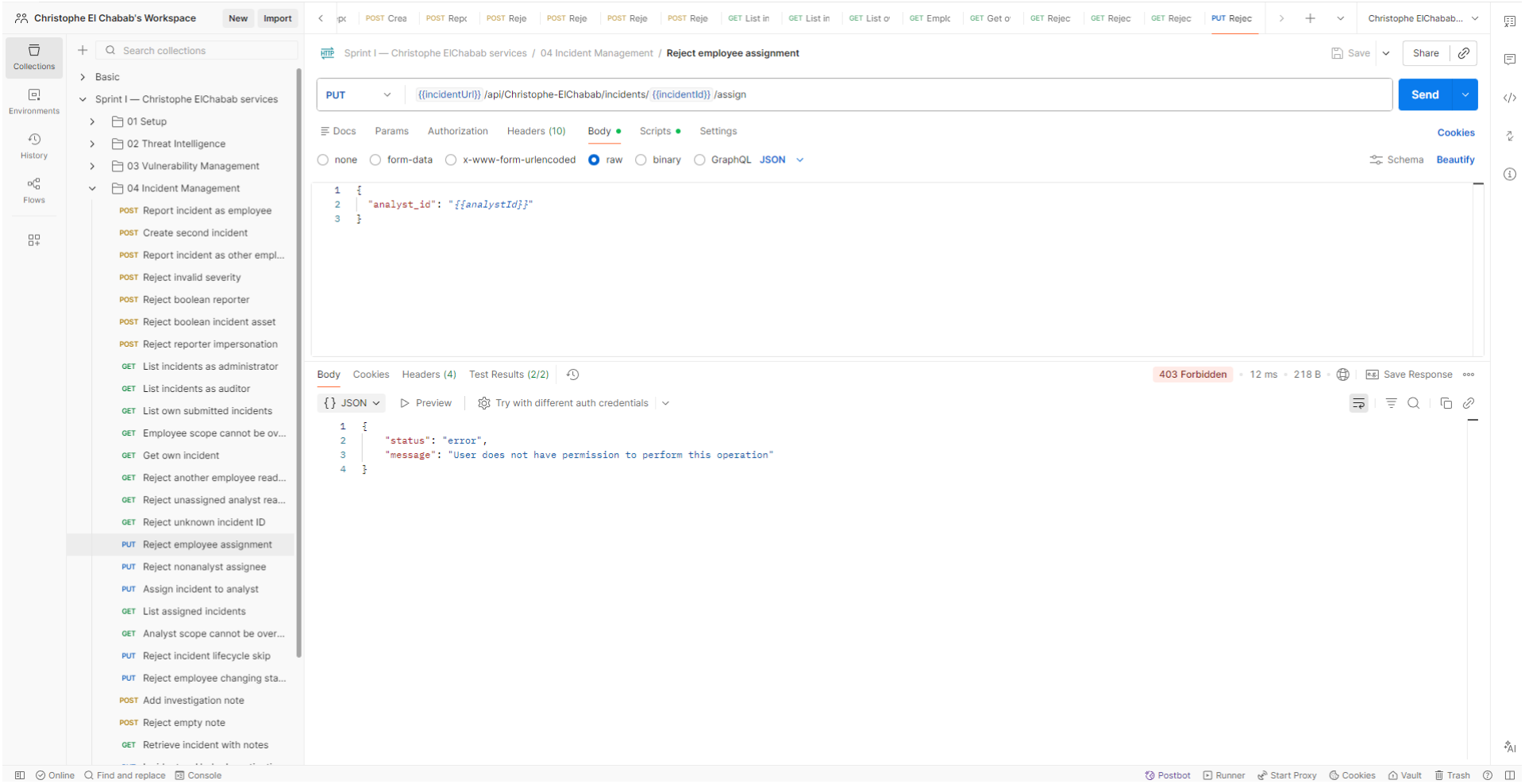

Figure 68. Christophe ElChabab - PUT Reject employee assignment - Shows that the unauthorized operation is refused with HTTP 403.

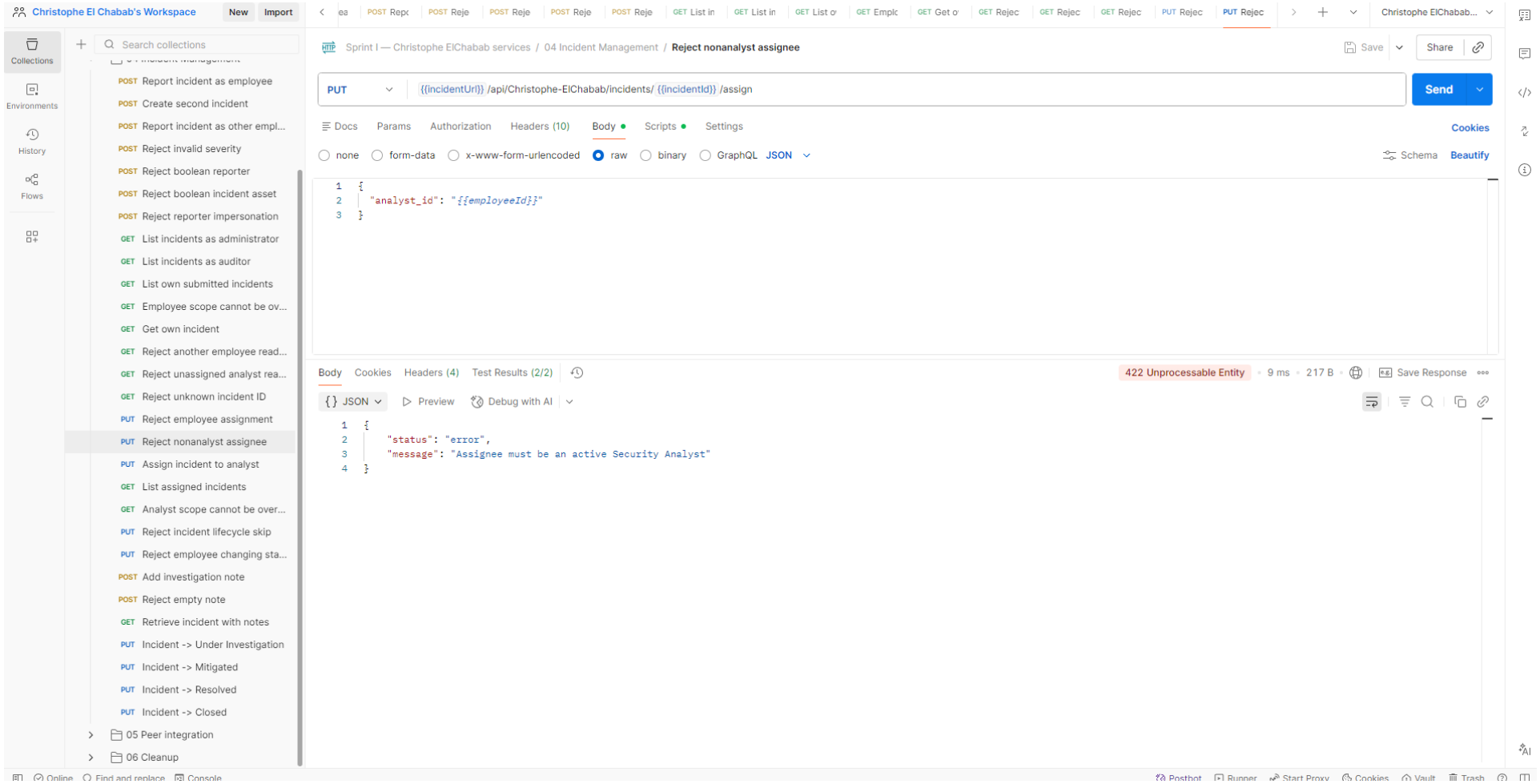

Figure 69. Christophe ElChabab - PUT Reject nonanalyst assignee - Shows that invalid input or a reference is rejected with HTTP 422.

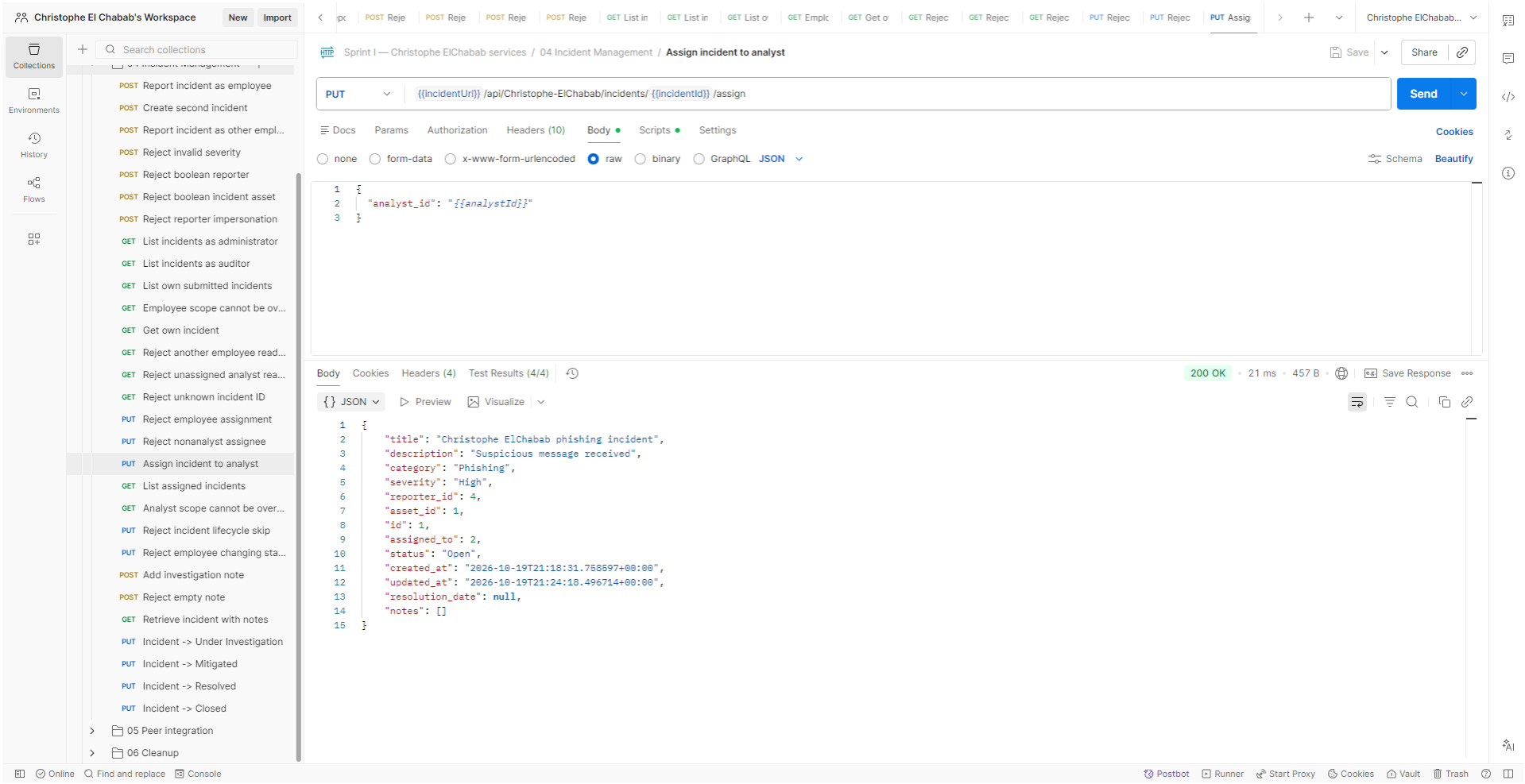

Figure 70. Christophe ElChabab - PUT Assign incident to analyst - Shows the named request and its successful HTTP 200 result.

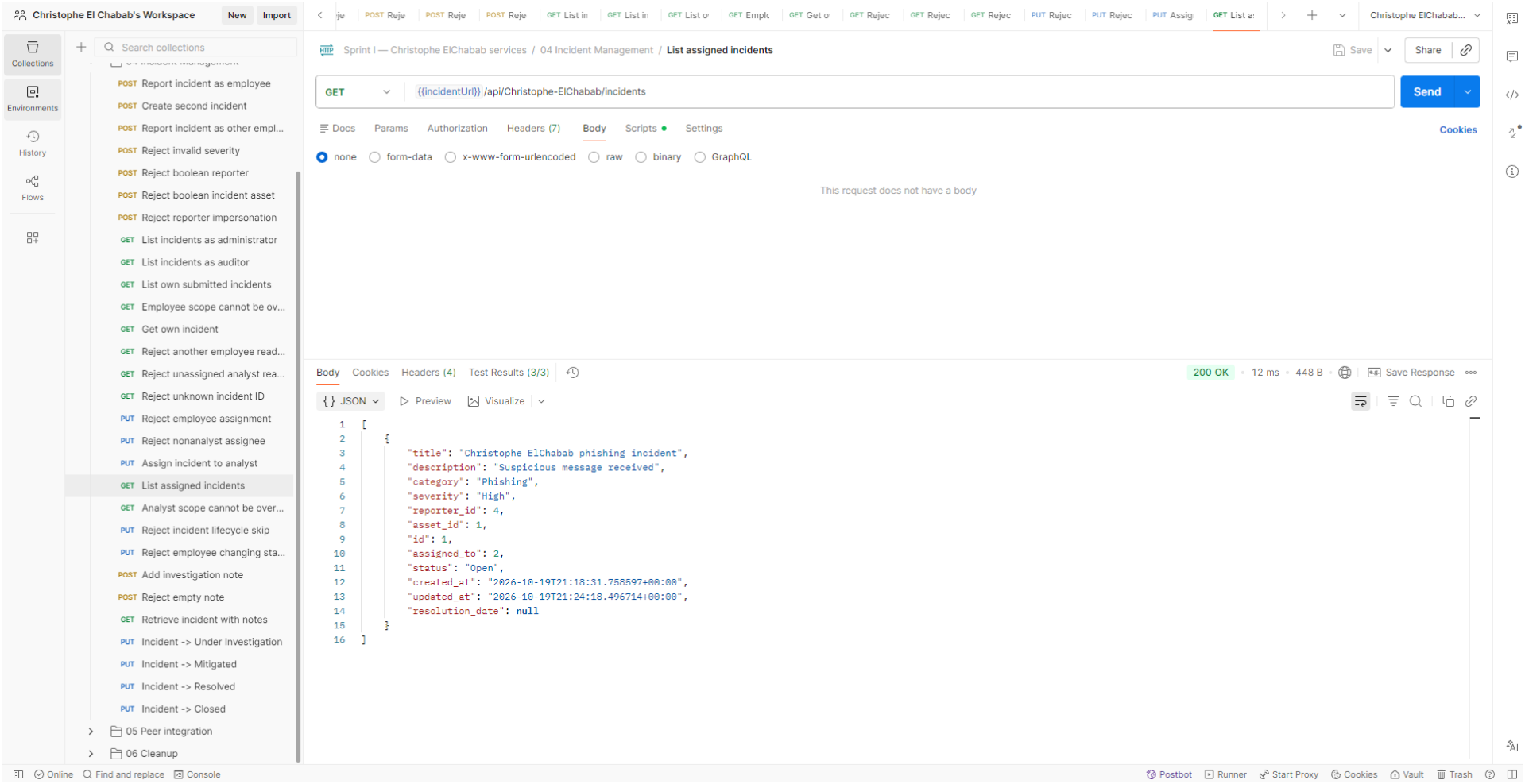

Figure 71. Christophe ElChabab - GET List assigned incidents - Shows the named request and its successful HTTP 200 result.

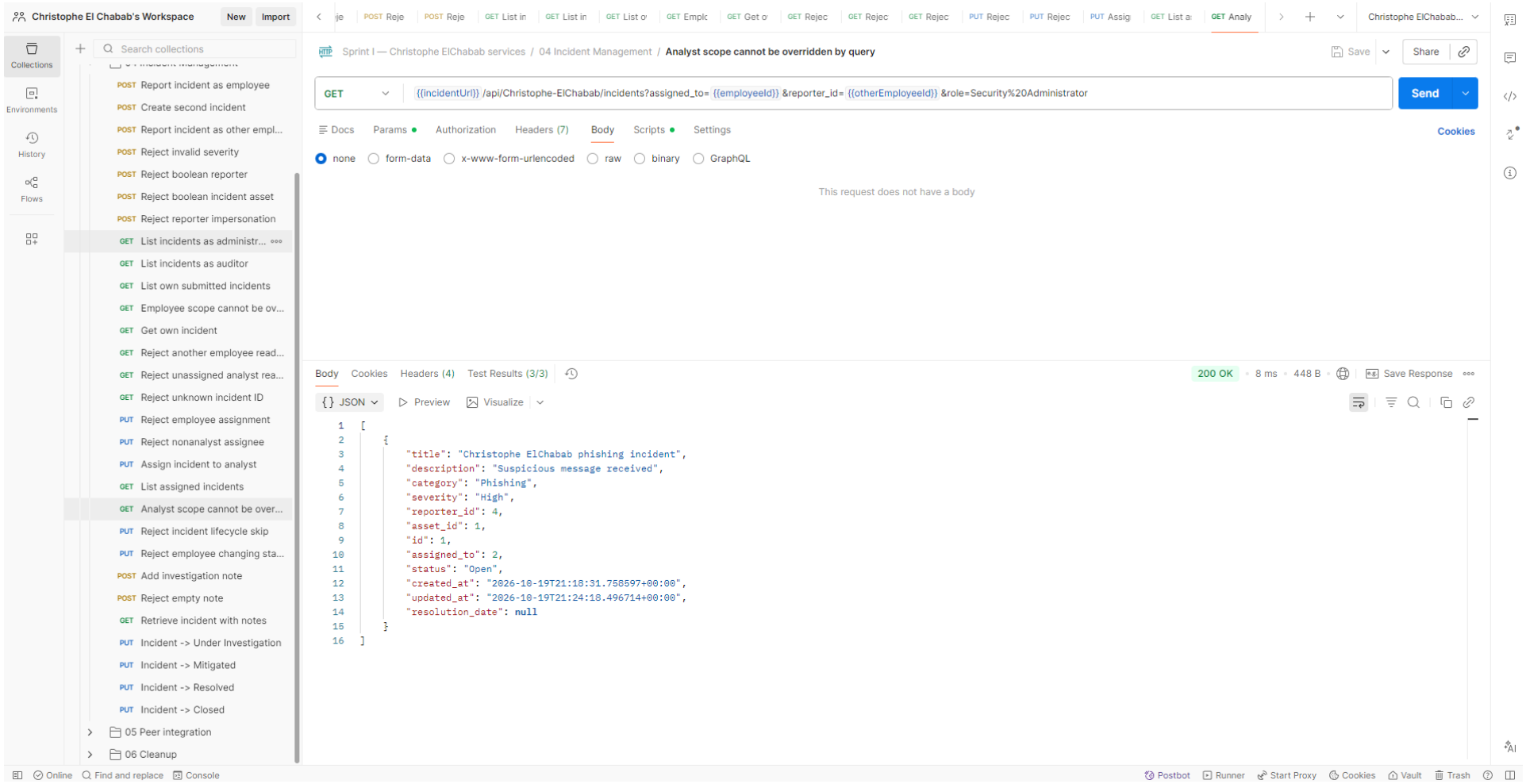

Figure 72. Christophe ElChabab - GET Analyst scope cannot be overridden by query - Shows that forged query parameters do not expand the authenticated user's incident scope.

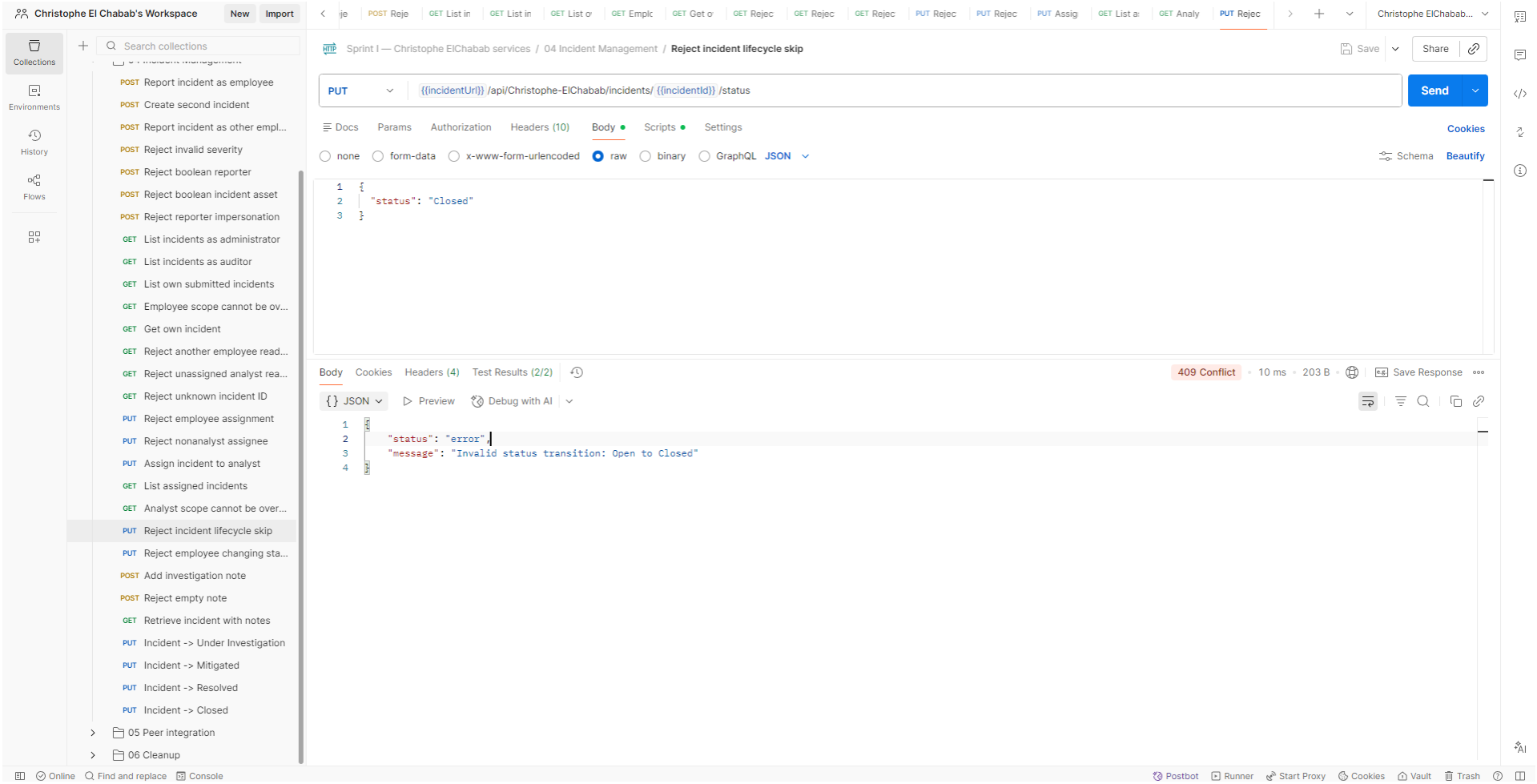

Figure 73. Christophe ElChabab - PUT Reject incident lifecycle skip - Shows that a duplicate indicator or invalid lifecycle change is rejected with HTTP 409.

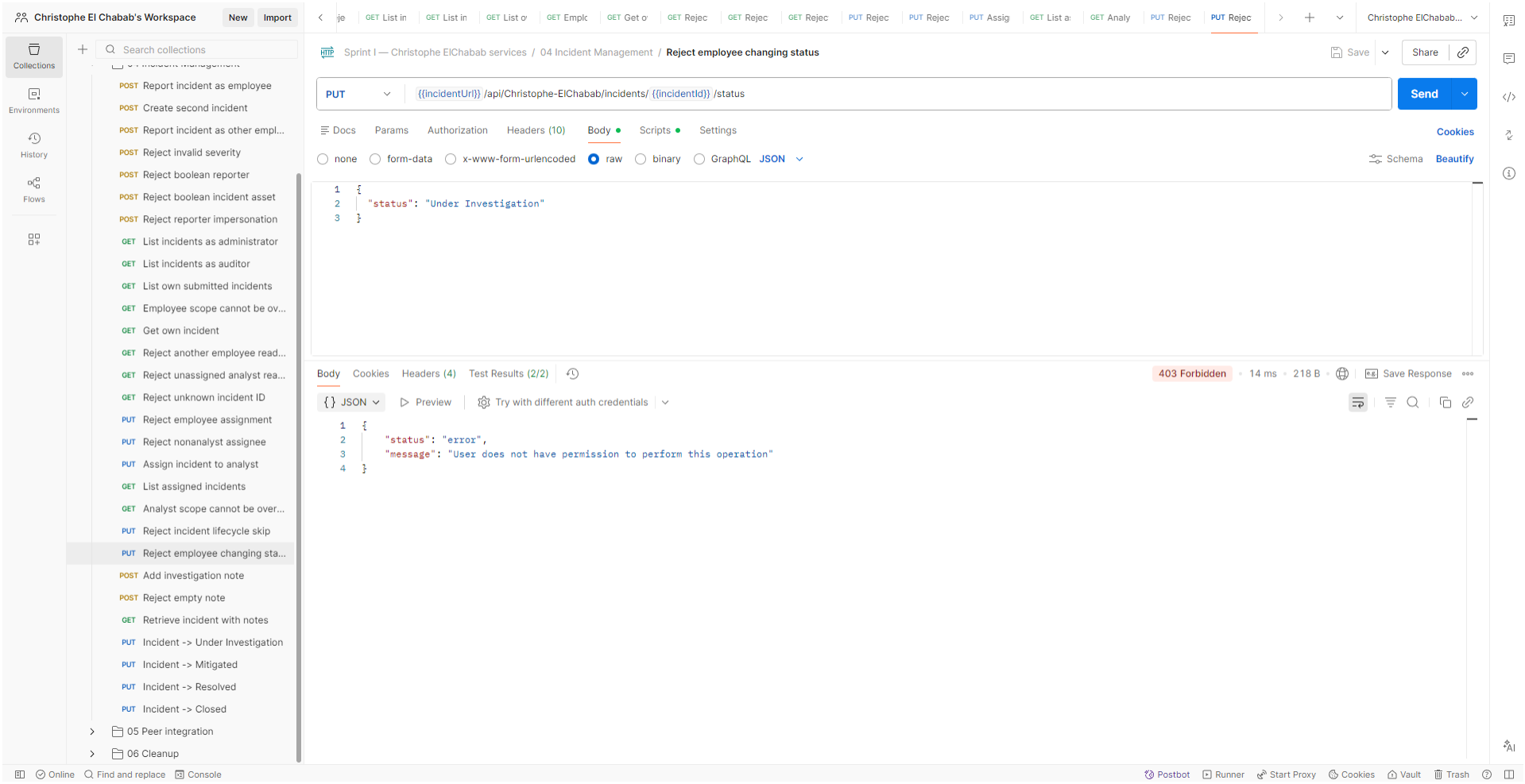

Figure 74. Christophe ElChabab - PUT Reject employee changing status - Shows that the unauthorized operation is refused with HTTP 403.

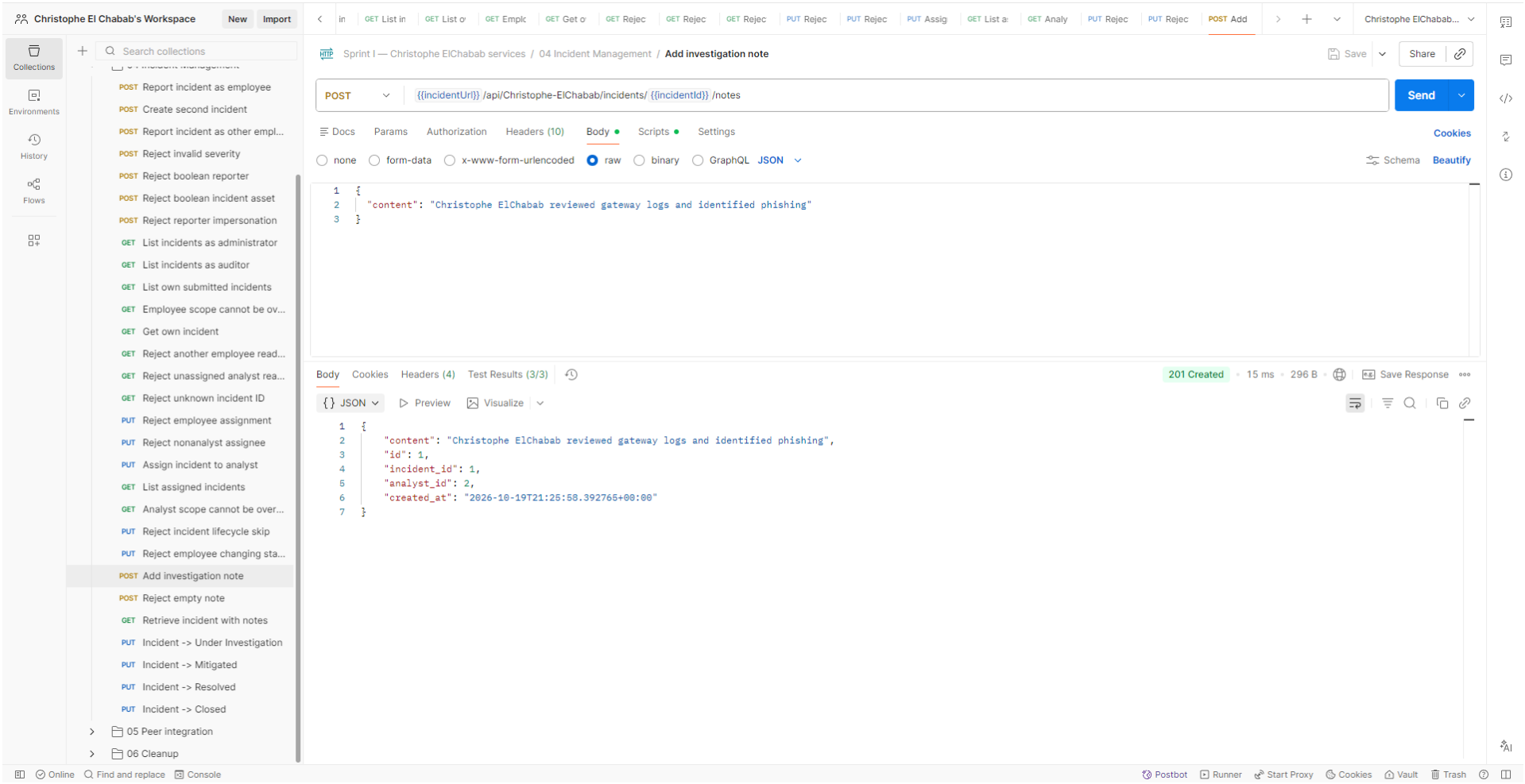

Figure 75. Christophe ElChabab - POST Add investigation note - Shows the submitted request and the newly created record returned with HTTP 201.

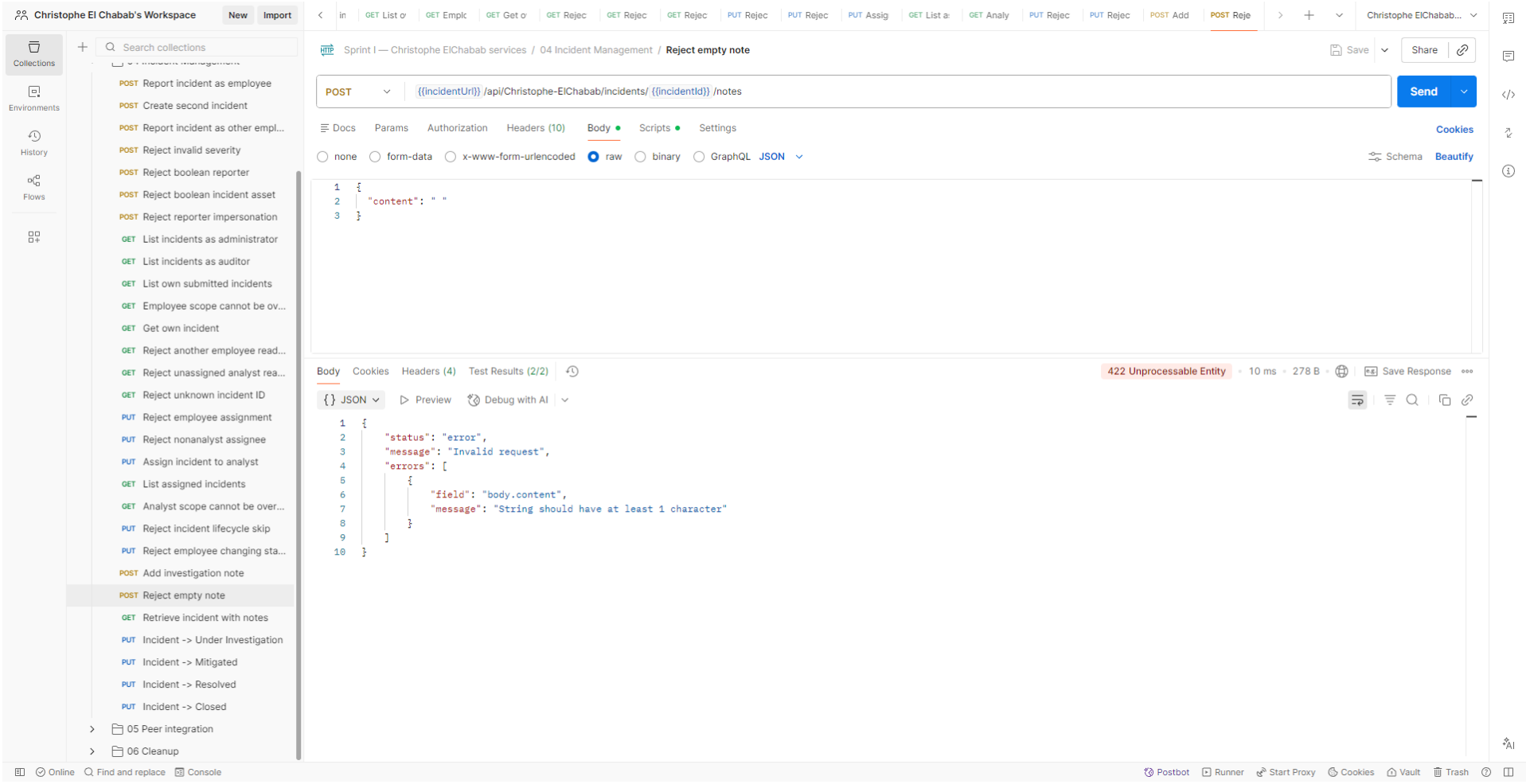

Figure 76. Christophe ElChabab - POST Reject empty note - Shows that invalid input or a reference is rejected with HTTP 422.

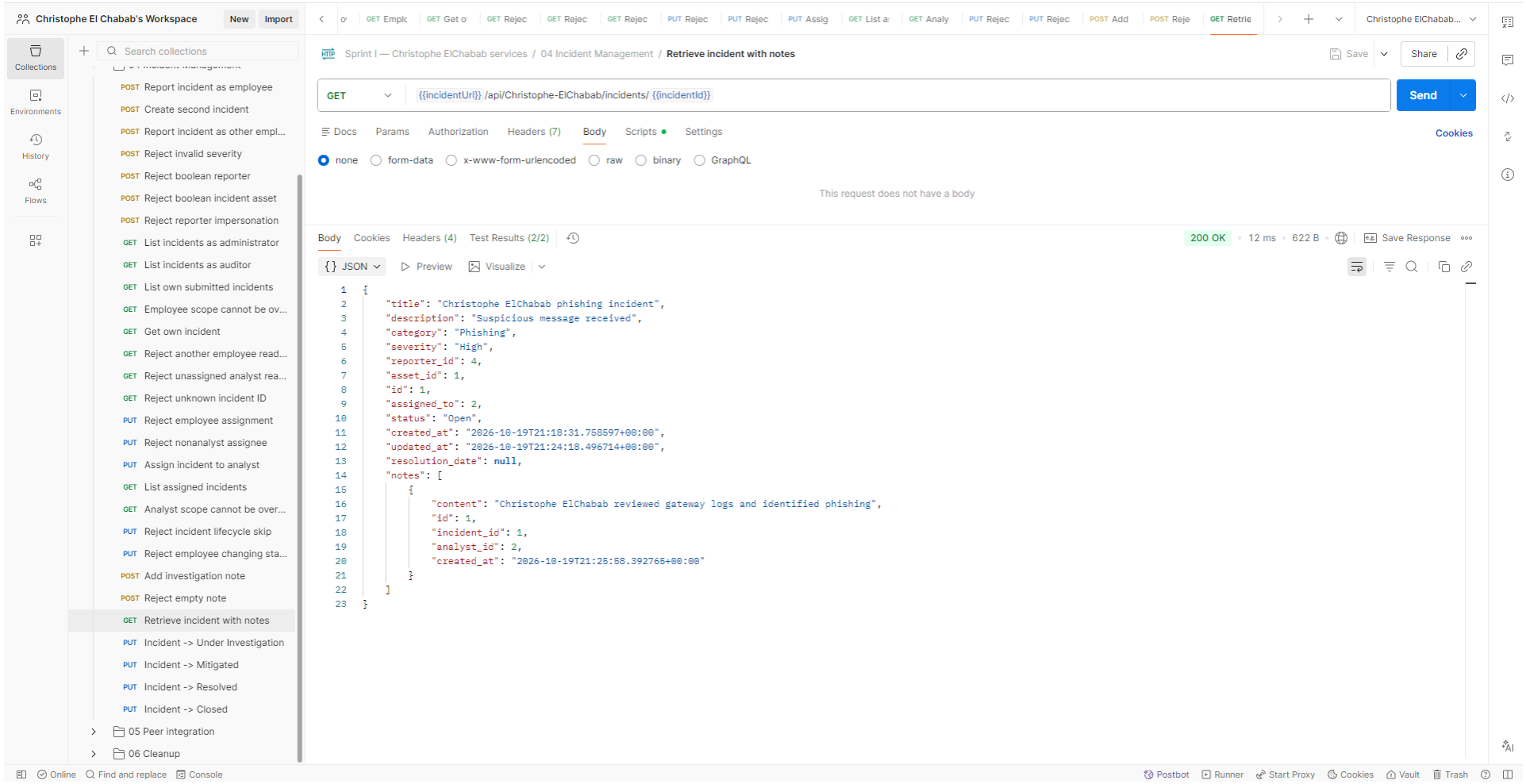

Figure 77. Christophe ElChabab - GET Retrieve incident with notes - Shows the named request and its successful HTTP 200 result.

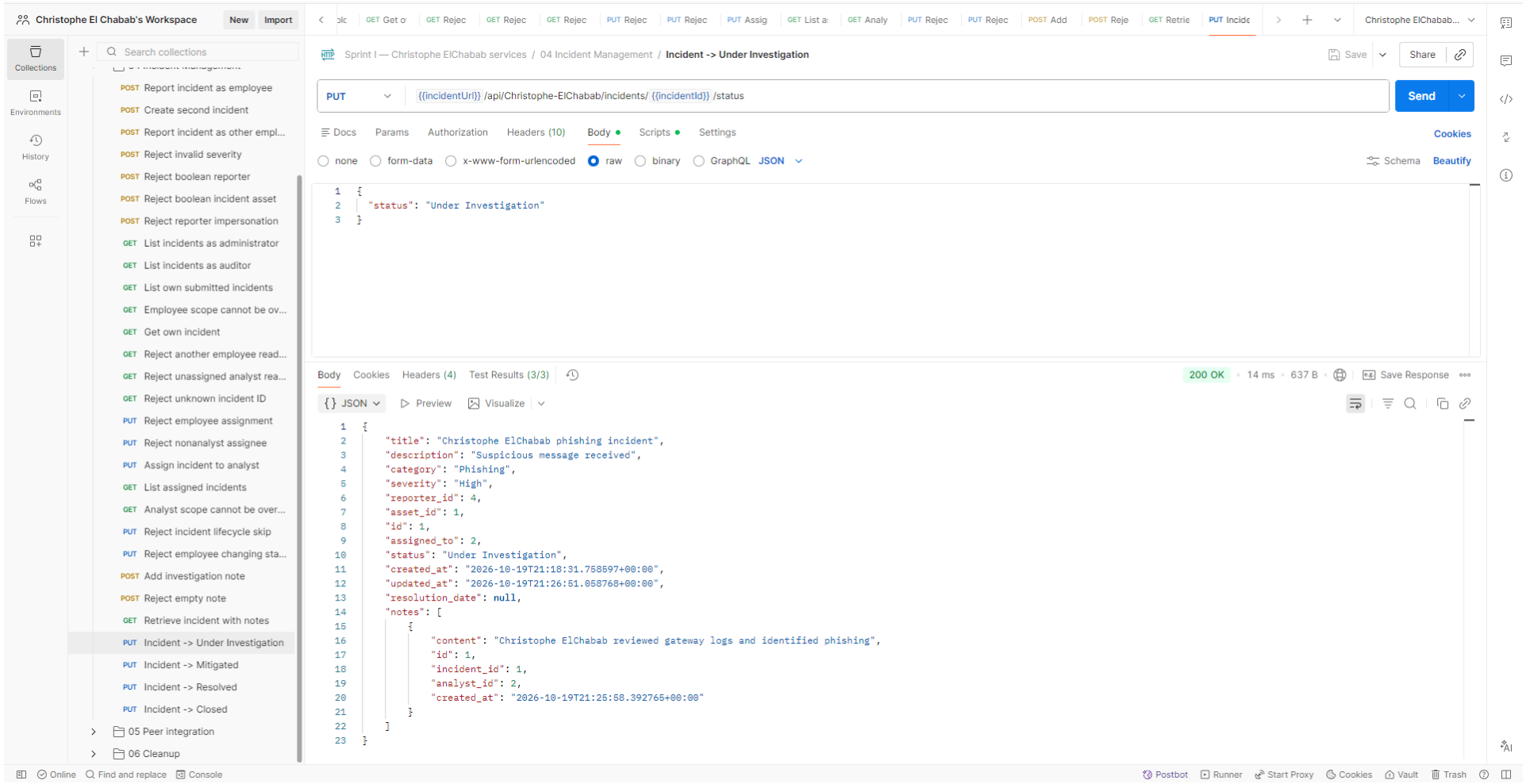

Figure 78. Christophe ElChabab - PUT Incident Under Investigation - Shows the named request and its successful HTTP 200 result.

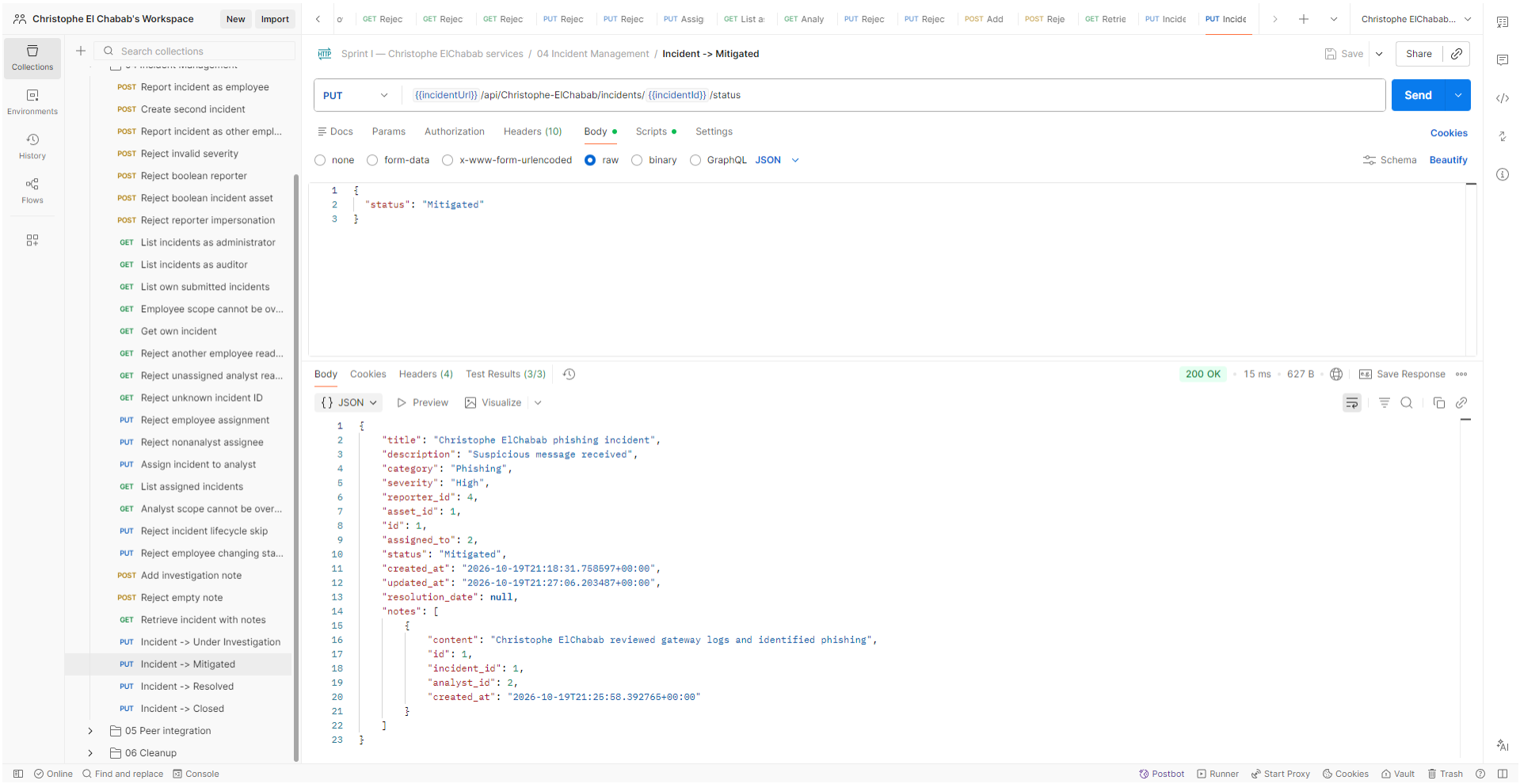

Figure 79. Christophe ElChabab - PUT Incident Mitigated - Shows the named request and its successful HTTP 200 result.

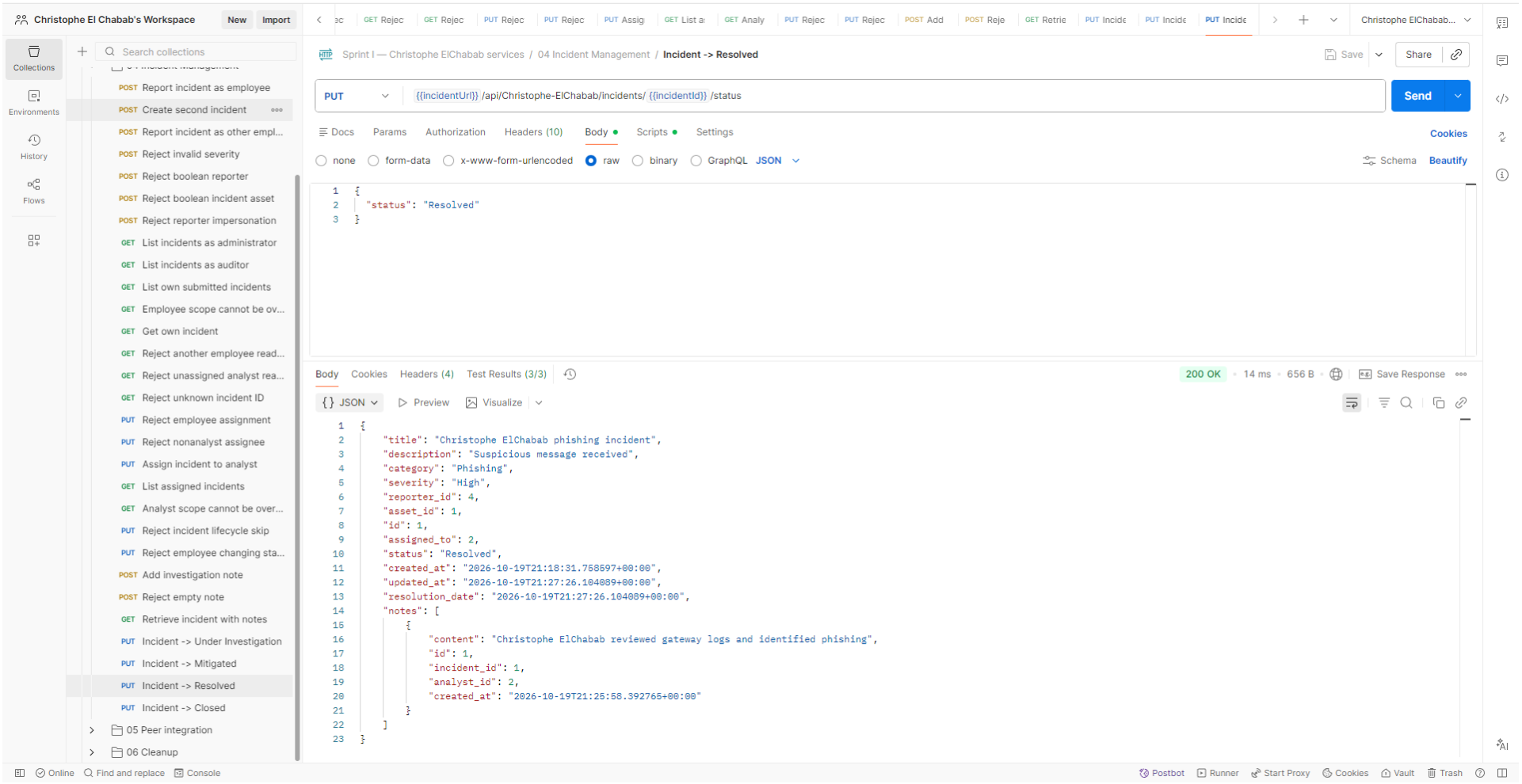

Figure 80. Christophe ElChabab - PUT Incident Resolved - Shows the named request and its successful HTTP 200 result.

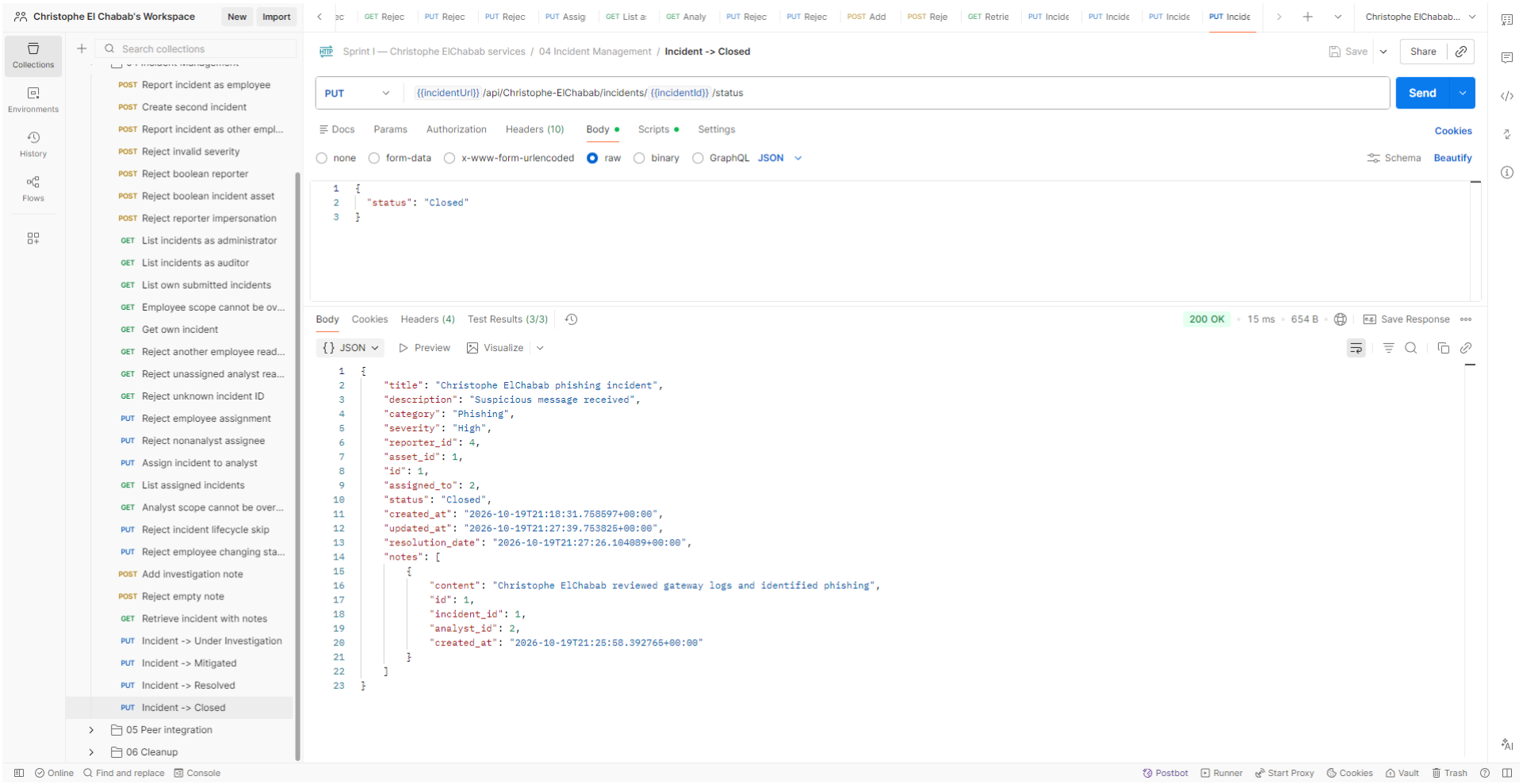

Figure 81. Christophe ElChabab - PUT Incident Closed - Shows the named request and its successful HTTP 200 result.

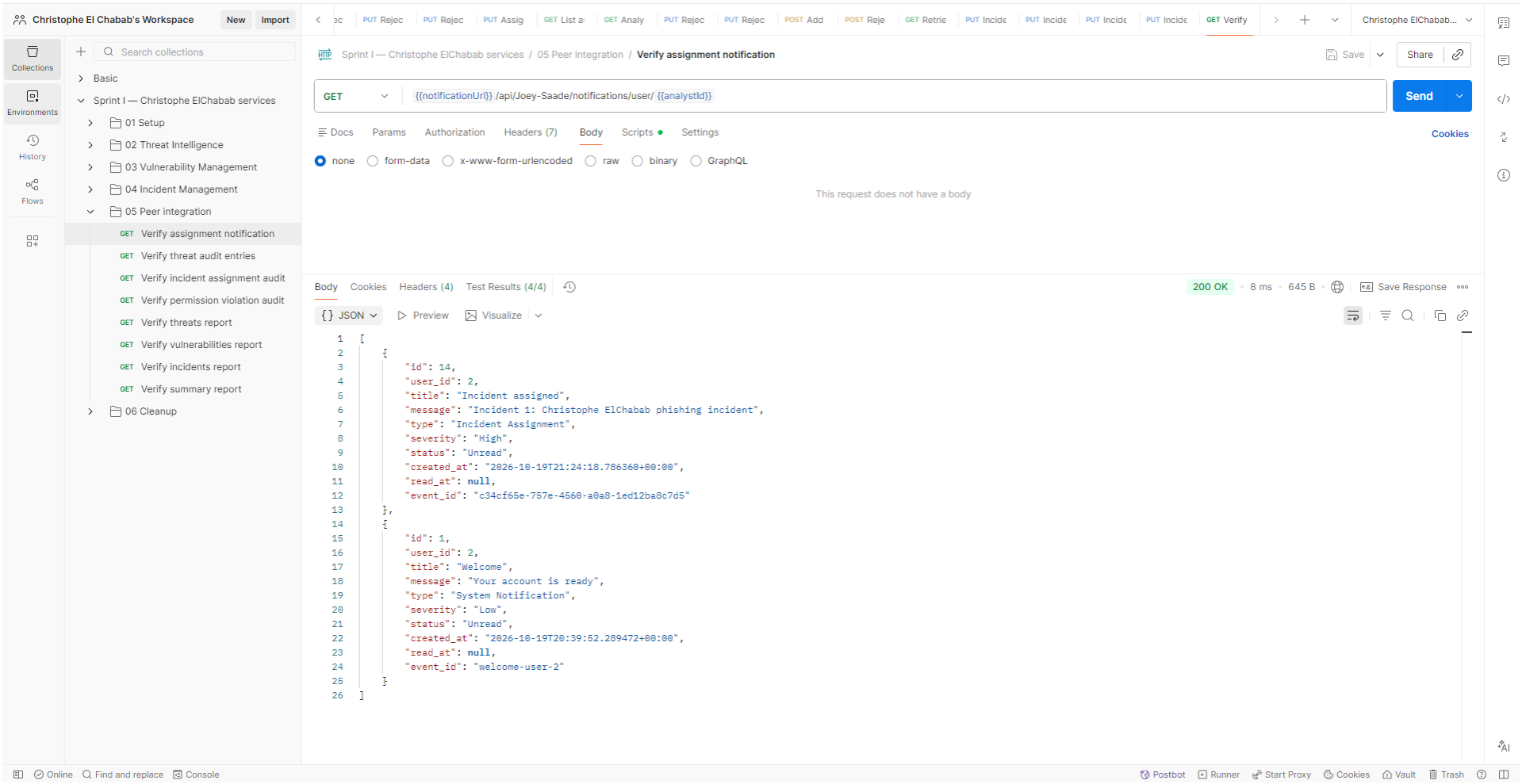

Figure 82. Christophe ElChabab - GET Verify assignment notification - Shows the named request and its successful HTTP 200 result.

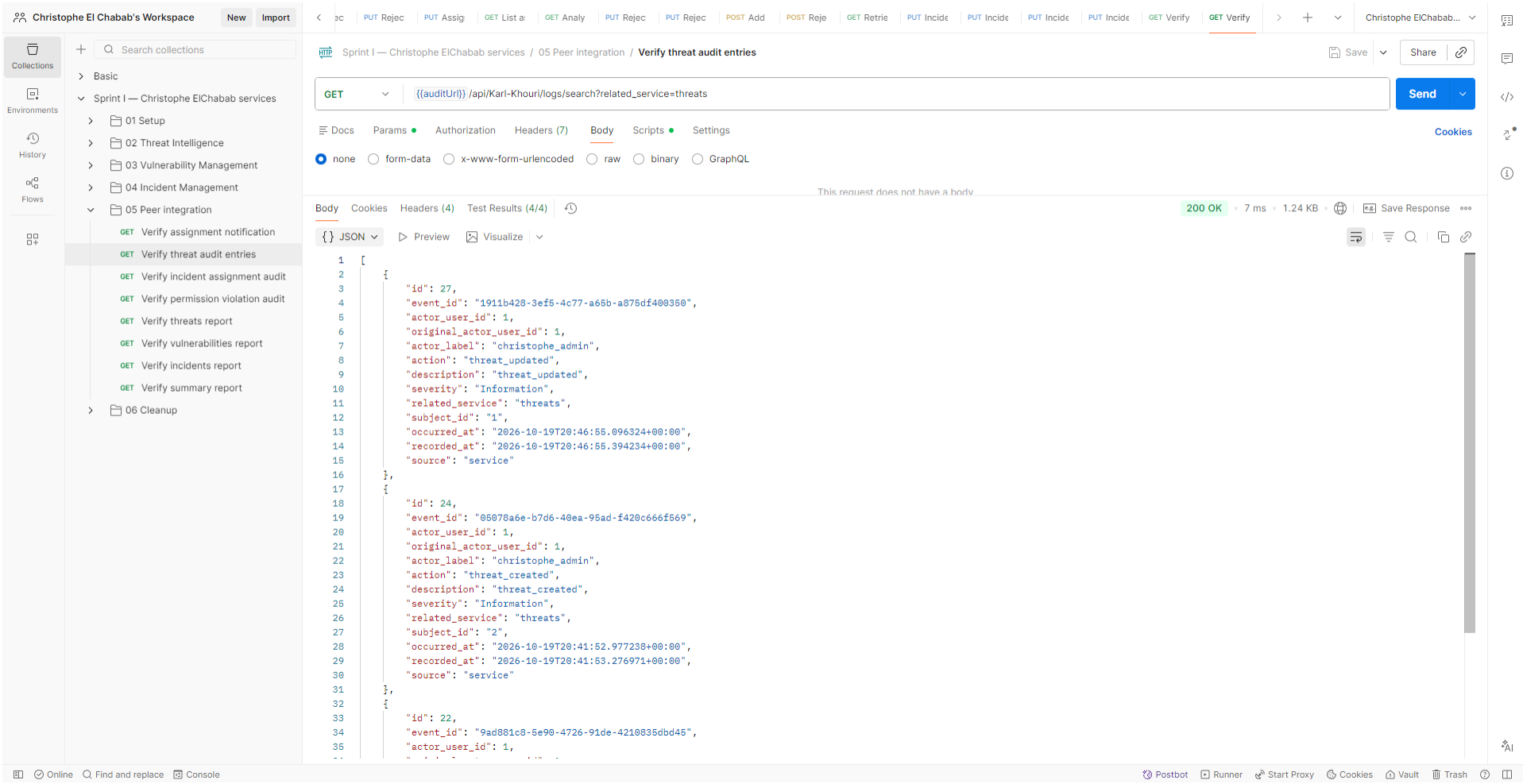

Figure 83. Christophe ElChabab - GET Verify threat audit entries - Shows the named request and its successful HTTP 200 result.

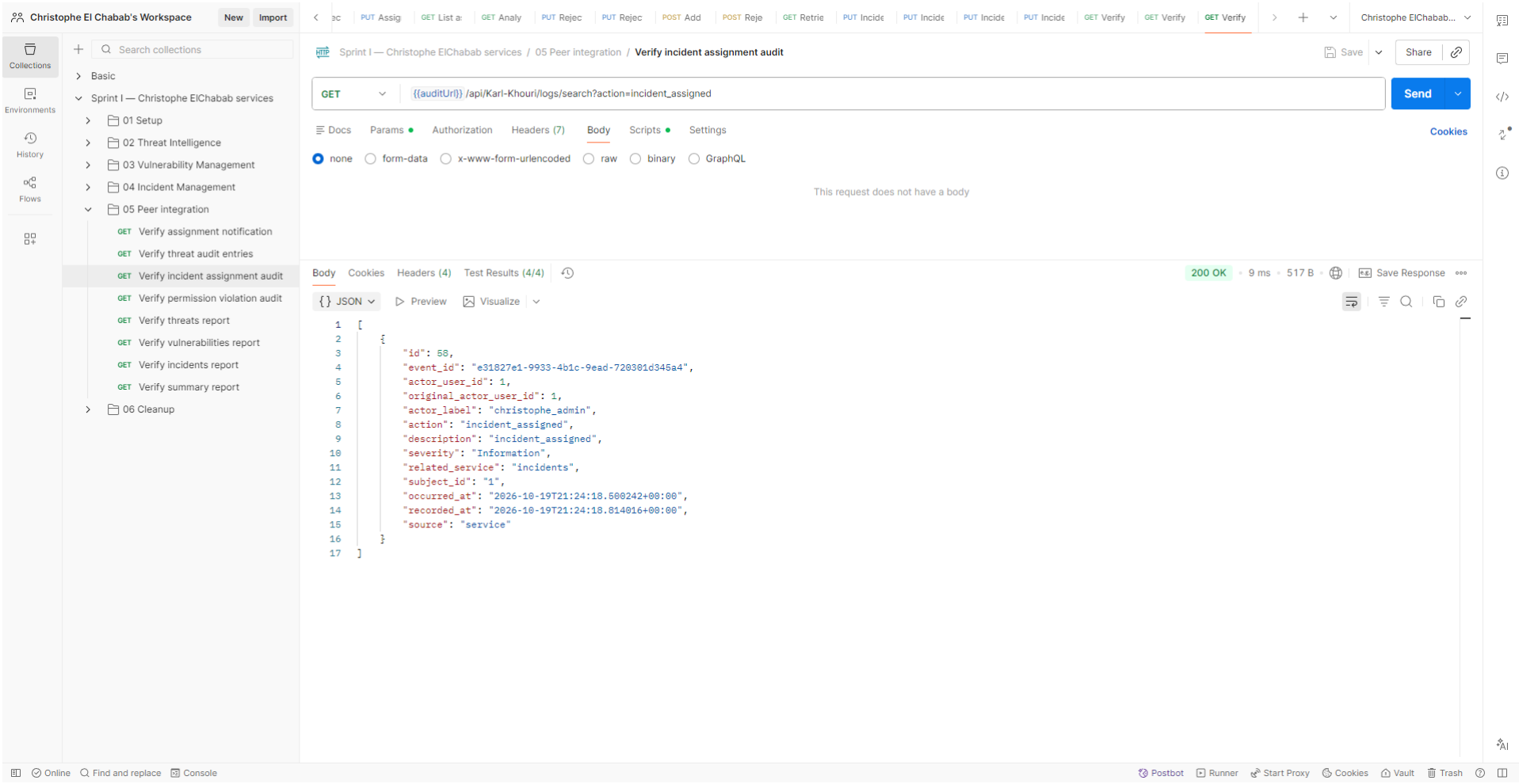

Figure 84. Christophe ElChabab - GET Verify incident assignment audit - Shows the named request and its successful HTTP 200 result.

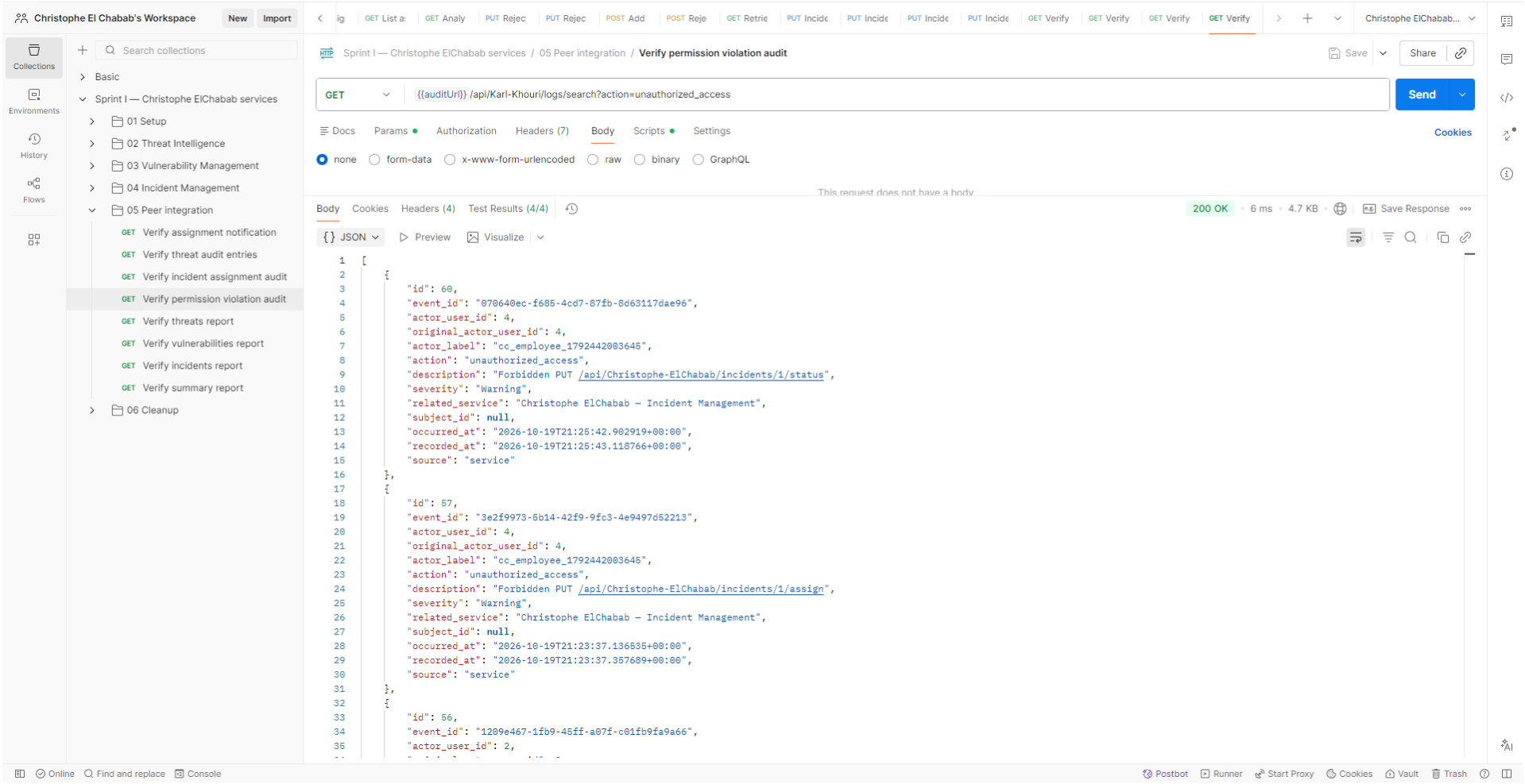

Figure 85. Christophe ElChabab - GET Verify permission violation audit - Shows the named request and its successful HTTP 200 result.

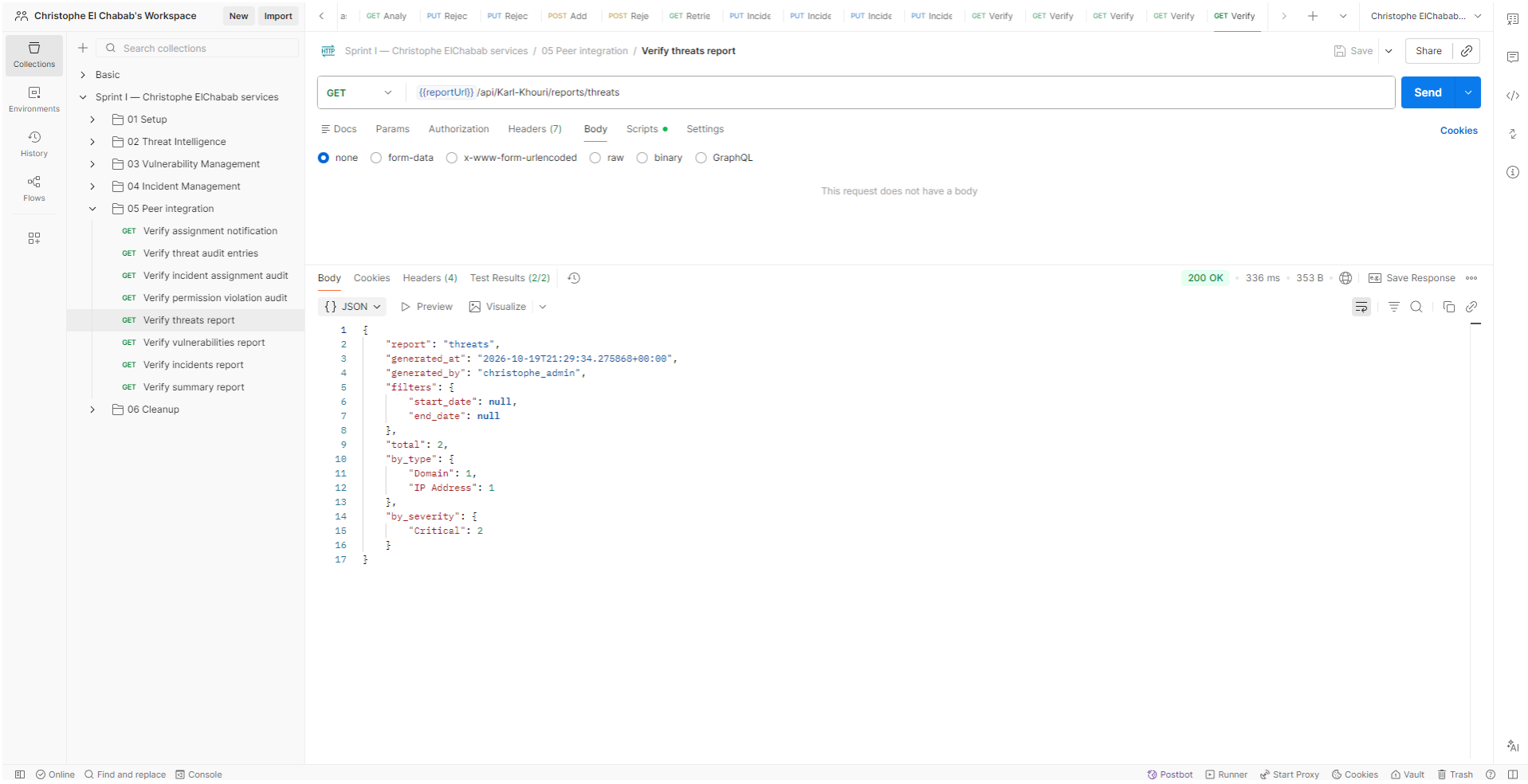

Figure 86. Christophe ElChabab - GET Verify threats report - Shows the named request and its successful HTTP 200 result.

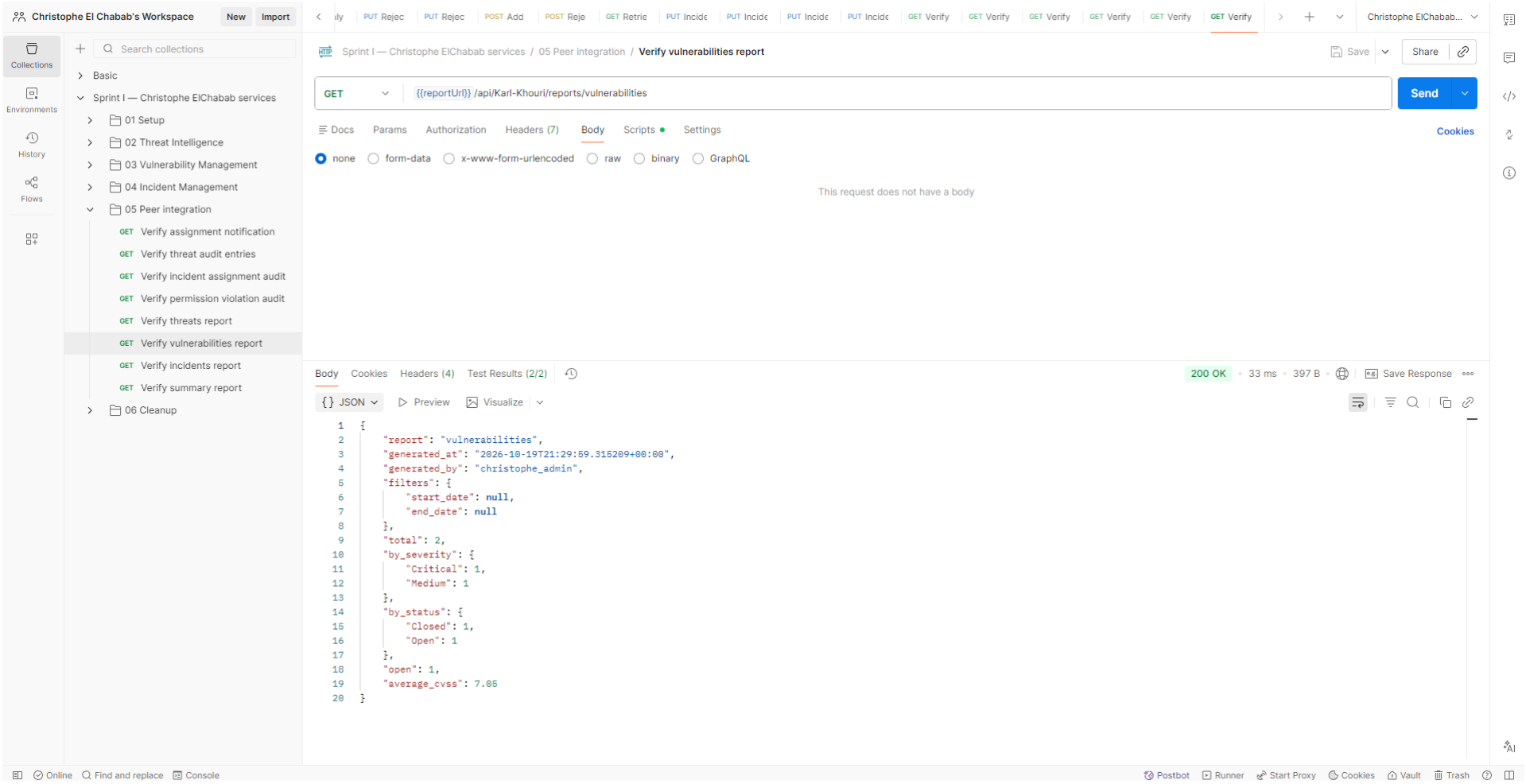

Figure 87. Christophe ElChabab - GET Verify vulnerabilities report - Shows the named request and its successful HTTP 200 result.

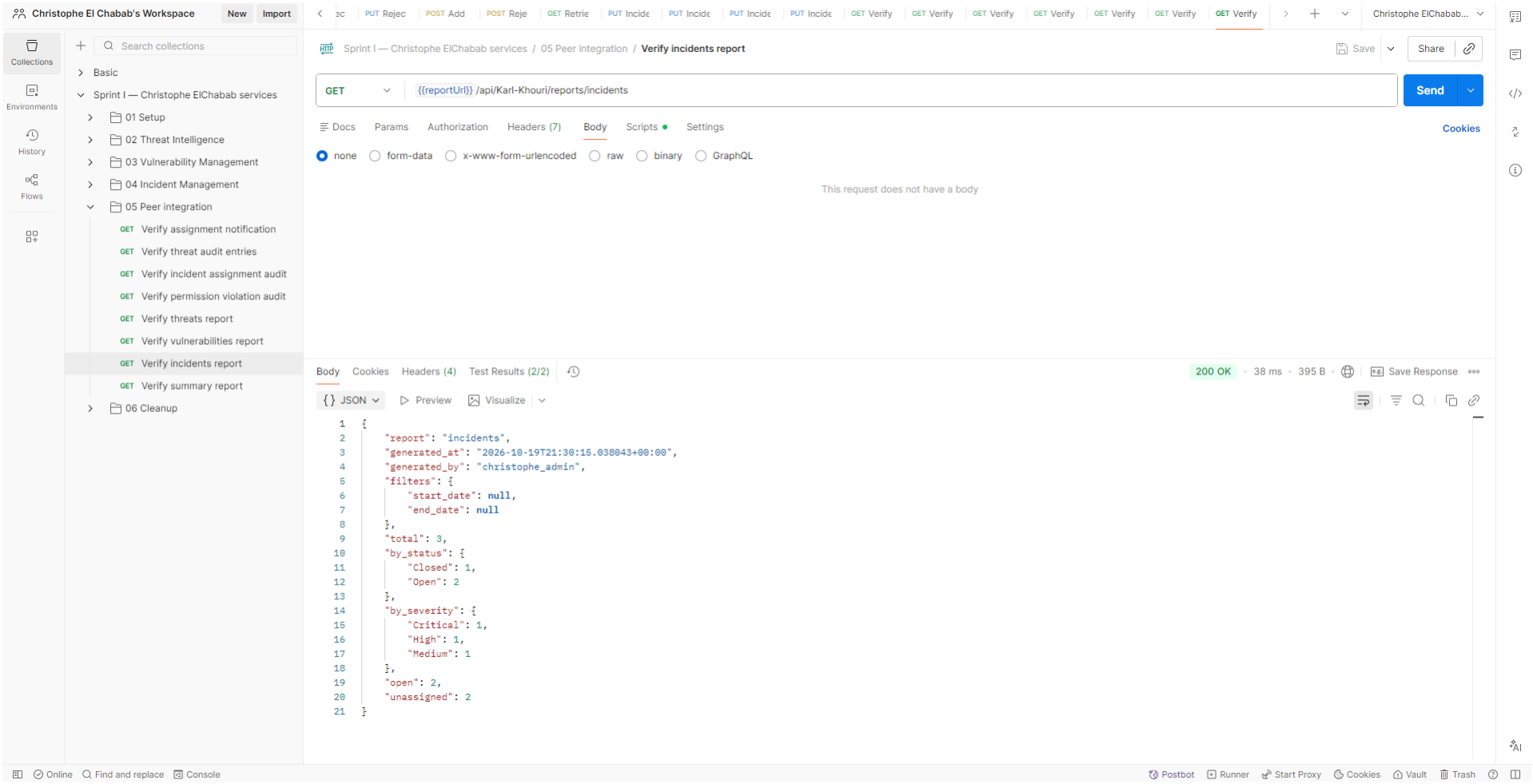

Figure 88. Christophe ElChabab - GET Verify incidents report - Shows the named request and its successful HTTP 200 result.

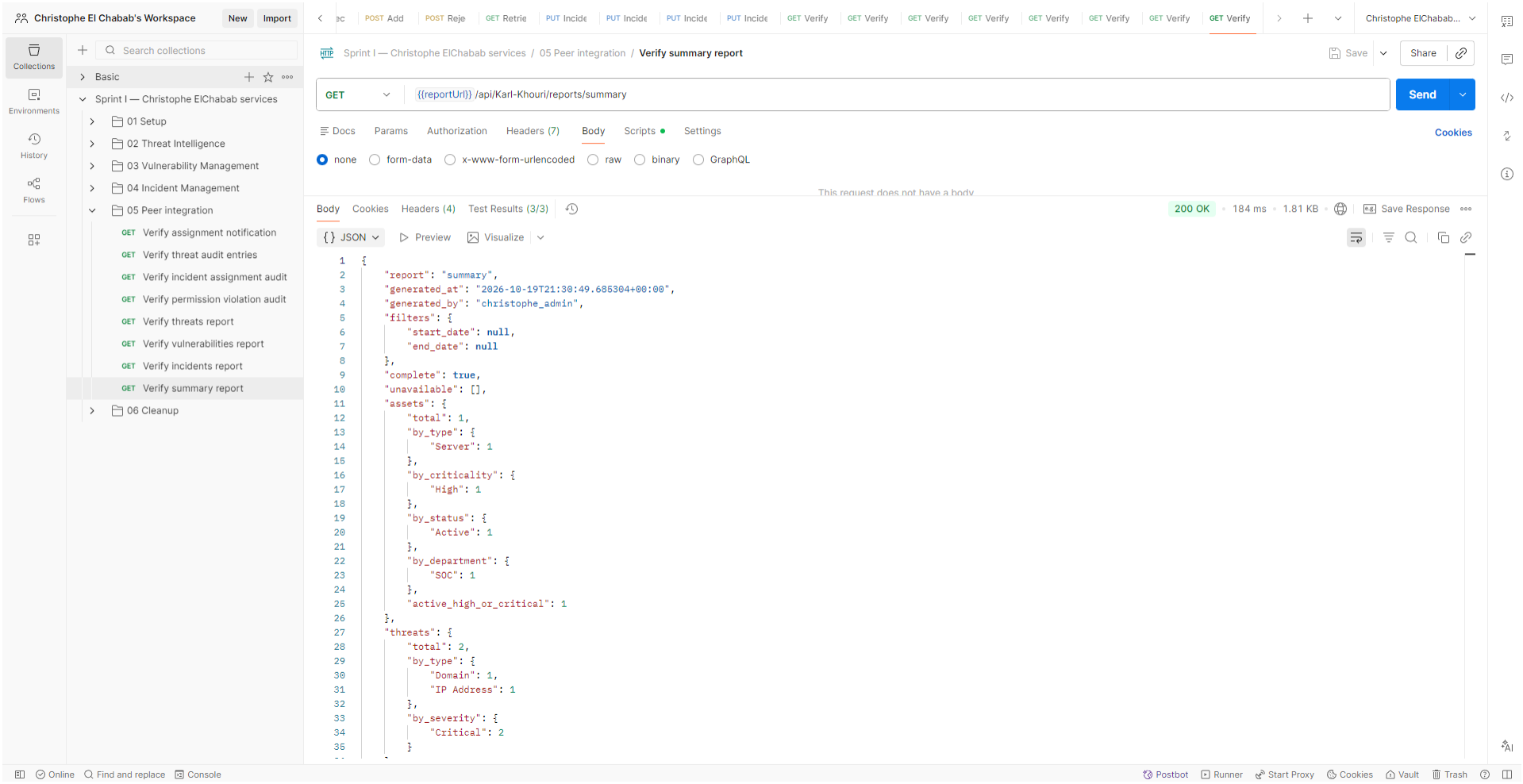

Figure 89. Christophe ElChabab - GET Verify summary report - Shows the named request and its successful HTTP 200 result.

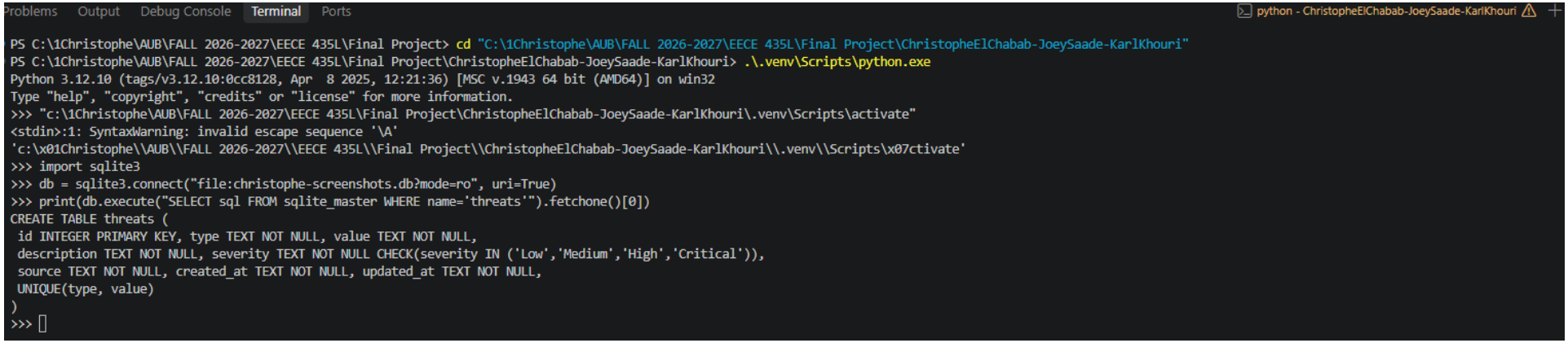

Figure 90. Christophe ElChabab - Database Threats Schema - Displays the actual table definition and its keys and constraints.

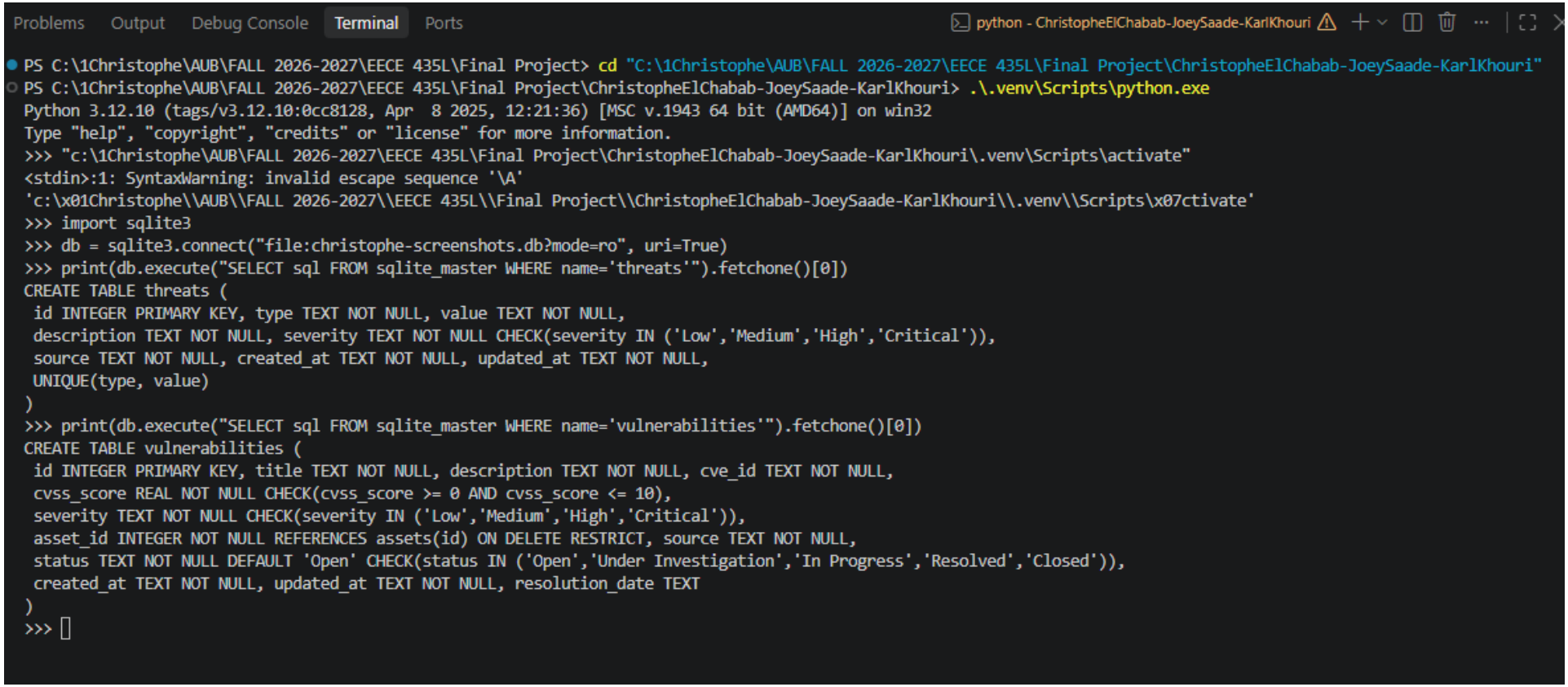

Figure 91. Christophe ElChabab - Database Vulnerabilities Schema - Displays the actual table definition and its keys and constraints.

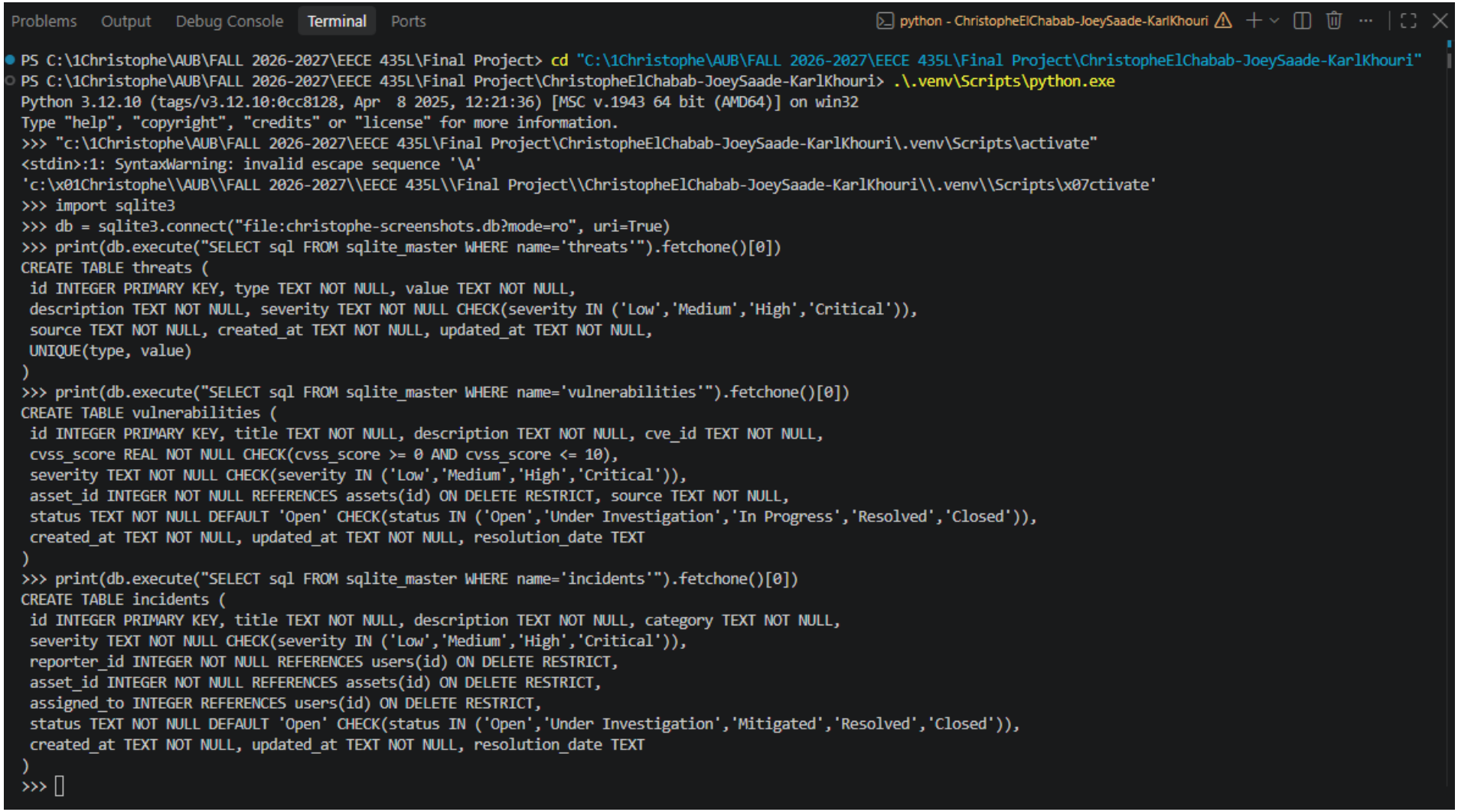

Figure 92. Christophe ElChabab - Database Incidents Schema - Displays the actual table definition and its keys and constraints.

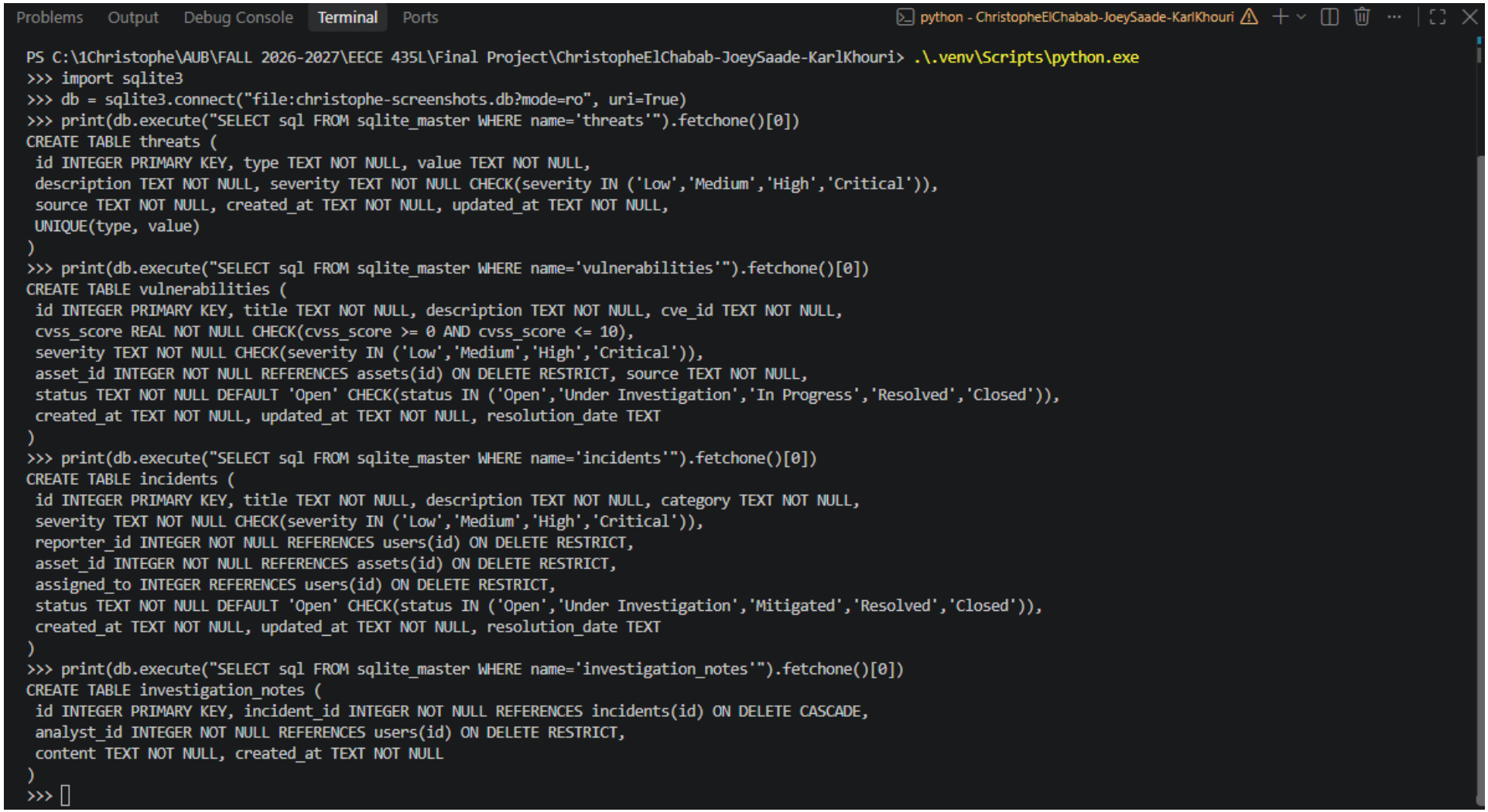

Figure 93. Christophe ElChabab - Database Investigation Notes Schema - Displays the actual table definition and its keys and constraints.

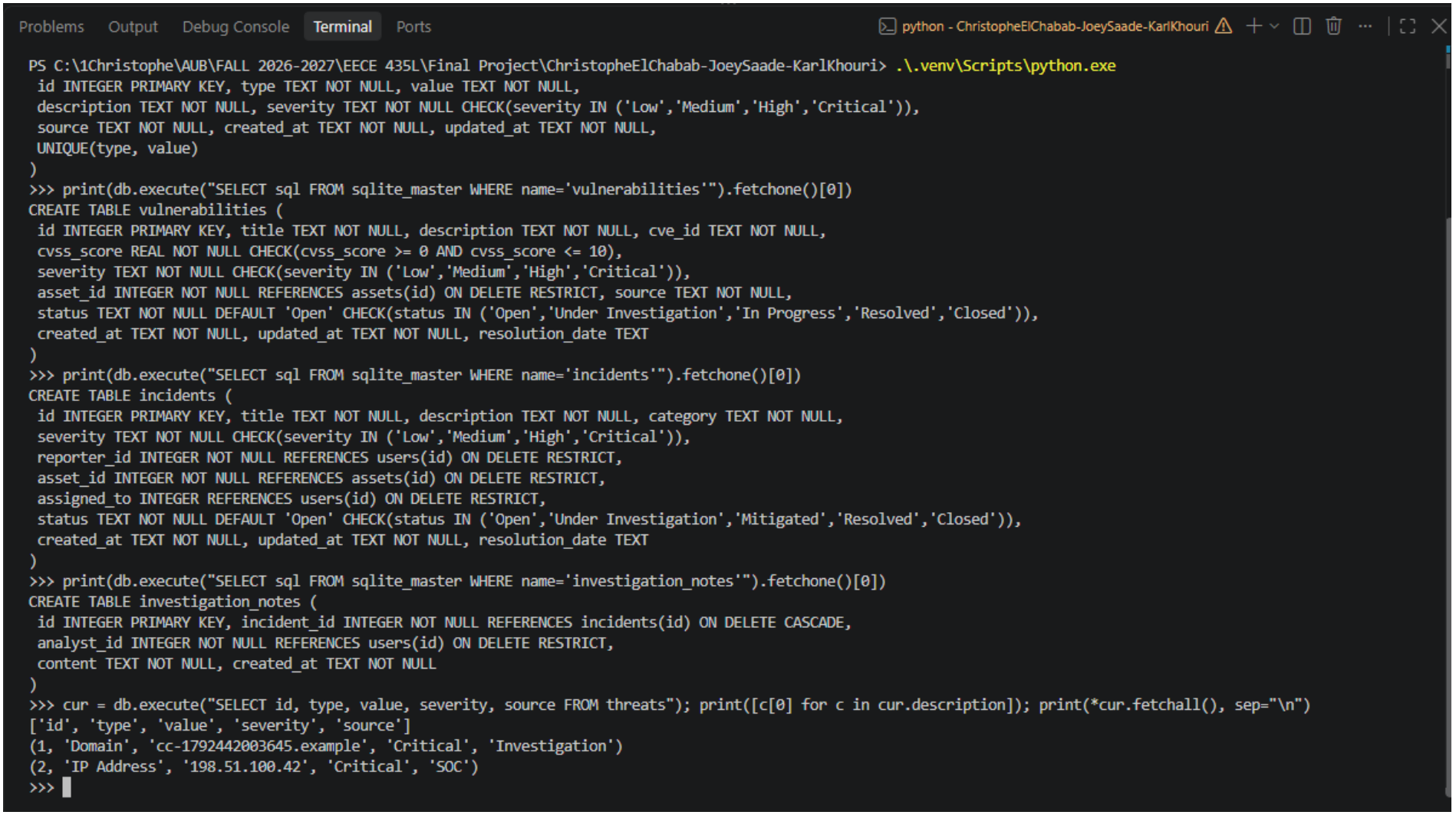

Figure 94. Christophe ElChabab - Database Threats Records - Displays the stored rows and their column names before Cleanup.

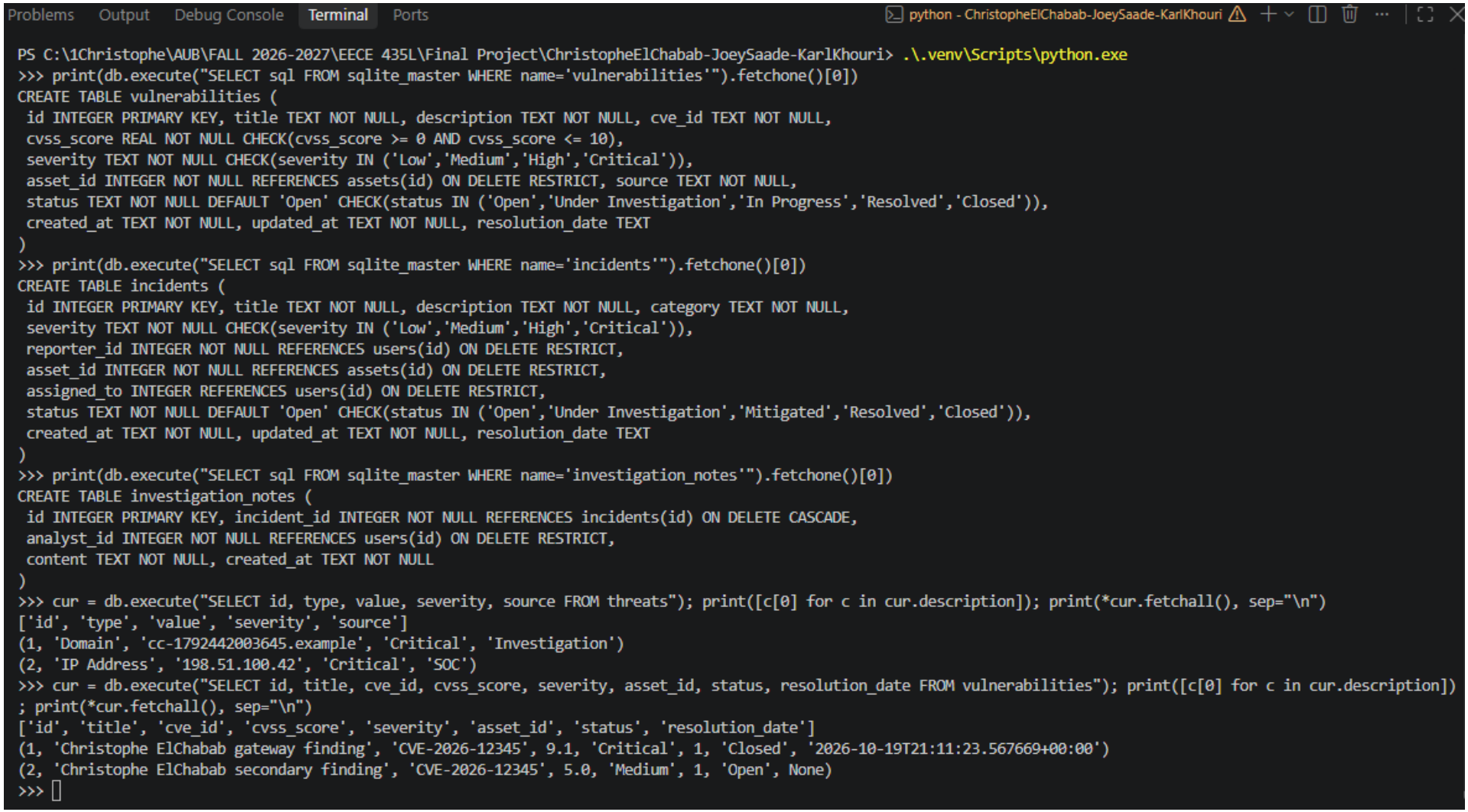

Figure 95. Christophe ElChabab - Database Vulnerabilities Records - Displays the stored rows and their column names before Cleanup.

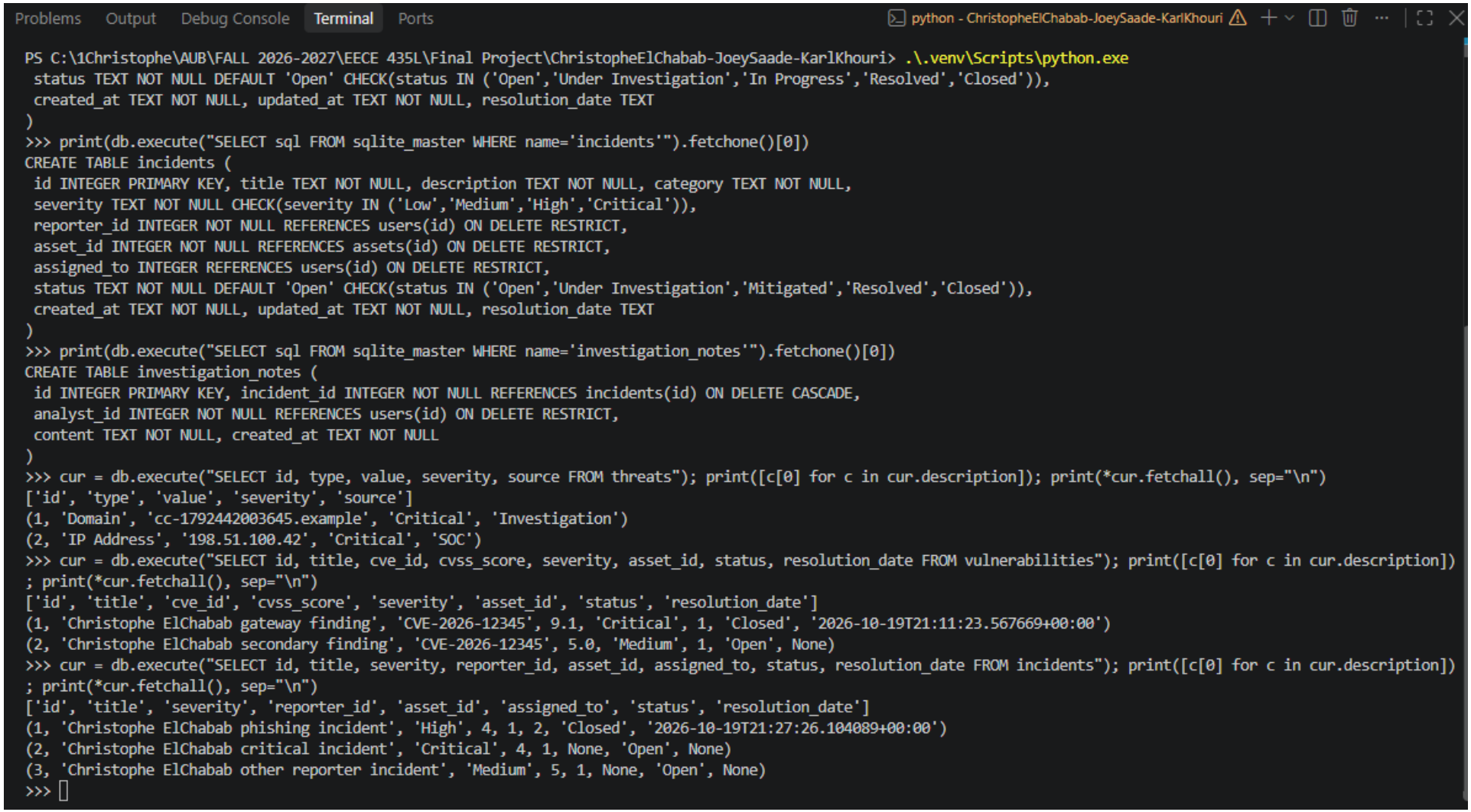

Figure 96. Christophe ElChabab - Database Incidents Records - Displays the stored rows and their column names before Cleanup.

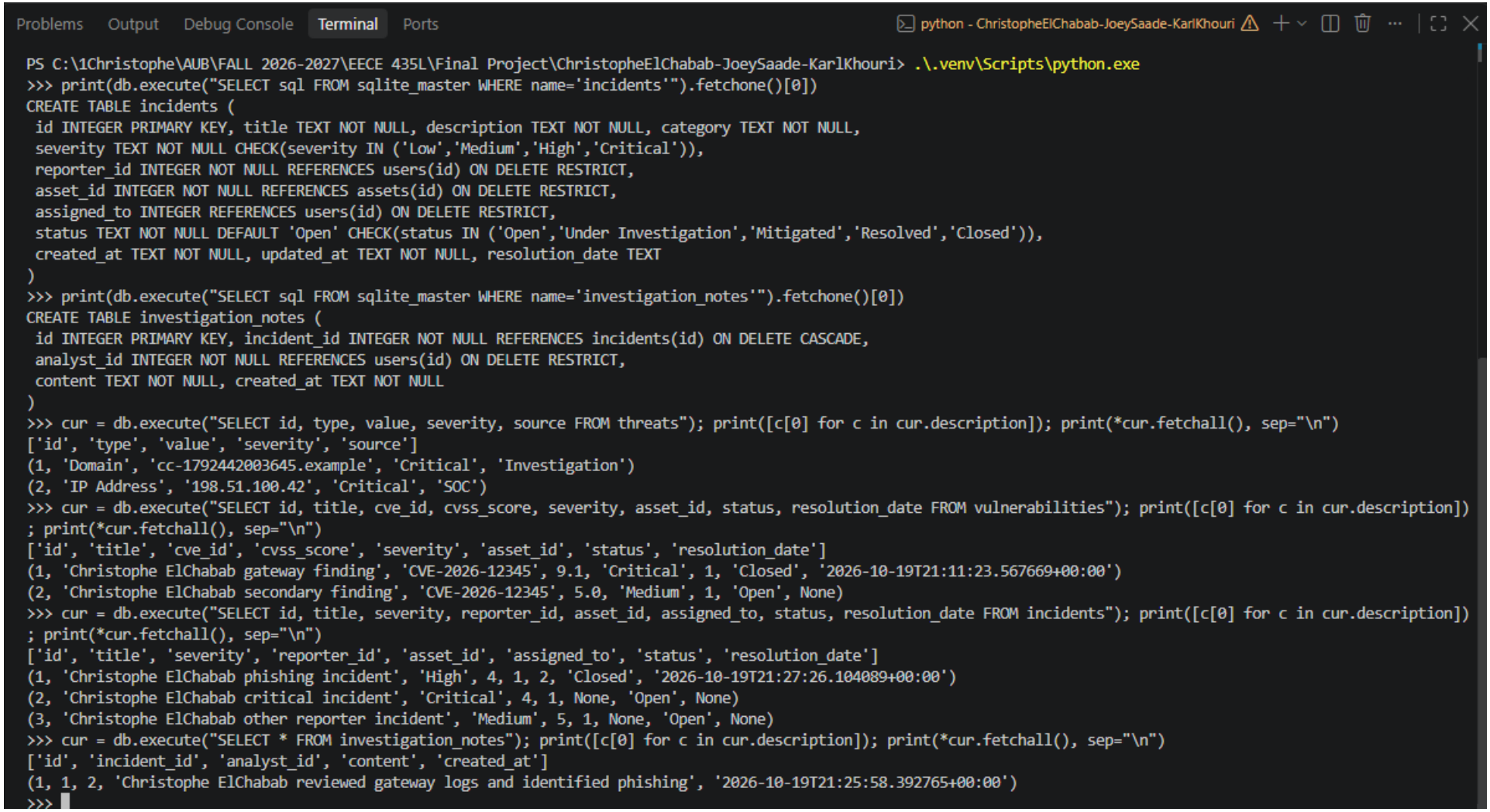

Figure 97. Christophe ElChabab - Database Investigation Notes Records - Displays the stored rows and their column names before Cleanup.

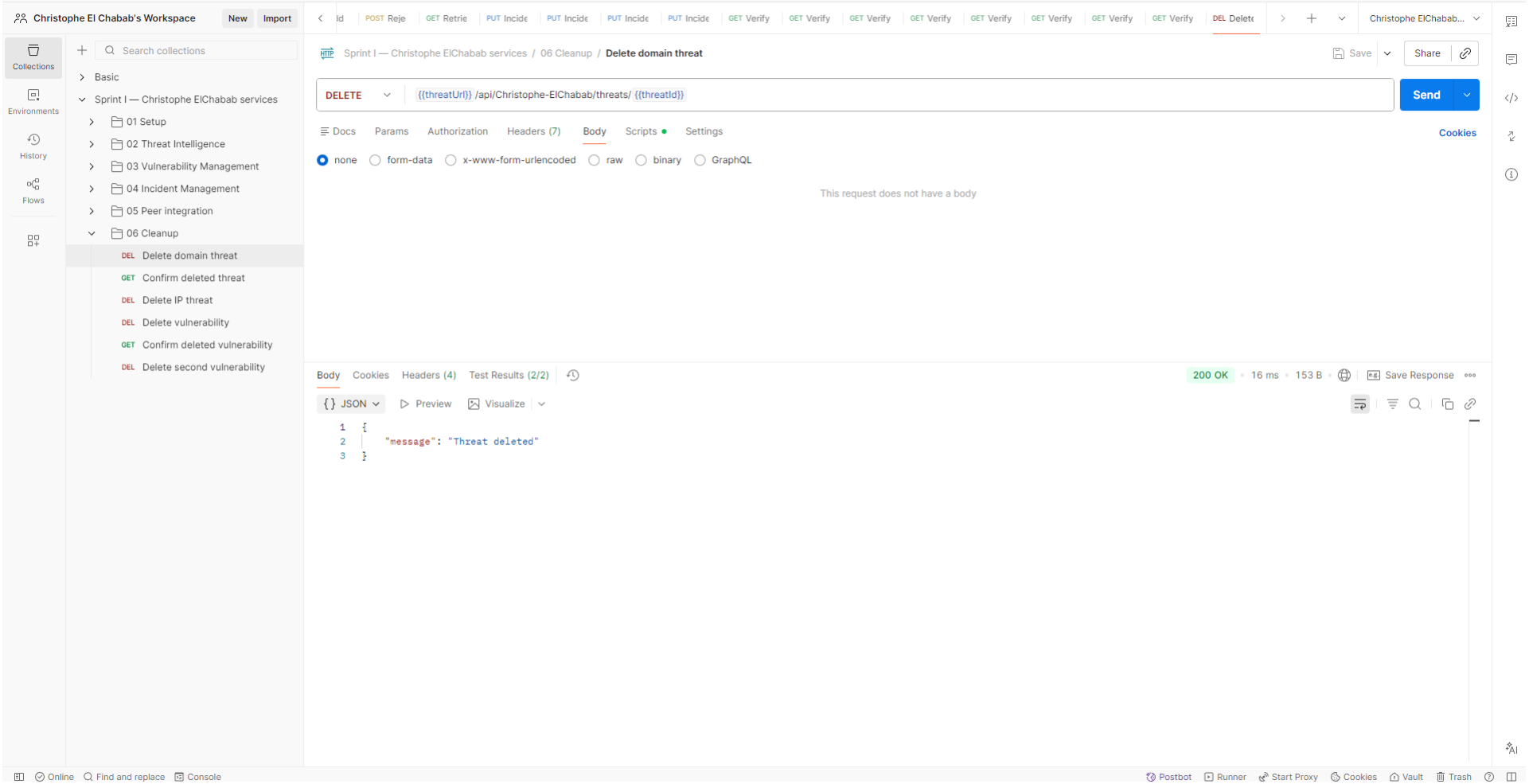

Figure 98. Christophe ElChabab - DELETE Delete Domain Threat - Shows the named request and its successful HTTP 200 result.

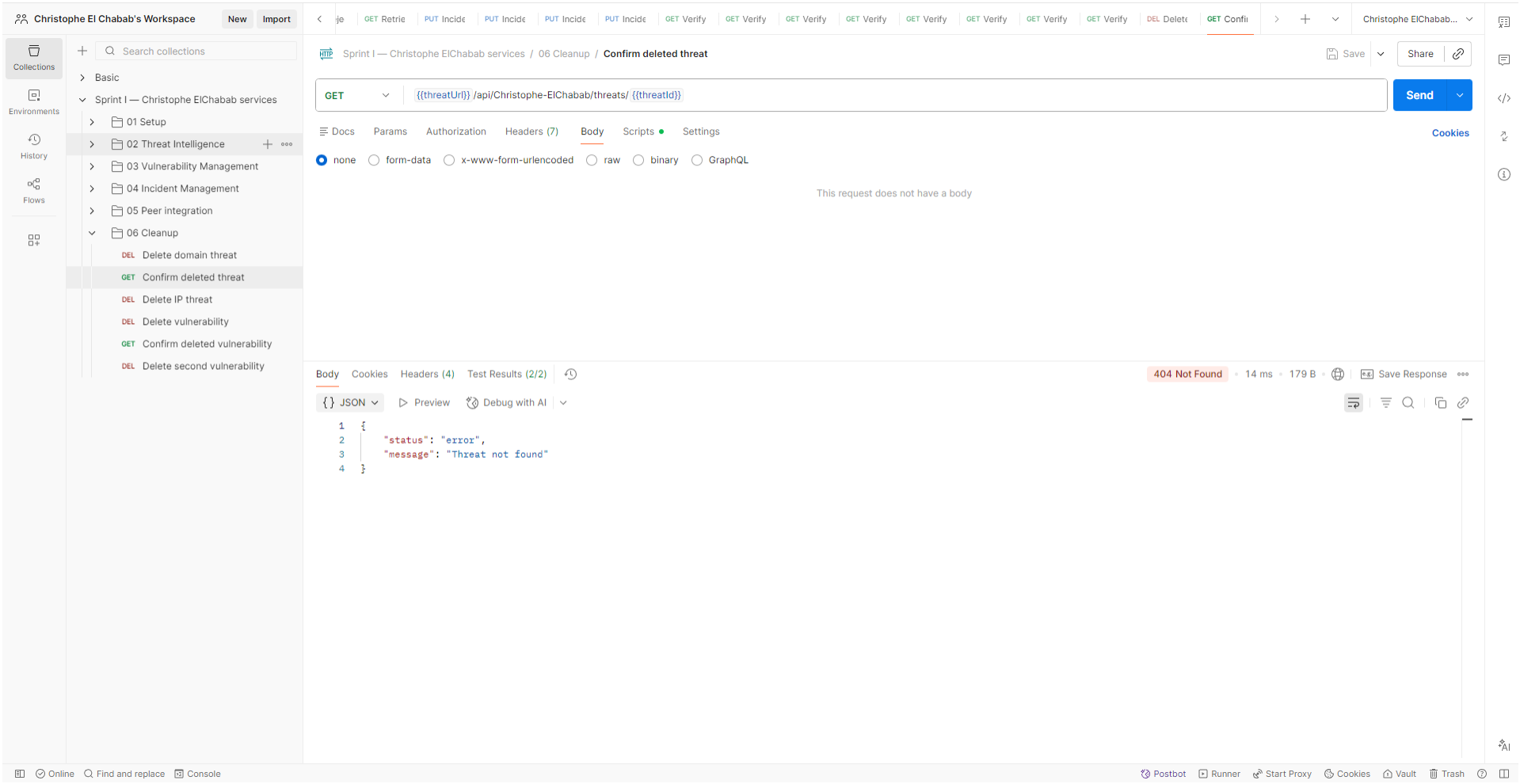

Figure 99. Christophe ElChabab - GET Confirm Deleted Threat - Confirms that retrieving the deleted record returns HTTP 404.

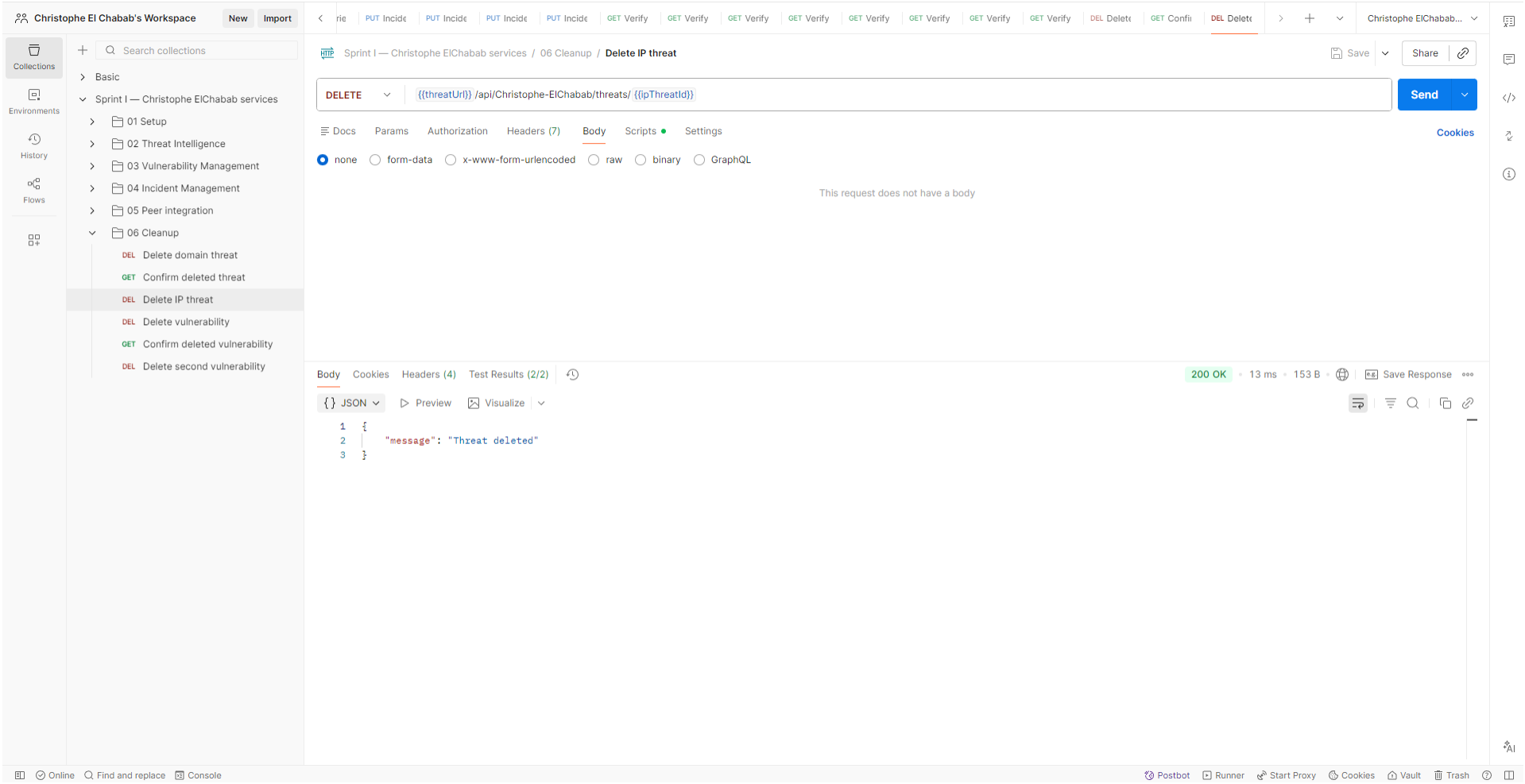

Figure 100. Christophe ElChabab - DELETE Delete IP Threat - Shows the named request and its successful HTTP 200 result.

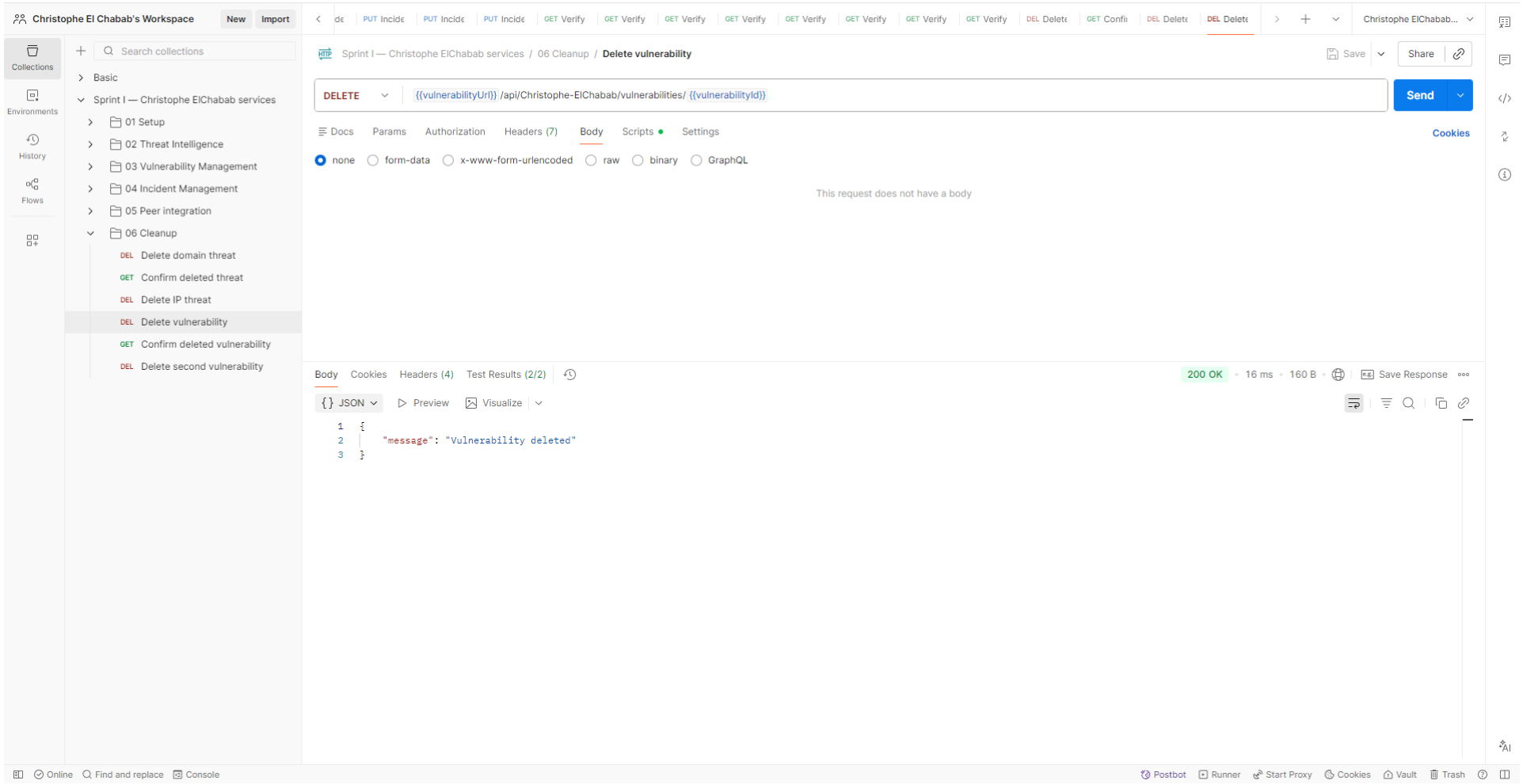

Figure 101. Christophe ElChabab - DELETE Delete Vulnerability - Shows the named request and its successful HTTP 200 result.

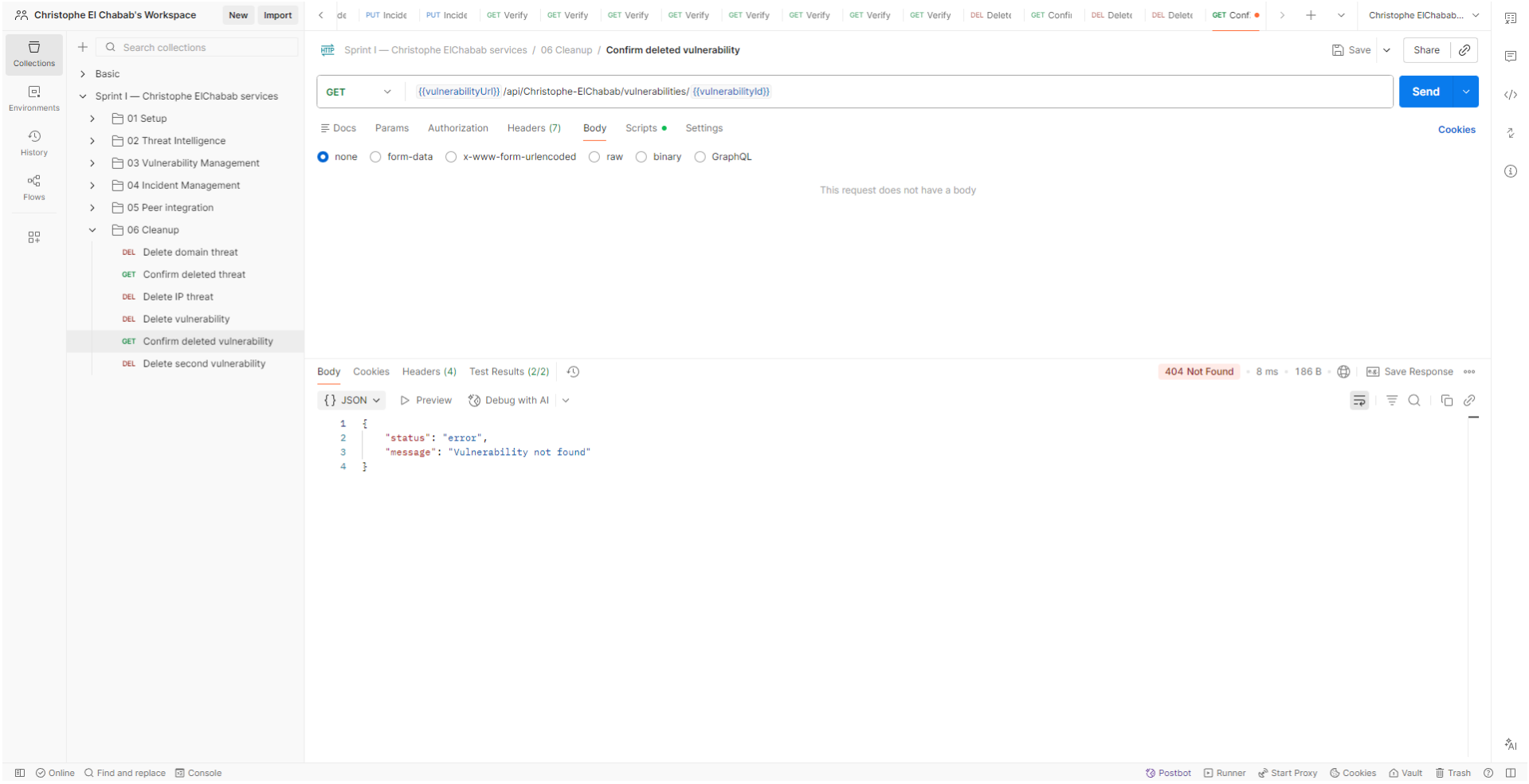

Figure 102. Christophe ElChabab - GET Confirm Deleted Vulnerability - Confirms that retrieving the deleted record returns HTTP 404.

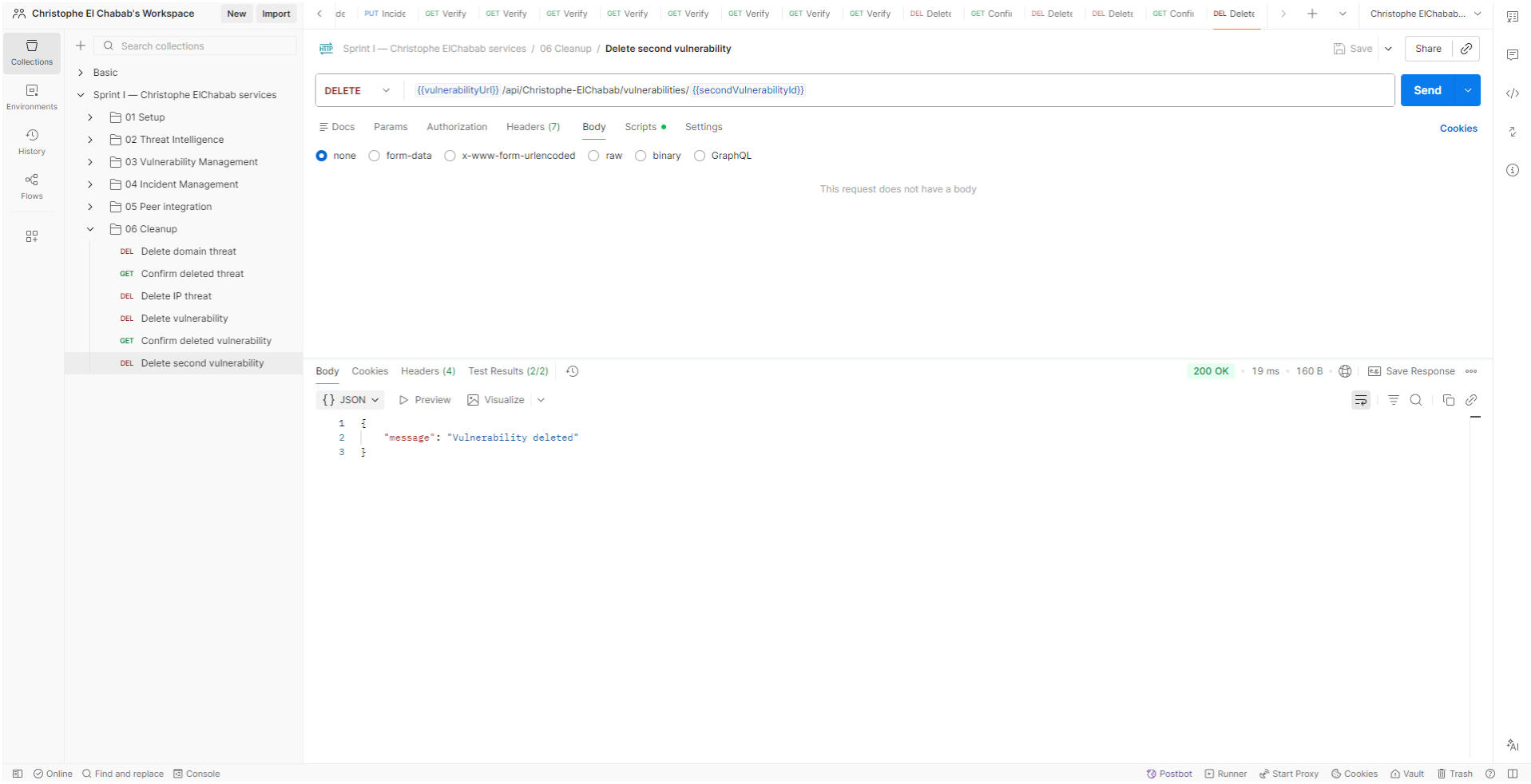

Figure 103. Christophe ElChabab - DELETE Delete Second Vulnerability - Shows the named request and its successful HTTP 200 result.

B Postman and Database Evidence (Joey Saade)

Joey's 32 captures include 27 individual Postman request/response views and five database record queries. They cover the three services, validation and missing-authentication errors, and ordered cleanup. The API captures use sample user 4; the database views show an earlier run with sample user 2. These are separate executions. The asset-history response includes retained entries from earlier runs, and the outbox view shows queued audit events.

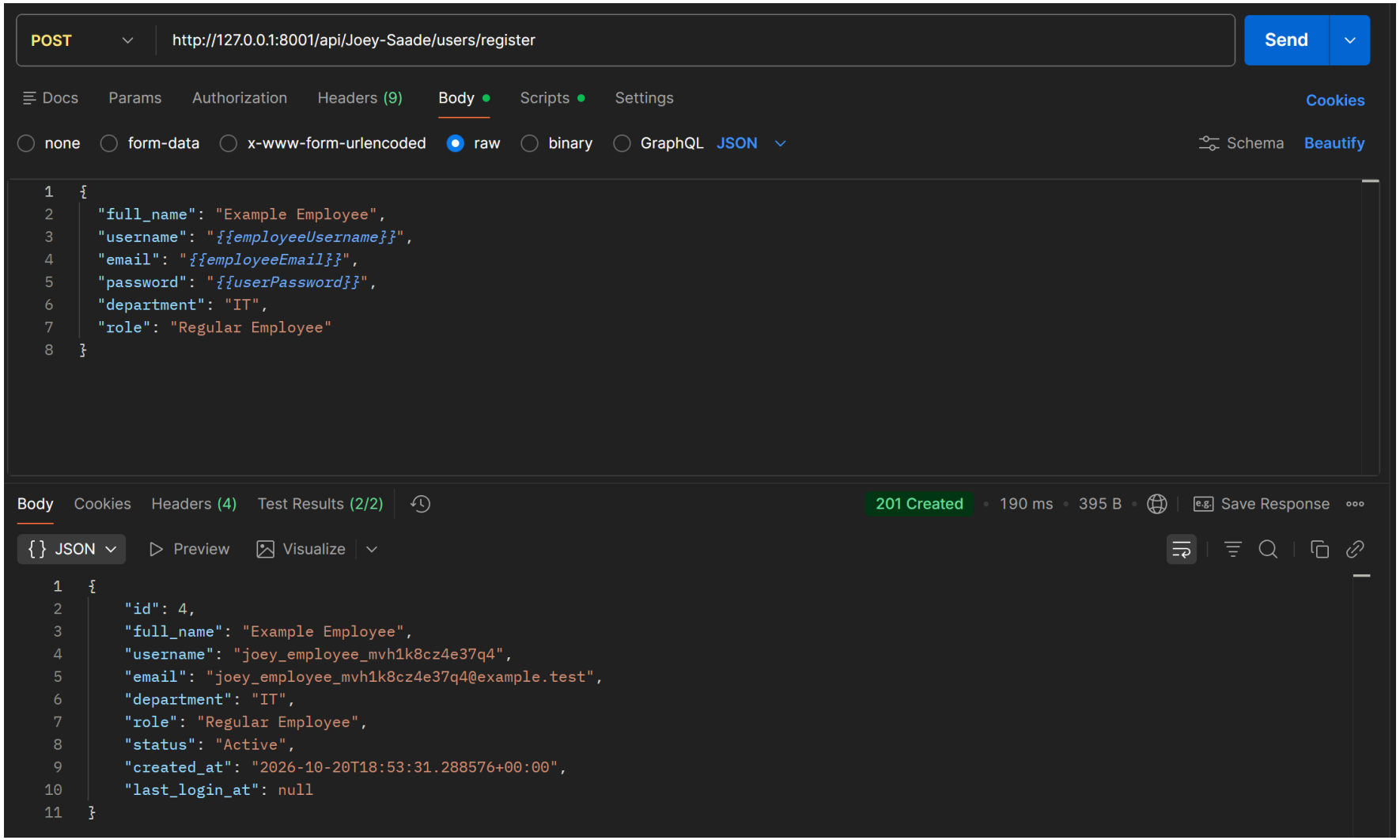

Figure 104. Joey Saade - Register employee - POST creates a regular employee and omits the password hash from the response.

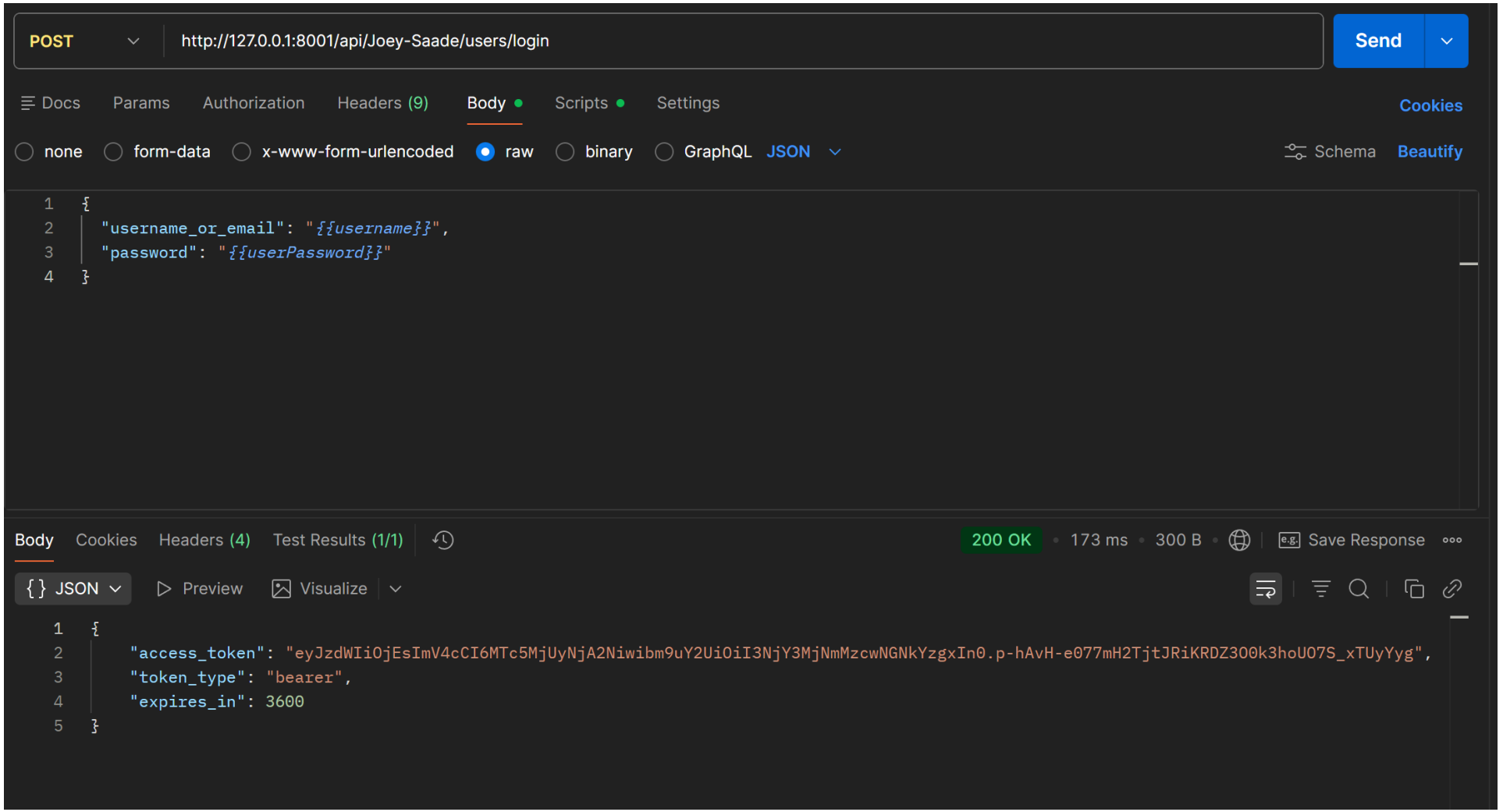

Figure 105. Joey Saade - Login - POST authenticates the administrator and returns HTTP 200 with a bearer token and expiry.

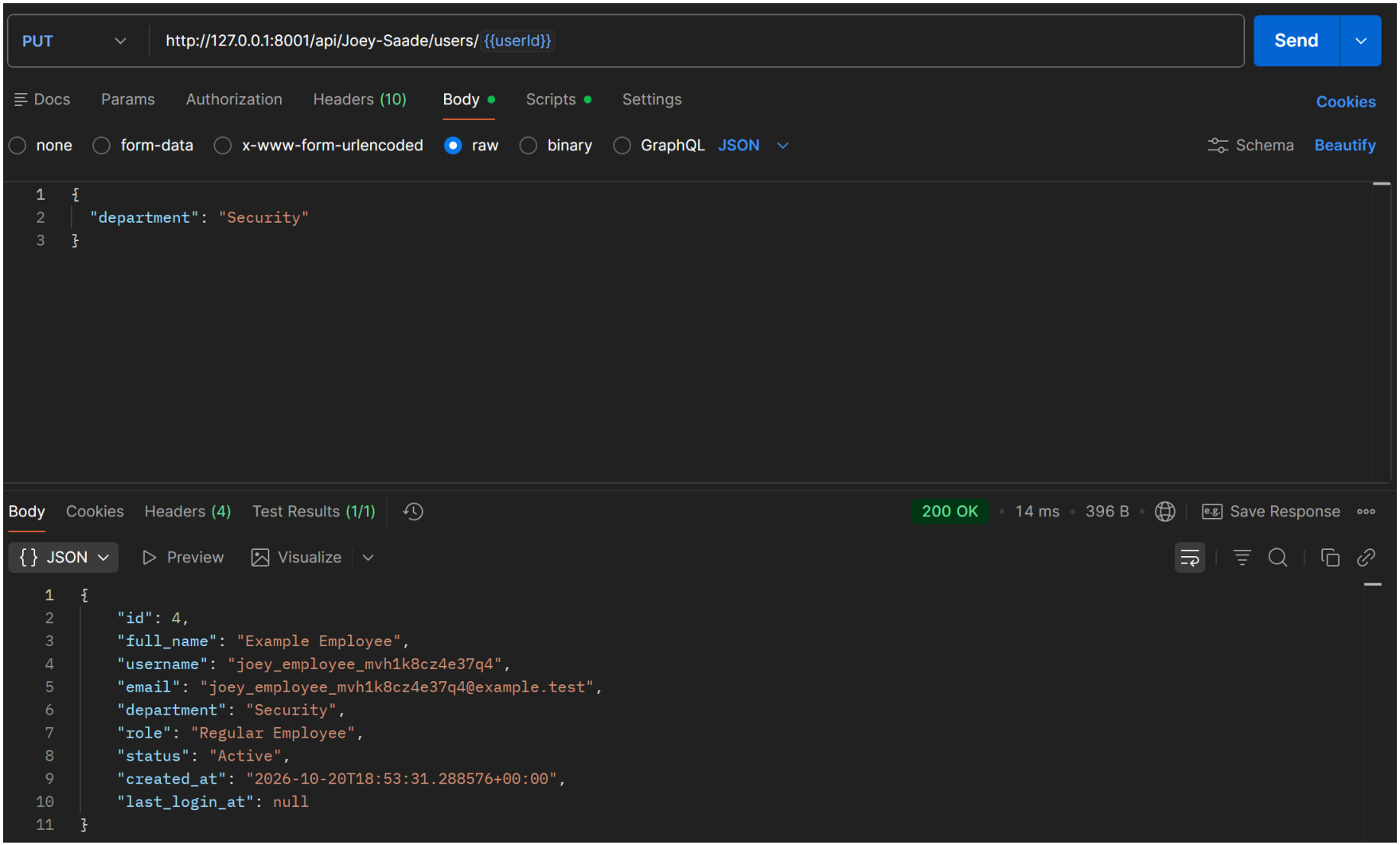

Figure 106. Joey Saade - Update profile - PUT changes the employee department.

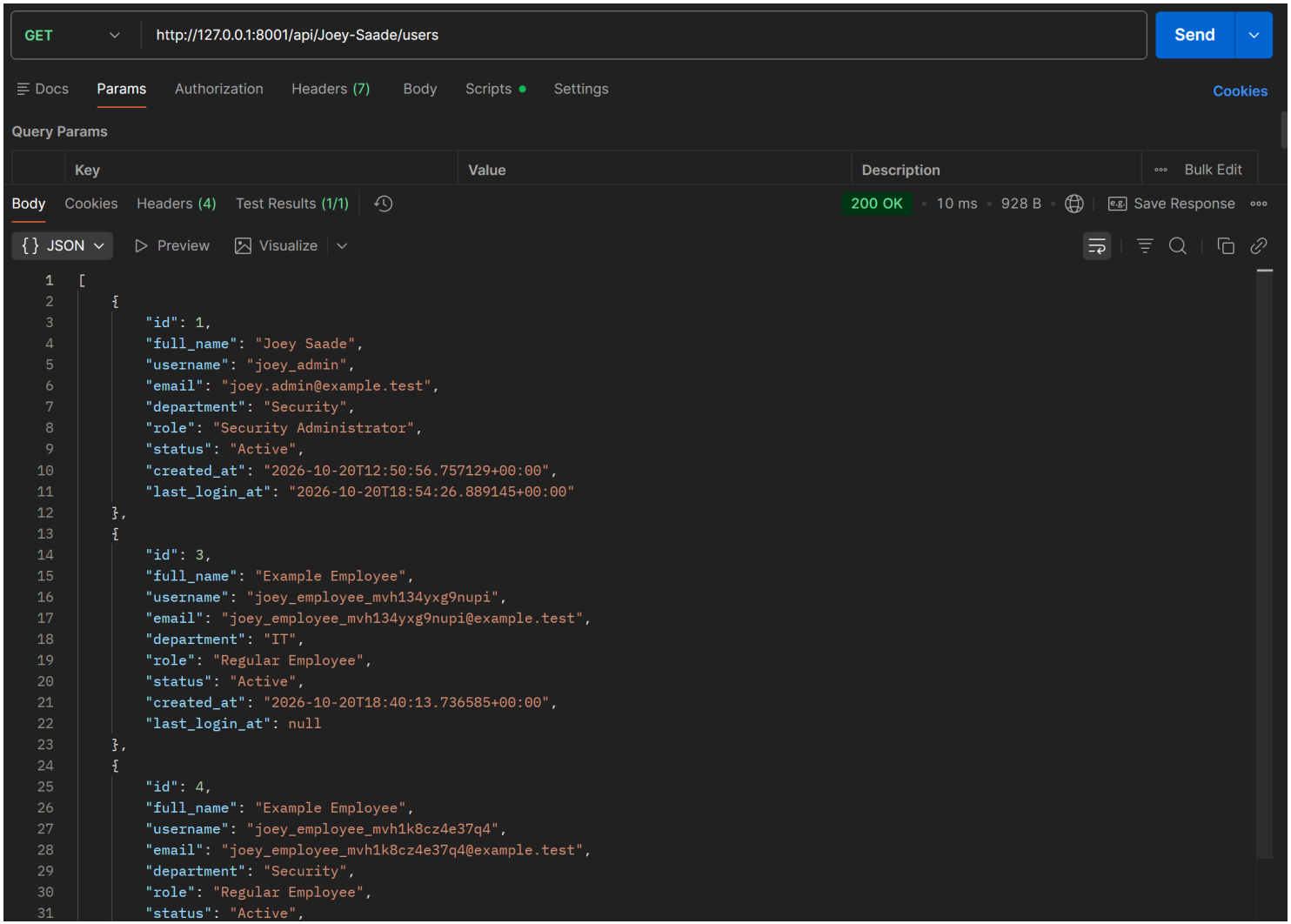

Figure 107. Joey Saade - List users - GET returns the user records.

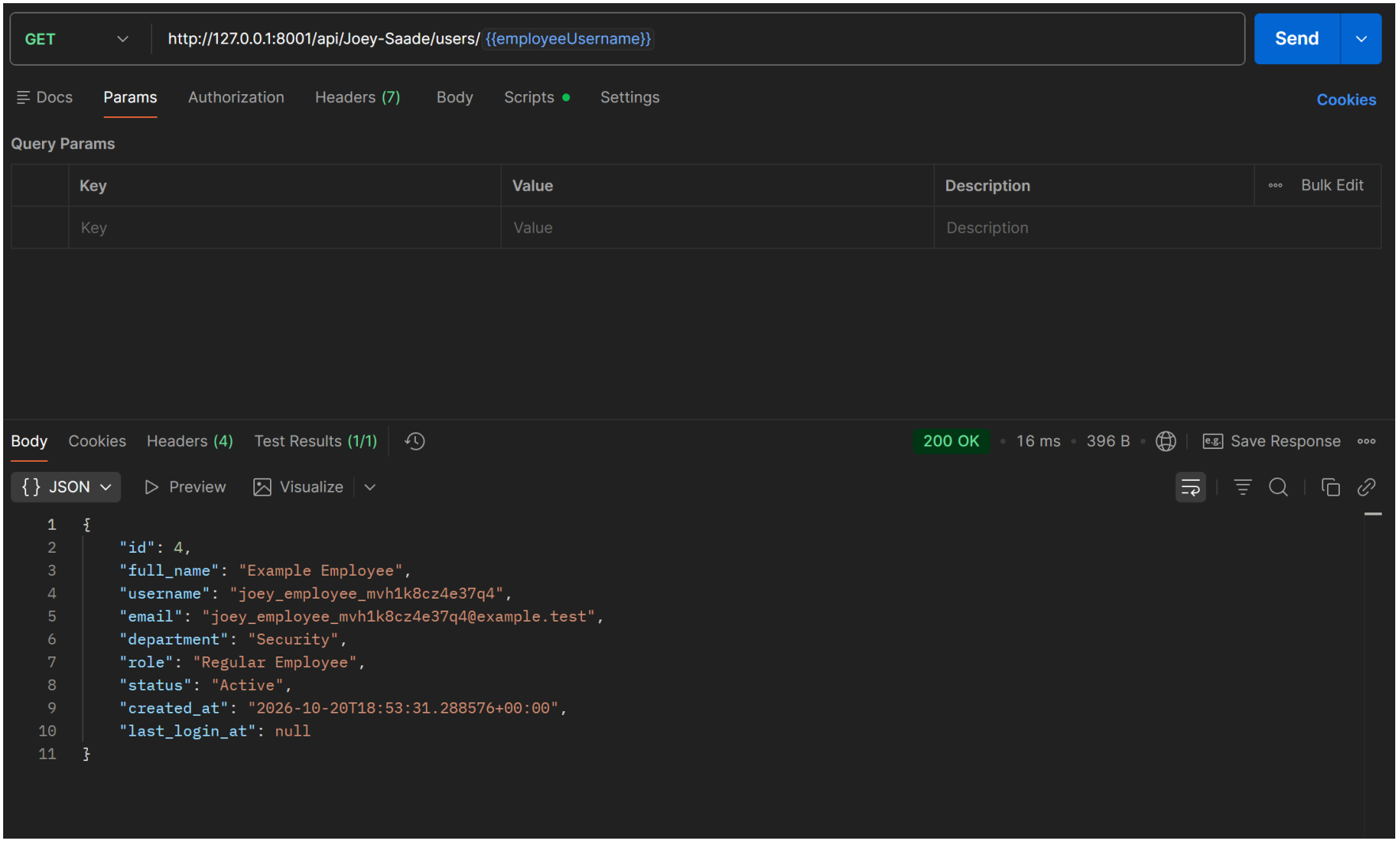

Figure 108. Joey Saade - Get user by username - GET returns the registered employee.

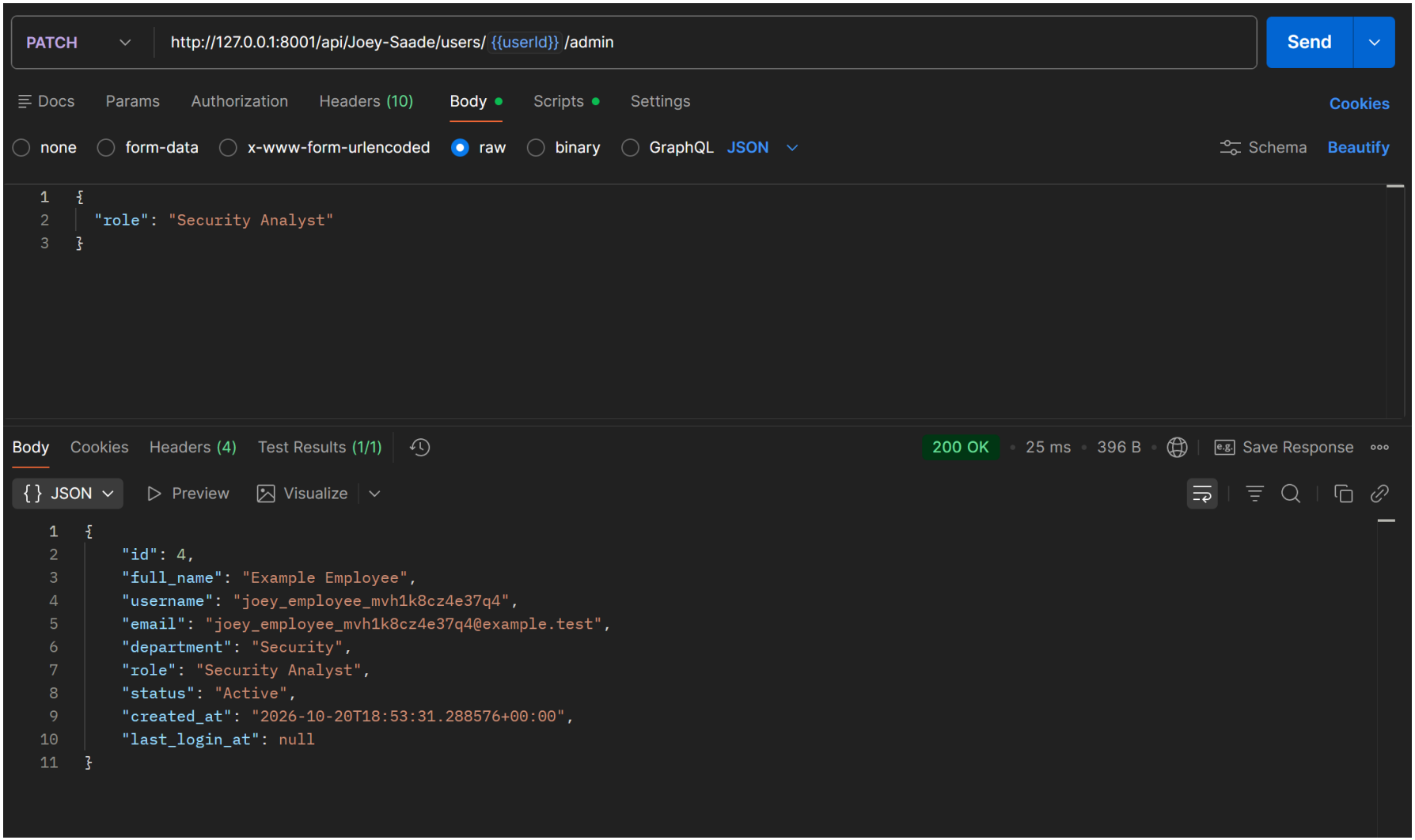

Figure 109. Joey Saade - Admin role and status update - PATCH assigns the security analyst role.

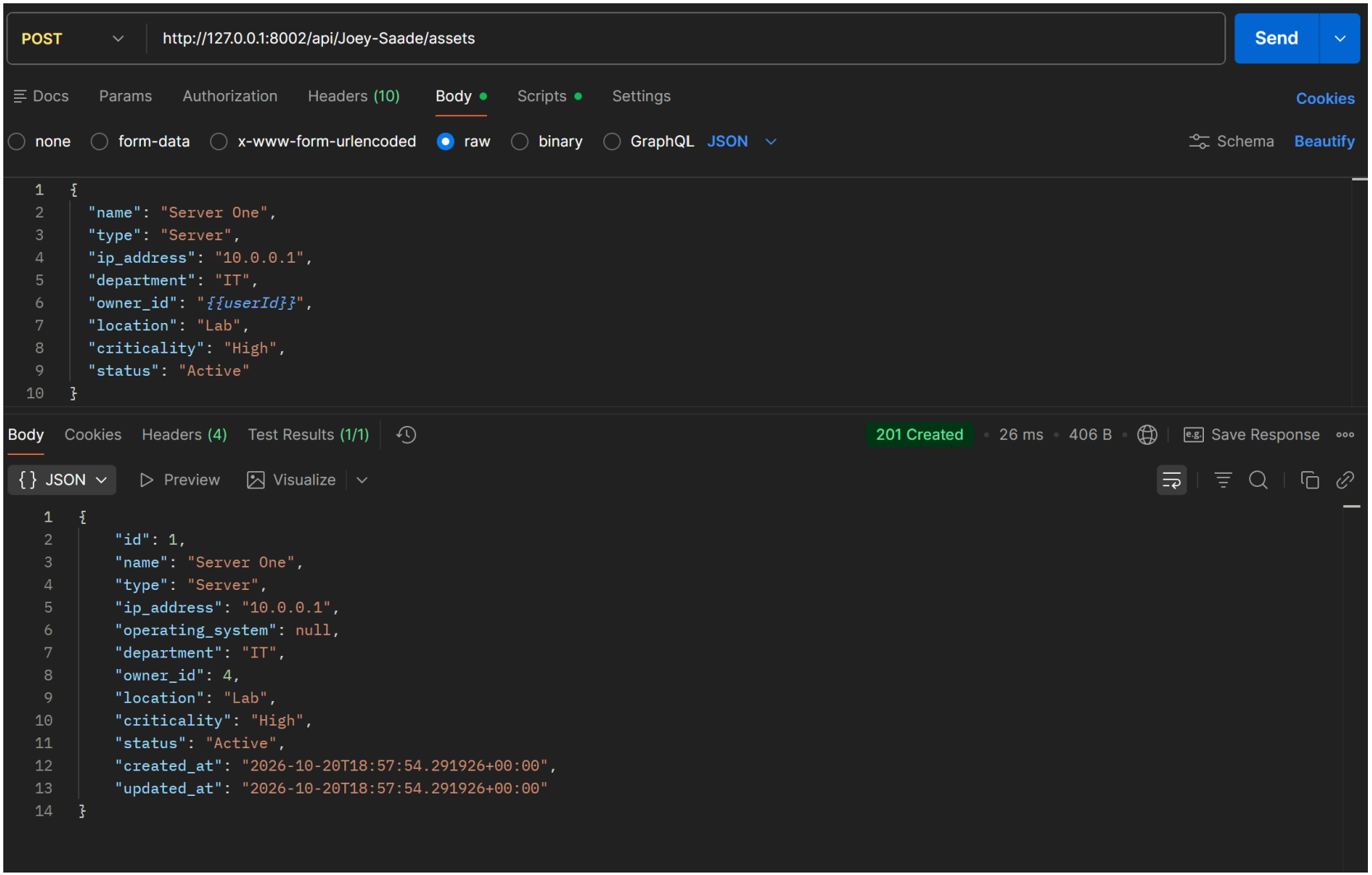

Figure 110. Joey Saade - Create asset - POST stores an asset owned by the registered user.

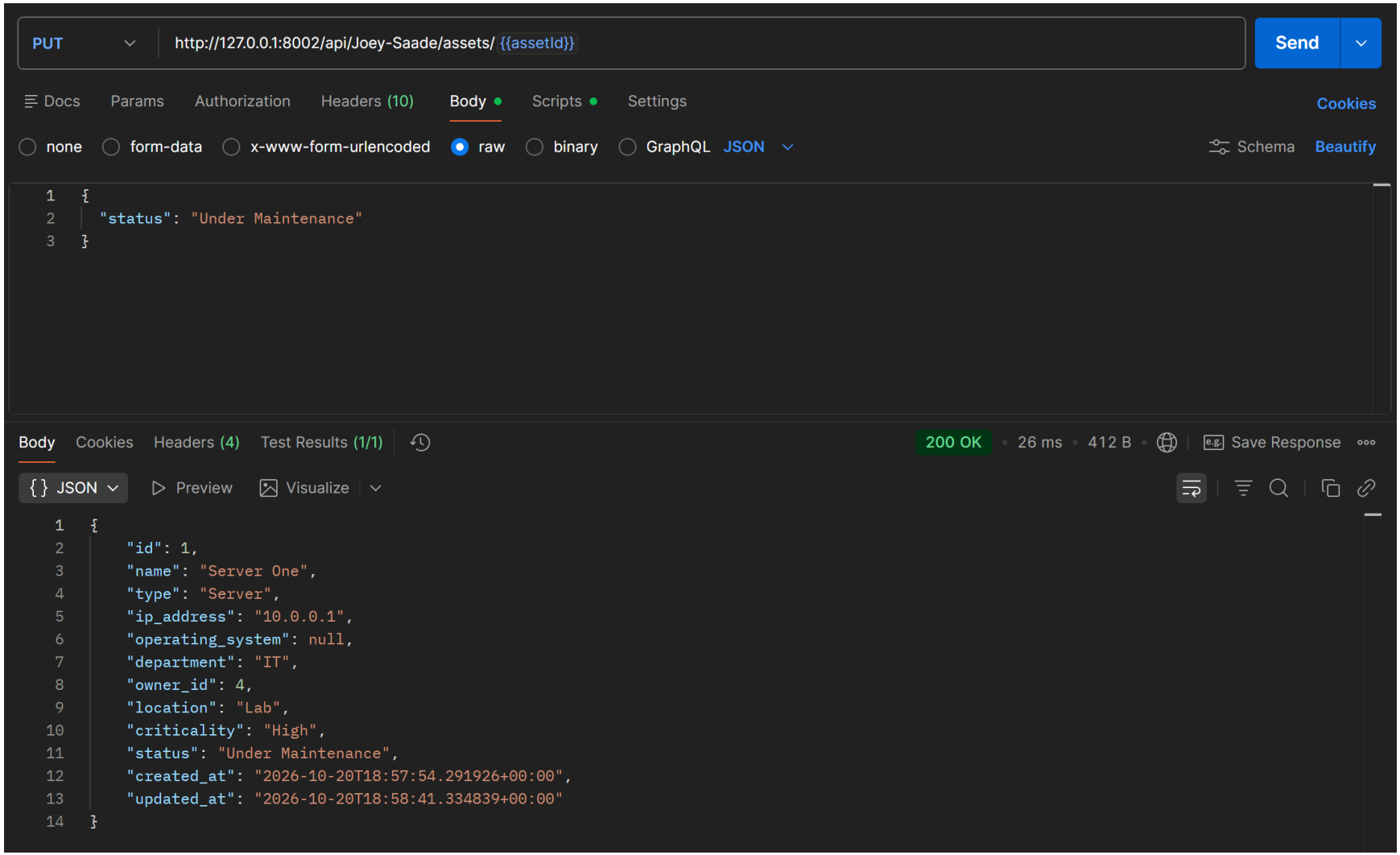

Figure 111. Joey Saade - Update asset - PUT changes its status.

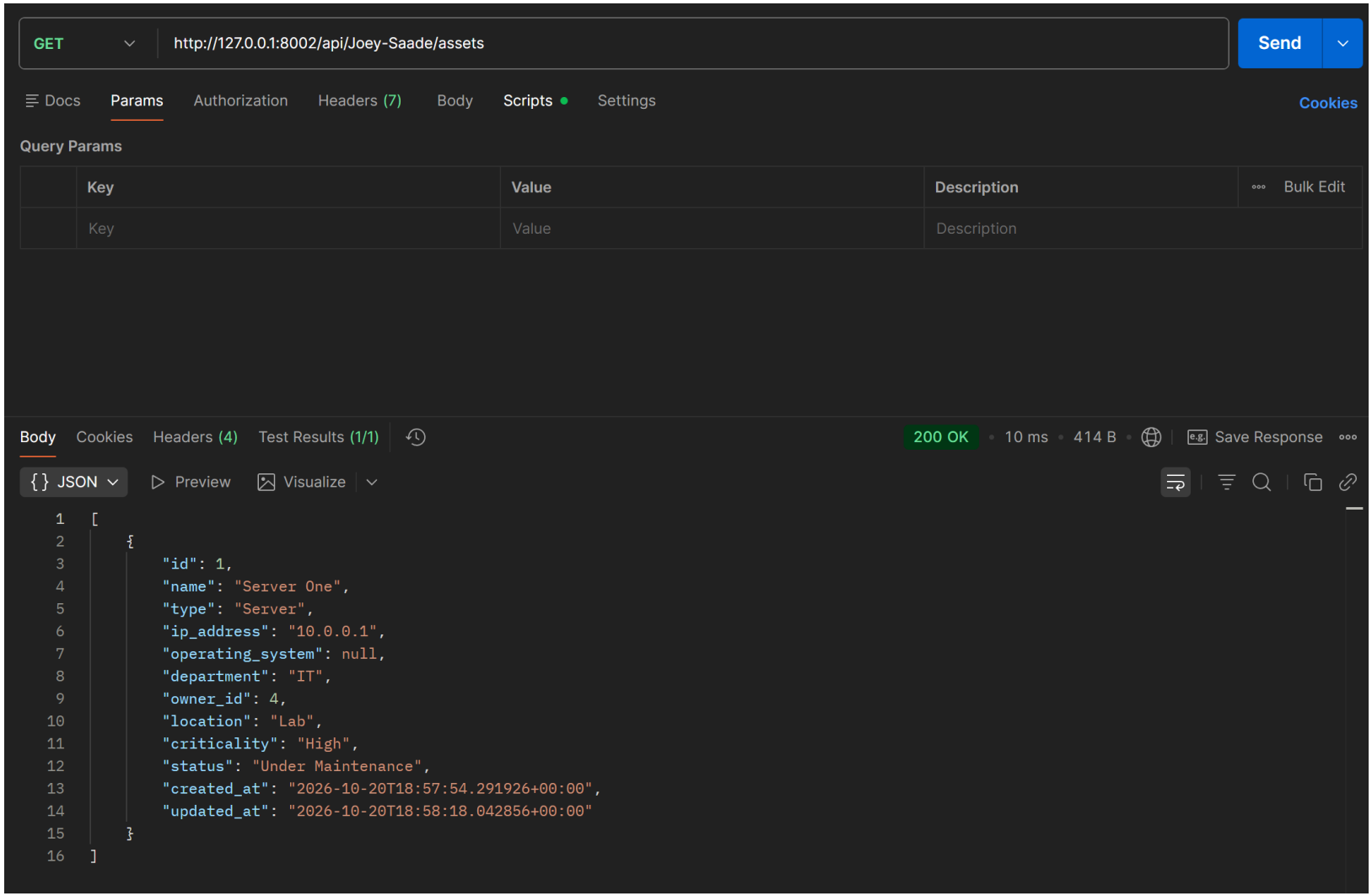

Figure 112. Joey Saade - List assets - GET returns the stored assets.

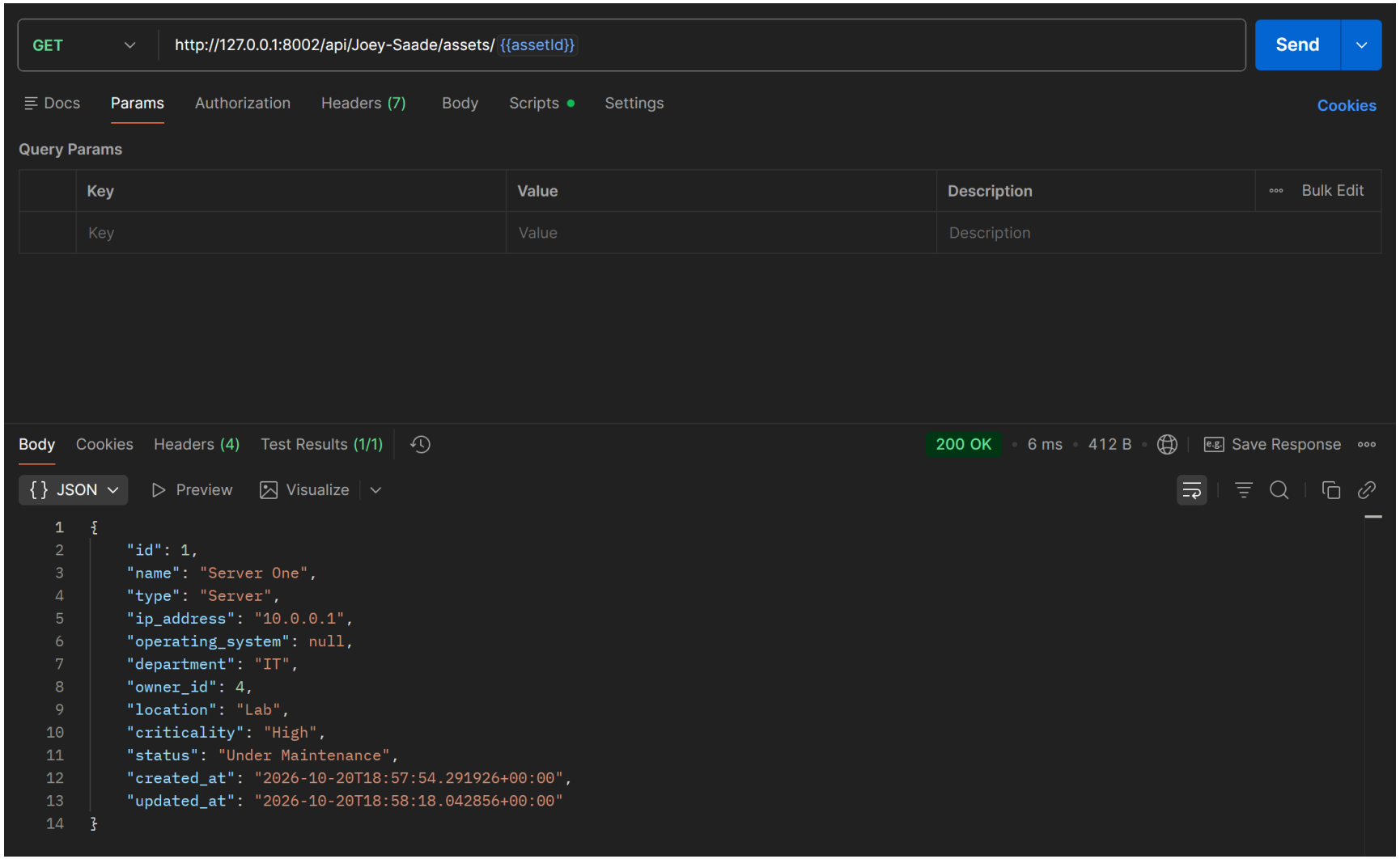

Figure 113. Joey Saade - Get asset - GET returns the asset before Cleanup.

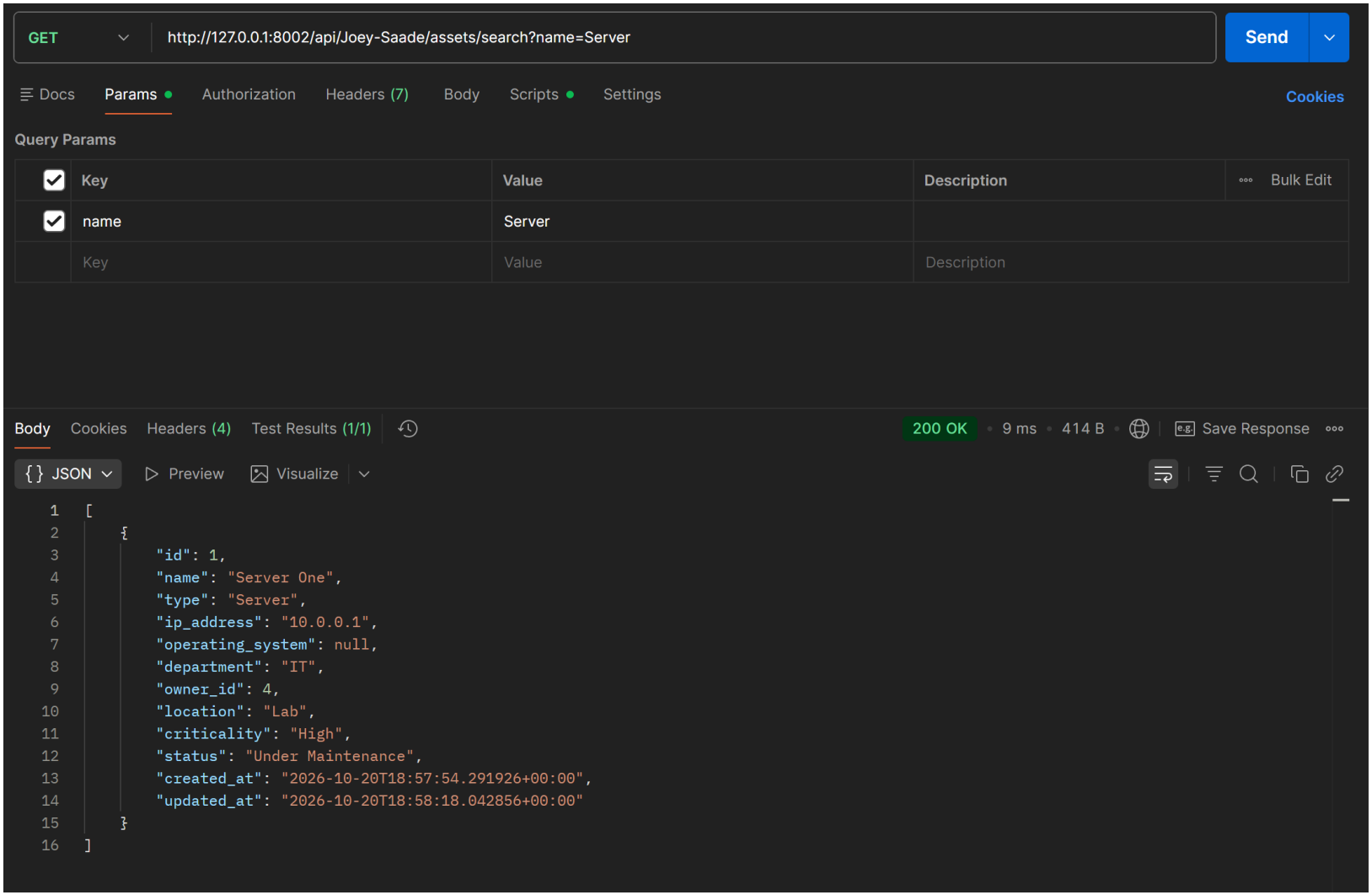

Figure 114. Joey Saade - Search assets - GET filters by name.

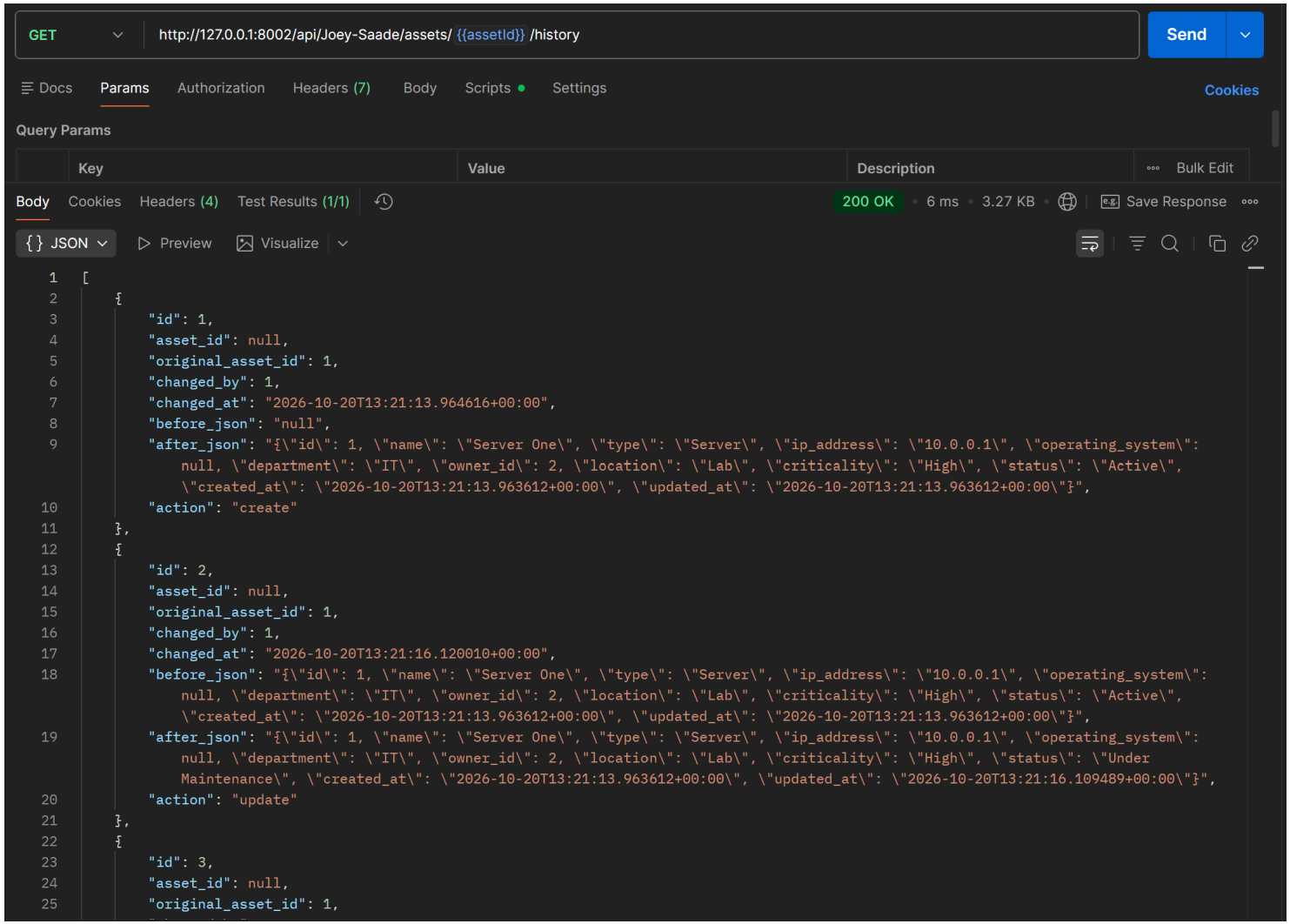

Figure 115. Joey Saade - Asset history - GET returns the asset's recorded changes.

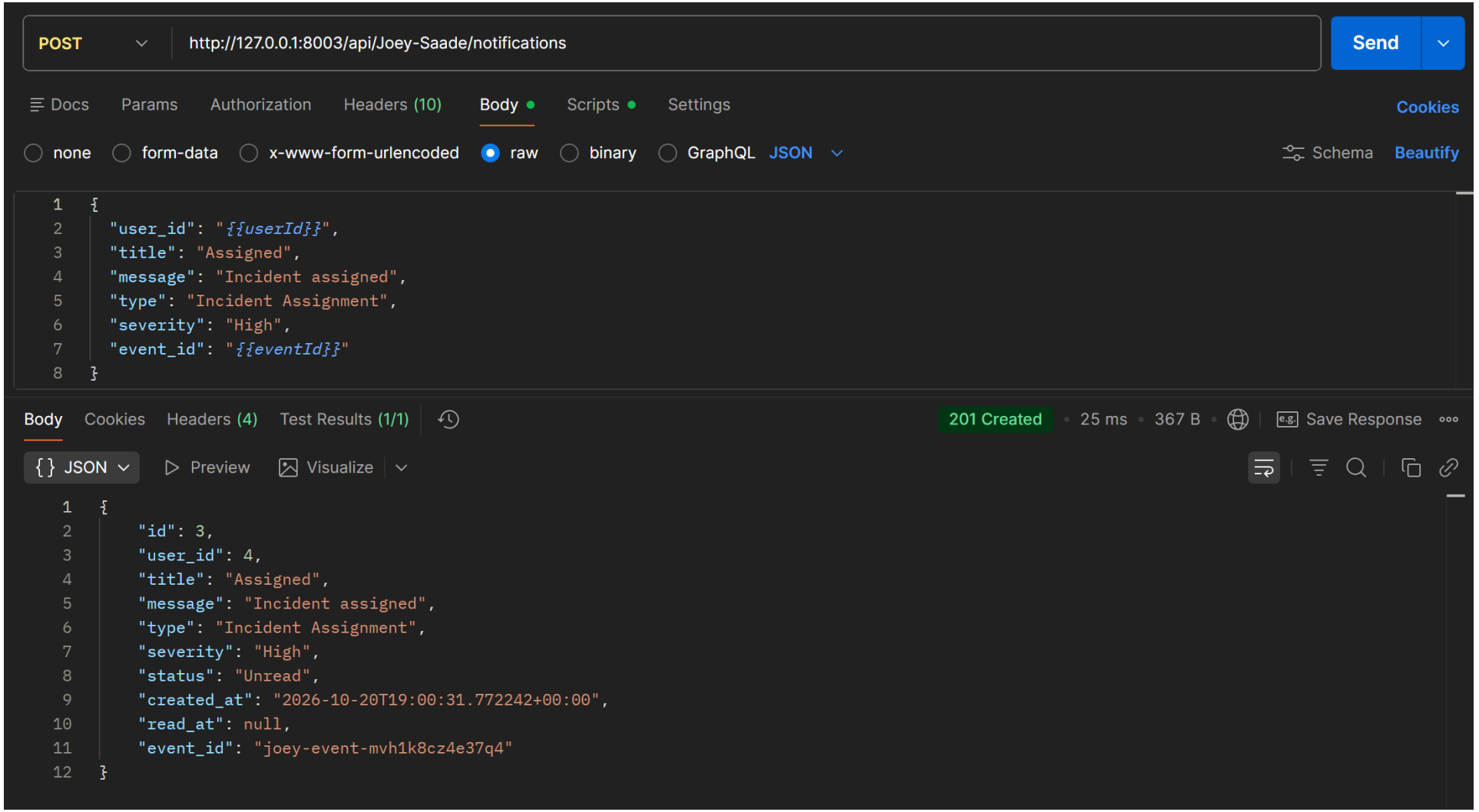

Figure 116. Joey Saade - Create notification as service - POST stores an Unread assignment notification for user 4 with HTTP 201.

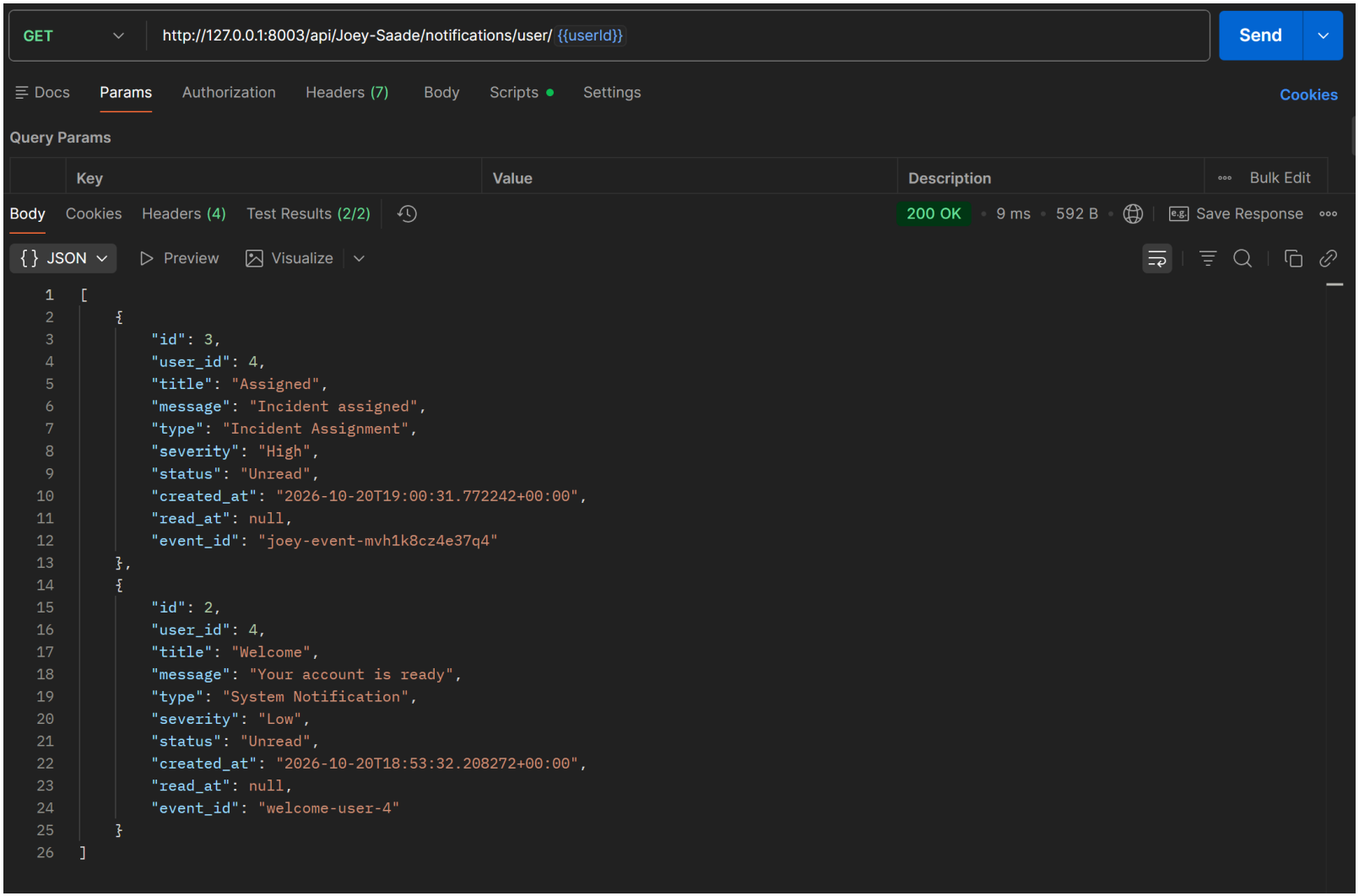

Figure 117. Joey Saade - List user notifications - GET includes the automatic Welcome record.

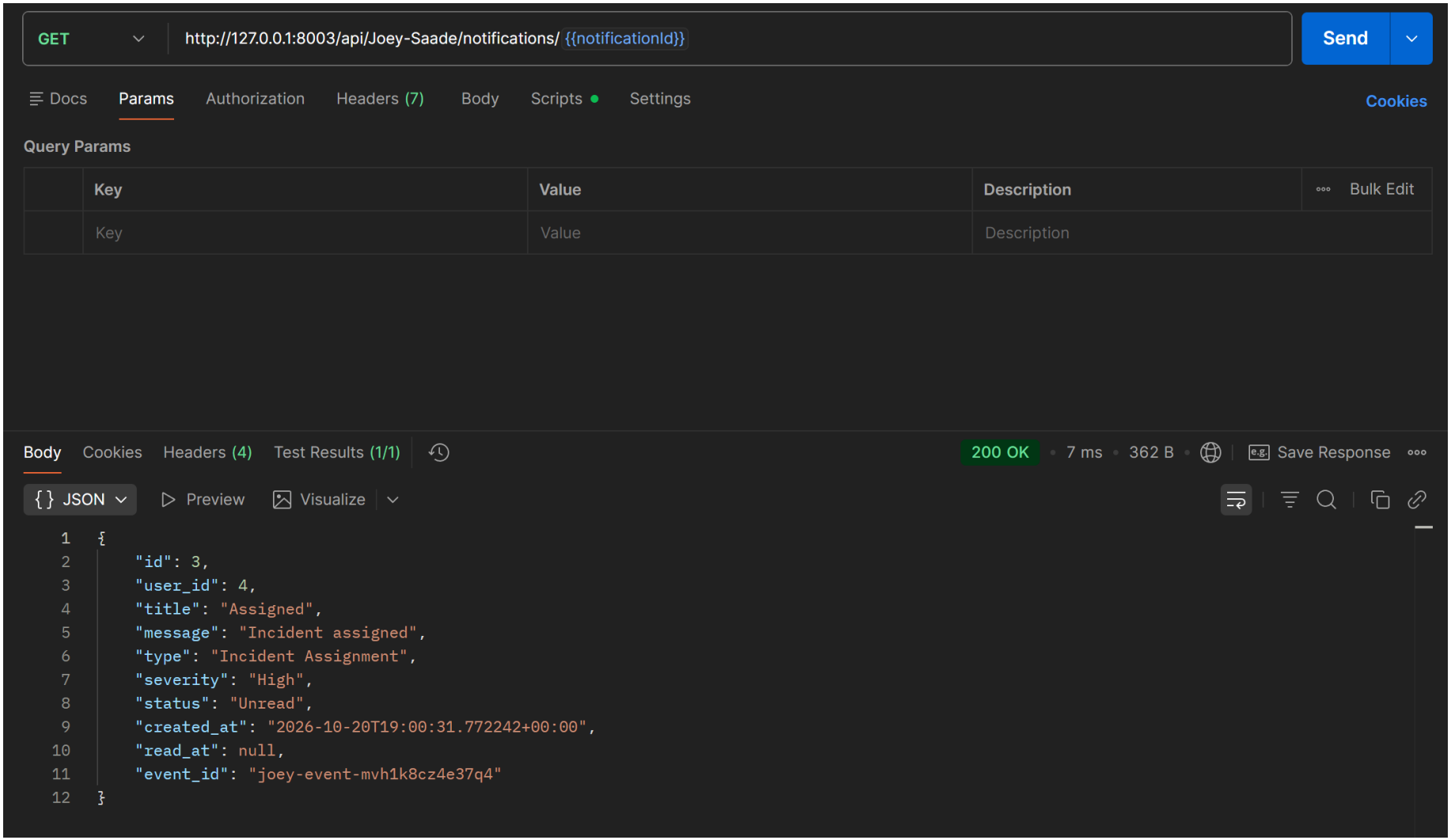

Figure 118. Joey Saade - Get notification - GET returns the created assignment.

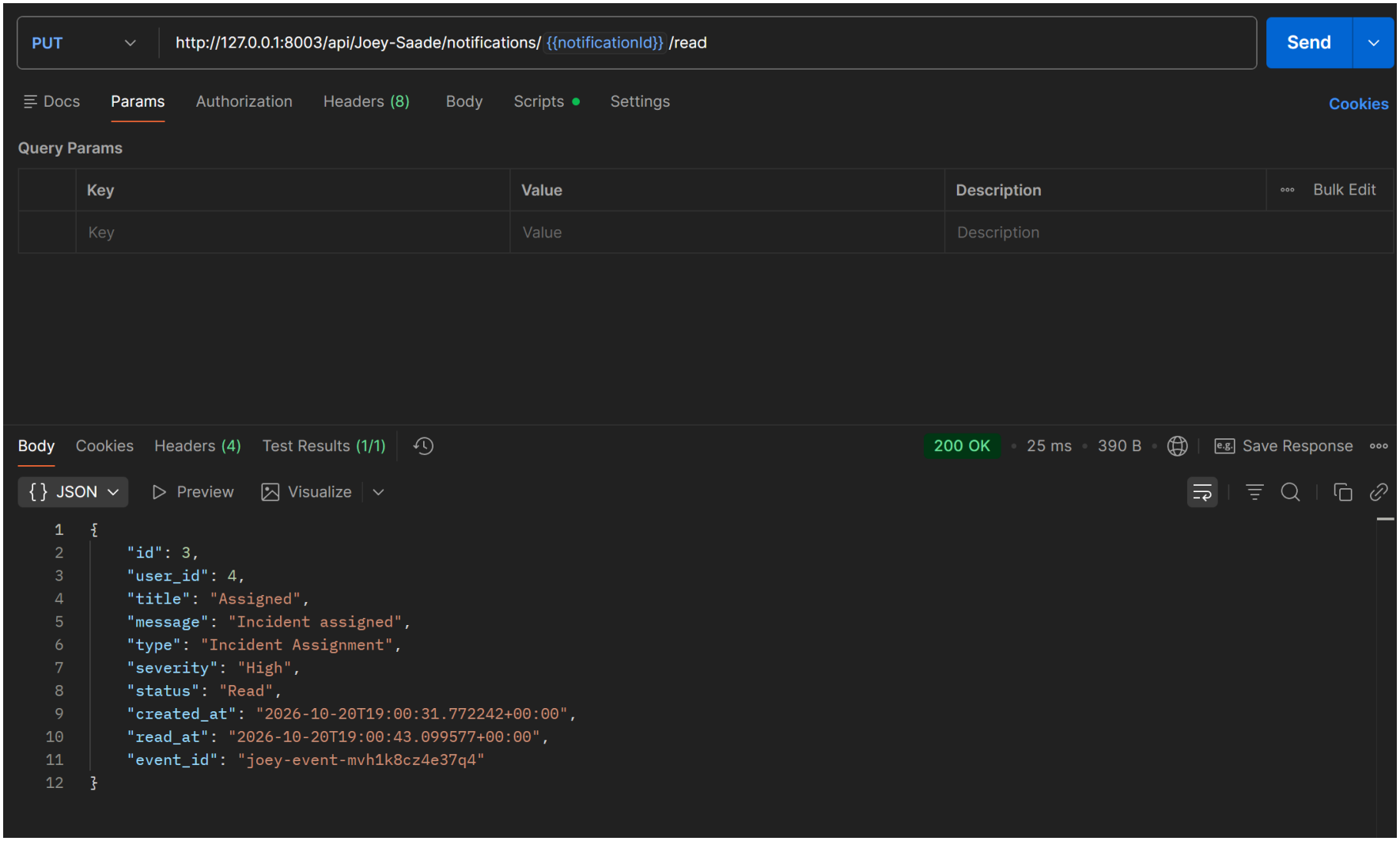

Figure 119. Joey Saade - Mark notification read - PUT updates its status.

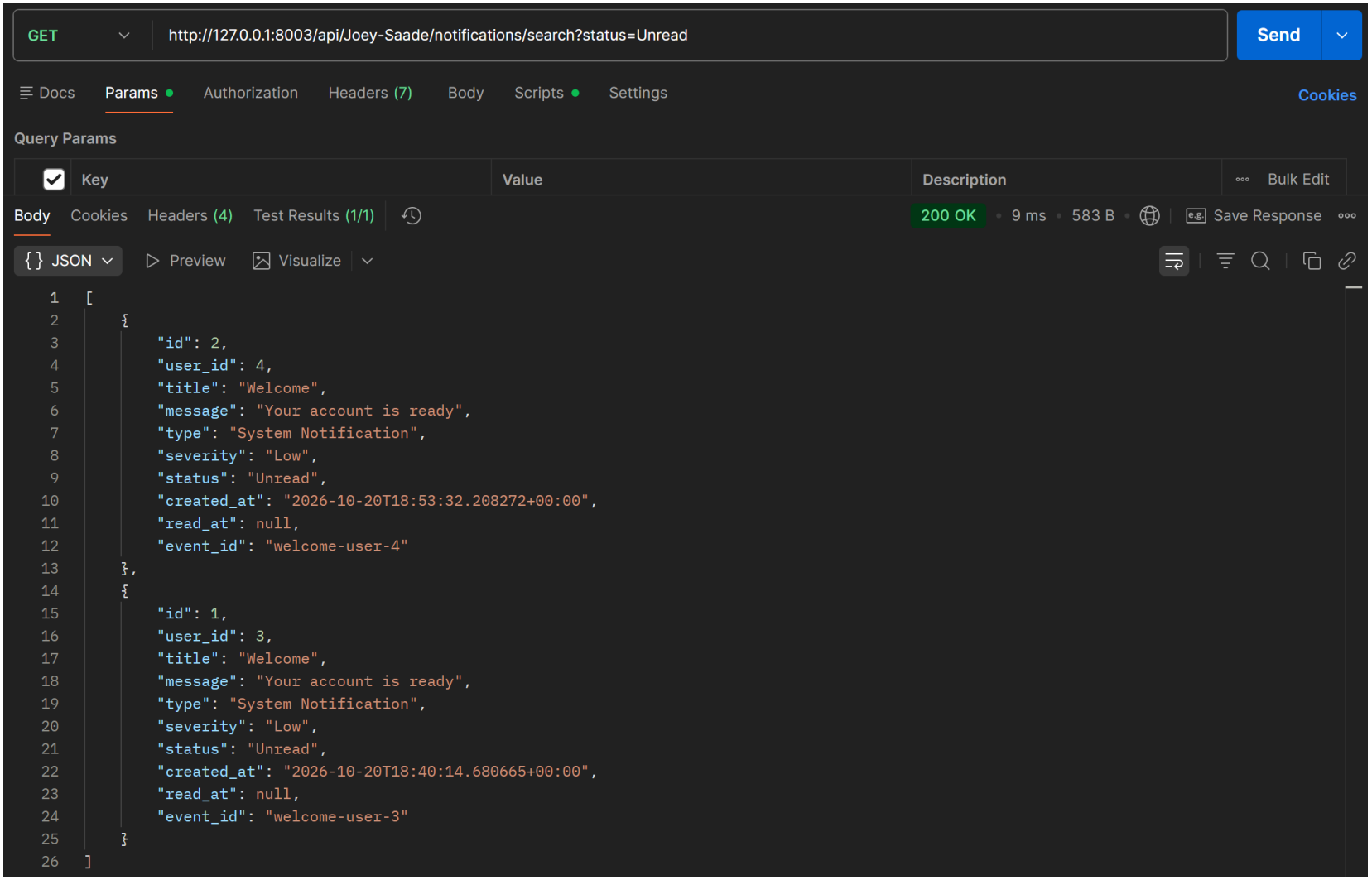

Figure 120. Joey Saade - Search notifications - GET filters unread records.

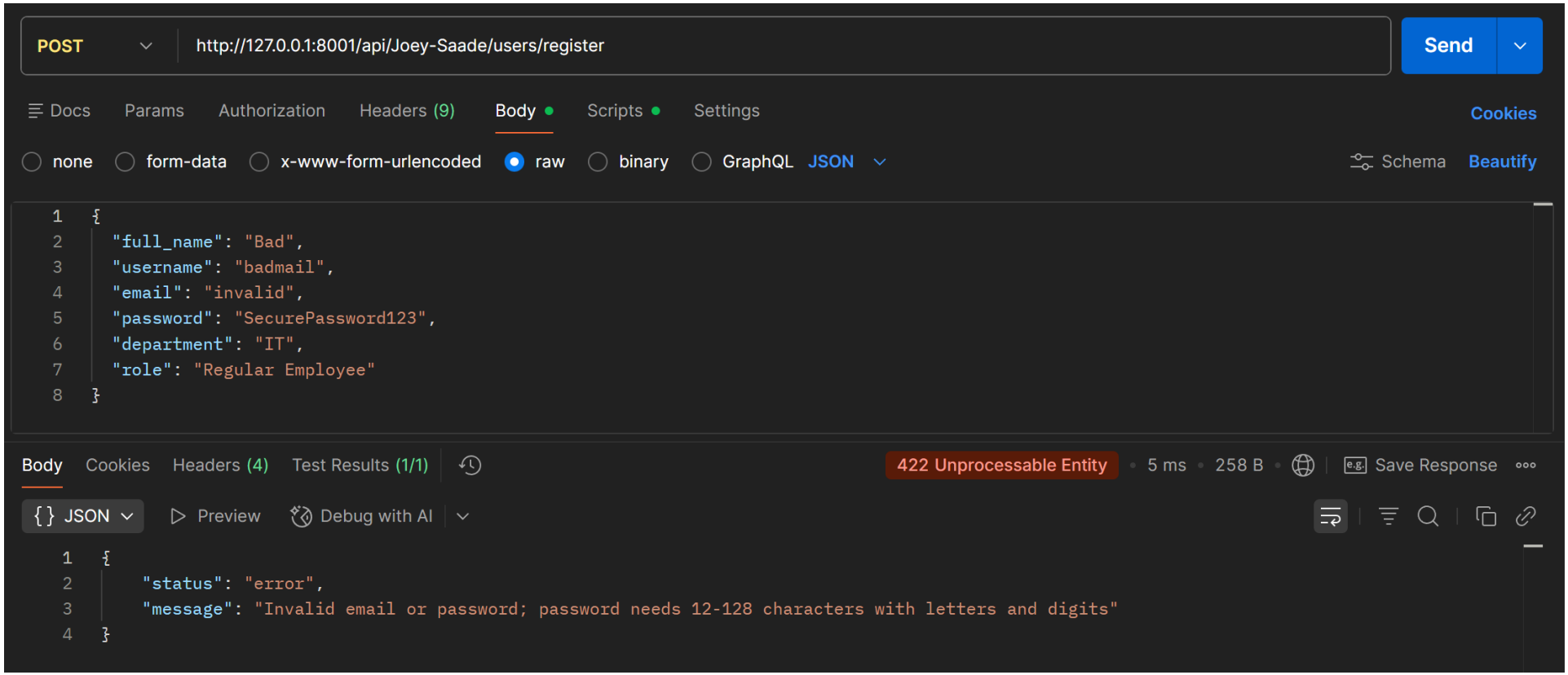

Figure 121. Joey Saade - Invalid registration email - POST returns validation error 422.

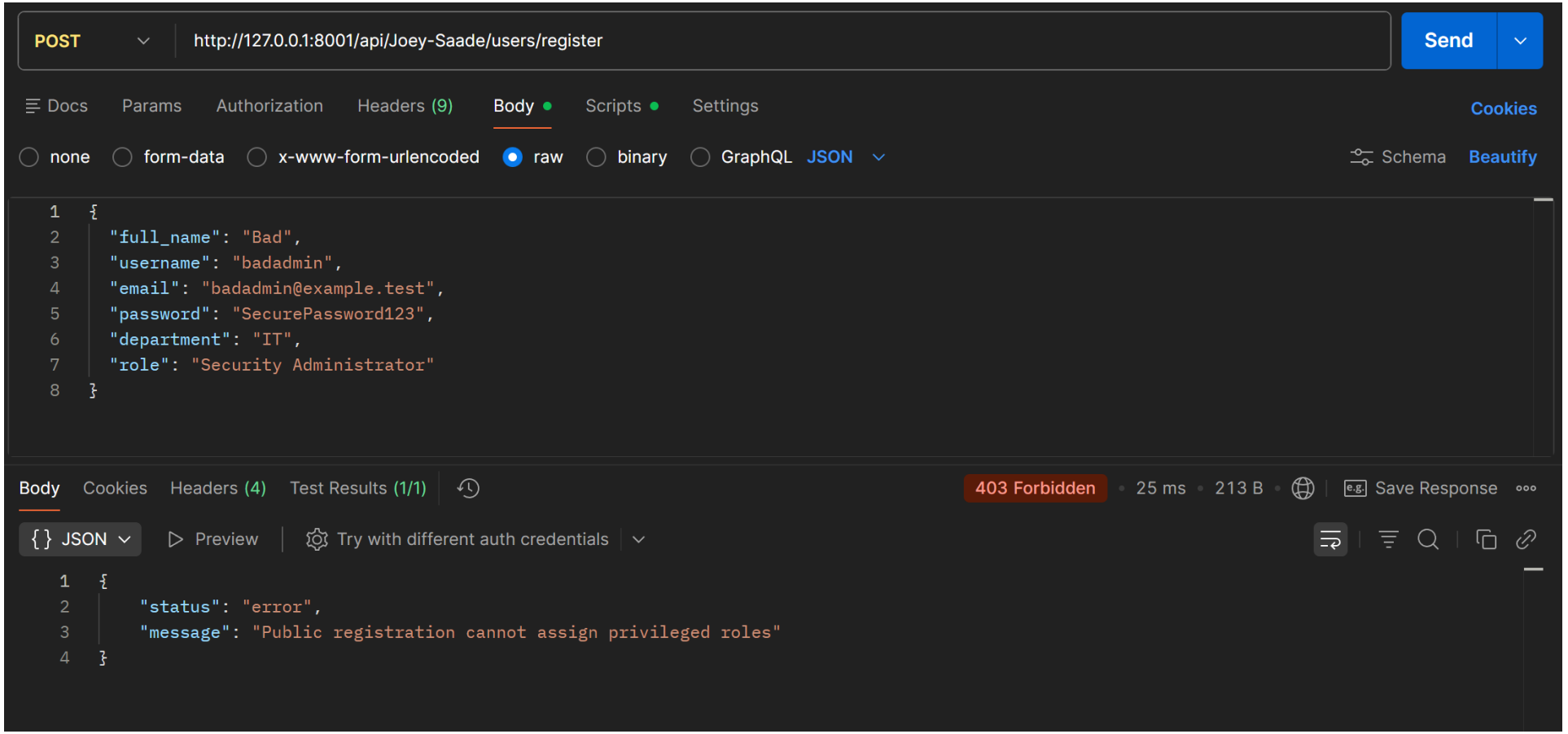

Figure 122. Joey Saade - Privileged public registration - POST rejects role escalation with 403.

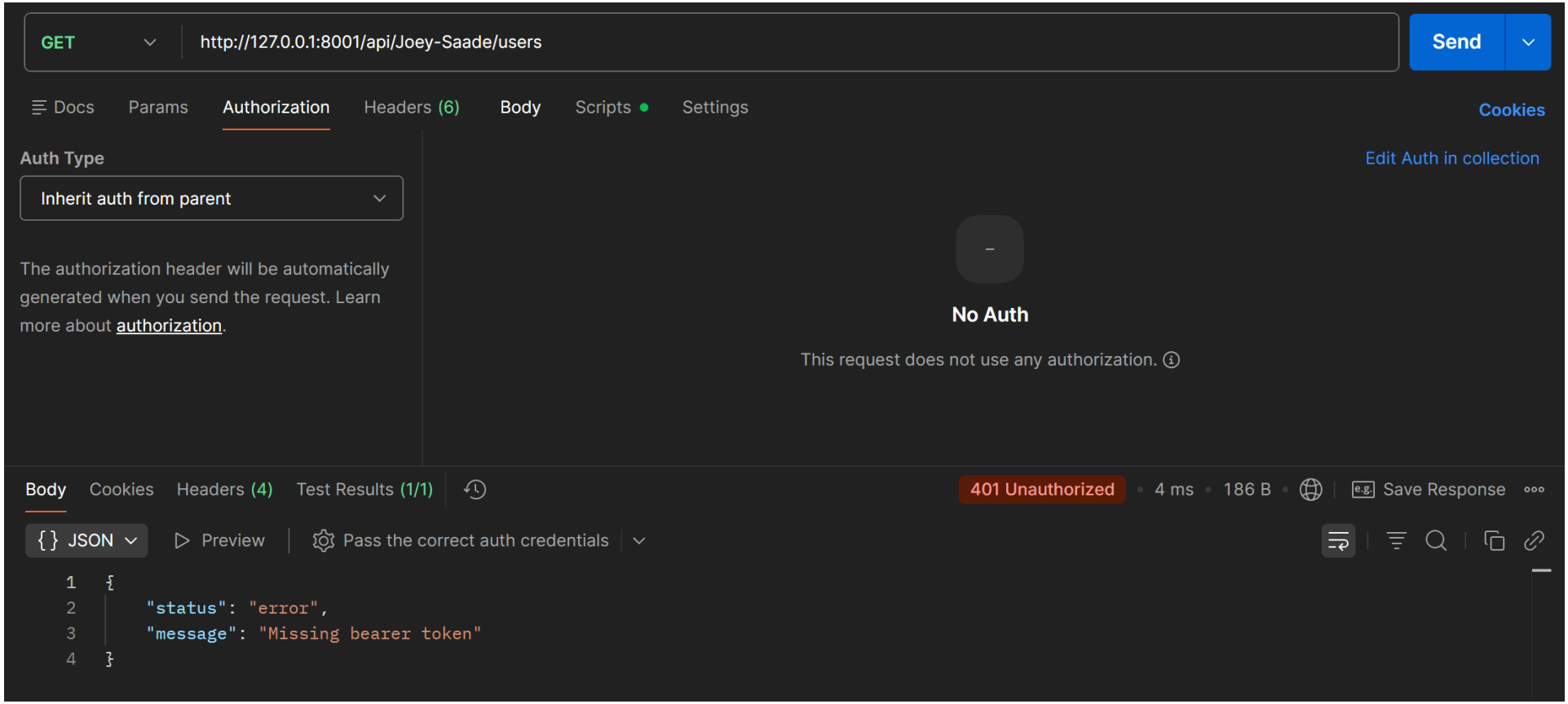

Figure 123. Joey Saade - Users listing without authentication - GET returns 401.

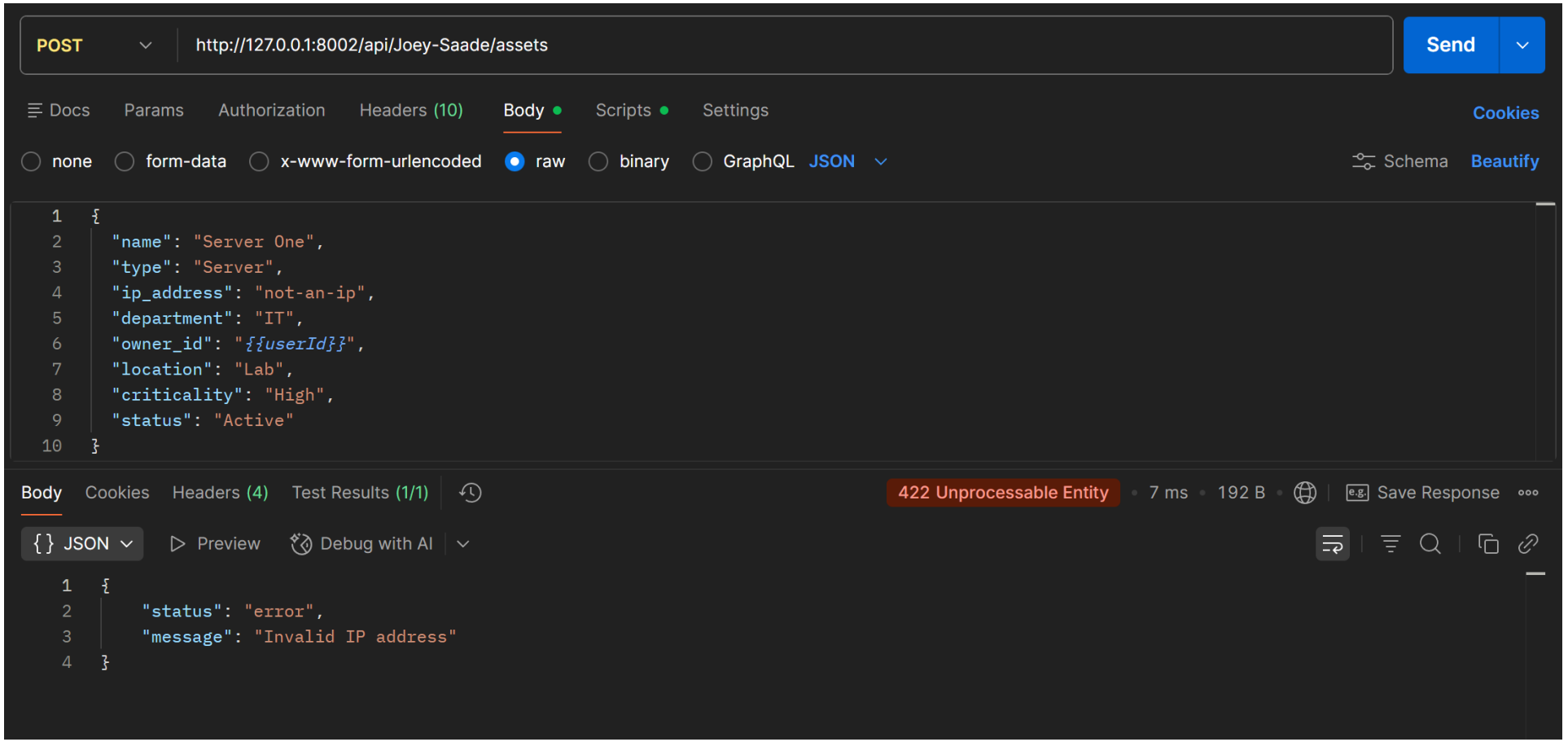

Figure 124. Joey Saade - Invalid asset IP - POST returns validation error 422.

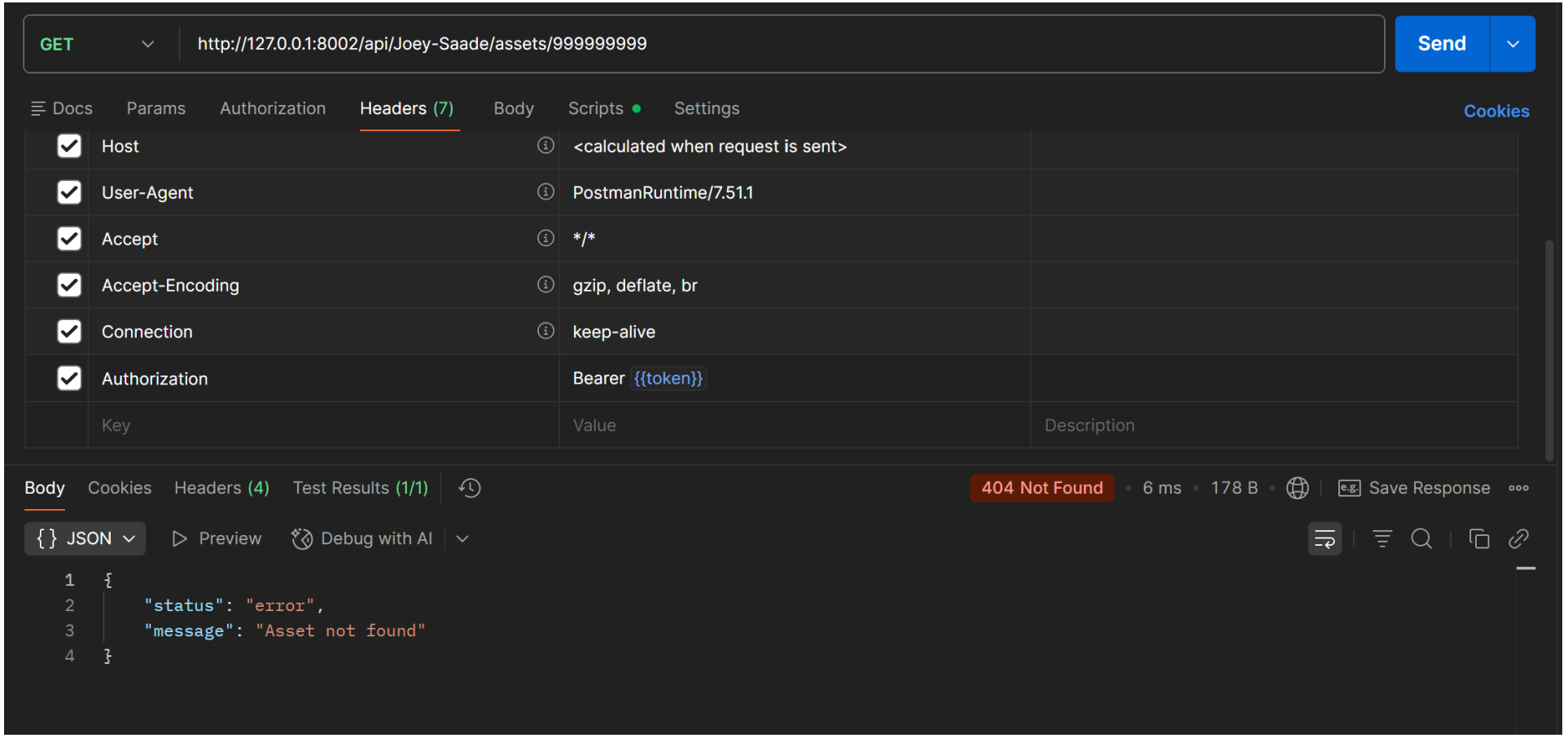

Figure 125. Joey Saade - Missing asset - GET returns 404.

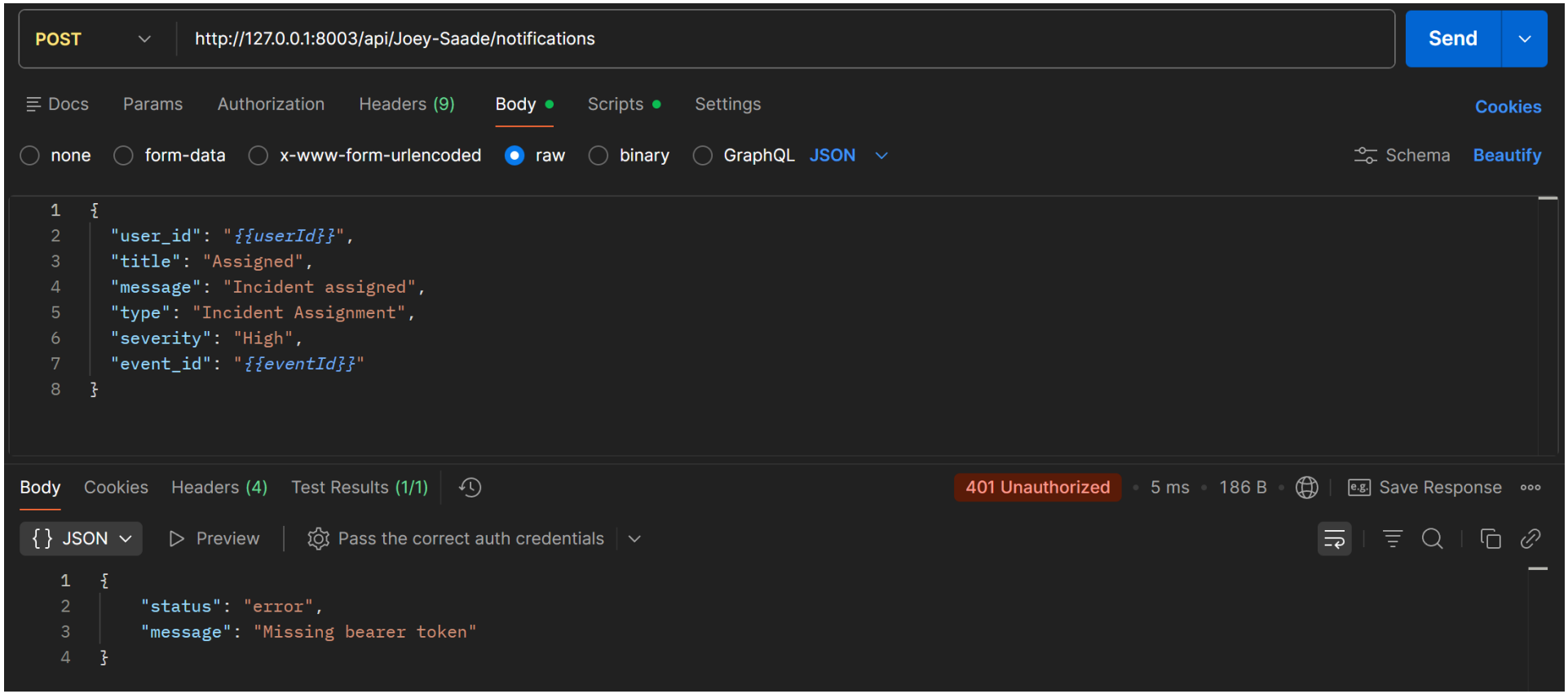

Figure 126. Joey Saade - Notification without authentication - POST returns 401.

```
sqlite> SELECT id, username, role, status FROM users ORDER BY id;
id  username                        role                      status
--  ------------------------------  ------------------------  ------
1   joey_admin                      Security Administrator    Active
2   joey_employee_mvgow1rjhci8gw    Security Analyst          Active
```

Figure 127. Joey Saade - Users database records - Selected columns show the administrator and sample account without displaying password hashes.

```
sqlite> SELECT id, name, owner_id, status FROM assets;
id  name        owner_id  status
--  ----------  --------  -----------------
1   Server One  2         Under Maintenance
sqlite> SELECT original_asset_id, action FROM asset_history ORDER BY id;
original_asset_id  action
-----------------  ------
1                  create
1                  update
sqlite>
```

Figure 128. Joey Saade - Asset records and history - The sample asset belongs to the registered user and has create/update history.

```
sqlite> SELECT id, user_id, title, type, status, event_id FROM notifications ORDER BY id;
id  user_id  title     type                 status  event_id
--  -------  --------  -------------------  ------  ------------------------
1   2        Welcome   System Notification  Unread  welcome-user-2
2   2        Assigned  Incident Assignment  Read    joey-event-mvgow1rjhci8gw
sqlite>
```

Figure 129. Joey Saade - Notification records - The sample user has Welcome and incident-assignment notifications with event IDs.

```
sqlite> SELECT id, destination, event_id, attempts, delivered_at FROM outbox ORDER BY id DESC LIMIT 10;
id  destination  event_id                              attempts  delivered_at
--  -----------  ------------------------------------  --------  ------------
11  audit        de13ee88-47f8-4290-9b7e-4171f7a70947  0
10  audit        cf17cf9f-6db1-44a0-84b3-fb8662721582  0
9   audit        5b2a1674-2c51-4cb3-a387-77025cafcb45  0
8   audit        0117c9c0-1491-464a-a008-d4ab41bb4153  0
7   audit        89803fe8-3077-4d22-a774-ab3cca49f7dc  0
6   audit        59137ede-4a1f-4440-a07f-00ac11873d03  0
5   audit        2bf84054-27eb-45d1-9096-0845655e9ed1  0
4   audit        8db4424e-e4f0-4200-bab5-f61e96592378  0
3   audit        bb043804-96d4-4342-ba17-5dc0eedaf3b3  0
2   audit        1486dca0-4755-4b01-8f47-04c3e240fc94  1
sqlite> SELECT id FROM user_id_allocations ORDER BY id;
id
--
1
2
sqlite>
```

Figure 130. Joey Saade - Queued events and issued user IDs - The query shows queued audit events, their delivery fields and the retained user ID allocations.

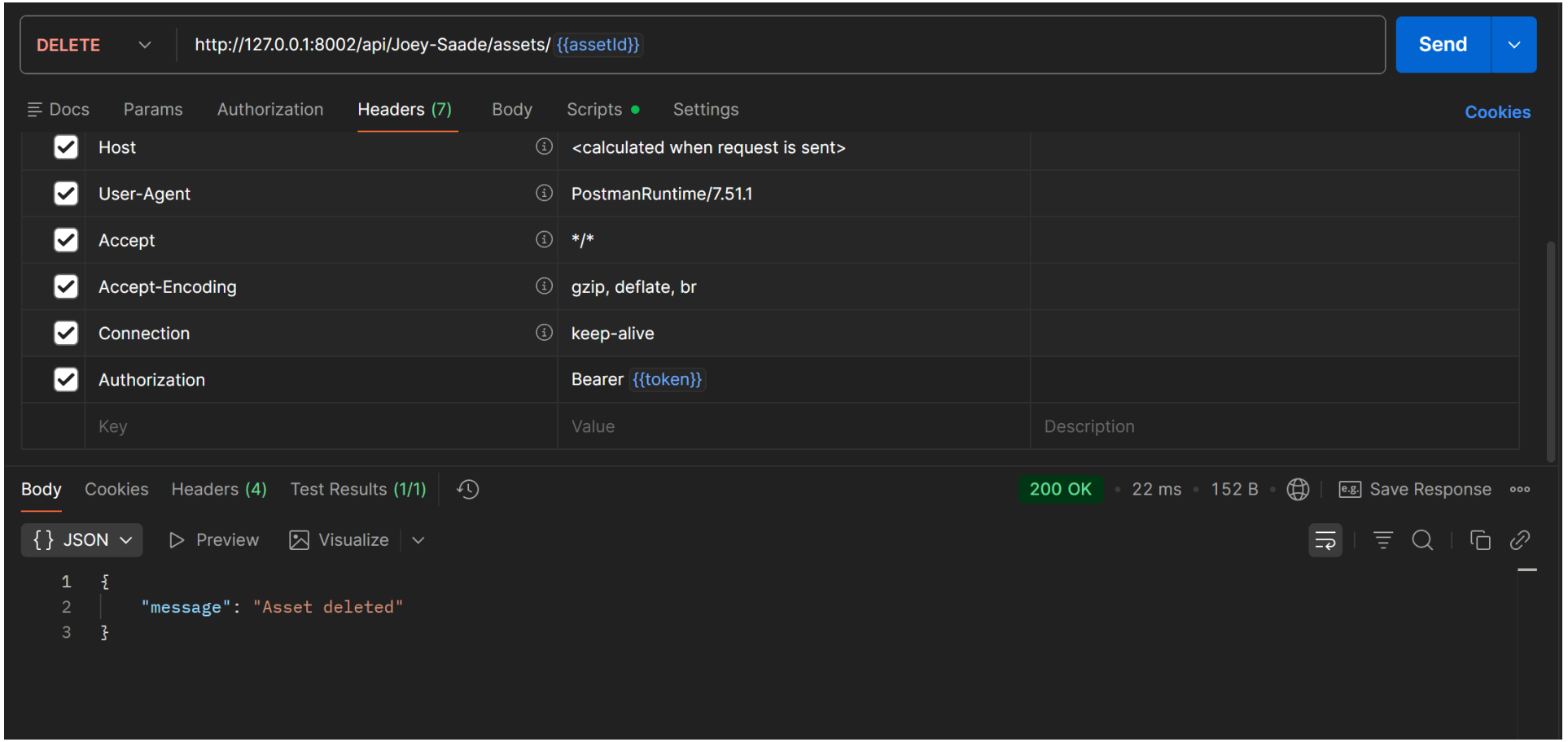

Figure 131. Joey Saade - Delete asset - DELETE succeeds before user deletion.

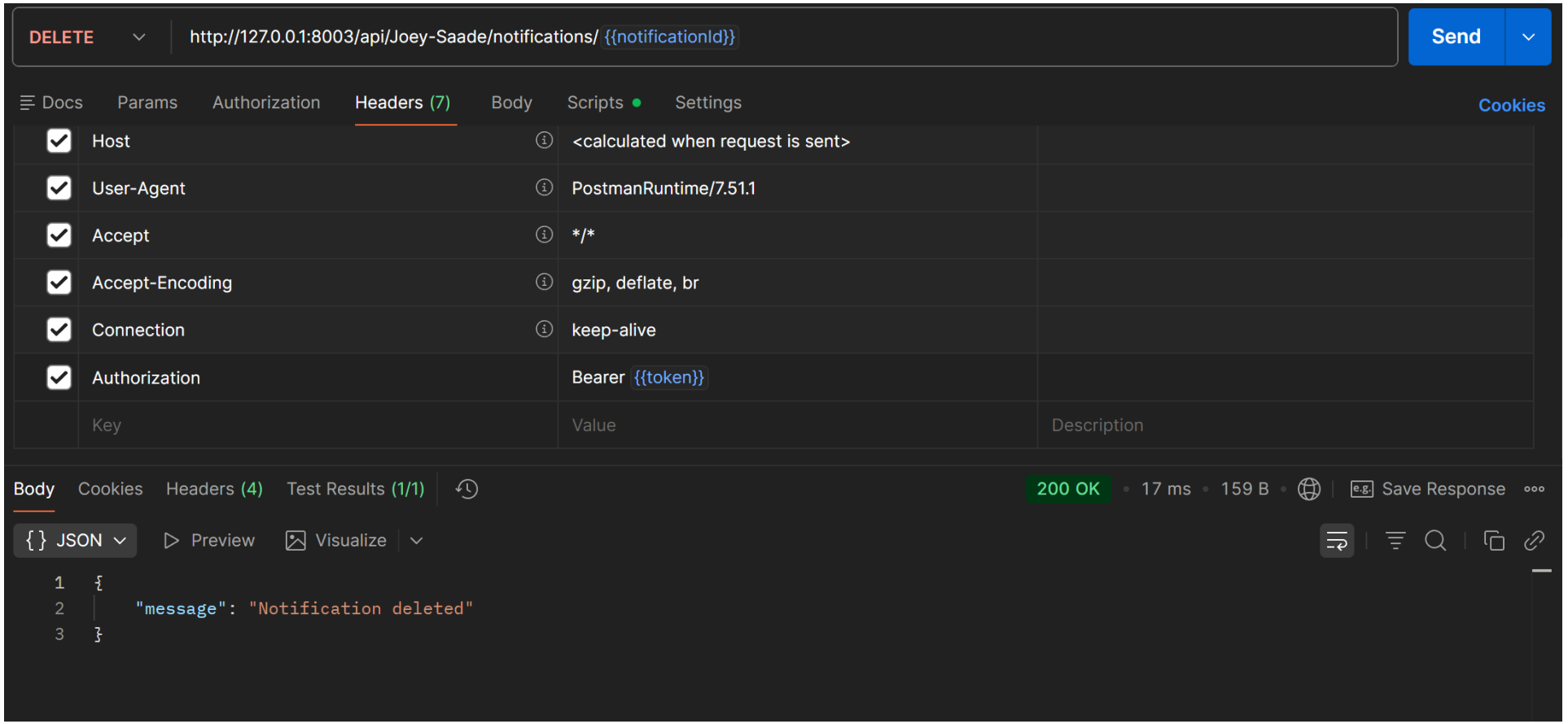

Figure 132. Joey Saade - Delete created notification - DELETE removes the assignment.

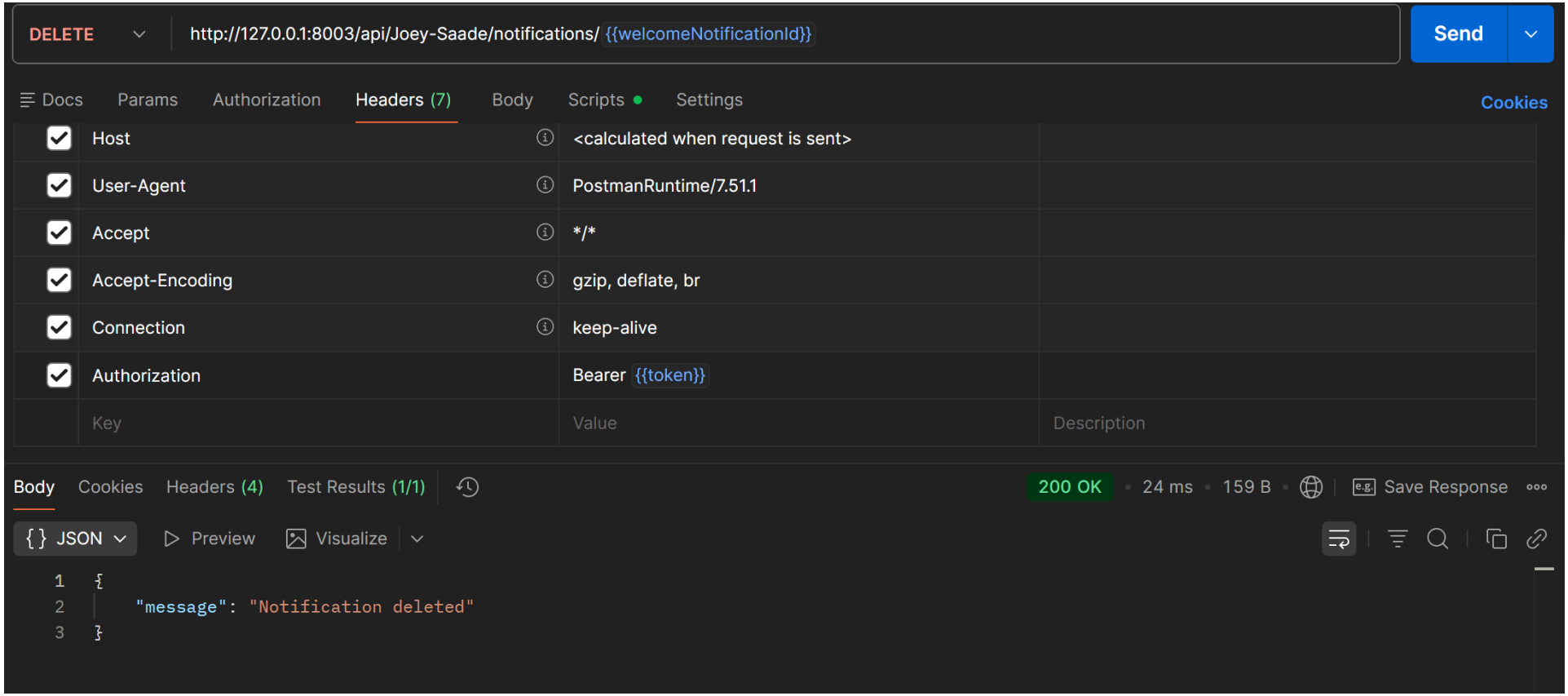

Figure 133. Joey Saade - Delete welcome notification - DELETE removes the user's remaining reference.

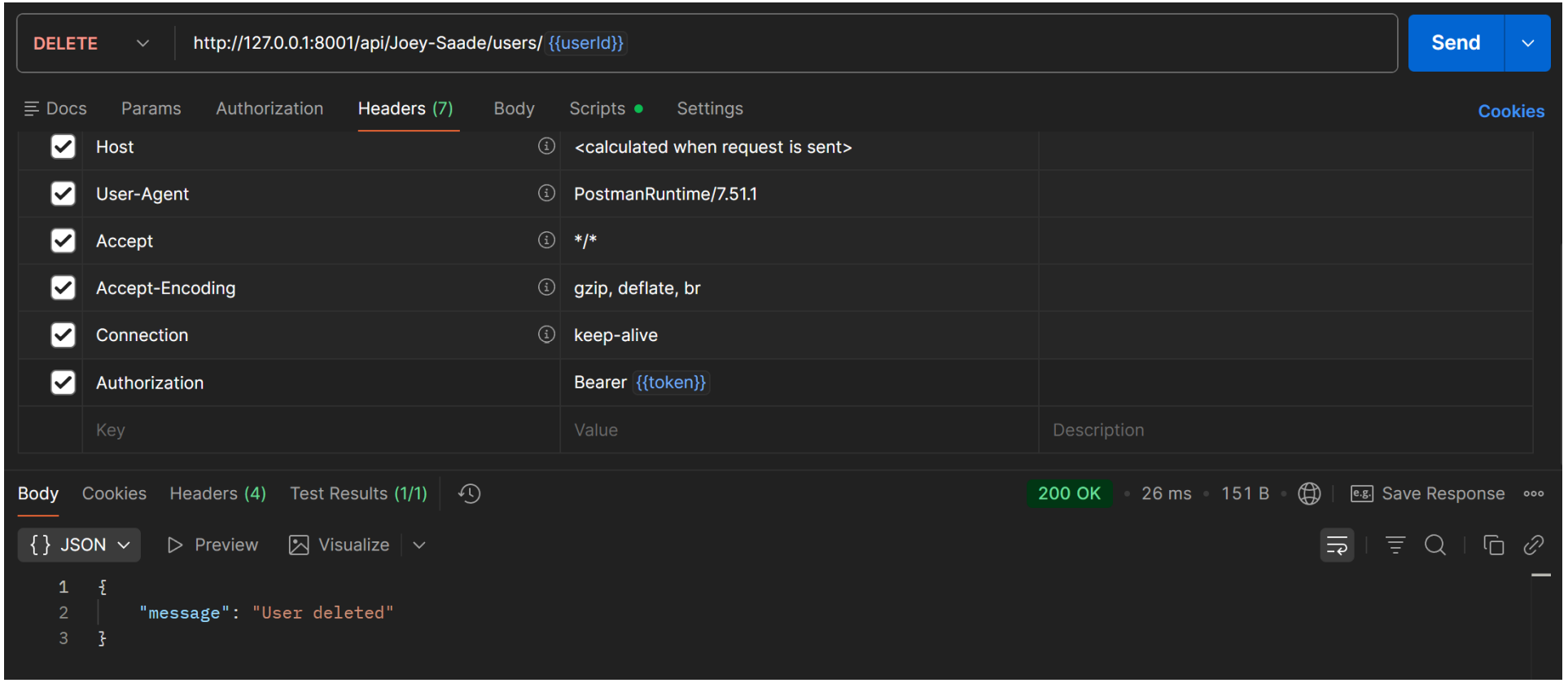

Figure 134. Joey Saade - Delete user - DELETE succeeds after dependent records are removed.

```
sqlite> SELECT id, username FROM users ORDER BY id;
id  username
--  ----------
1   joey_admin
sqlite> SELECT id, name FROM assets ORDER BY id;
sqlite> SELECT original_asset_id, action FROM asset_history ORDER BY id;
original_asset_id  action
-----------------  ------
1                  create
1                  update
1                  delete
sqlite> SELECT id FROM user_id_allocations ORDER BY id;
id
--
1
2
sqlite>
```

Figure 135. Joey Saade - Database after Cleanup - Only the administrator remains; asset history and allocated IDs are retained.

C Postman and Database Evidence (Karl Khouri)

Karl's 50 captures cover Audit Logging, Playbooks, Reports and database queries from his local nine-service run. The Postman images are unchanged; the database images were only trimmed of empty console space. Captures from different members describe their respective local runs; record IDs and totals should not be pooled as one shared execution.

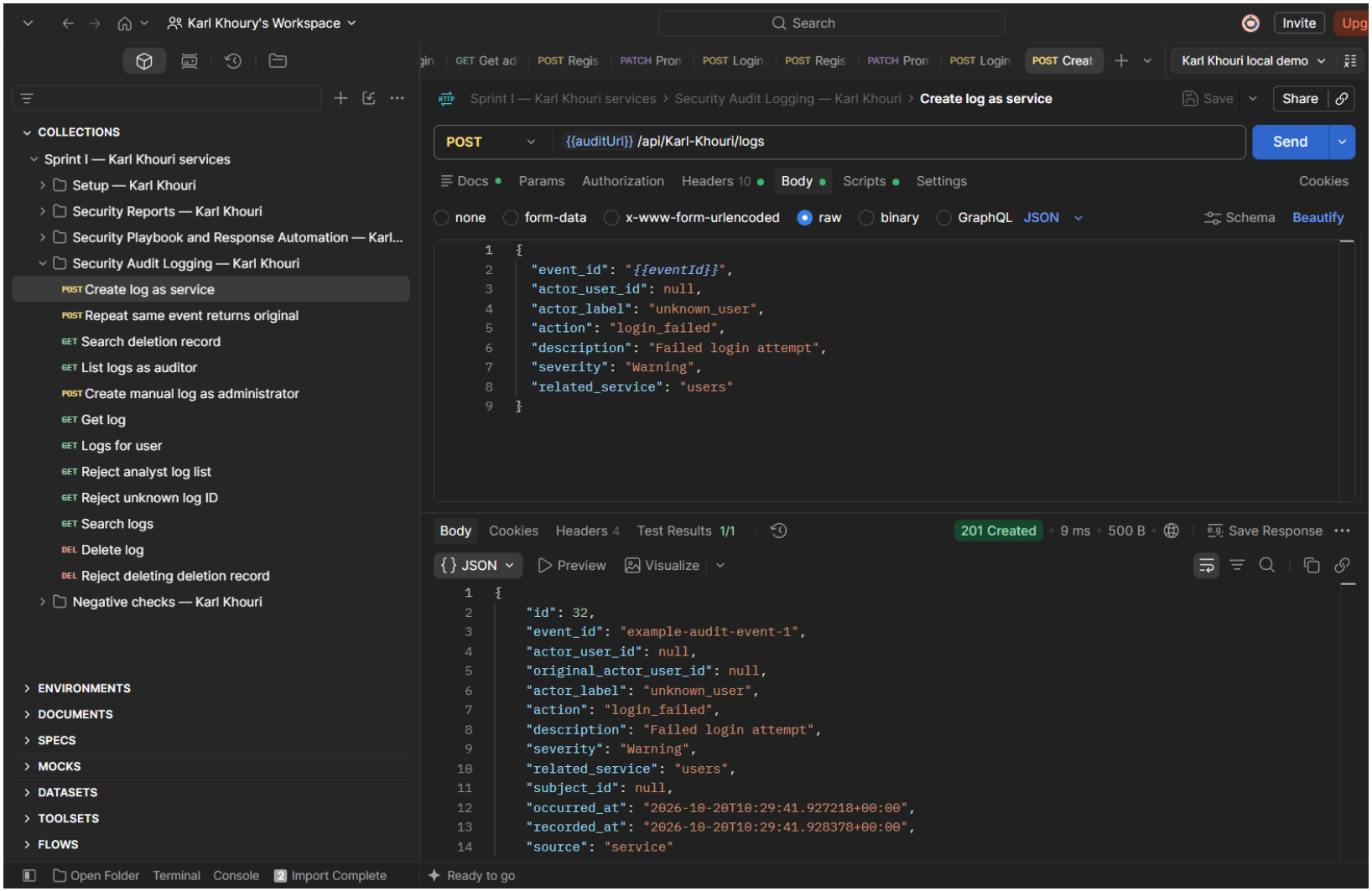

Figure 136. Karl Khouri - POST Create Log as Service - A peer service stores a failed-login event with the audit service key; HTTP 201 and the stored entry are shown.

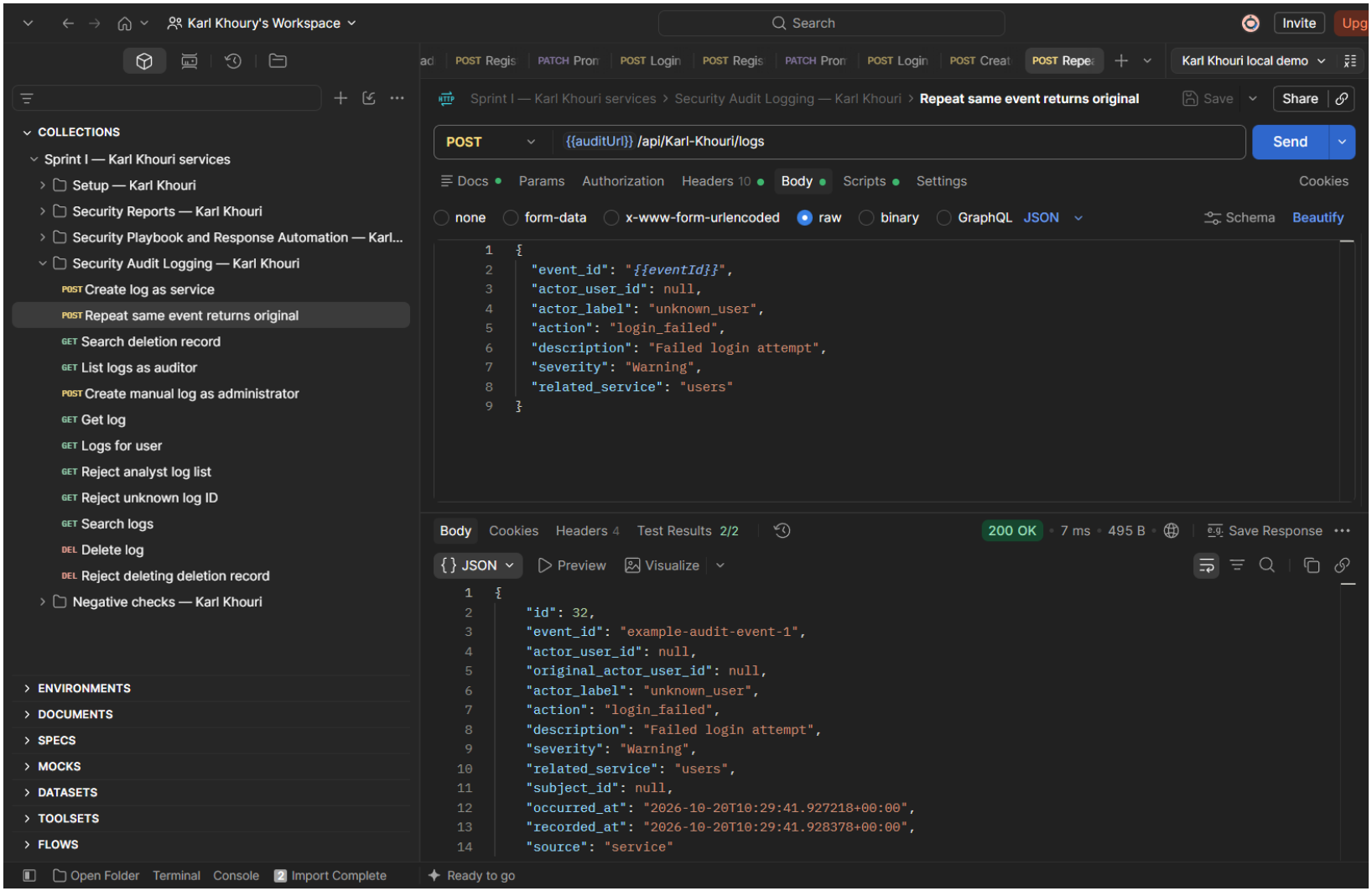

Figure 137. Karl Khouri - POST Repeat Same Event - Sending the same event ID again returns the original entry with HTTP 200 instead of a duplicate.

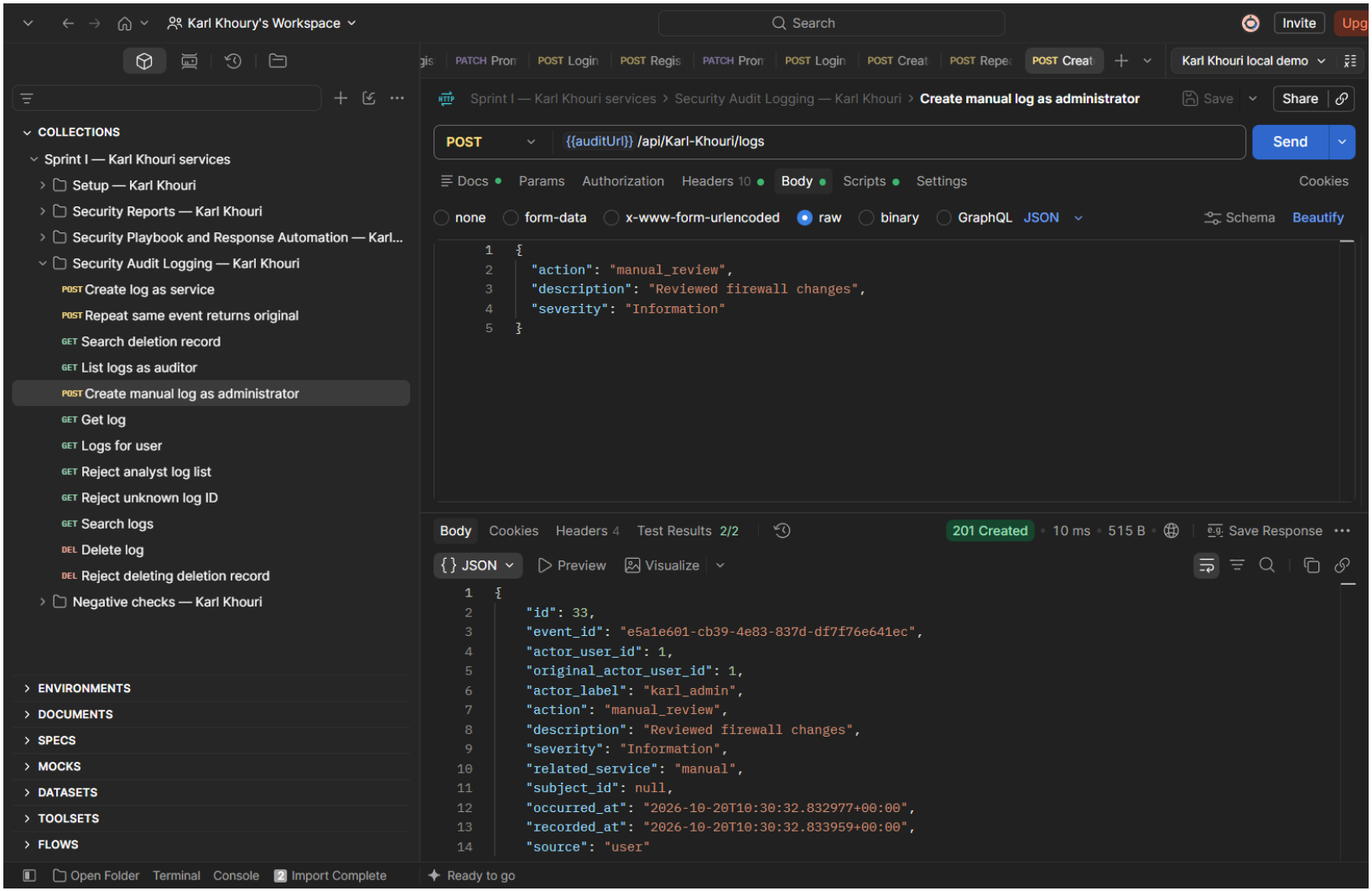

Figure 138. Karl Khouri - POST Create Manual Log as Administrator - The actor and time come from the token, shown by source user, with HTTP 201.

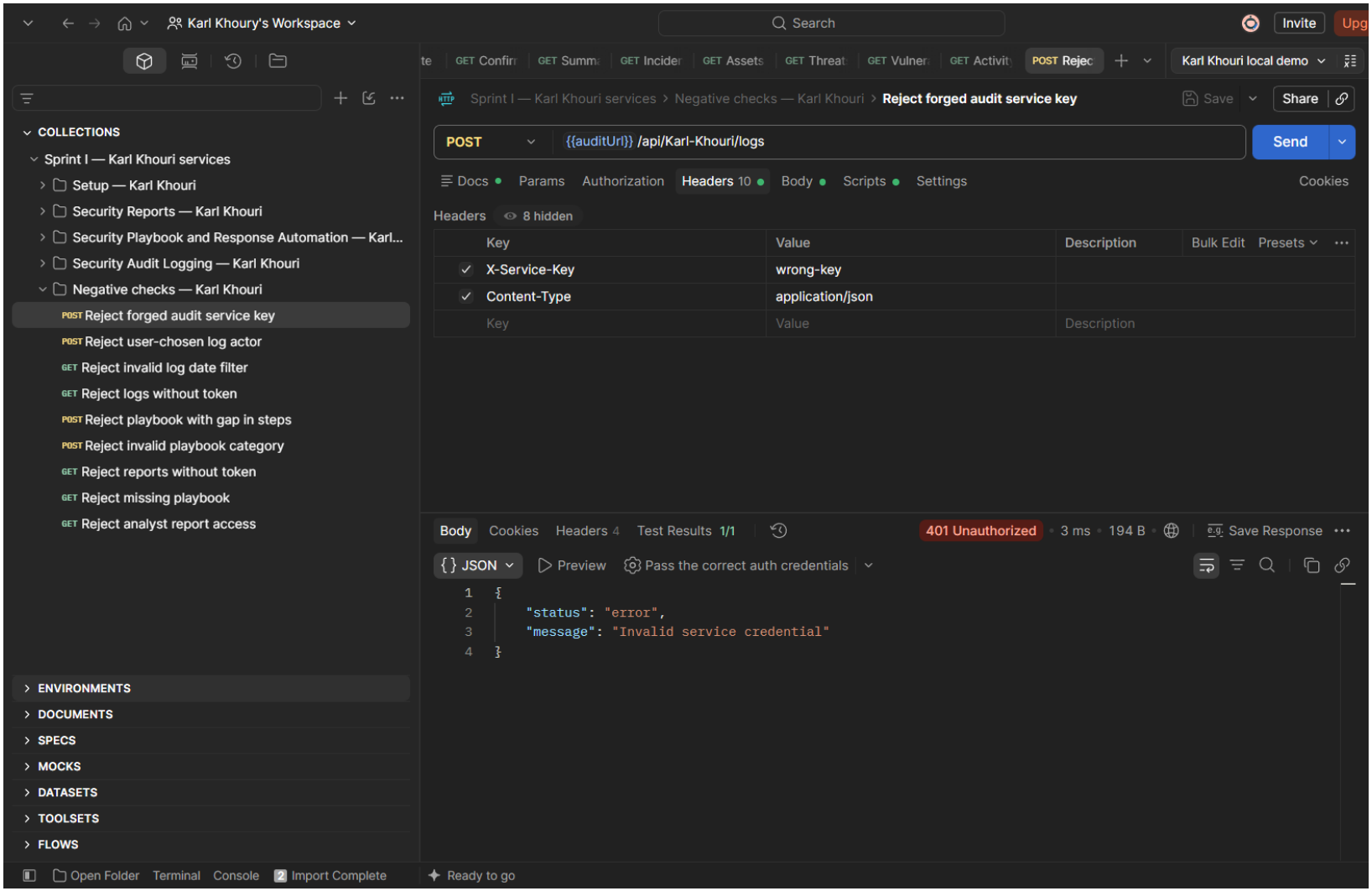

Figure 139. Karl Khouri - POST Reject Forged Service Key - The X-Service-Key header holds a wrong key (Headers tab) and the event is rejected with HTTP 401.

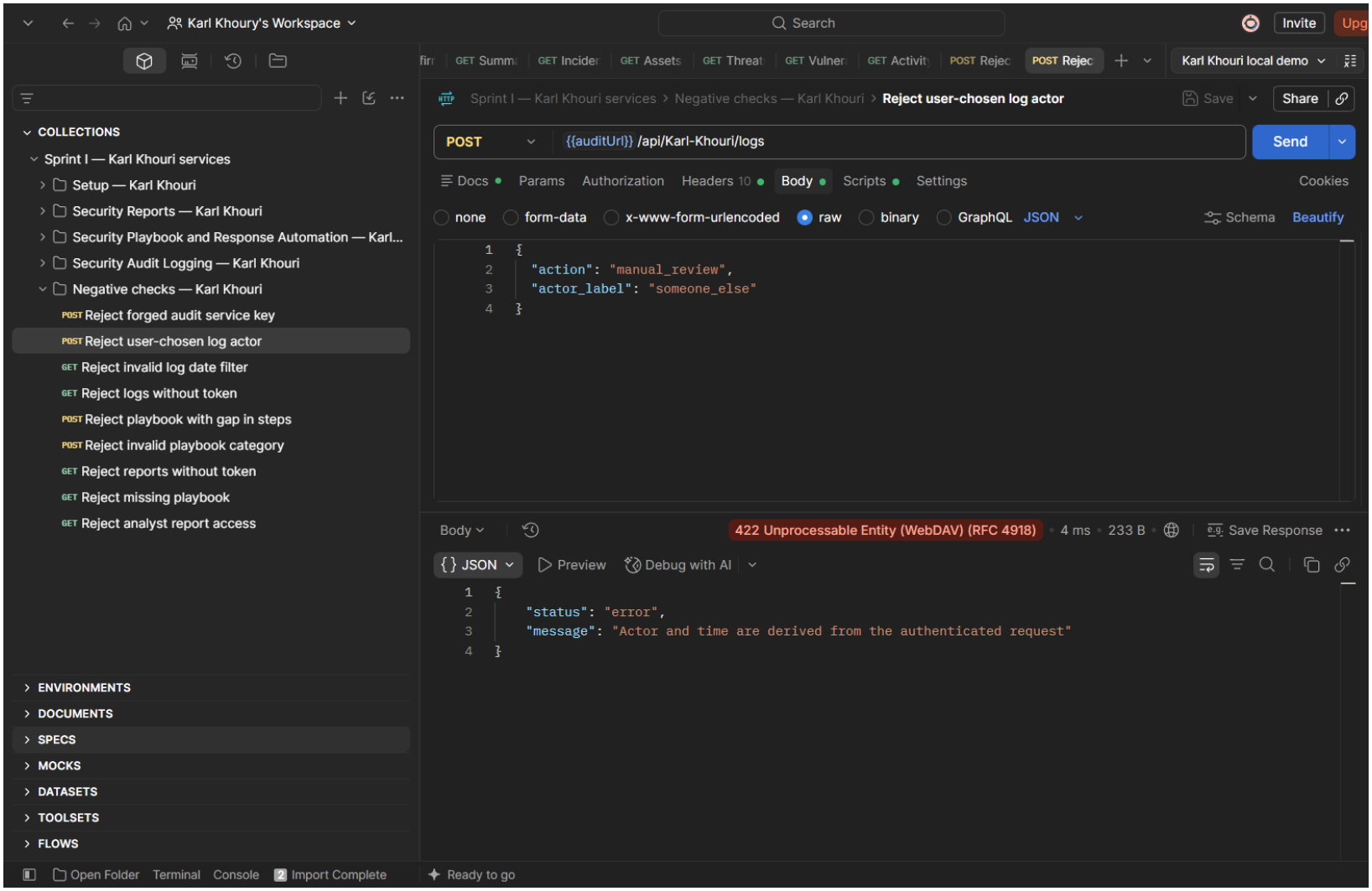

Figure 140. Karl Khouri - POST Reject User-Chosen Actor - An administrator cannot attribute an entry to someone else; HTTP 422.

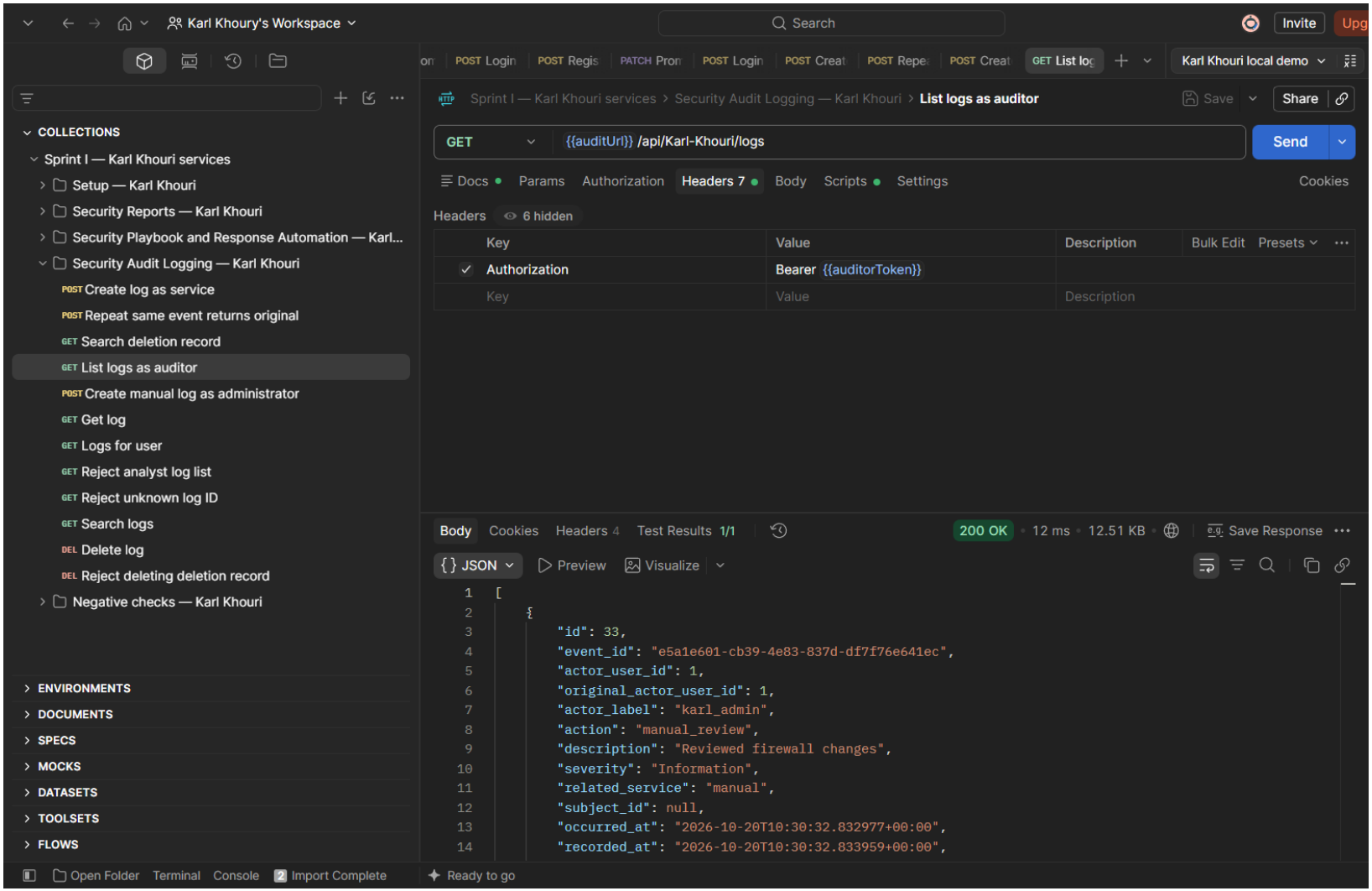

Figure 141. Karl Khouri - GET List Logs as Auditor - The request carries the auditor's token (Headers tab) and every audit entry is returned with HTTP 200.

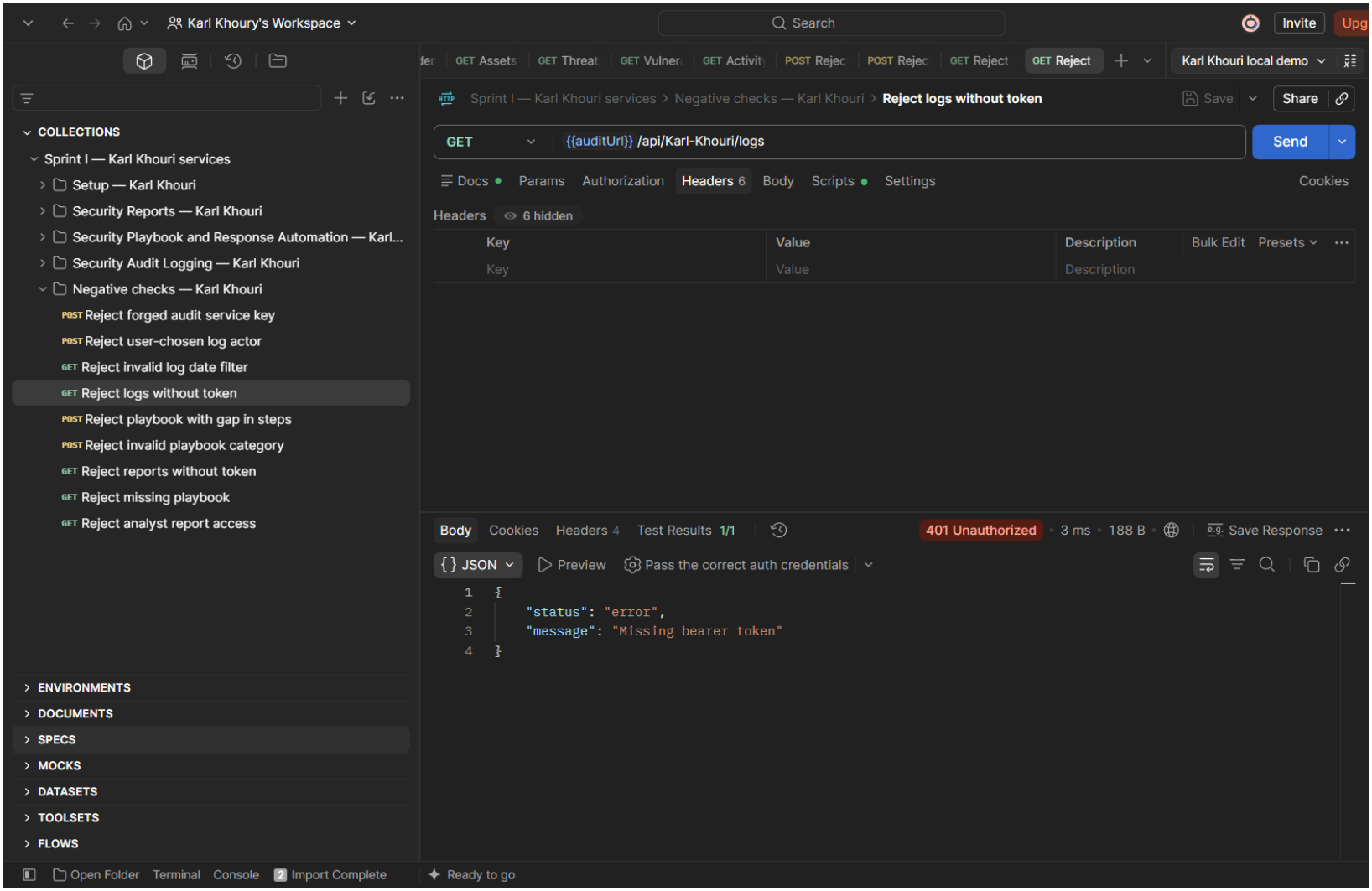

Figure 142. Karl Khouri - GET Reject Logs Without Token - An unauthenticated request is rejected with HTTP 401.

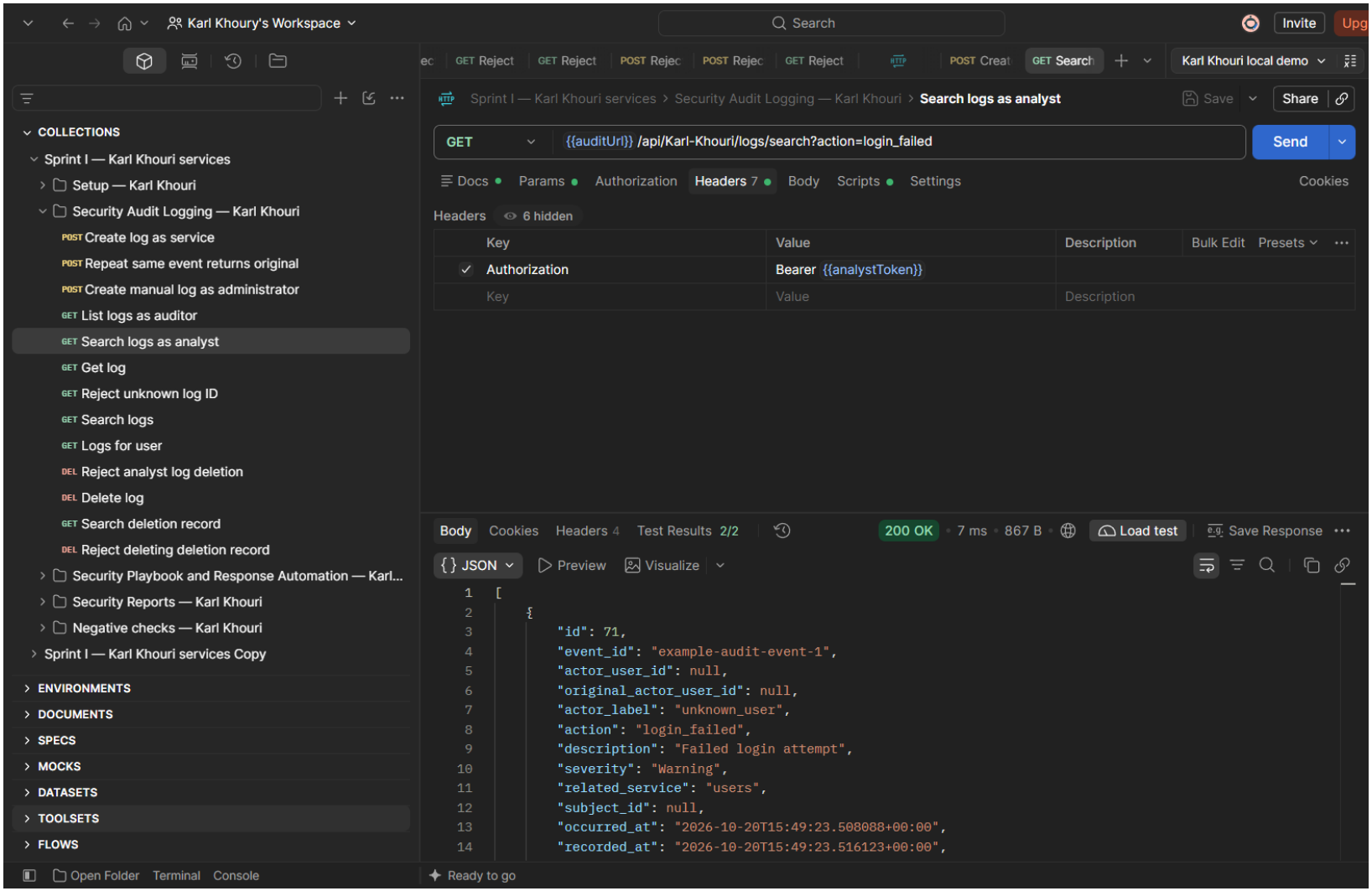

Figure 143. Karl Khouri - GET Search Logs as Analyst - With the analyst's token (Headers tab) the search succeeds with HTTP 200, as the handout allows analysts to retrieve and search audit logs.

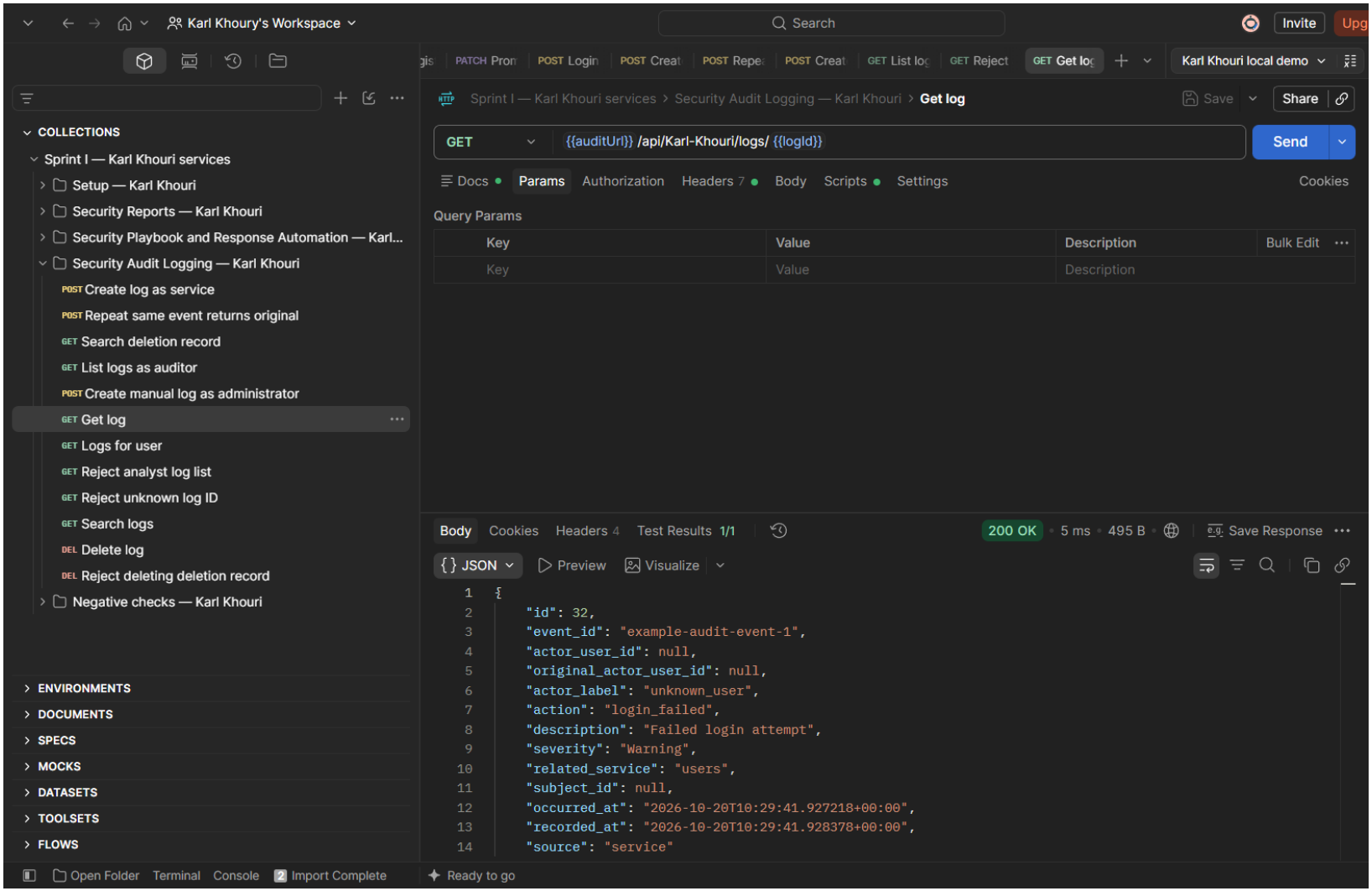

Figure 144. Karl Khouri - GET Get Log by ID - One stored entry is returned with HTTP 200.

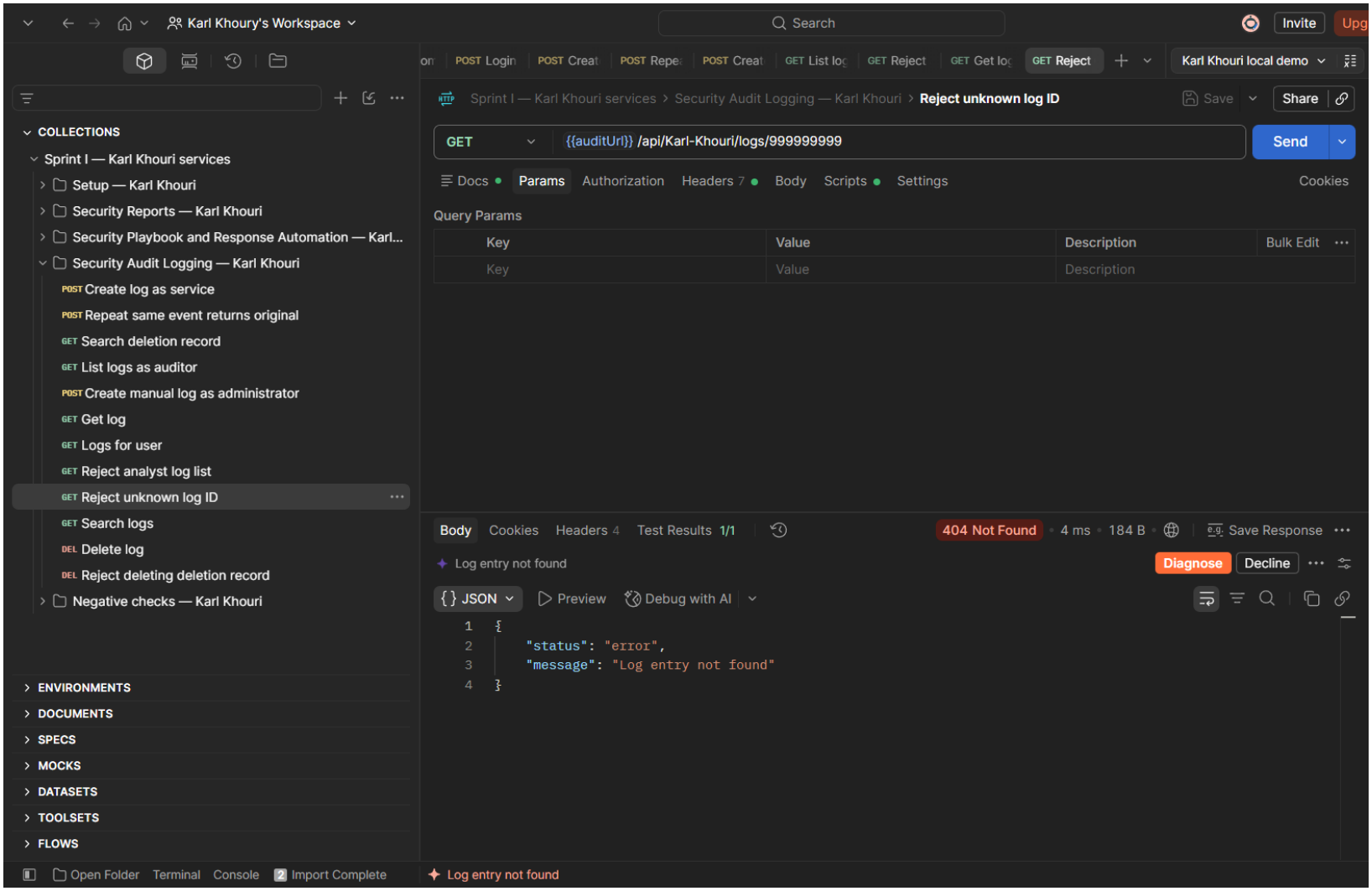

Figure 145. Karl Khouri - GET Reject Unknown Log ID - An unknown entry returns HTTP 404.

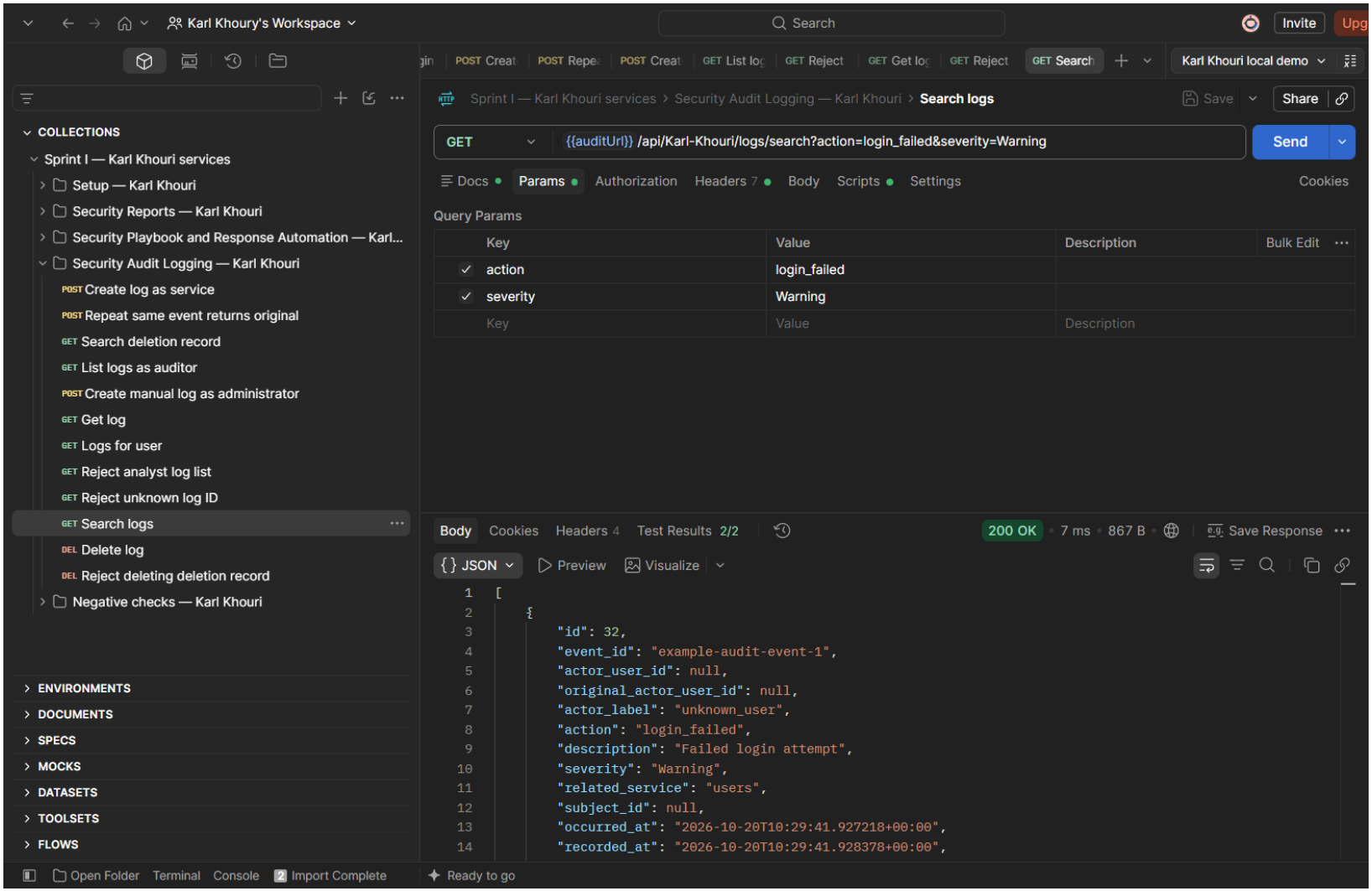

Figure 146. Karl Khouri - GET Search Logs by Action and Severity - Only failed-login warnings are returned; a failed login has a null actor ID and the attempted username as label.

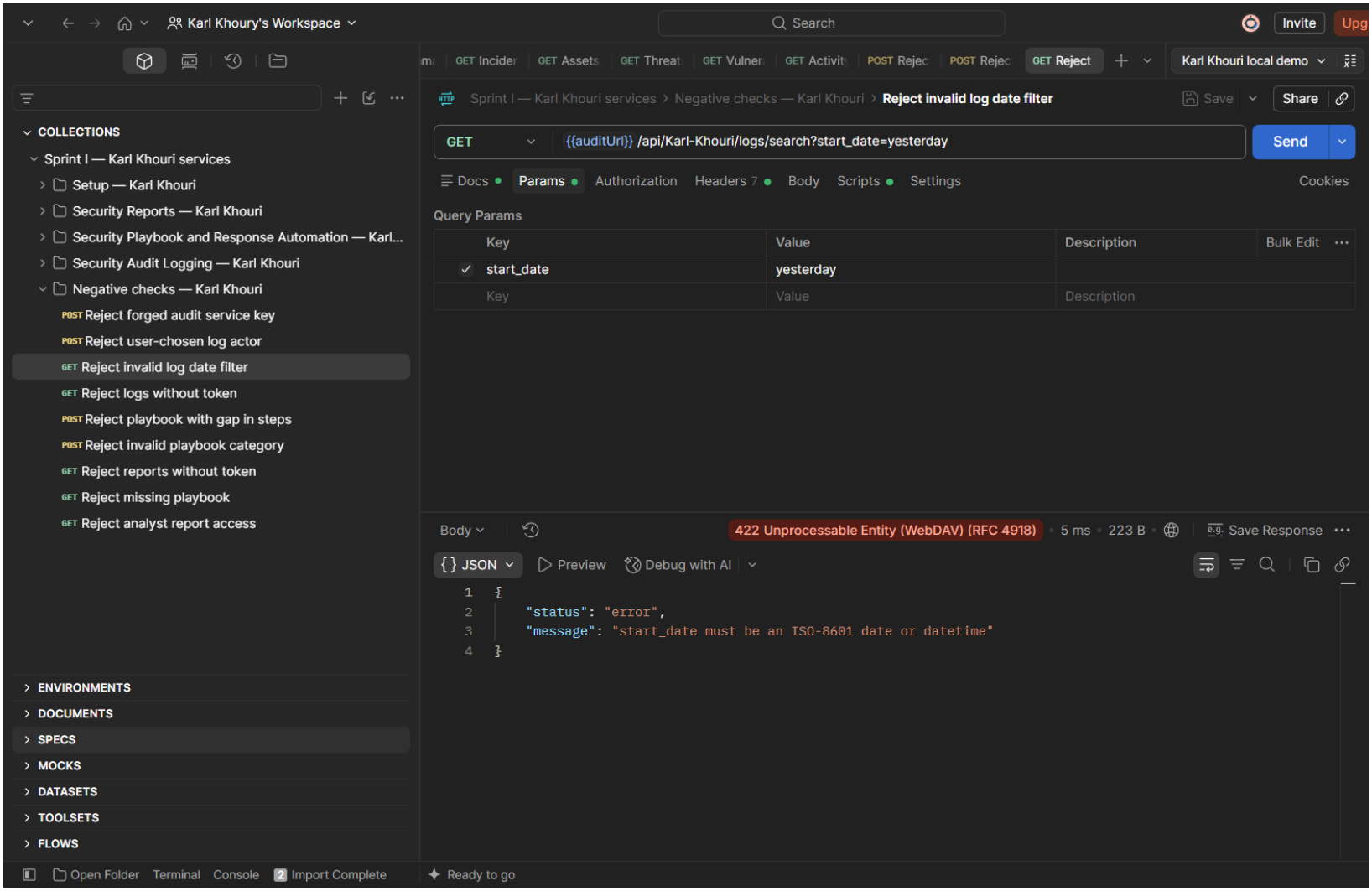

Figure 147. Karl Khouri - GET Reject Invalid Date Filter - A malformed start date is rejected with HTTP 422.

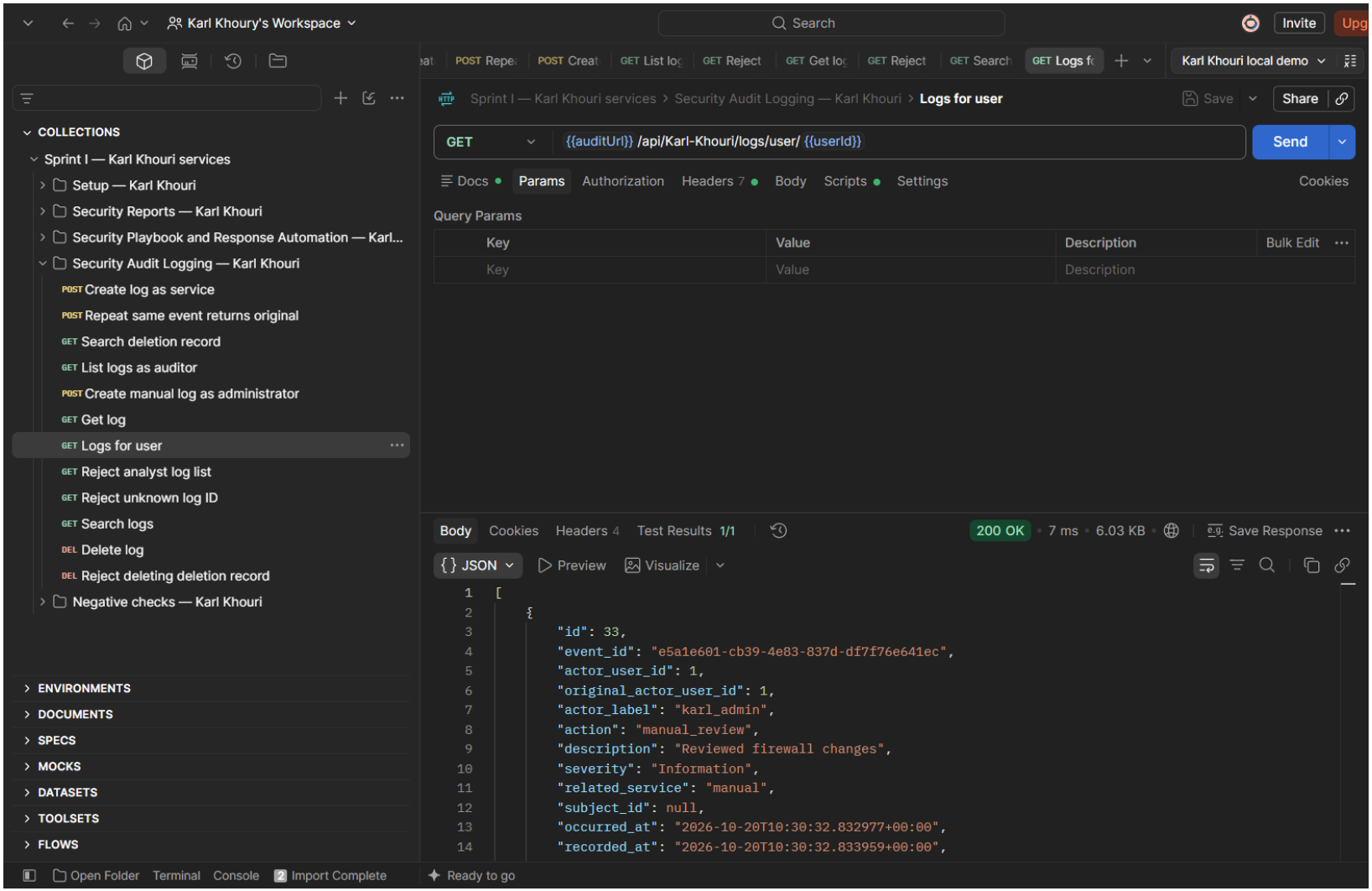

Figure 148. Karl Khouri - GET Logs for User - An administrator retrieves one user's recorded activity; only entries by that user are returned with HTTP 200.

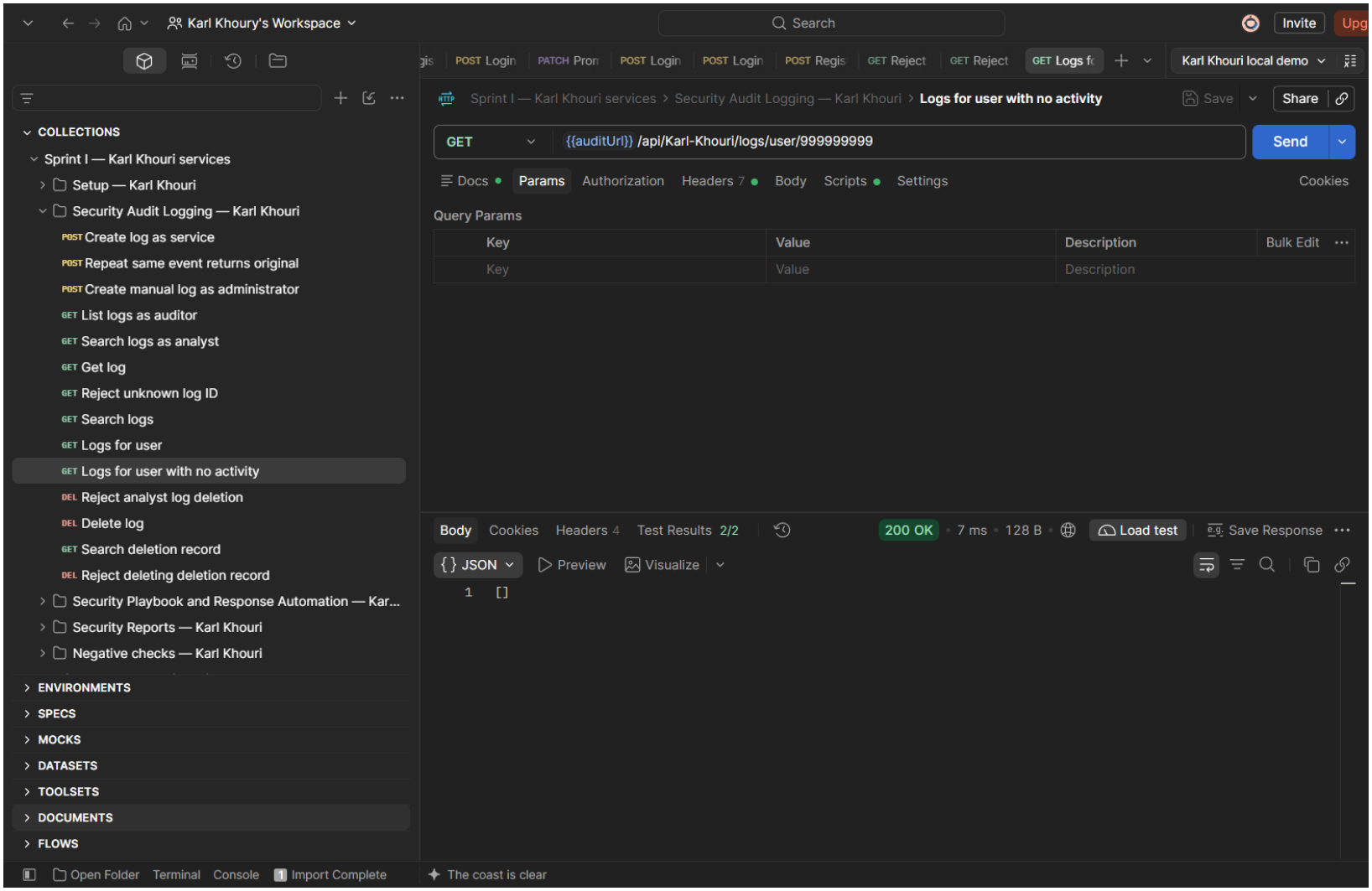

Figure 149. Karl Khouri - GET Logs for User With No Activity - A user ID with no recorded activity returns HTTP 200 with an empty list.

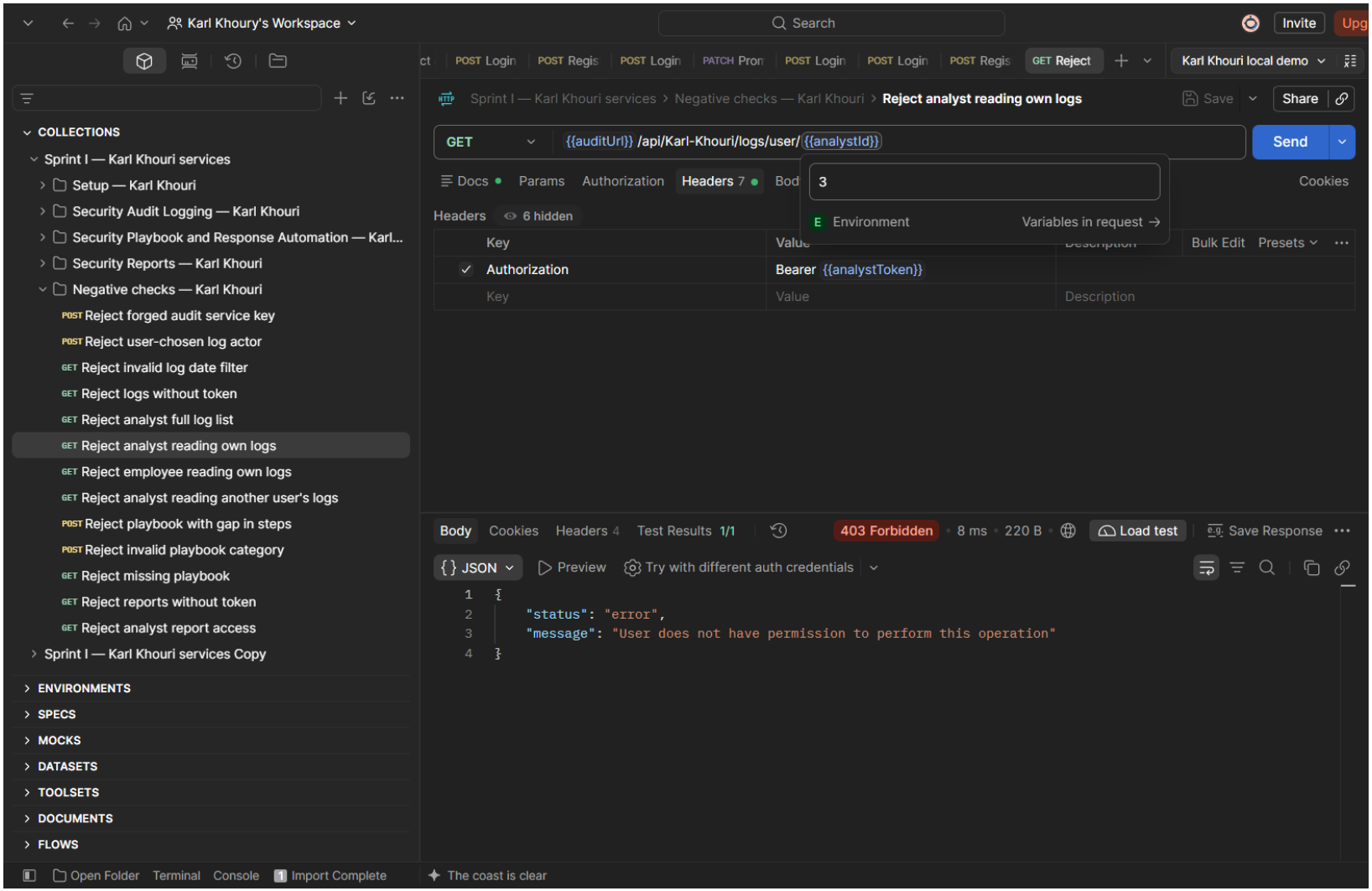

Figure 150. Karl Khouri - GET Reject Analyst Own Logs - The analyst's token requests the analyst's own ID (analystId = 3, shown in the variable popup) and gets HTTP 403: user histories are Administrator/Auditor-only.

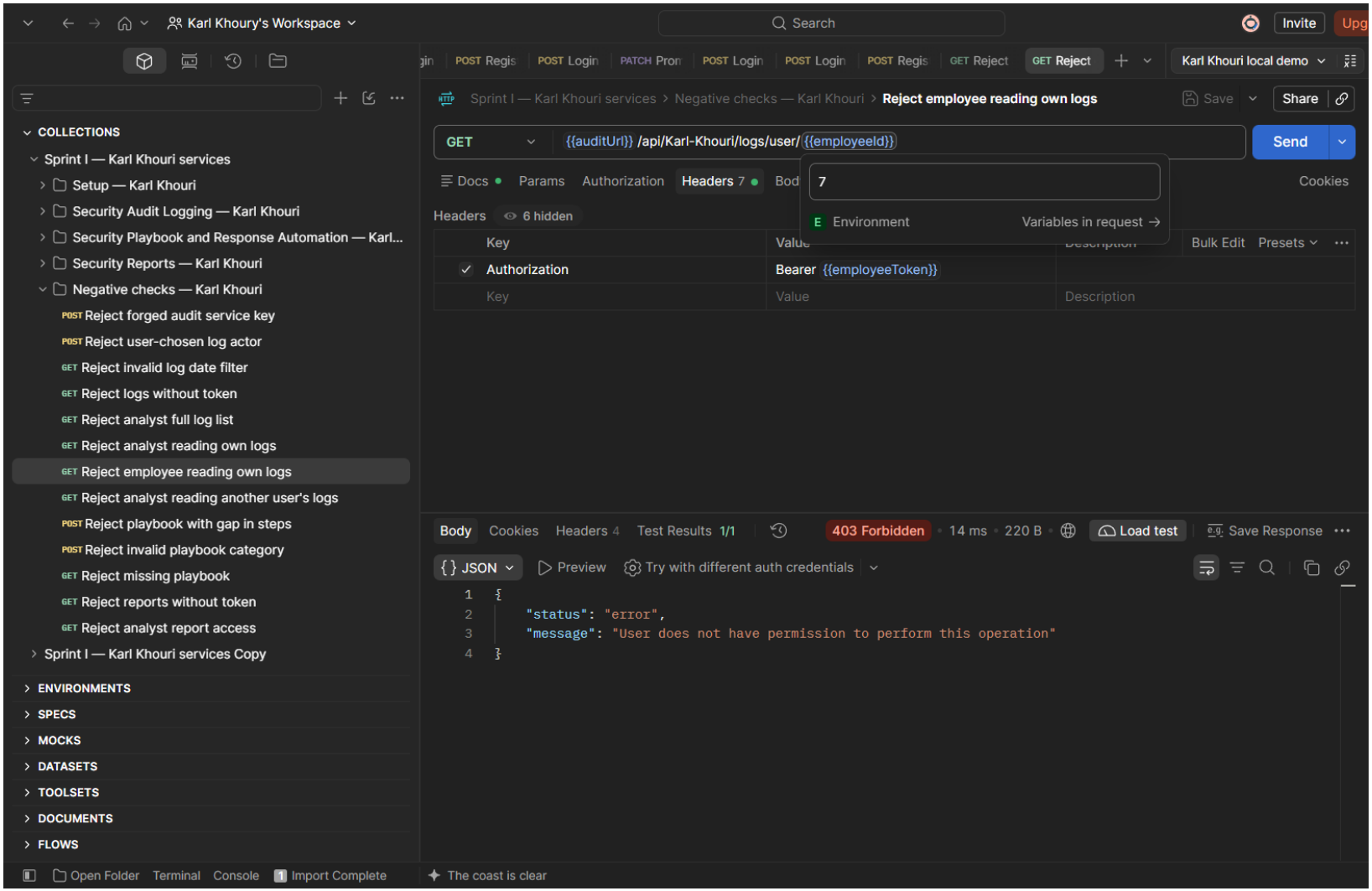

Figure 151. Karl Khouri - GET Reject Employee Own Logs - The employee's token requests the employee's own ID (employeeId = 7) and gets HTTP 403: employees have no audit-log access.

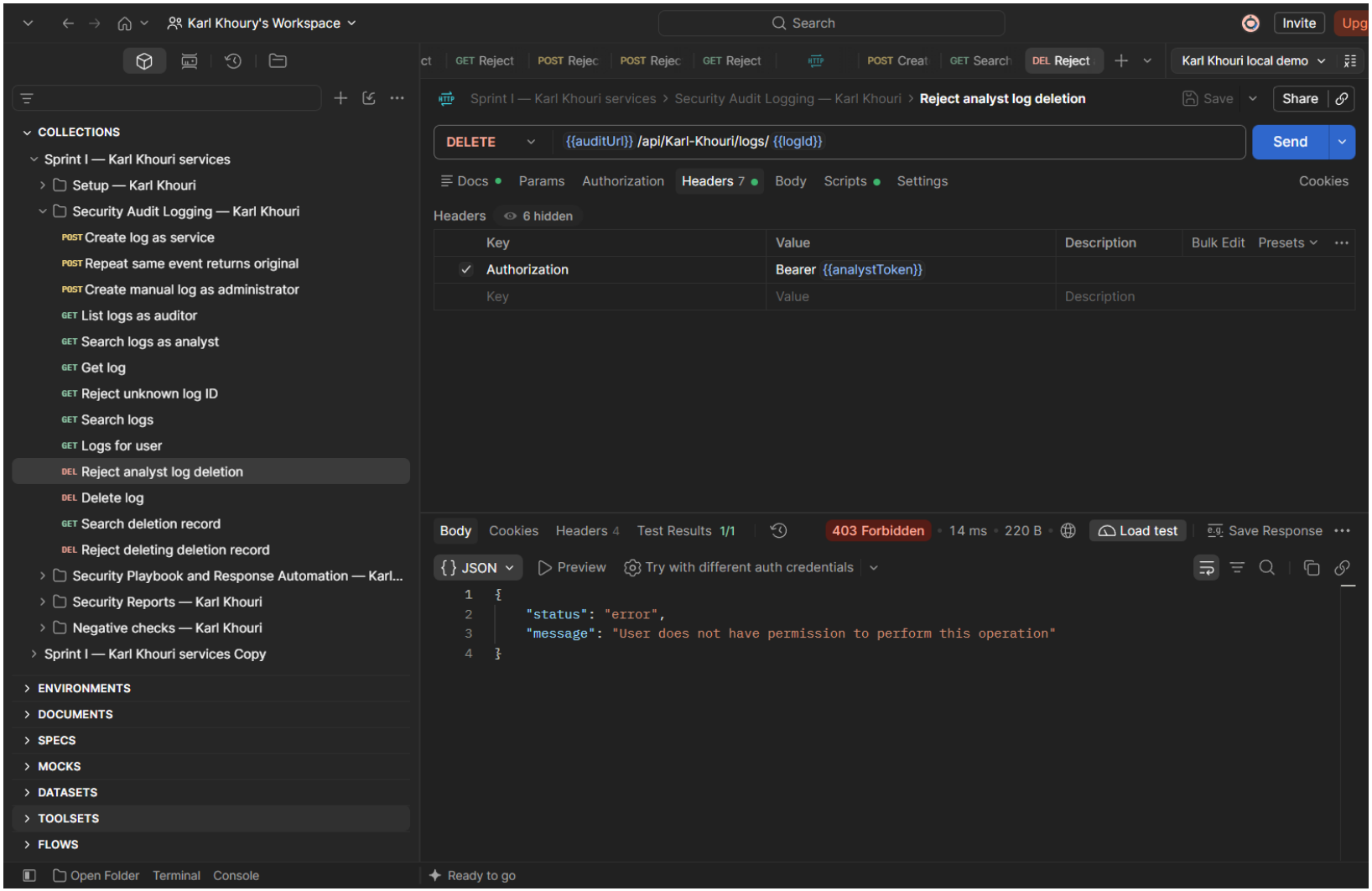

Figure 152. Karl Khouri - DELETE Reject Analyst Log Deletion - Analysts may read logs but not delete them; the analyst's token gets HTTP 403.

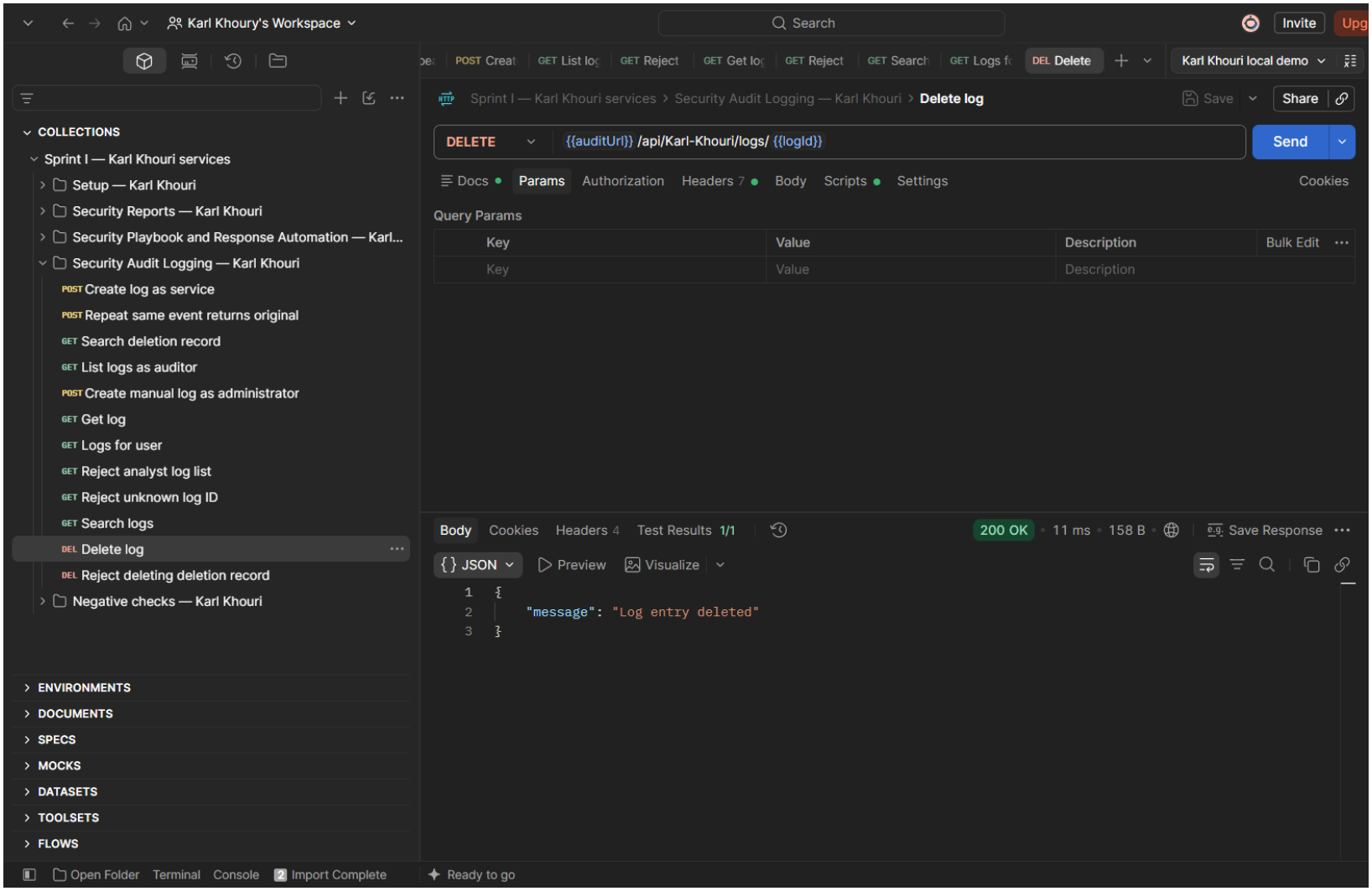

Figure 153. Karl Khouri - DELETE Delete Log - An administrator deletes an entry with HTTP 200.

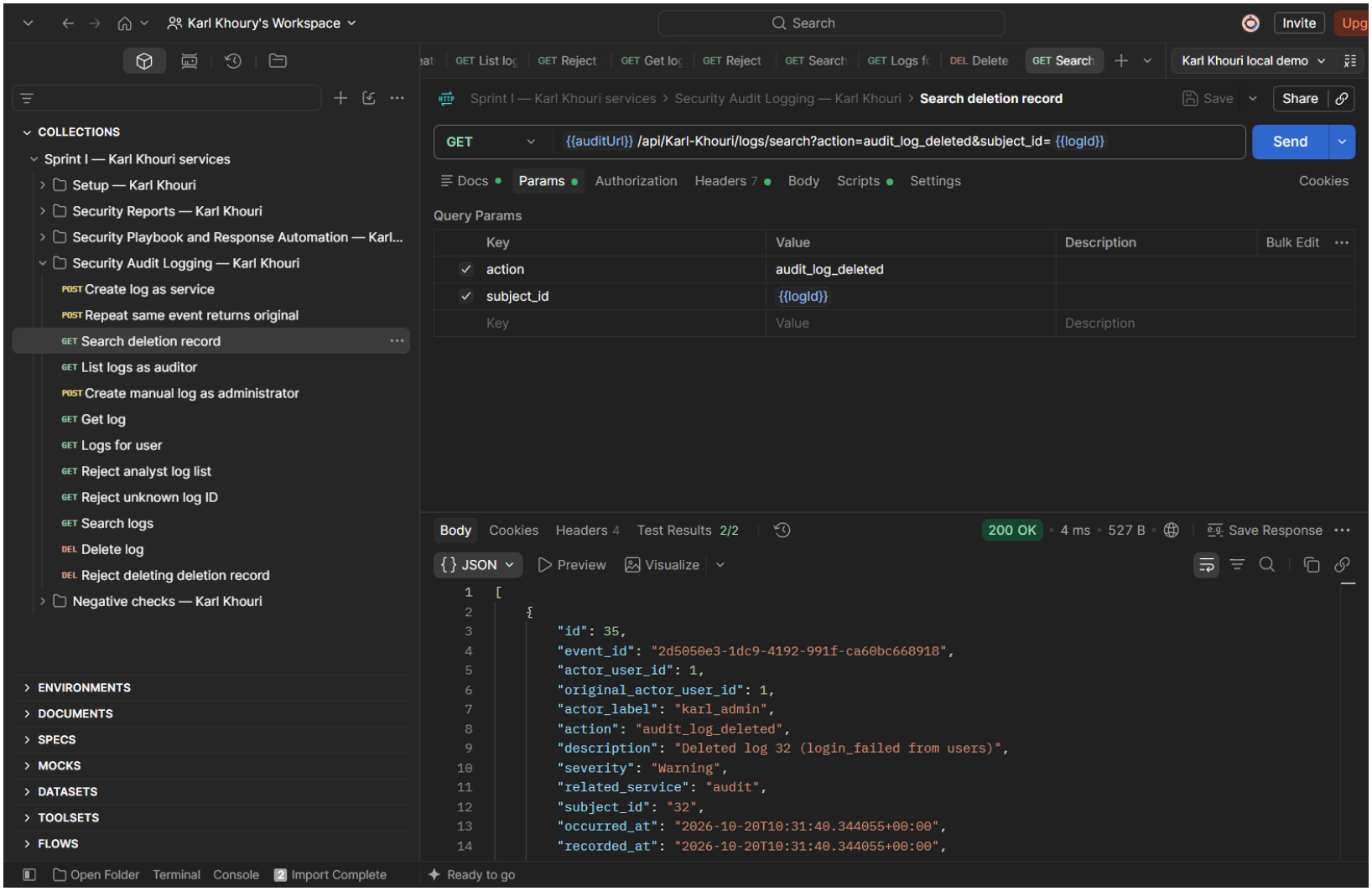

Figure 154. Karl Khouri - GET Search Deletion Record - The deletion itself was recorded as audit_log_deleted, naming the removed entry.

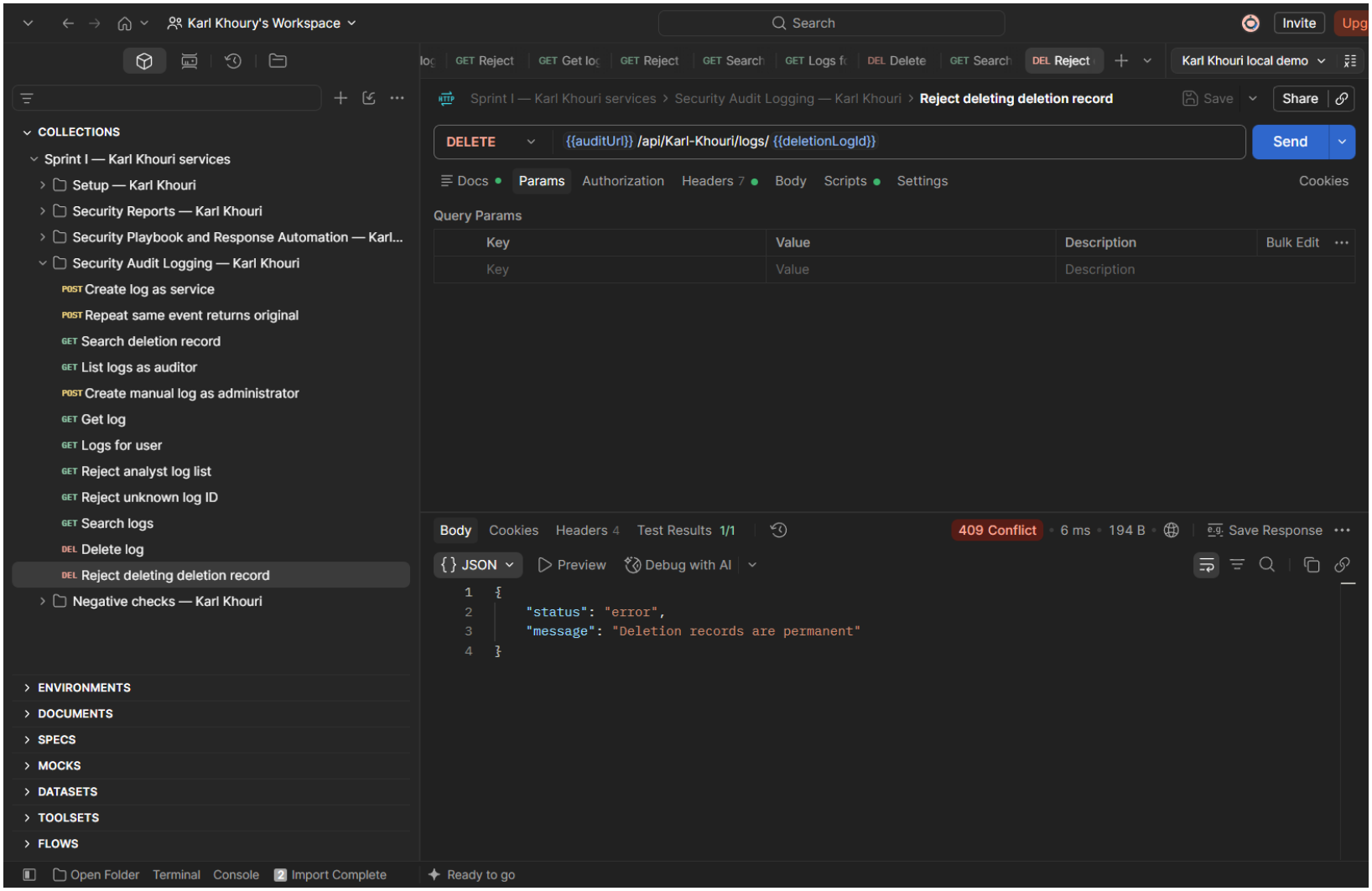

Figure 155. Karl Khouri - DELETE Reject Deleting Deletion Record - Deletion records are permanent; HTTP 409.

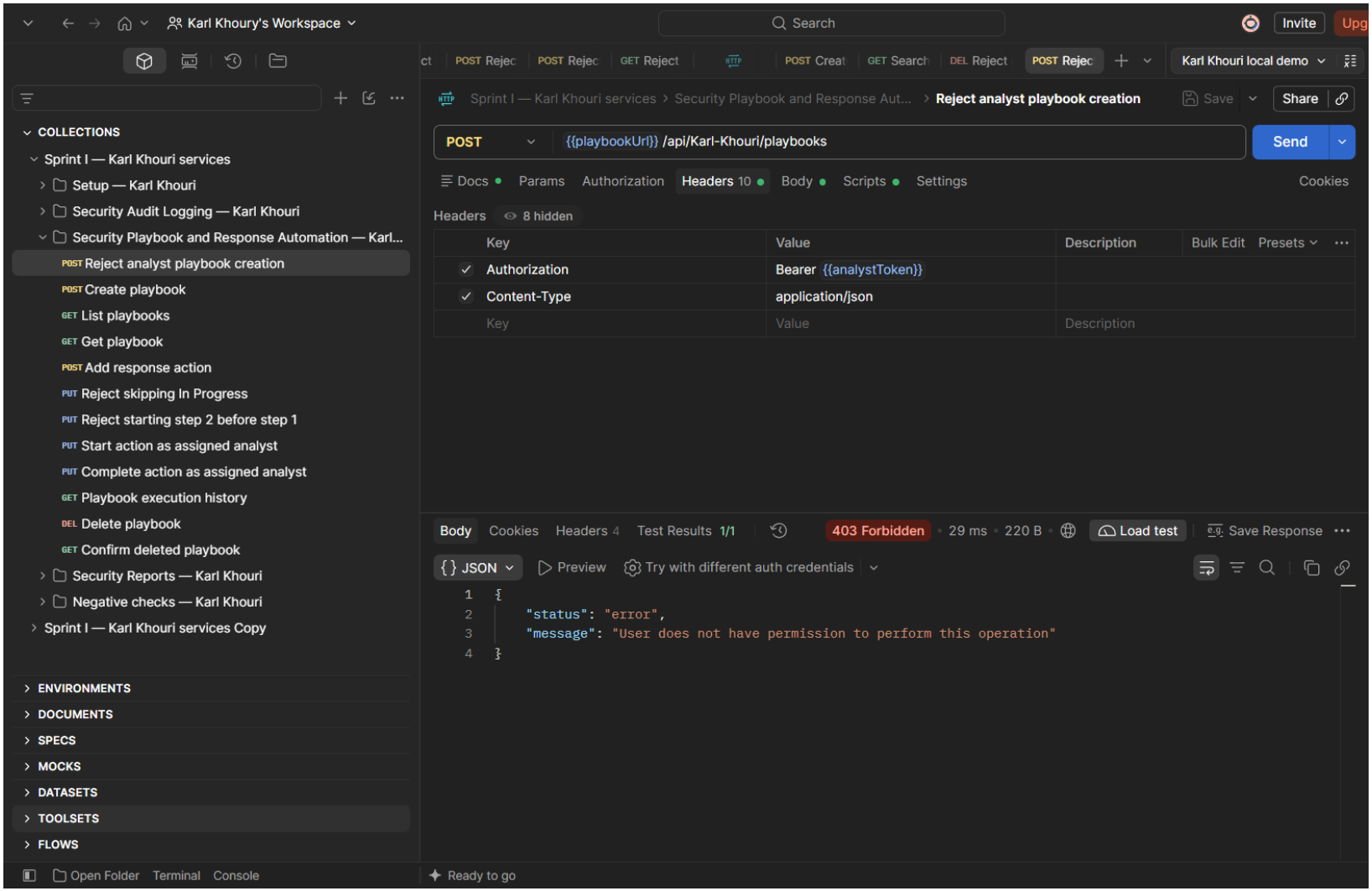

Figure 156. Karl Khouri - POST Reject Analyst Playbook Creation - Playbook creation is administrator-only; the analyst's token (Headers tab) gets HTTP 403.

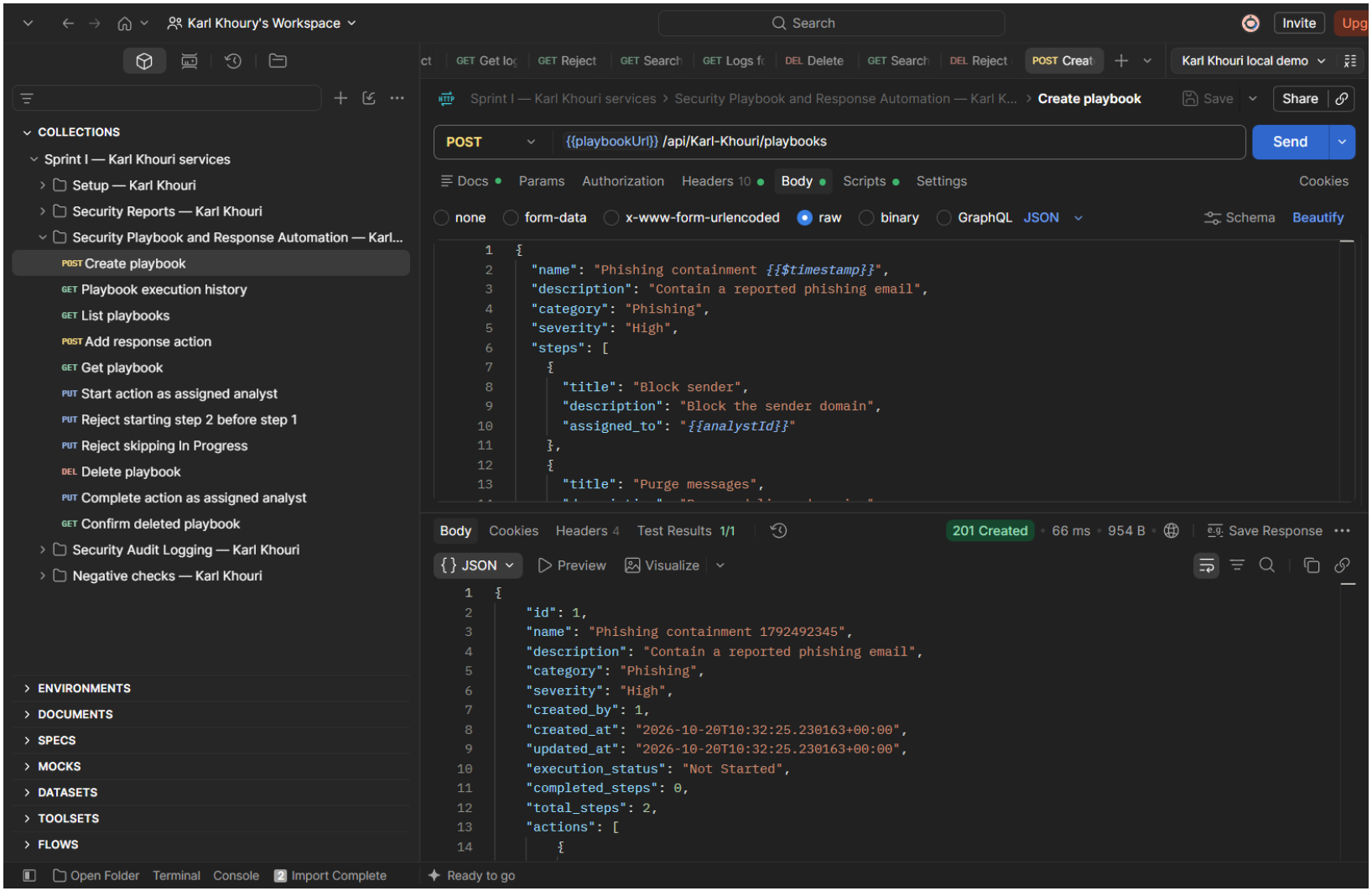

Figure 157. Karl Khouri - POST Create Playbook with Two Steps - A playbook and its ordered steps assigned to an analyst are created with HTTP 201.

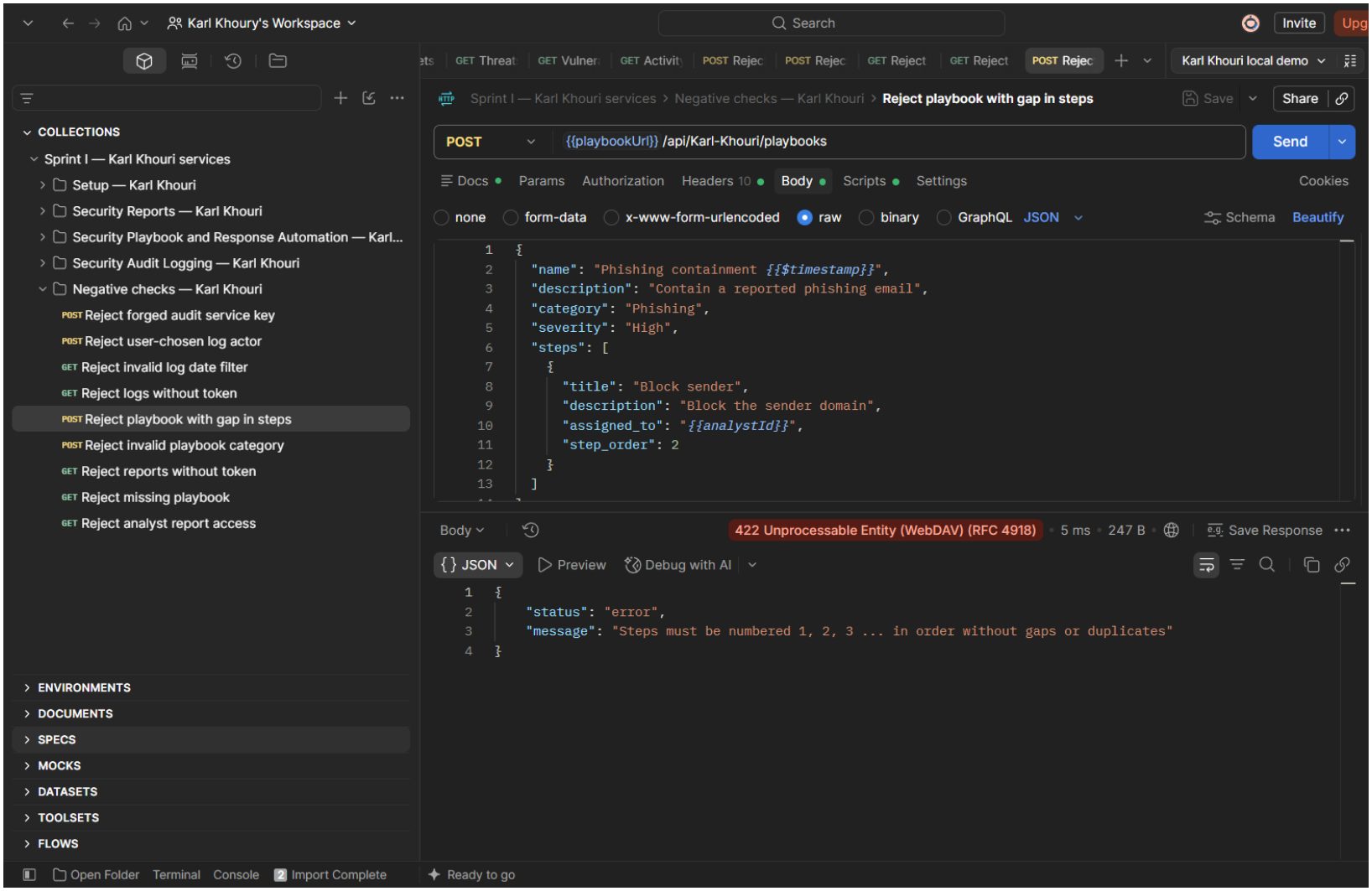

Figure 158. Karl Khouri - POST Reject Gap in Steps - Steps that do not start at 1 or skip a number are rejected with HTTP 422.

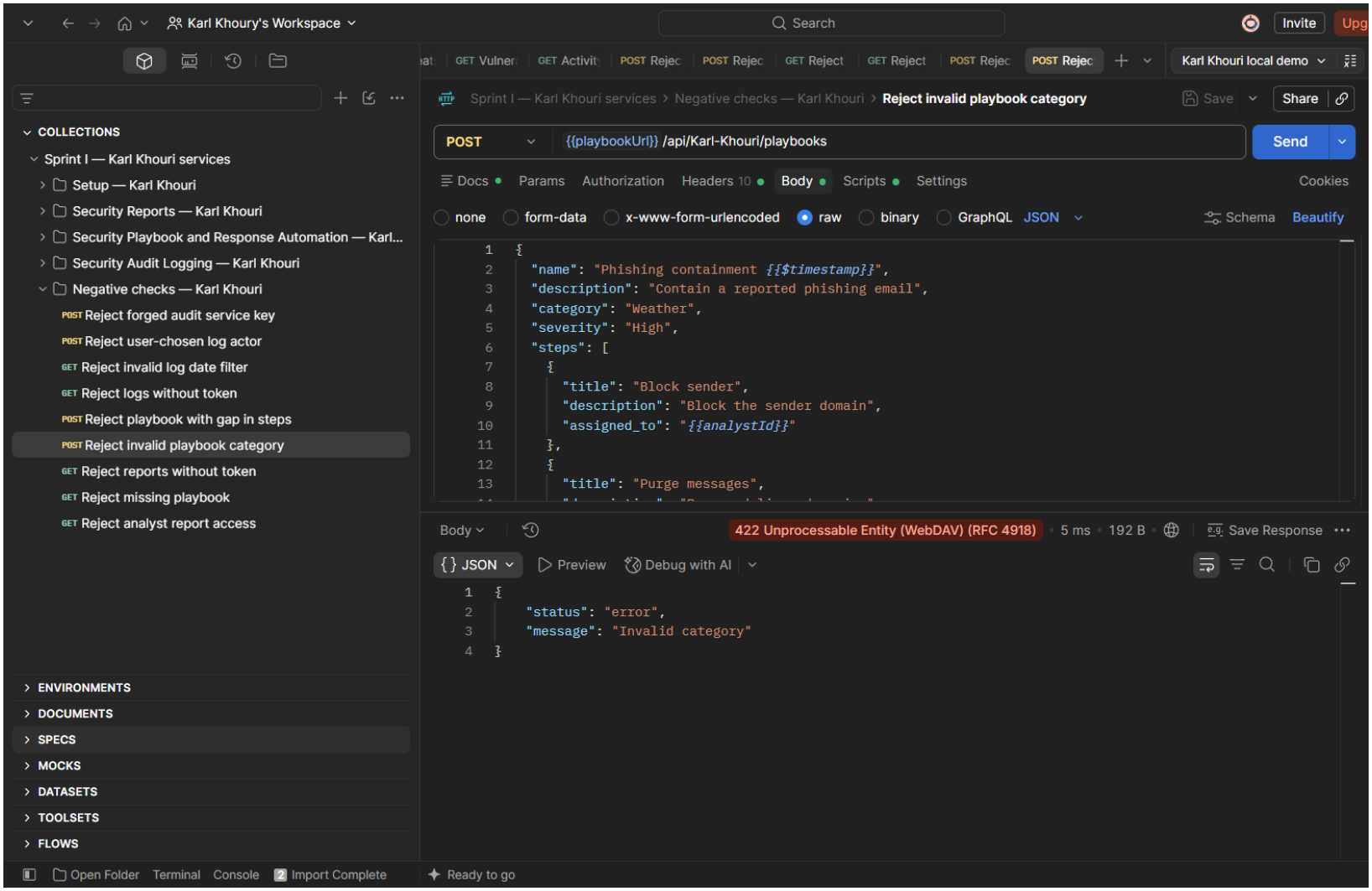

Figure 159. Karl Khouri - POST Reject Invalid Category - An unsupported category is rejected with HTTP 422.

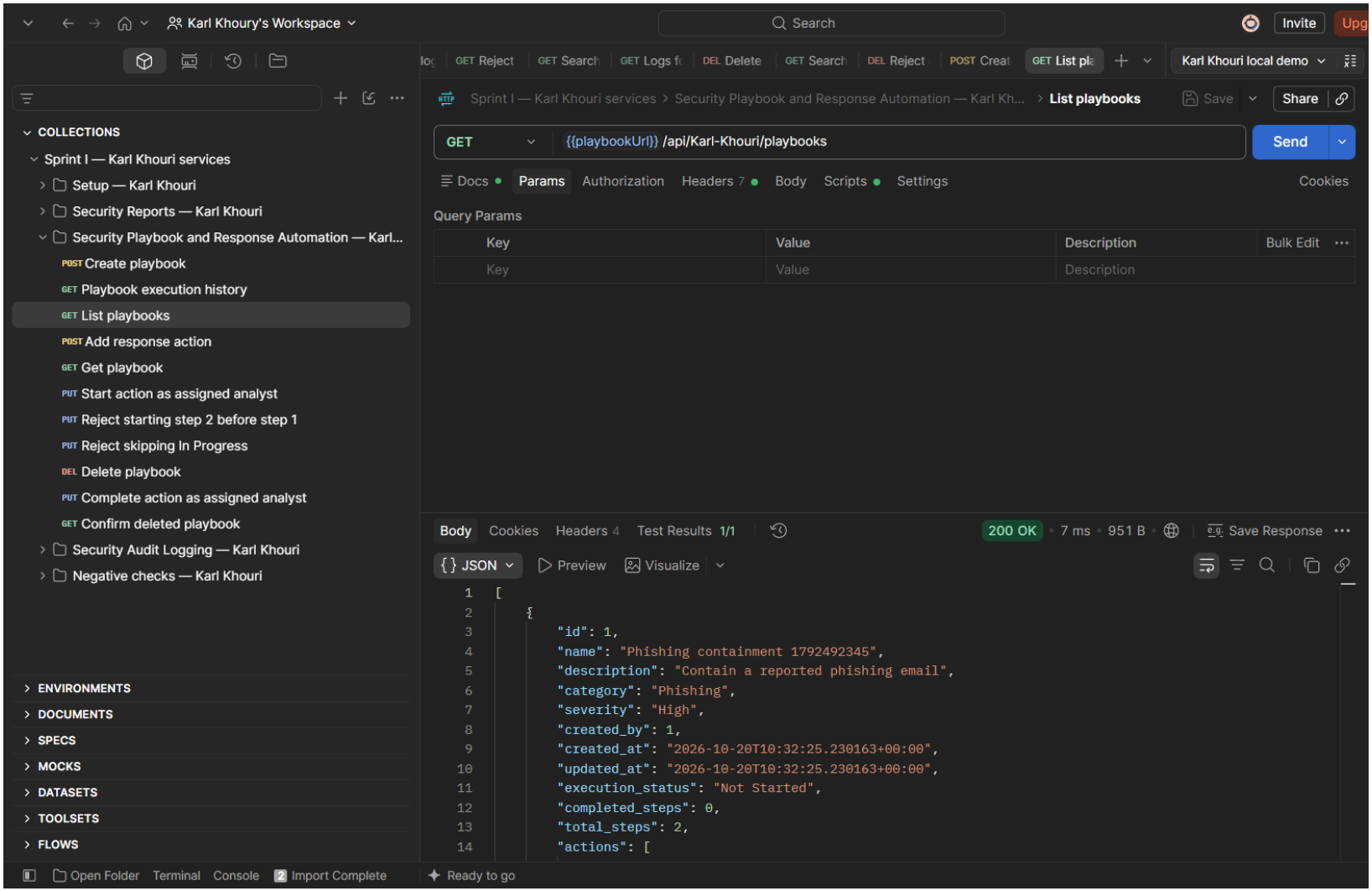

Figure 160. Karl Khouri - GET List Playbooks - Playbooks are listed with their steps and progress, HTTP 200.

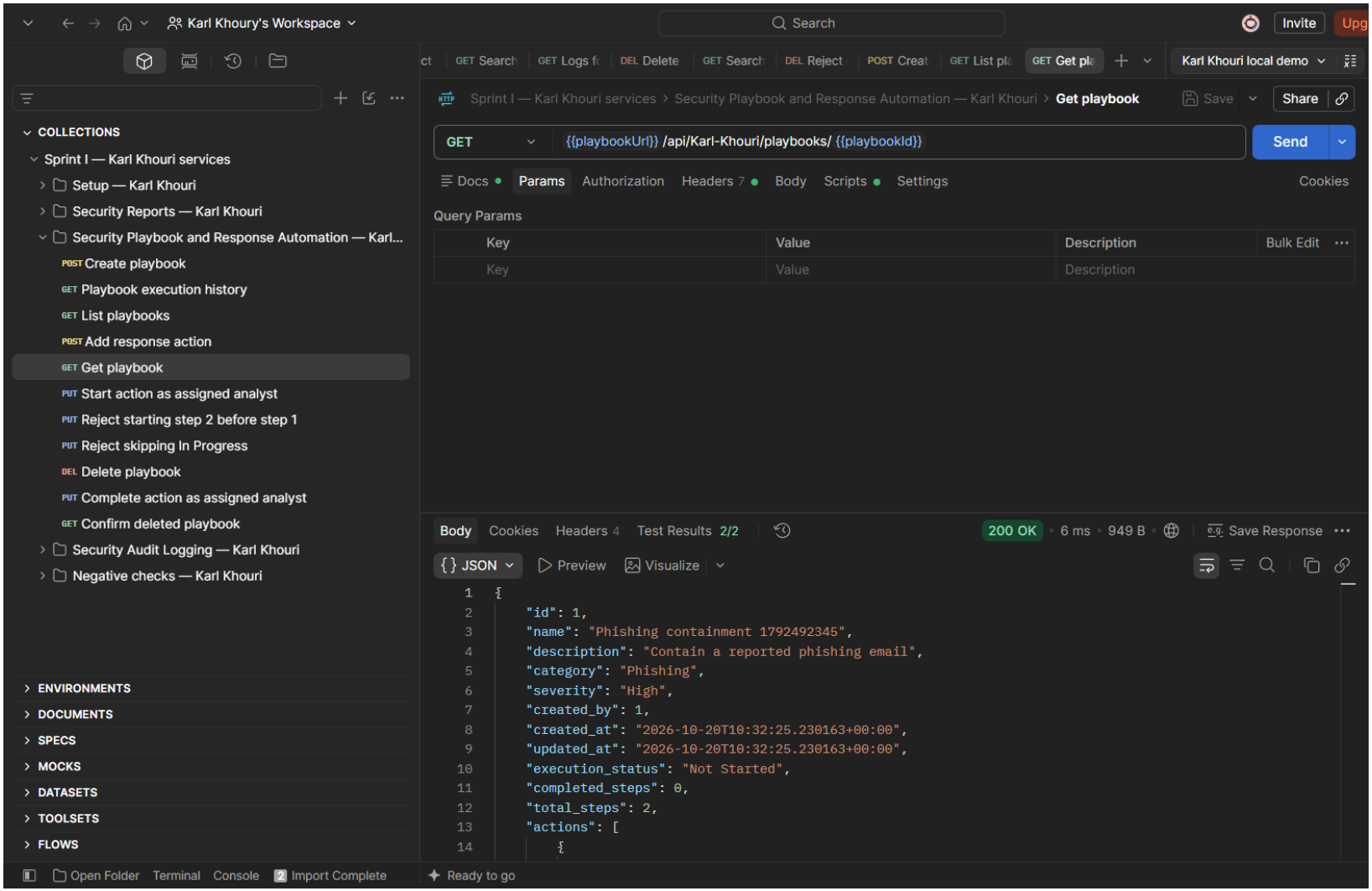

Figure 161. Karl Khouri - GET Get Playbook - Steps are returned in order with execution state Not Started, HTTP 200.

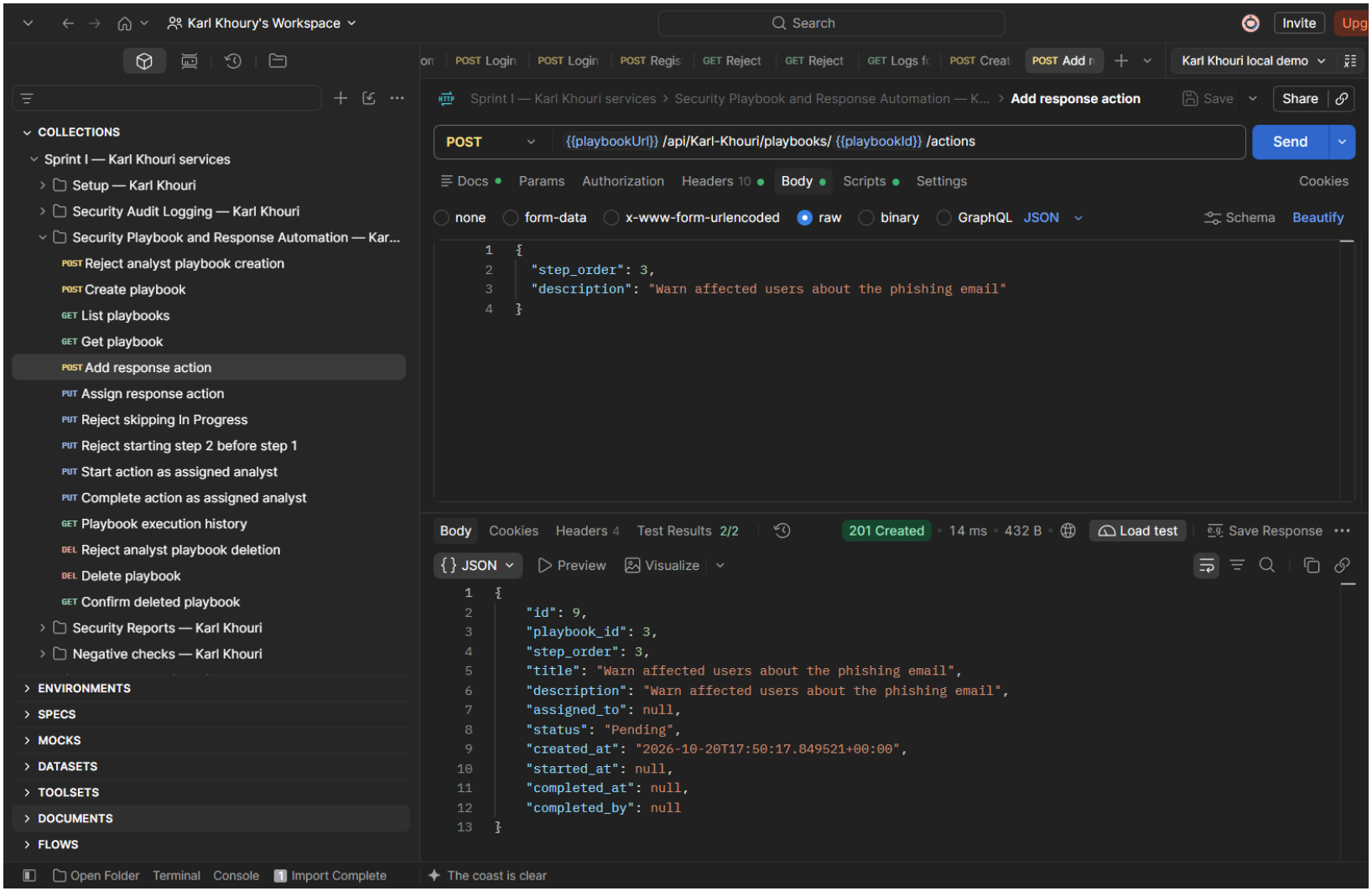

Figure 162. Karl Khouri - POST Add Response Action - Only the handout's required information (description and step number) is sent; step 3 is stored unassigned with HTTP 201.

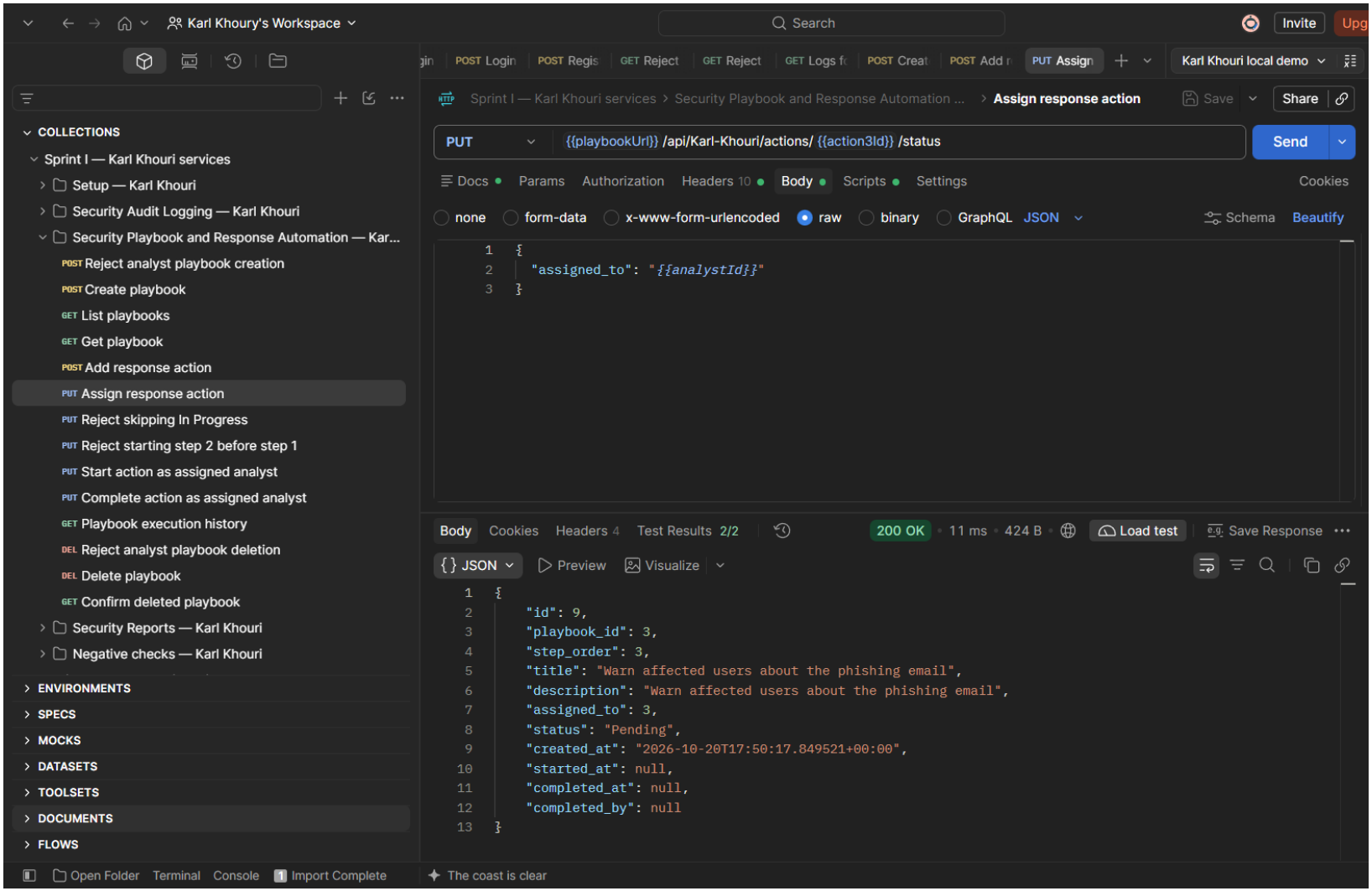

Figure 163. Karl Khouri - PUT Assign Response Action - The assigned user is updated through the status route, one of the handout's possible updates; HTTP 200.

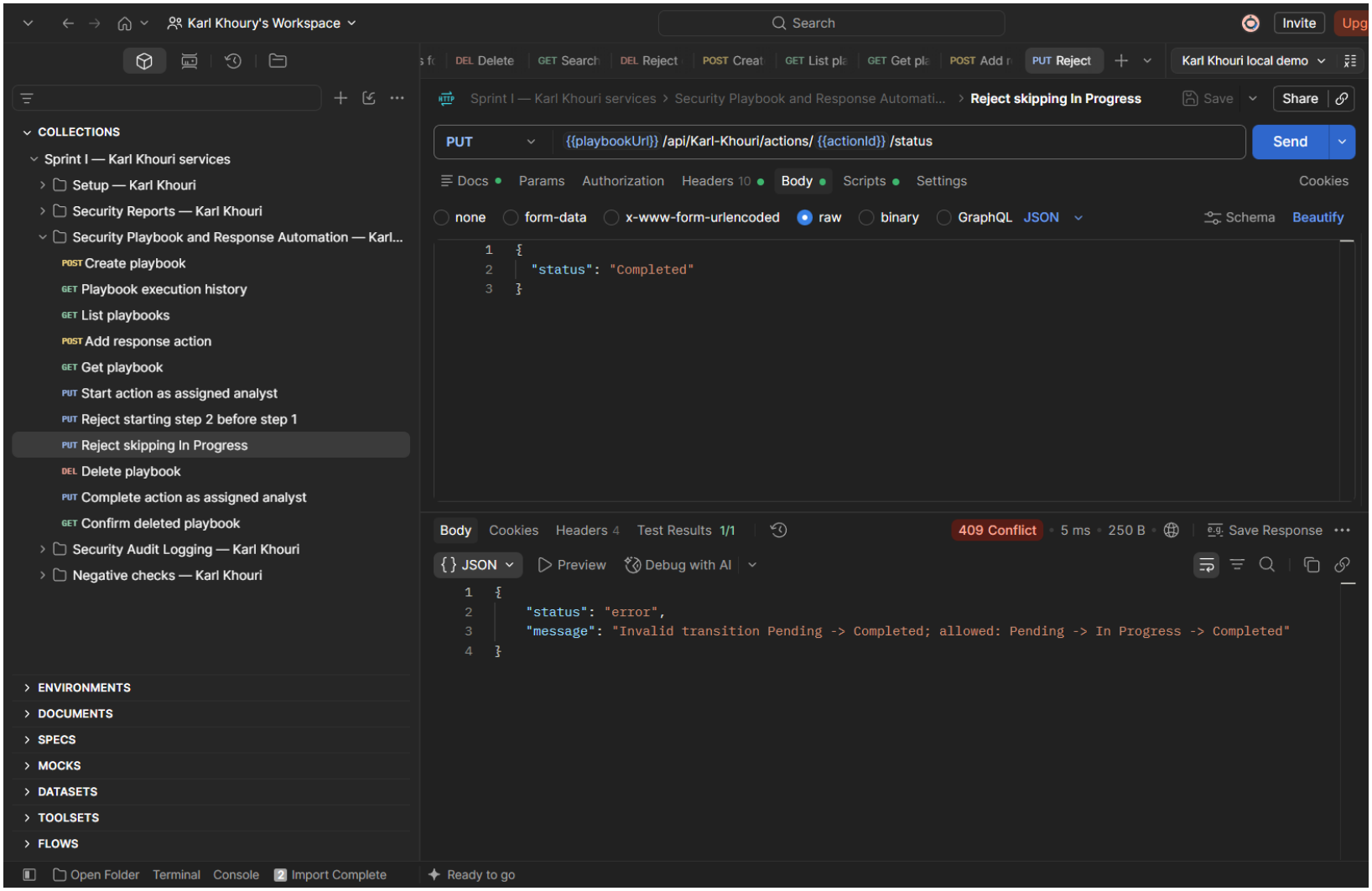

Figure 164. Karl Khouri - PUT Reject Skipping In Progress - A Pending step cannot jump to Completed; HTTP 409.

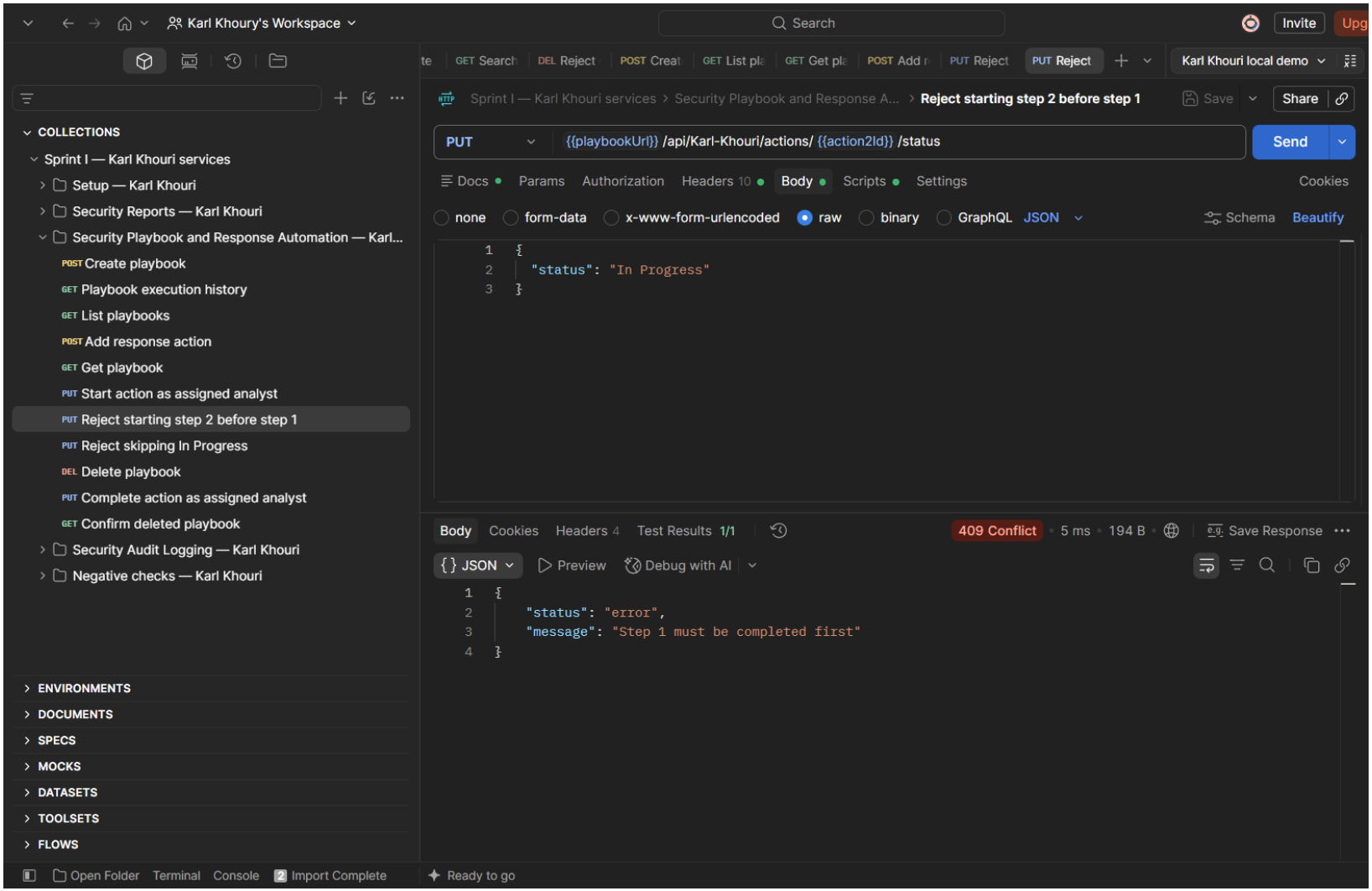

Figure 165. Karl Khouri - PUT Reject Starting Step 2 Before Step 1 - Ordering is enforced; HTTP 409 names the unfinished step.

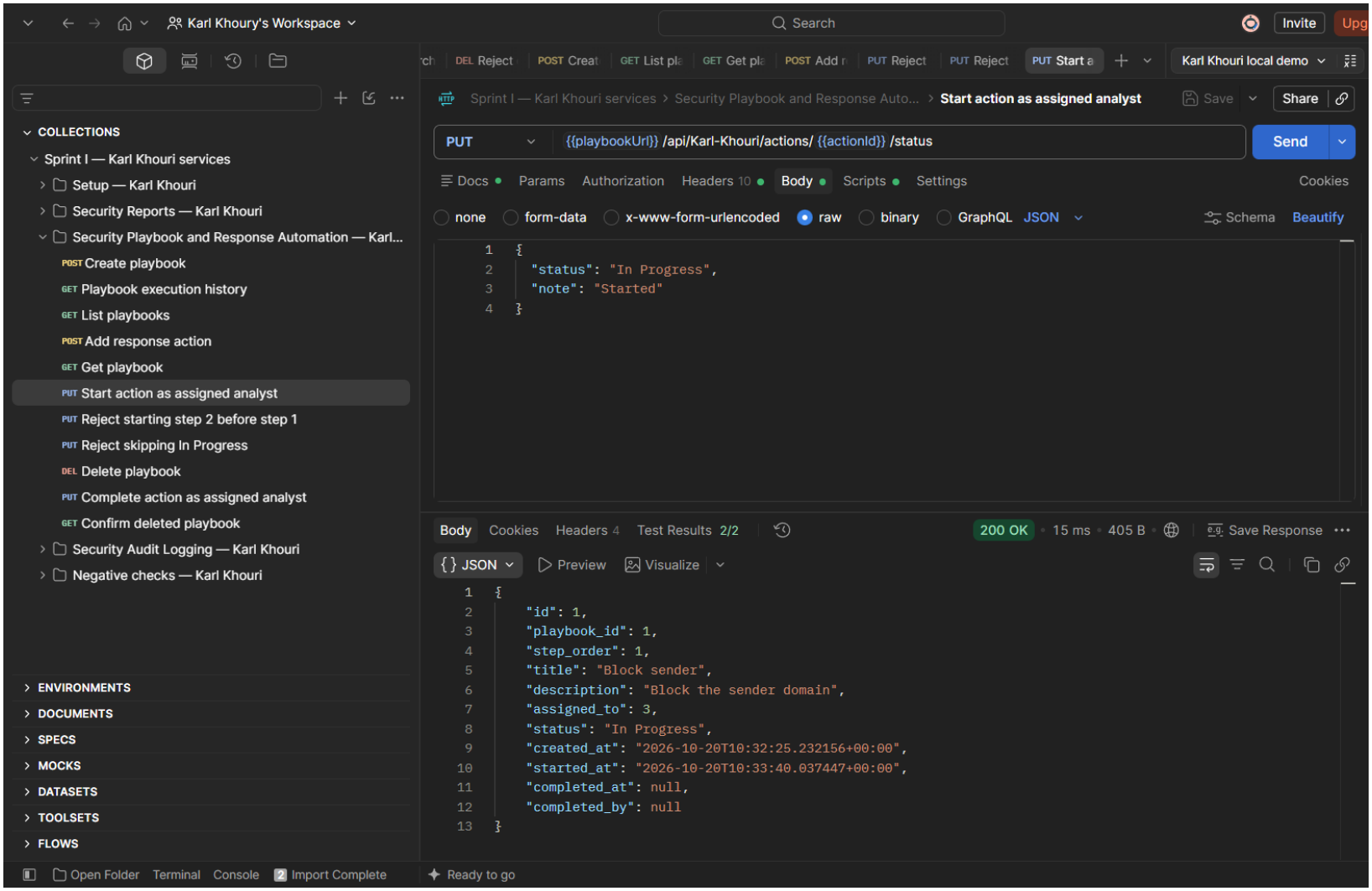

Figure 166. Karl Khouri - PUT Start Action as Assigned Analyst - The assigned analyst moves step 1 to In Progress, HTTP 200, with the start time recorded.

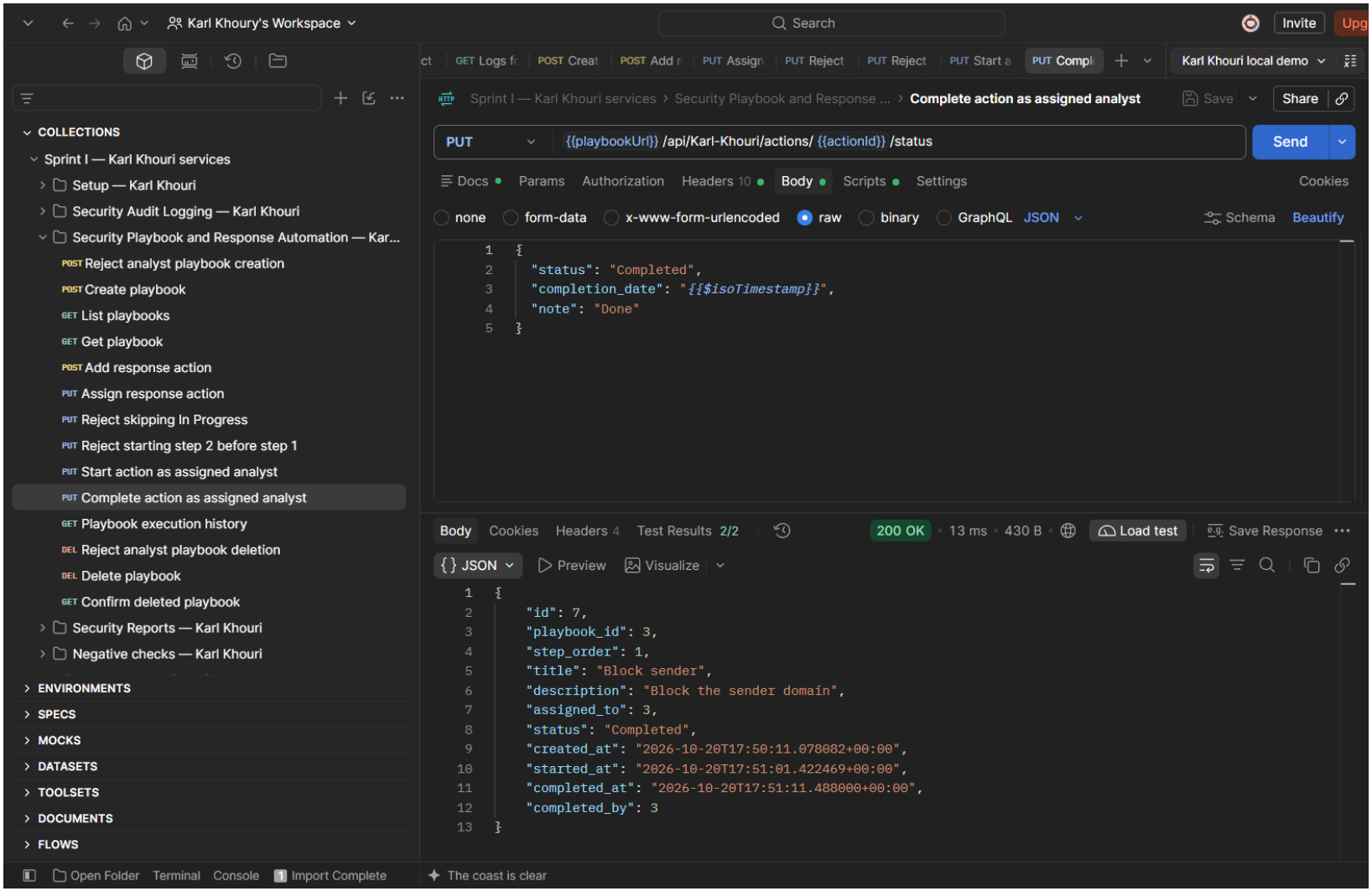

Figure 167. Karl Khouri - PUT Complete Action as Assigned Analyst - Step 1 becomes Completed with the supplied completion date and the completing user recorded; HTTP 200.

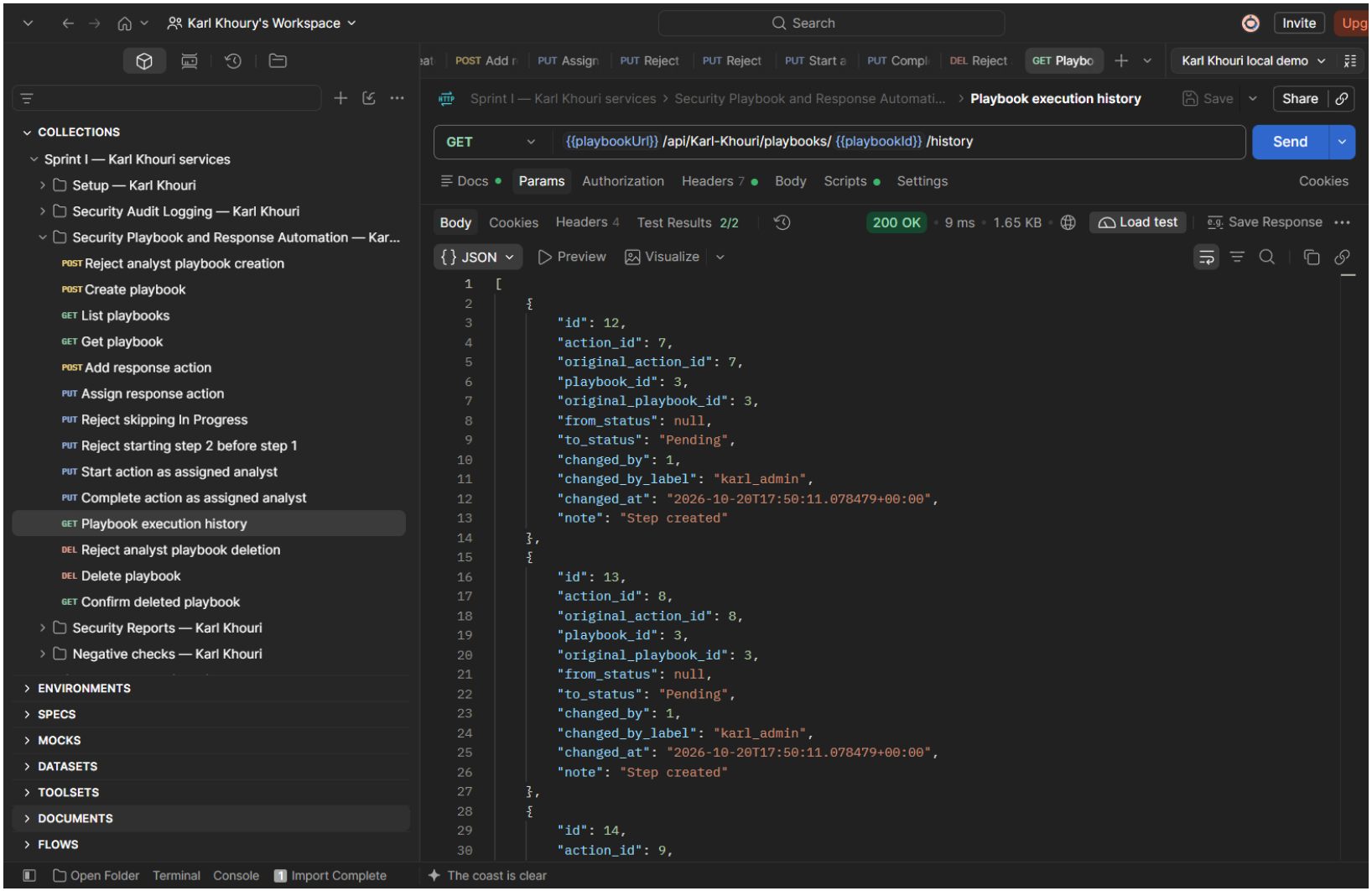

Figure 168. Karl Khouri - GET Playbook Execution History - Step creations, the later assignment and each status change are listed with user and time.

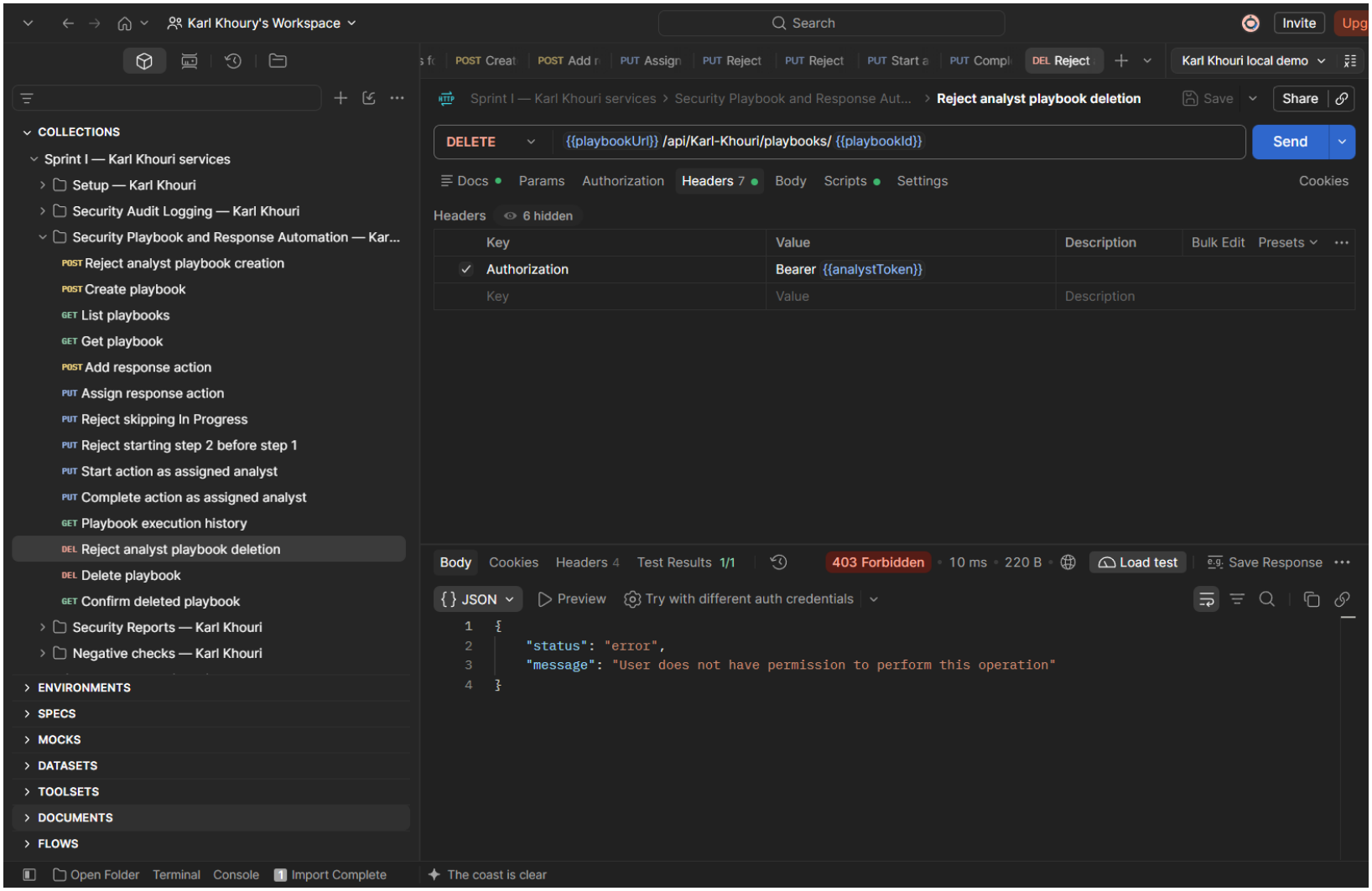

Figure 169. Karl Khouri - DELETE Reject Analyst Playbook Deletion - Unauthorized deletion is prevented: the analyst's token gets HTTP 403.

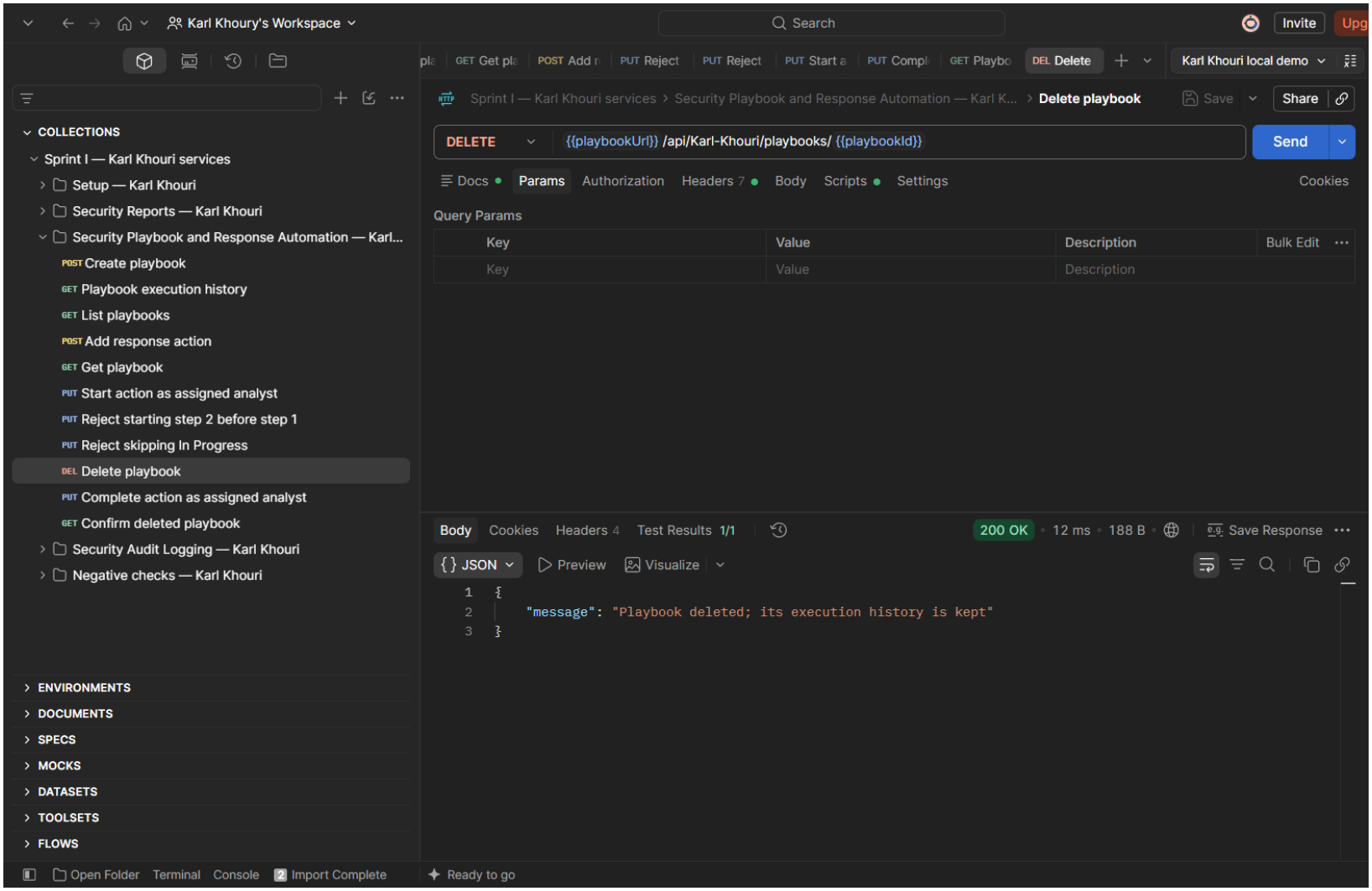

Figure 170. Karl Khouri - DELETE Delete Playbook - An administrator deletes the playbook with HTTP 200.

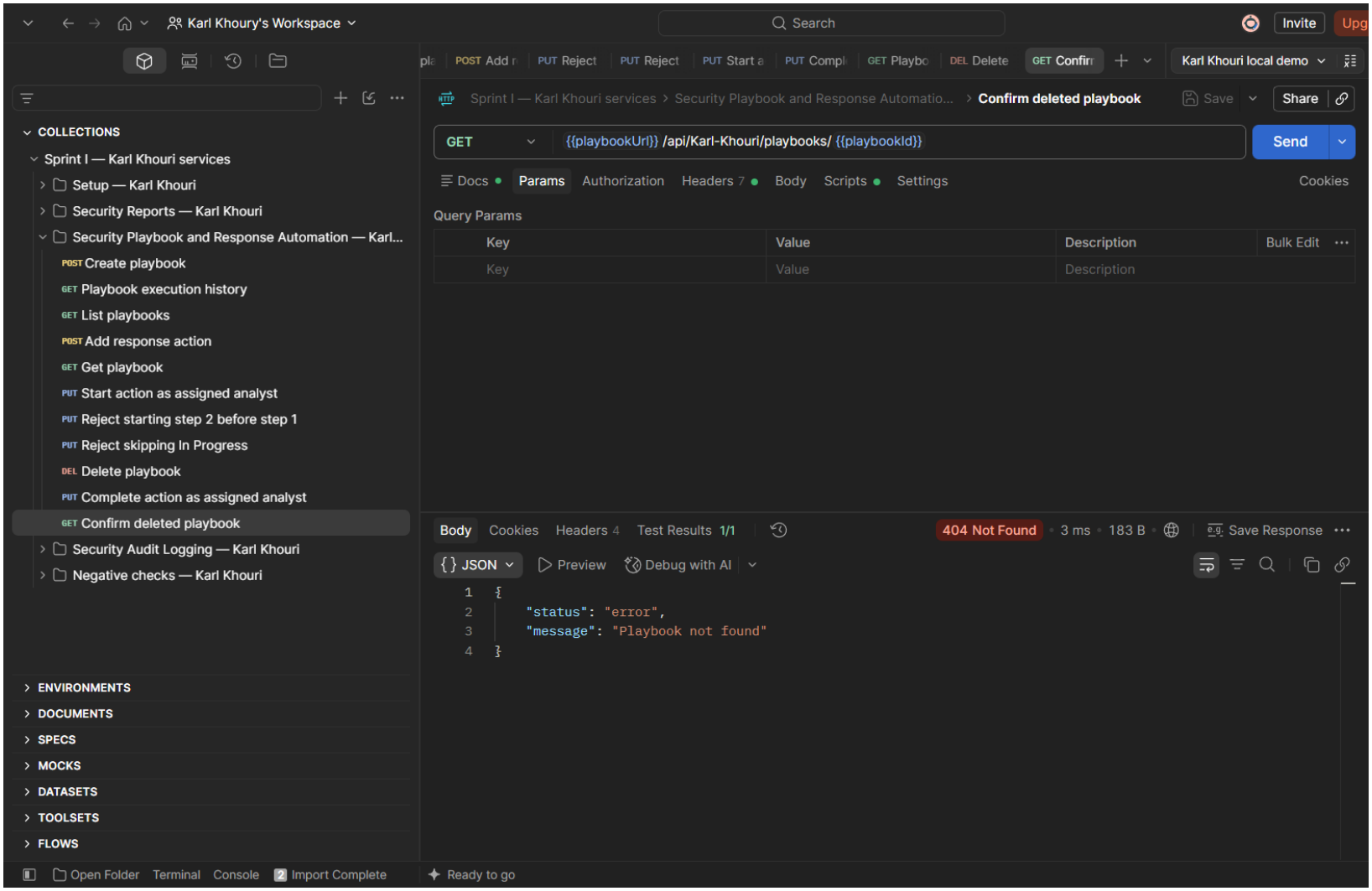

Figure 171. Karl Khouri - GET Confirm Deleted Playbook - The deleted playbook now returns HTTP 404.

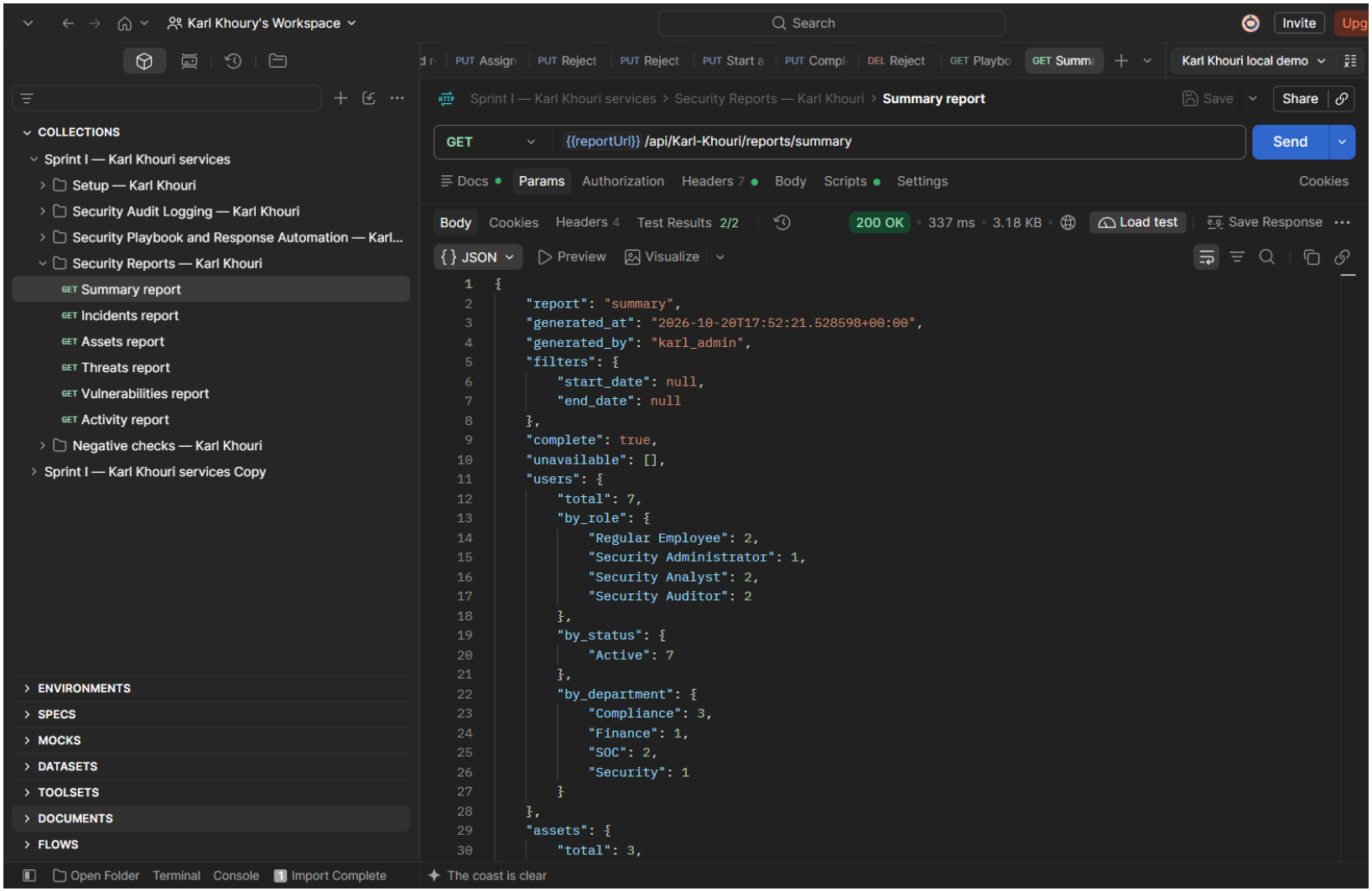

Figure 172. Karl Khouri - GET Summary Report - All six sources are reached (complete is true, unavailable is empty), starting with the total number of users by role, status and department.

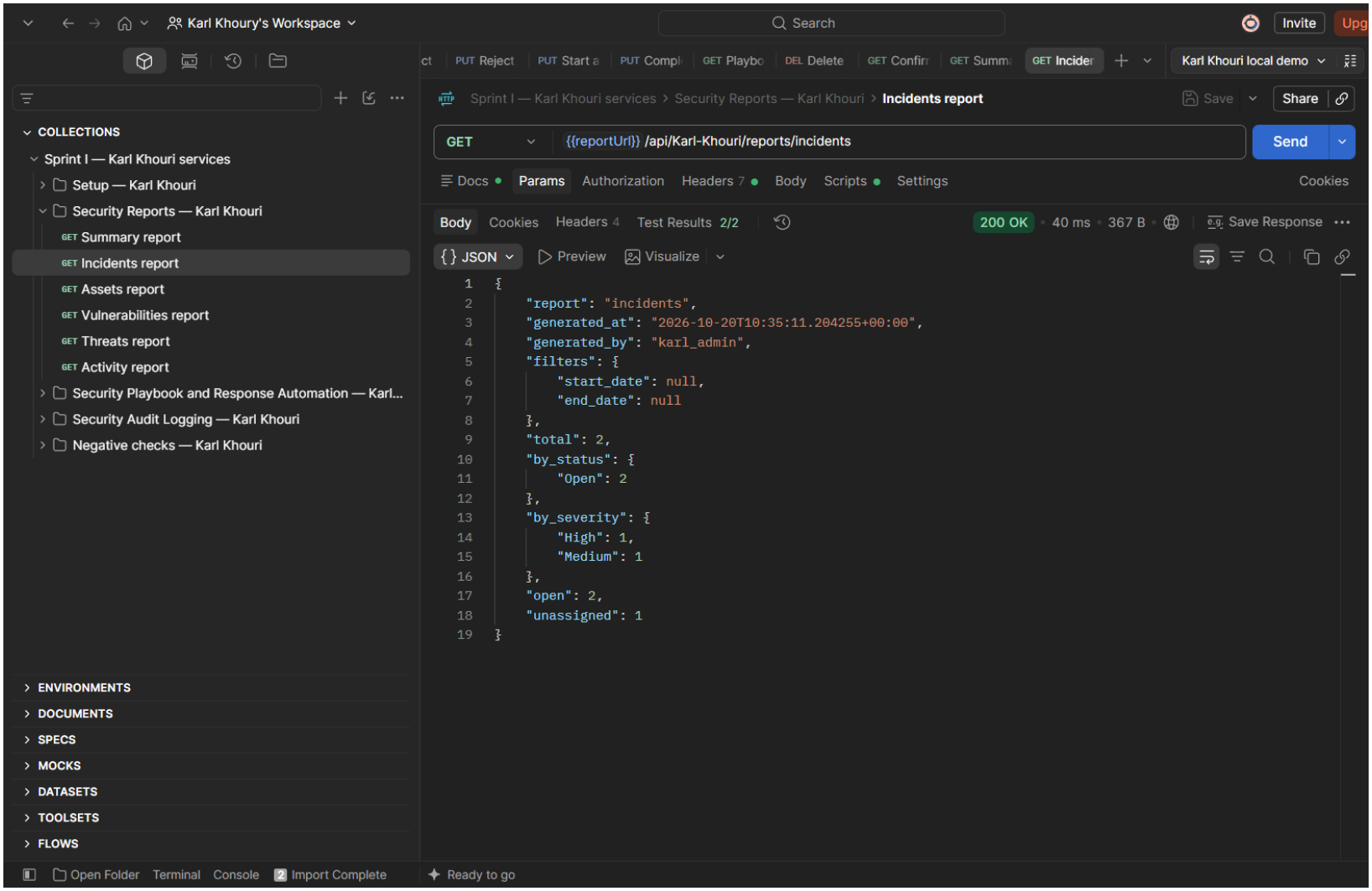

Figure 173. Karl Khouri - GET Incidents Report - Totals by status and severity, open and unassigned counts from the incident API.

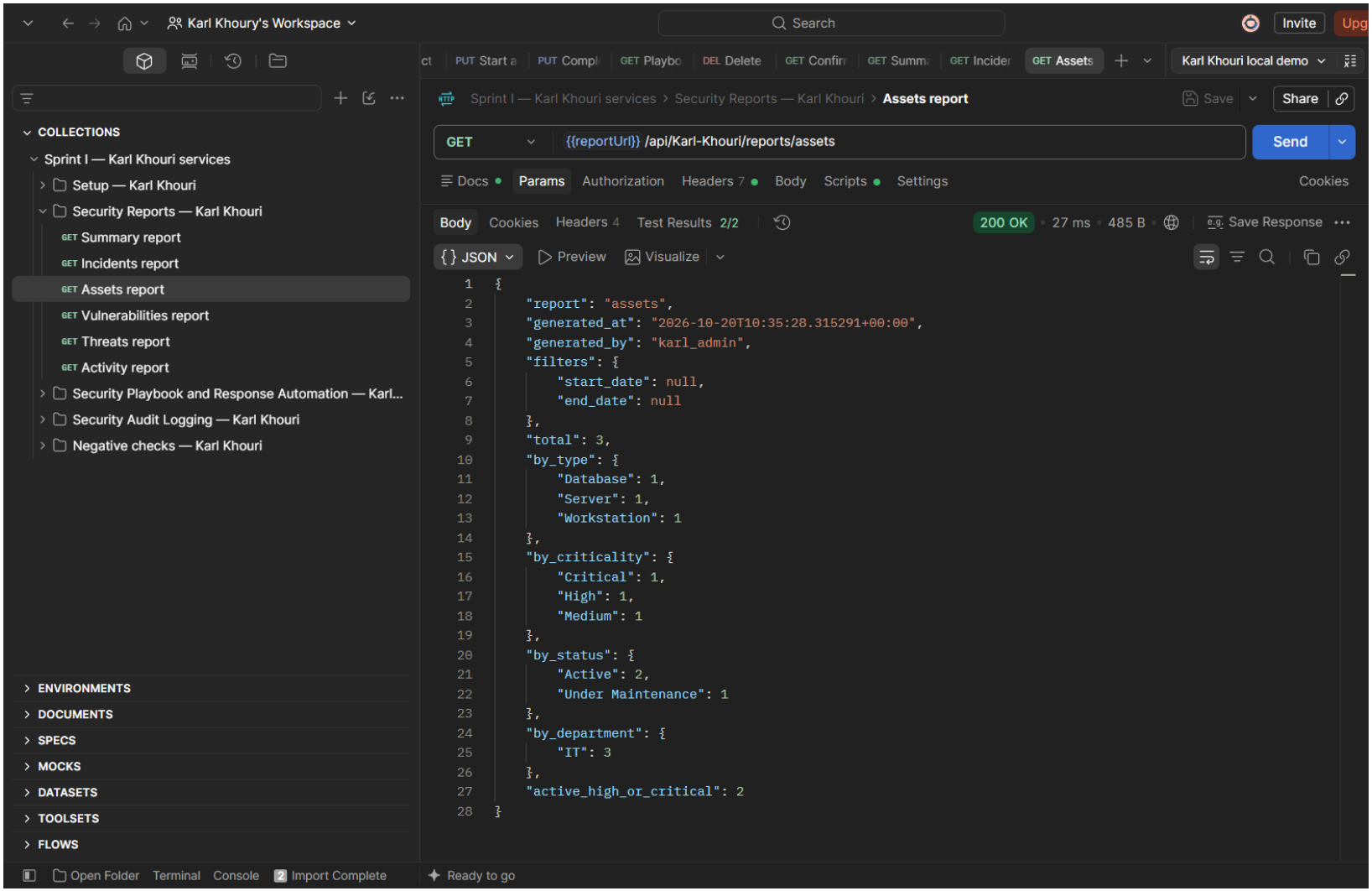

Figure 174. Karl Khouri - GET Assets Report - Asset totals by type, criticality, status and department from the asset API.

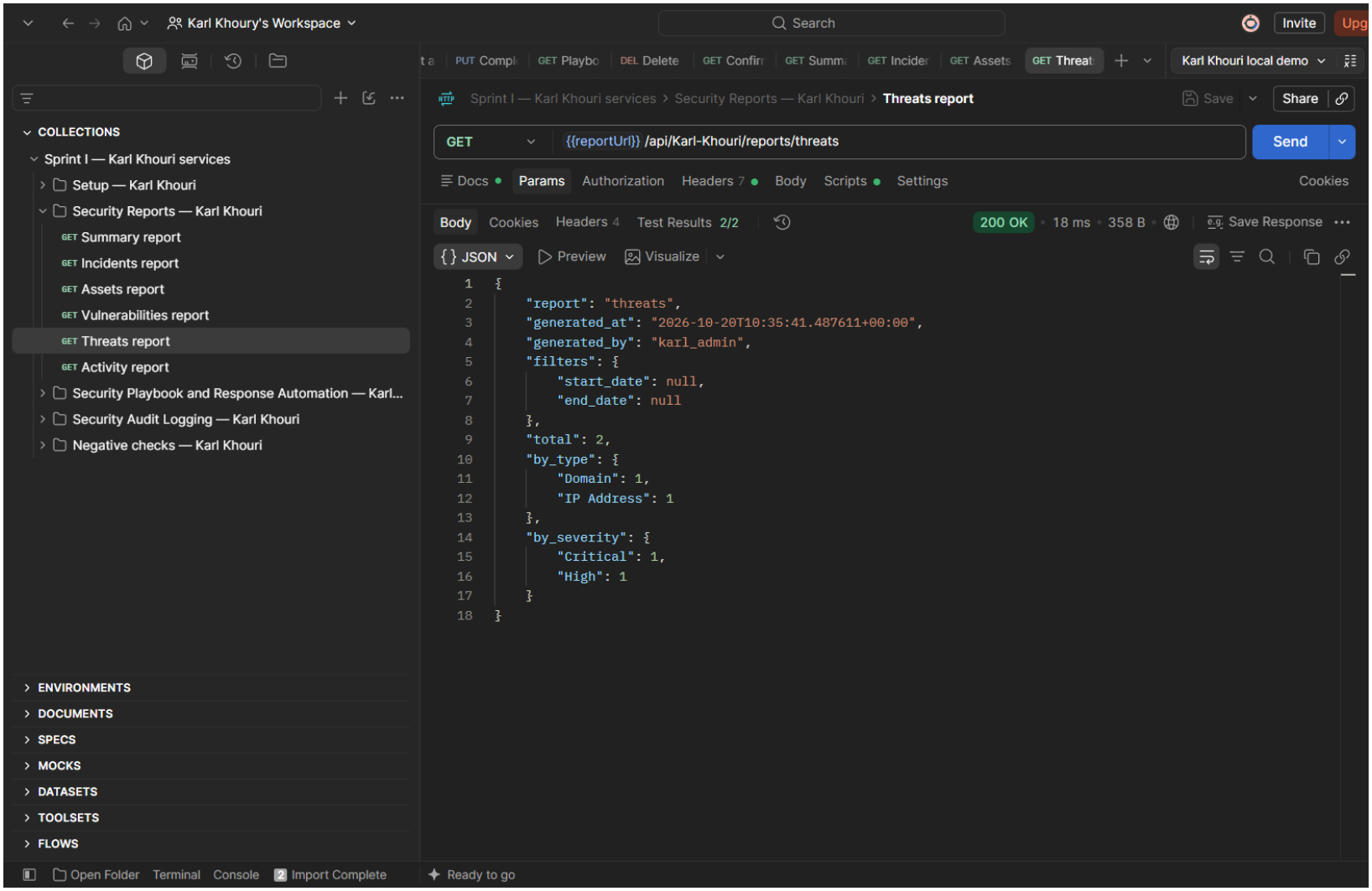

Figure 175. Karl Khouri - GET Threats Report - Threat totals by type and severity from the threat API.

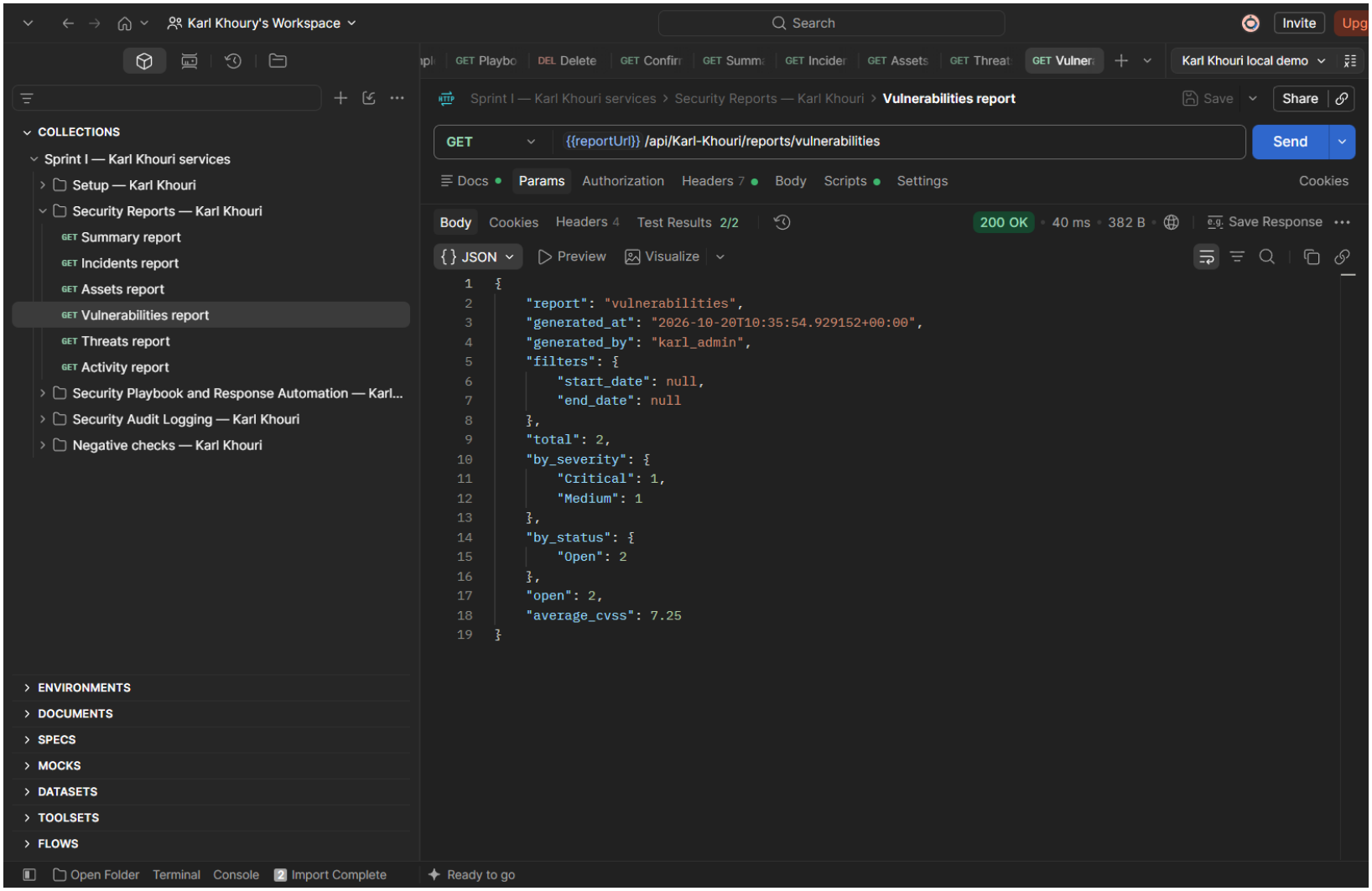

Figure 176. Karl Khouri - GET Vulnerabilities Report - Vulnerability totals by severity and status, open count and average CVSS.

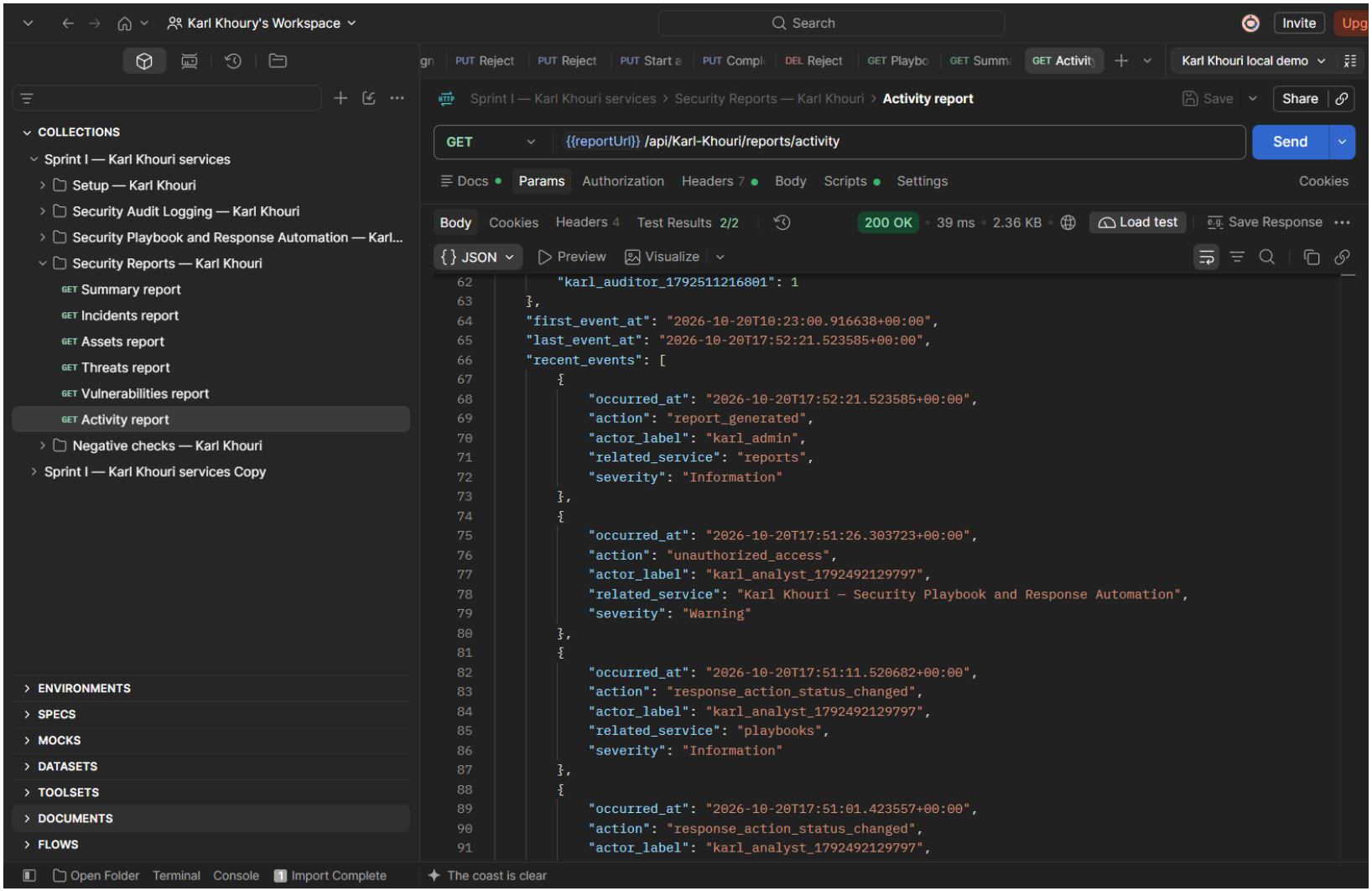

Figure 177. Karl Khouri - GET Activity Report - Audit activity counts plus the most recent security activities, newest first.

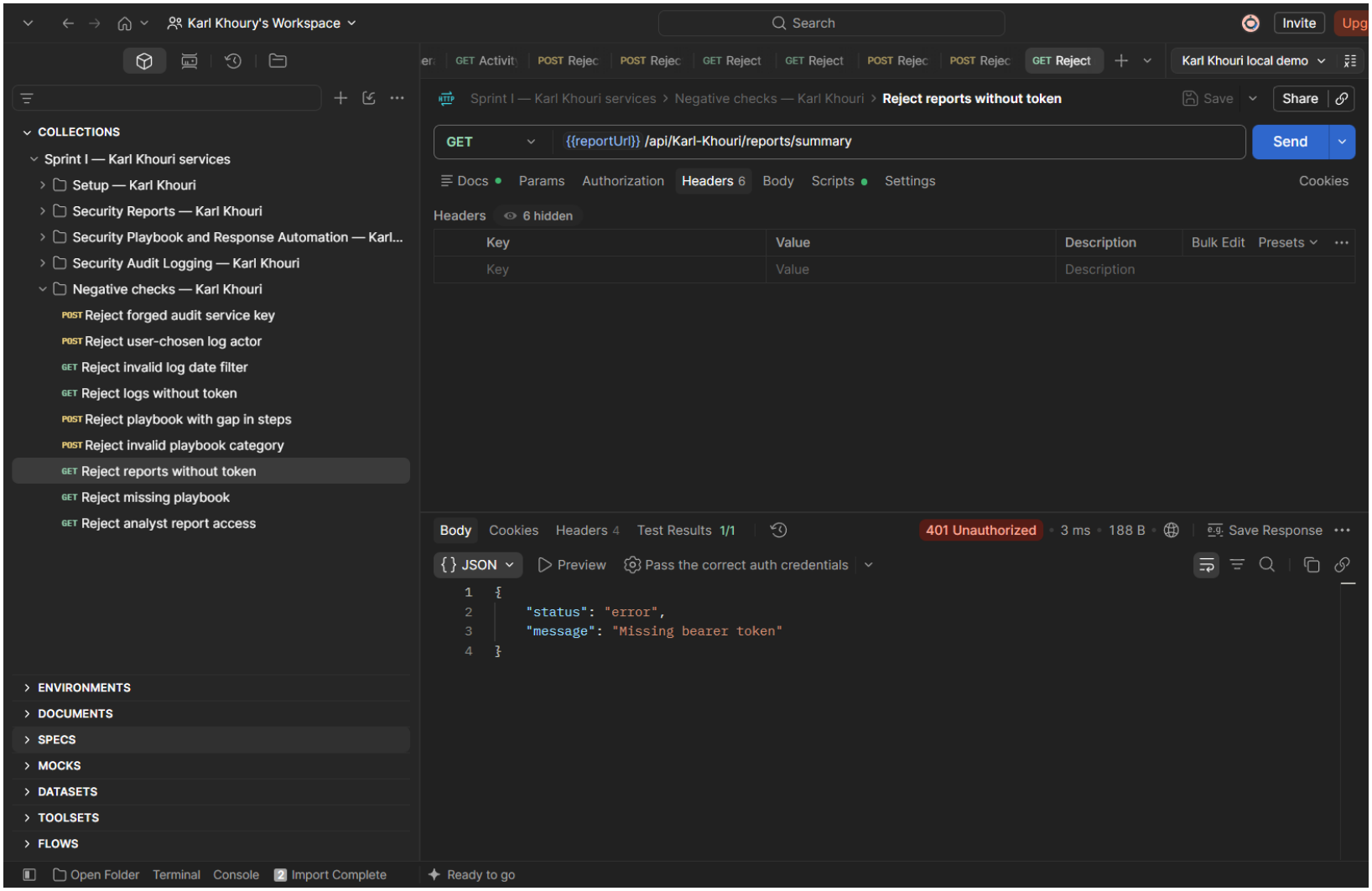

Figure 178. Karl Khouri - GET Reject Reports Without Token - An unauthenticated report request is rejected with HTTP 401.

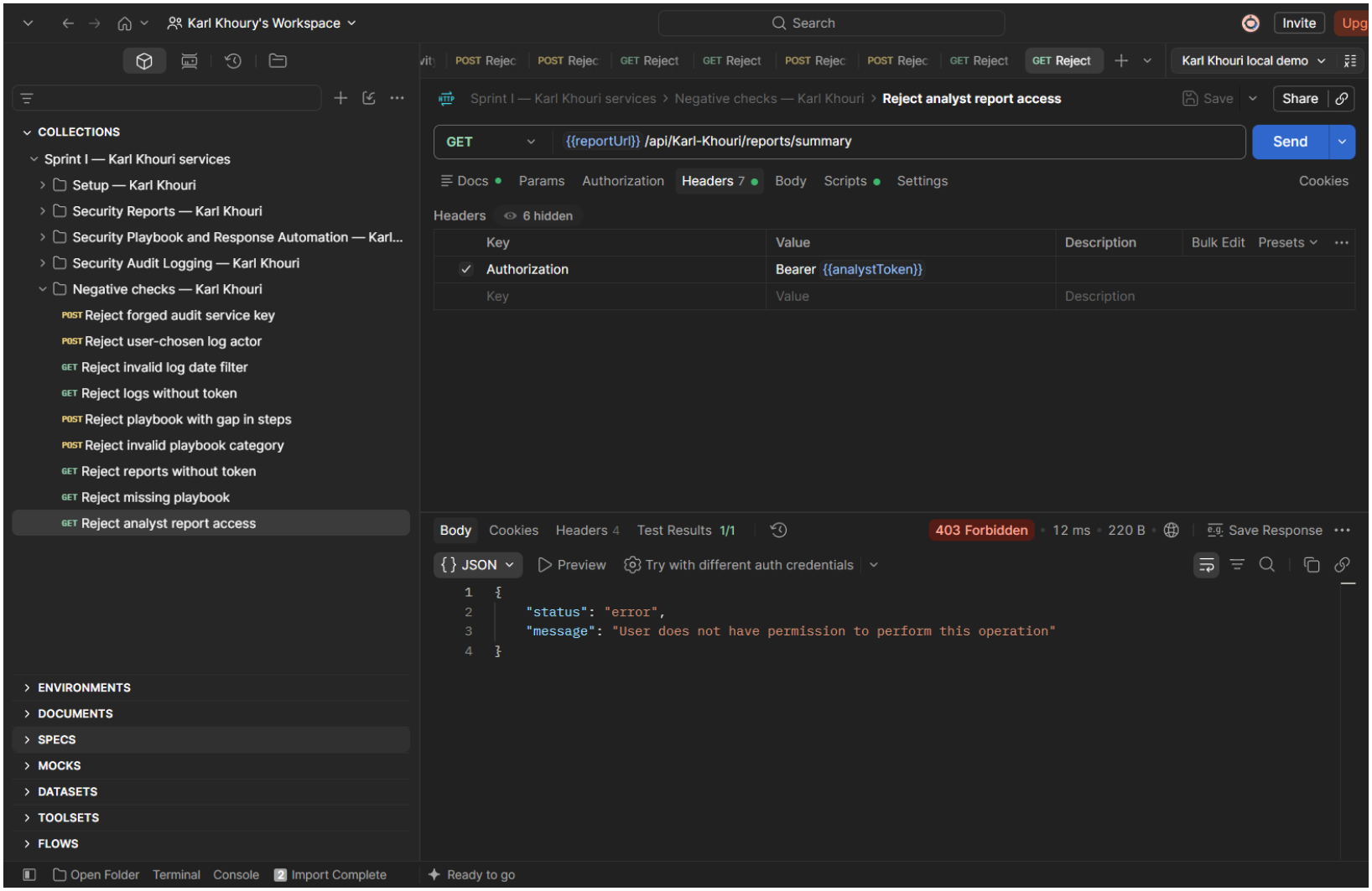

Figure 179. Karl Khouri - GET Reject Analyst Report Access - Only administrators and auditors can generate reports; HTTP 403.

Karl Khouri - SQLite karl-demo.db - 01

```
Python sqlite3 module (SQLite 3.45.1) - queries run against C:\Users\user\Desktop\435L\karl-demo\karl-demo.db

SQL> SELECT sql FROM sqlite_master WHERE name IN ('audit_logs', 'audit_logs_actor', 'audit_logs_occurred', 'audit_logs_immutable');

CREATE TABLE "audit_logs" (
 id INTEGER PRIMARY KEY AUTOINCREMENT, event_id TEXT NOT NULL UNIQUE,
 actor_user_id INTEGER REFERENCES users(id) ON DELETE SET NULL, original_actor_user_id INTEGER, actor_label TEXT,
 action TEXT NOT NULL, description TEXT NOT NULL,
 severity TEXT NOT NULL CHECK(severity IN ('Information','Warning','Error','Critical')),
 related_service TEXT NOT NULL, subject_id TEXT, occurred_at TEXT NOT NULL, recorded_at TEXT NOT NULL,
 source TEXT NOT NULL CHECK(source IN ('service','user','audit'))
);

CREATE INDEX audit_logs_actor ON audit_logs(original_actor_user_id);

CREATE INDEX audit_logs_occurred ON audit_logs(occurred_at);

CREATE TRIGGER audit_logs_immutable BEFORE UPDATE ON audit_logs
WHEN NOT (NEW.actor_user_id IS NULL AND NEW.id IS OLD.id AND NEW.event_id IS OLD.event_id AND NEW.original_actor_user_id IS OLD.original_actor_user_id
 AND NEW.actor_label IS OLD.actor_label AND NEW.action IS OLD.action AND NEW.description IS OLD.description AND NEW.severity IS OLD.severity
 AND NEW.related_service IS OLD.related_service AND NEW.subject_id IS OLD.subject_id AND NEW.occurred_at IS OLD.occurred_at
 AND NEW.recorded_at IS OLD.recorded_at AND NEW.source IS OLD.source)
BEGIN SELECT RAISE(ABORT, 'Audit log entries are immutable'); END;
```

Figure 180. Karl Khouri - audit_logs schema - Table definition with primary key, actor foreign key, unique event ID, CHECK constraints and the immutability trigger.

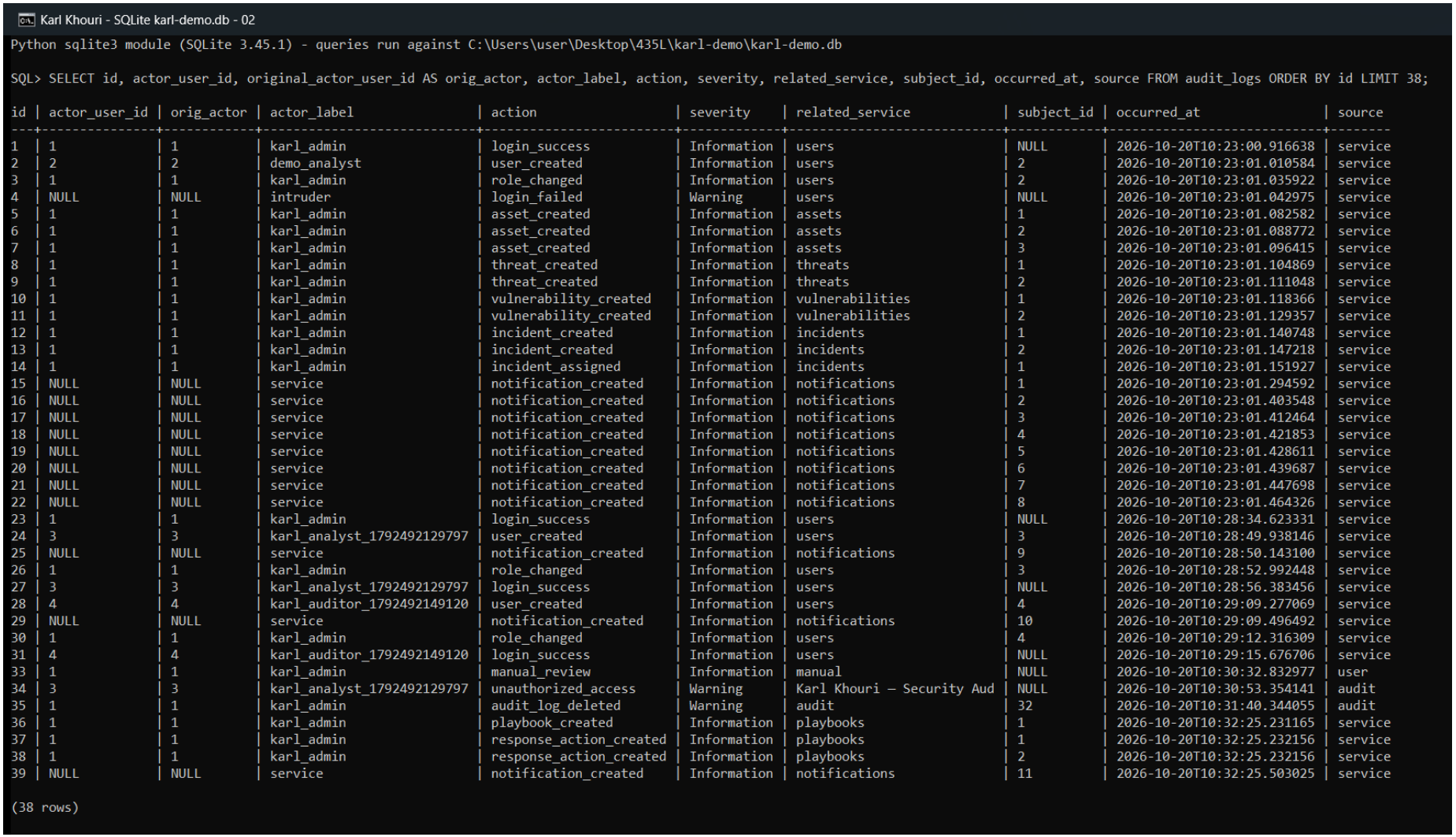

Figure 181. Karl Khouri - audit_logs records - Entries delivered by the users, assets, threat, vulnerability, incident, notification and playbook services. Row 4 is a failed login with a null actor; entry 32 was deleted through the API, and deletion record 35 names it.

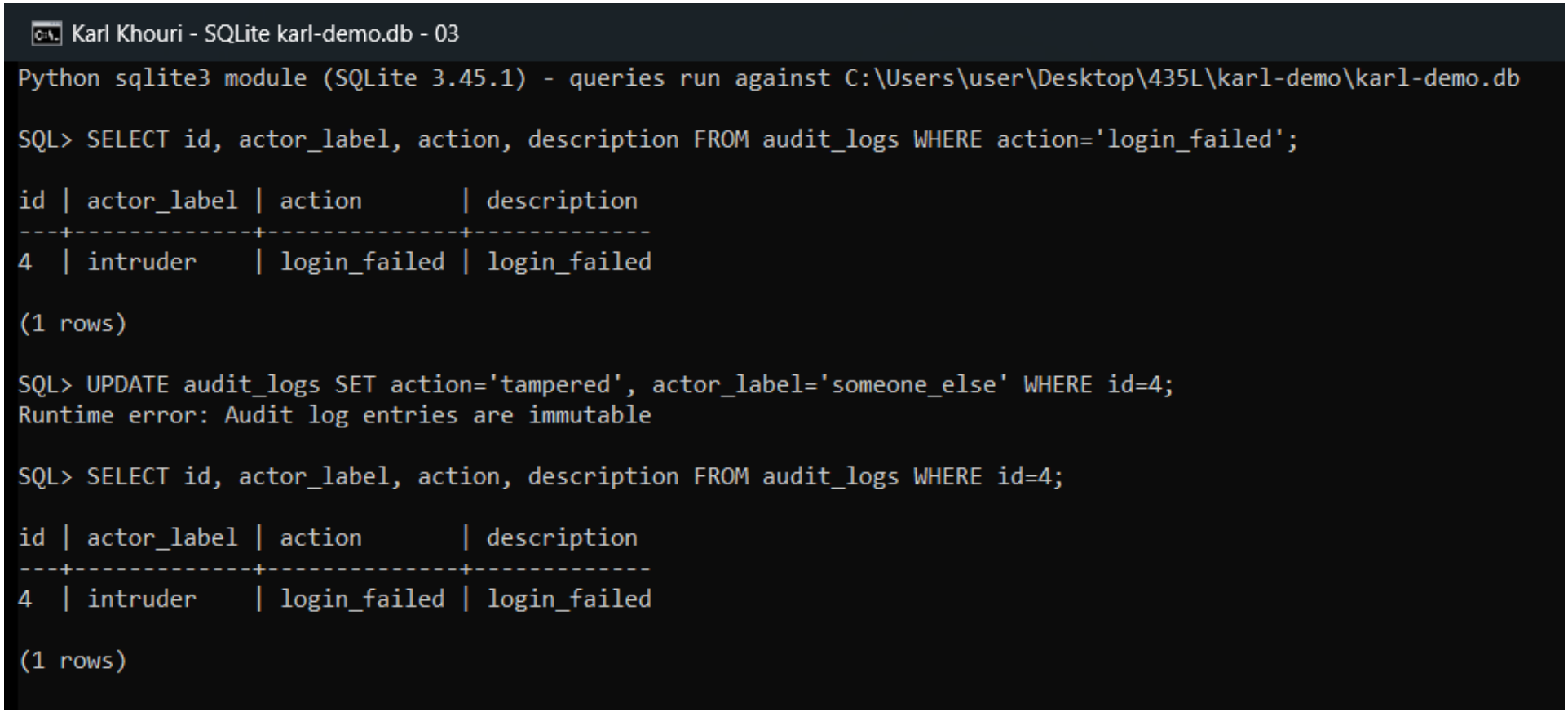

Figure 182. Karl Khouri - Reject audit log update - A direct SQL UPDATE, run through Python's sqlite3 module against the demo database, is aborted by the immutability trigger and the row is unchanged.

```
Karl Khouri - SQLite karl-demo.db - 04
Python sqlite3 module (SQLite 3.45.1) - queries run against C:\Users\user\Desktop\435L\karl-demo\karl-demo.db

SQL> SELECT sql FROM sqlite_master WHERE name IN ('playbooks', 'response_actions', 'action_history');

CREATE TABLE action_history (
 id INTEGER PRIMARY KEY, action_id INTEGER REFERENCES response_actions(id) ON DELETE SET NULL, original_action_id INTEGER NOT NULL,
 playbook_id INTEGER REFERENCES playbooks(id) ON DELETE SET NULL, original_playbook_id INTEGER NOT NULL,
 from_status TEXT, to_status TEXT NOT NULL, changed_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
 changed_by_label TEXT NOT NULL, changed_at TEXT NOT NULL, note TEXT
);

CREATE TABLE "playbooks" (
 id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL COLLATE NOCASE UNIQUE, description TEXT NOT NULL,
 category TEXT NOT NULL CHECK(category IN ('Malware','Phishing','Ransomware','Unauthorized Access','Data Breach','Denial of Service','Insider Threat','Vulnerability Remediation','Other')),
 severity TEXT NOT NULL CHECK(severity IN ('Low','Medium','High','Critical')),
 created_by INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT, created_at TEXT NOT NULL, updated_at TEXT NOT NULL
);

CREATE TABLE "response_actions" (
 id INTEGER PRIMARY KEY AUTOINCREMENT, playbook_id INTEGER NOT NULL REFERENCES playbooks(id) ON DELETE CASCADE,
 step_order INTEGER NOT NULL CHECK(step_order >= 1), title TEXT NOT NULL, description TEXT NOT NULL,
 assigned_to INTEGER REFERENCES users(id) ON DELETE RESTRICT,
 status TEXT NOT NULL DEFAULT 'Pending' CHECK(status IN ('Pending','In Progress','Completed')),
 created_at TEXT NOT NULL, started_at TEXT, completed_at TEXT, completed_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
 UNIQUE(playbook_id, step_order)
);
```

Figure 183. Karl Khouri - playbooks, response_actions and action_history schema - Primary keys, foreign keys, cascade rule, the unique step order and the nullable assignee.

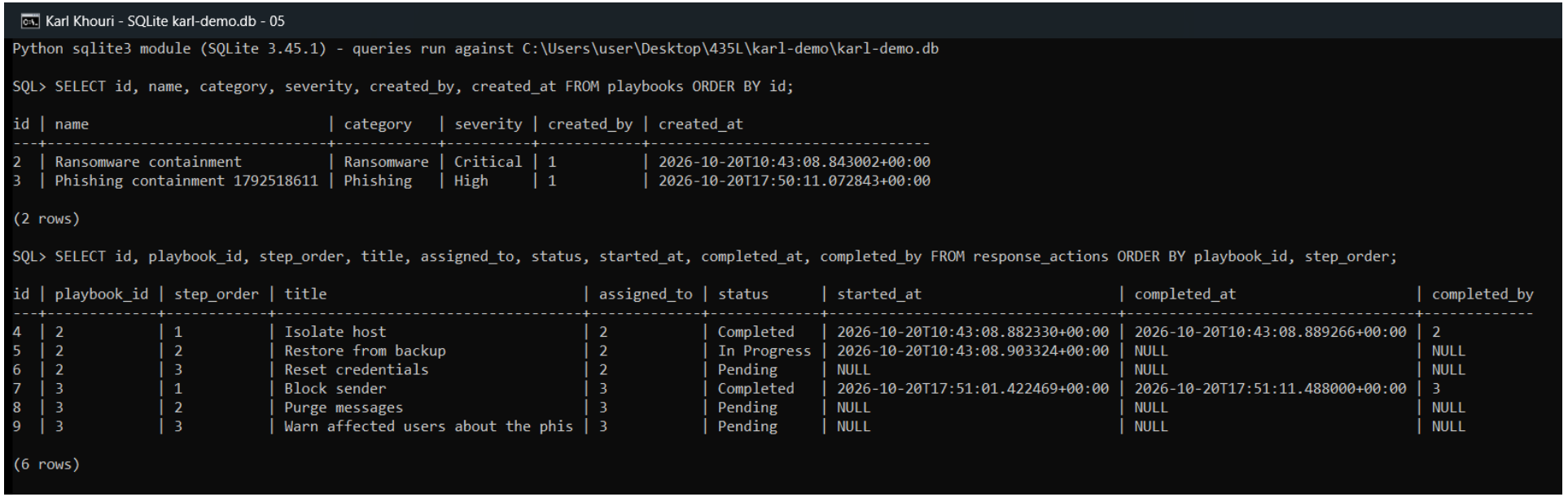

Figure 184. Karl Khouri - playbooks and response_actions records - Two stored playbooks with ordered steps, assignees, statuses and completion dates; IDs continue after the deleted playbook 1 without reuse.

Karl Khouri - SQLite karl-demo.db - 06

```
Python sqlite3 module (SQLite 3.45.1) - queries run against C:\Users\user\Desktop\435L\karl-demo\karl-demo.db

SQL> SELECT id, action_id, original_action_id AS orig_action, playbook_id, original_playbook_id AS orig_playbook, from_status, to_status, changed_by_label, changed_at, note FROM action_history ORDER BY id;

id | action_id | orig_action | playbook_id | orig_playbook | from_status | to_status   | changed_by_label           | changed_at                     | note
---+-----------+-------------+-------------+---------------+-------------+-------------+----------------------------+--------------------------------+--------------------------------
1  | NULL      | 1           | NULL        | 1             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:32:25.232156+00:00 | Step created
2  | NULL      | 2           | NULL        | 1             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:32:25.232156+00:00 | Step created
3  | NULL      | 3           | NULL        | 1             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:32:52.883487+00:00 | Step created
4  | NULL      | 1           | NULL        | 1             | Pending     | In Progress | karl_analyst_1792492129797 | 2026-10-20T10:33:40.037447+00:00 | Started
5  | NULL      | 1           | NULL        | 1             | In Progress | Completed   | karl_analyst_1792492129797 | 2026-10-20T10:33:52.031866+00:00 | Done
6  | 4         | 4           | 2           | 2             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:43:08.846019+00:00 | Step created
7  | 5         | 5           | 2           | 2             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:43:08.846019+00:00 | Step created
8  | 6         | 6           | 2           | 2             | NULL        | Pending     | karl_admin                 | 2026-10-20T10:43:08.846019+00:00 | Step created
9  | 4         | 4           | 2           | 2             | Pending     | In Progress | demo_analyst               | 2026-10-20T10:43:08.882330+00:00 | In Progress by analyst
10 | 4         | 4           | 2           | 2             | In Progress | Completed   | demo_analyst               | 2026-10-20T10:43:08.889266+00:00 | Completed by analyst
11 | 5         | 5           | 2           | 2             | Pending     | In Progress | demo_analyst               | 2026-10-20T10:43:08.903324+00:00 | NULL
12 | 7         | 7           | 3           | 3             | NULL        | Pending     | karl_admin                 | 2026-10-20T17:50:11.078479+00:00 | Step created
13 | 8         | 8           | 3           | 3             | NULL        | Pending     | karl_admin                 | 2026-10-20T17:50:11.078479+00:00 | Step created
14 | 9         | 9           | 3           | 3             | NULL        | Pending     | karl_admin                 | 2026-10-20T17:50:17.850252+00:00 | Step created
15 | 9         | 9           | 3           | 3             | Pending     | Pending     | karl_analyst_1792492129797 | 2026-10-20T17:50:33.366103+00:00 | Assigned to karl_analyst_1792492
16 | 7         | 7           | 3           | 3             | Pending     | In Progress | karl_analyst_1792492129797 | 2026-10-20T17:51:01.422469+00:00 | Started
17 | 7         | 7           | 3           | 3             | In Progress | Completed   | karl_analyst_1792492129797 | 2026-10-20T17:51:11.520682+00:00 | Done

(17 rows)
```

Figure 185. Karl Khouri - action_history records - Every step creation, assignment and status change with user and time. Rows for the deleted playbook 1 keep their original IDs with null links.